

NEUMANN JÁNOS
INFORMATIKAI KAR

Szénási Sándor

Algoritmusok, adatszerkezetek II.

ÓE-NIK 5013

Budapest, 2018.

Készült az Óbudai Egyetem Neumann János Informatikai Karán az ÓE-NIK 5013. sz.
jegyzetszerződés keretein belül 2014-ben.

Szerző: Dr. Szénási Sándor
egyetemi docens
szenasi.sandor@nik.uni-obuda.hu

Lektor: Dr. Vámossy Zoltán
egyetemi docens
vamossy.zoltan@nik.uni-obuda.hu

1.3.3. verzió
2020. június 20.

A jegyzet legfrissebb változata letölthető az alábbi címről:
http://users.nik.uni-obuda.hu/prog2/ProgramozasII.pdf

Ez a jegyzet LaTeX segítségével készült.

A mű egyéni tanulmányozás céljára szabadon letölthető. Minden egyéb felhasználás csak a szerző írásos engedélyével lehetséges.

ISBN 978-615-5460-43-2

Tartalomjegyzék

Bevezetés **1**

1. Visszalépéses keresés **5**
1.1. Visszalépéses keresés alapgondolata . . . 5
1.1.1. Alapelv . . . 5
1.1.2. Példa visszalépéses keresésre . . . 6
1.2. Visszalépéses keresés általános alakja . . . 12
1.2.1. Bemenet és kimenet . . . 12
1.2.2. Tetszőleges megoldás keresése . . . 13
1.2.3. Összes megoldás keresése . . . 14
1.2.4. Optimális megoldás keresése . . . 16
1.2.5. Egymást kölcsönösen kizáró részfeladatok . . . 17
1.3. Visszalépéses keresés példák . . . 19
1.3.1. Megoldható feladatok osztályozása . . . 19
1.3.2. Logikai, egymást kizáró részmegoldás - Időzítési probléma . . . 20
1.3.3. M darab, egymást kizáró részmegoldás - Sudoku feladat . . . 21
1.3.4. Előzményfüggő, egymást kizáró részmegoldás - Szólánc játék . . . 22
1.3.5. Logikai, nem kizáró részmegoldás - Hátizsák-pakolás . . . 23
1.3.6. M darab, nem kizáró részmegoldás - Feladatok kiosztása . . . 24
1.3.7. Előzményfüggő, nem kizáró részmegoldás - Huszár útja a sakktáblán . . . 25

2. Optimalizációs stratégiák **27**
2.1. Alapfogalmak . . . 27
2.1.1. Optimalizáció . . . 27
2.1.2. A 0-1 hátizsák probléma . . . 27
2.2. Nyers erő módszere . . . 30
2.3. Oszd meg és uralkodj módszer . . . 32
2.4. Feljegyzéses módszer . . . 35
2.5. Dinamikus programozás . . . 38
2.6. Mohó algoritmusok . . . 54
2.7. Visszalépéses keresés . . . 58
2.8. Szétválasztás és korlátozás . . . 61
2.9. A módszerek összehasonlítása . . . 64

3. Hasító táblázat **65**
3.1. Alternatívák elemzése . . . 65
3.1.1. Tömbök felépítése . . . 65
3.1.2. Közvetlen címzés . . . 66
3.2. Hasító táblázatok felépítése . . . 68
3.3. Hasítófüggvény megvalósítások . . . 70
3.3.1. Az ideális hasítófüggvény . . . 70
3.3.2. Néhány hasítófüggvény megvalósítás . . . 70
3.3.3. További megfontolások . . . 72
3.4. Kulcsütközések kezelése . . . 74
3.4.1. Túlcsordulási terület . . . 74

3.4.2. Láncolás használata . . . 75
3.4.3. Nyílt címzés . . . 76
3.4.4. Többszörös hasítás . . . 80

4. Láncolt lista **81**
4.1. Láncolt listák felépítése . . . 81
4.1.1. Lista elemeinek szerkezete . . . 81
4.2. Egyirányú, egyszerű láncolt lista alapműveletei . . . 84
4.2.1. Inicializálás . . . 84
4.2.2. Új elem felvétele . . . 84
4.2.3. Bejárás és keresés . . . 89
4.2.4. Törlés láncolt listából . . . 91
4.3. Rendezett láncolt lista . . . 96
4.3.1. Rendezett láncolt lista felépítése . . . 96
4.3.2. Beszúrás rendezett láncolt listába . . . 96
4.3.3. Keresés rendezett láncolt listába . . . 103
4.4. Láncolt lista strázsa elemekkel . . . 105
4.4.1. Beszúrás és törlés helyi műveletekkel . . . 105
4.4.2. Strázsa elemek használata . . . 107
4.5. Speciális láncolt listák . . . 109
4.5.1. Kétirányú láncolt lista . . . 109
4.5.2. Többszörösen láncolt lista . . . 109
4.5.3. Ciklikus láncolt lista . . . 110
4.6. Láncolt listák implementációja . . . 112
4.6.1. Implementáció tömbökkel . . . 112
4.6.2. Implementáció mutatókkal/referenciákkal . . . 113

5. Bináris keresőfa **115**
5.1. Bináris keresőfa felépítése . . . 115
5.2. Bejárások . . . 118
5.2.1. Preorder bejárás . . . 118
5.2.2. Inorder bejárás . . . 122
5.2.3. Postorder bejárás . . . 125
5.2.4. Keresés . . . 129
5.3. Új elem felvétele . . . 131
5.4. Elem törlése . . . 135

6. B-fa **145**
6.1. Fák kiegyensúlyozottsága . . . 145
6.2. B-fa felépítése . . . 146
6.3. Beszúrás B-fába . . . 148
6.3.1. Beszúrás pszeudokódja . . . 153
6.3.2. Beszúrás segédalgoritmusok . . . 154
6.4. Törlés B-fából . . . 157
6.4.1. Kulcs eltávolítása . . . 157
6.4.2. Legalább t darab kulcs biztosítása . . . 163
6.4.3. Törlés pszeudokódja . . . 171
6.4.4. Törlés segéd algoritmusok . . . 174

7. Gráf **177**
7.1. Gráfok felépítése . . . 177
7.1.1. Alapvető jellemzők . . . 177
7.1.2. Alapműveletek . . . 177
7.2. Gráfok implementációja . . . 179
7.2.1. Dinamikus tárolás . . . 179
7.2.2. Tárolás csúcsmátrixban . . . 179
7.2.3. Szomszédsági listában tárolás . . . 180

7.3. Bejárások . . . 182
7.3.1. Szélességi bejárás . . . 182
7.3.2. Mélységi bejárás . . . 187
7.4. Legrövidebb út keresése (Dijkstra algoritmus) . . . 193
7.4.1. Az alapelv . . . 193
7.4.2. Dijkstra algoritmus pszeudokódja . . . 193
7.4.3. Dijkstra algoritmus kimenetének feldolgozása . . . 200
7.5. Minimális feszítőfa keresése . . . 203
7.5.1. Prim algoritmusa . . . 203
7.5.2. Kruskal algoritmusa . . . 203

Irodalomjegyzék **203**

Bevezetés

Ez a jegyzet az Óbudai Egyetem Neumann János Informatikai Karán tanuló mérnökinformatikus hallgatók számára íródott, hogy a Programozás II. tárgy előadásai és gyakorlatai mellett egy írásos anyag is segítse a felkészülésüket. Bár a jegyzet anélkül is érthető, mégis erősen javasolt az előadások látogatása, illetve a gyakorlatokon való aktív részvétel, mivel mindkettő tartalmazhat olyan témaköröket, amelyek itt nem jelennek meg.

A jegyzet erőteljesen épít az „Algoritmusok, adatszerkezetek I." jegyzet [9] anyagára, az ott bevezetett fogalmakat (típusok, strukturált programozás, stb.) és technikákat (programozási tételek, rendezések, stb.) itt már ismertnek tekintjük, így azok újbóli bemutatására már nem kerül sor. Hasonló módon feltételezzük, hogy az olvasó rendelkezik legalább egy objektum-orientált programozási nyelvben közepes szintű ismeretekkel. Bár a jegyzet írásakor fontos szempont volt, hogy csak általános, konkrét programozási nyelvektől független leírásokat tartalmazzon, a megértést mégis nagyban segíti, ha az itt megjelenő fogalmakat és pszeudokódokat hozzá tudjuk kötni konkrét programozási nyelvi elemekhez (különösen kiemelve ez utóbbiakat, amelyeket esetenként célszerű lehet implementálni is egy választott nyelven).

A tananyag határainak megszabásakor arra törekedtünk, hogy az összhangban legyen a BSc képzésben résztvevő mérnökinformatikusok elvárásaival (illetve a tőlük elvártakkal). Emiatt a konkrét programozási nyelvek szintjénél jóval absztraktabb módon, általános formában tárgyaljuk a témaköröket. Az elméleti fejtegetések azonban csak olyan szinten jelennek meg, hogy azok rávilágítsanak az okokra és okozatokra, a legtöbb esetben nem ragaszkodunk a precíz matematikai definíciókhoz és bizonyításokhoz. Ez a szabatosságból tett engedmény remélhetőleg megtérül abban, hogy a jegyzet gördülékenyebben olvasható, könnyebben tanulható azok számára, akik most találkoznak először az itt leírtakkal. A szövegben található szakirodalmi hivatkozások pedig útmutatást adnak azoknak, akik mélyebben is el szeretnének merülni egy-egy témakörben.

Az egyes technikák bemutatása mindig az alábbi lépéseket követi majd: a probléma rövid bemutatása, ennek megfelelően a szükséges modell (fogalmak, stb.) felállítása, megoldás elvi hátterének áttekintése, egy lehetséges megoldást jelentő pszeudokód bemutatása és annak részletes elemzése, majd pedig egy-egy példán keresztül a fentiek gyakorlati vizsgálata (néhány, bonyolultabb esetben a példa megelőzheti a pszeudokódot, ezzel is elősegítve az alapelv megértését). Fontos kihangsúlyozni, hogy általában csak egy lehetséges megoldást mutatunk be a sok közül, a legtöbb esetben találhatunk ennél rövidebbet vagy akár hatékonyabbat. A jegyzet célja ugyanis nem a végletekig optimalizált, legkifinomultabb módszerek bemutatása, ehelyett megelégszünk az alapelvek áttekintésével.

A szöveg gyakran tartalmaz utalásokat az egyes pszeudokódok soraira, illetve egyes ábrákra, esetenként alábrákra. A hivatkozások túl nagy száma már az olvashatóság rovására menne, emiatt ezek csak a külön figyelmet érdemlő, illetve kulcspontokat jelentő részeknél jelennek meg. Célszerű emiatt az elvi háttér megértését követően mind a pszeudokódokat, mind pedig az ábrákat külön is áttekinteni. A megjegyzéseknek köszönhetően ezek önmagukban, a szövegtől függetlenül is könnyen értelmezhetők.

Fejezetek rövid áttekintése

A jegyzet felépítése a már említett Programozás II. tárgy tematikáját követi, és bár az egyes fejezetek másmás témaköröket dolgoznak fel, azok gyakran egymásra épülnek, így az egyes fejezetekben feltételezzük az előzőek ismeretét.

Az 1. fejezetben egy széleskörűen alkalmazható problémamegoldási technikát, a *visszalépéses keresést* ismerhetjük meg. Ez nagyon hatékonyan használható olyan esetekben, ha több részfeladatot kell megoldanunk, és az ezekre adható megoldások valamilyen formában kizárják egymást. A visszalépéses keresés

alapelvén túlmenően számos példát is tartalmaz a jegyzet, azt hangsúlyozva, hogy ugyanaz az algoritmus milyen sokrétűen használható egymástól jelentősen különböző feladatok megoldása során.

A későbbi fejezetek egy nagyobb egységet alkotnak, ugyanis különböző *adatszerkezeteket* mutatnak be. Adatszerkezetek alatt a számítógépben való tárolás lehetséges módjait értjük, amelyek alapvetően az alábbiak specifikálását jelentik:

- Az adatok memóriabeli tárolása: A számítógép memóriájában az adott adatszerkezet milyen formában valósítja meg az egyes elemek, illetve a köztük lévő kapcsolatok eltárolását.
- Az adatokon végezhető műveletek: Az adatok eltárolásán túl biztosítanunk kell az ezekhez való hozzáférést is. Ez a különböző tárolási módot használó adatszerkezetek esetében egészen más algoritmusokat igényelhet. Bár az egyes módszerek más-más műveleteket tesznek lehetővé, általában elmondható, hogy az alábbiakhoz hasonlókkal találkozhatunk:
  - Inicializálás: Alaphelyzetbe hozza az adatszerkezetet. Ilyenkor még nem tárol elemeket, azonban már alkalmas a későbbi műveletek végrehajtására.
  - Beszúrás: Eltárol egy új elemet az adatszerkezetben. Ez a konkrét megvalósítástól függően különböző kiegészítő paramétereket igényelhet (új elem helye, kulcsa, stb.).
  - Bejárás: Az adatszerkezet minden egyes elemének a feldolgozása. Maga a feldolgozás művelete tetszőleges lehet, valójában nem is ez érdekes számunkra, hanem az elemek egyenkénti elérése.
  - Keresés: Egy megadott elem (kulcs alapján, tulajdonság alapján, stb.) megkeresése az adatszerkezetben.
  - Törlés: A megadott kulcsú elem eltávolítása az adatszerkezetből.

A fenti leírások meglehetősen elnagyoltak, ez azonban ezen a szinten még szándékos. Célunk annak bemutatása, hogy az egymástól jelentősen különböző adatszerkezetek is alapvetően hasonló műveleteket támogatnak, még ha a háttérben egészen máshogy is hajtják azokat végre. Hasonló módon azt sem akarjuk pontosítani, hogy milyen típusú adatokat tárolunk el (számot, szöveget, objektumot, stb.), ugyanis az összes módszernél látni fogjuk, hogy az adatszerkezetek felépítése és működése alapvetően független attól, hogy mit tárolunk el bennük.

A 3. fejezetben található *hasító táblázat* valószínűleg ismerős lesz mindenki számára, ez ugyanis a hagyományos tömbök általánosításaként fogható fel. Legnagyobb előnye, hogy az elemek kulcsa egyértelműen (egy függvény segítségével) meghatározza azok memóriabeli helyét, ennek köszönhetően nagyon gyors beszúrás/keresés/törlés műveleteket tudunk készíteni. Hátránya, hogy a bejárás viszont nehezen oldható meg, illetve nagy számú kulcs esetén a remek teljesítményt se tudjuk mindig biztosítani.

A tömbökhöz képest jóval nagyobb változást jelent majd a *láncolt lista* adatszerkezet, amellyel a 4. fejezet foglalkozik. Bár a listákat is eltárolhatjuk a tömbökhöz hasonló módon, de általában dinamikus adatszerkezetként tekintünk rájuk, tehát az elemek által ténylegesen elfoglalt memóriaterület futás közben is változhat a beszúrások és törlések hatására. Ez utóbbi már önmagában is meglehetősen nagy előnyt jelenthet, és ezt még kiegészíti az a jó tulajdonsága, hogy a módosító műveletek (beszúrás, törlés) mindig csak a módosítandó elem közvetlen környezetét érintik, így gyorsan elvégezhetők. Az adatszerkezet sajnos számos hátránnyal is bír, megnövekszik a tárhelyigény, illetve elveszítjük a gyors véletlen elérés (és ezzel a hatékony keresés) lehetőségét. A jegyzet többféle láncolt lista megvalósítással is foglalkozik.

A láncolt listák számos problémájára megoldást nyújt az 5. fejezetben található *bináris keresőfa*. Ez is rendelkezik a dinamikus adatszerkezetek előnyeivel, sőt, ebben az esetben már a keresésre is találunk meglehetősen hatékony megoldást. A helyfoglalás itt azonban még nagyobb lesz, illetve látni fogjuk, hogy a keresés is csak bizonyos esetekben lesz kielégítő.

A 6. fejezetben található *B-fa* kiküszöböli a bináris keresőfa gyengeségeit is. A fa felépítése biztosítja a folyamatos gyors karbantartási és keresési műveleteket, továbbá a tárolás is optimális módon történik, kihasználva napjaink számítógépeinek (merevlemezeinek) felépítését. Működése ugyan az előzőekhez képest jóval bonyolultabb algoritmusokat igényel, azonban érdemes vele foglalkozni, hiszen napjainkban is számos helyen találkozhatunk ennek gyakorlati használatával.

Az utolsó tárgyalt adatszerkezet a 7. fejezetben bemutatott *gráf* lesz. Ez szintén ismerős lehet, hiszen ez tulajdonképpen megfelel a matematika tárgyakból már előzőleg megismert gráfoknak. Jelen jegyzetben emiatt nem is szeretnénk elmerülni ezek gyakorlati használatában, hiszen az nagyon messzire vezetne, pusztán az adatszerkezetként való használat alapjait tekintjük át: milyen formában tudjuk eltárolni a

gráfokat, illetve hogyan tudunk hozzáférni a bennük eltárolt adatokhoz. A fejezet tartalmaz egy részletes megoldást a legrövidebb utak keresésére, de ezt tekintsük egy önkényesen kiragadott (látványos) példának a szakirodalomban található számtalan gráf algoritmus közül.

A jegyzetben szereplő pszeudokódokról

Az algoritmusok bemutatása mindig pszeudokódok segítségével történik. Ezek használata gyakran felvet kényes kérdéseket, mivel a konkrét programozási nyelvekkel ellentétben nincsenek egységes szabályok a pszeudokódok írására, így nehéz körülhatárolni, hogy milyen szintű műveleteket (pl. csere?) tekintünk a kódban alapműveletnek, vagy külön kifejtendőnek (bár jegyezzük meg, hogy ez a „hátrány" adja a pszeudokódok használatának létjogosultságát is, hiszen így szabadon határozhatjuk meg a használni kívánt absztrakciós szintet). A jegyzetben található kódok alapvetően alacsony szintűek, már-már a programozási nyelvekhez egészen hasonlóak, hogy ez is elősegítse az esetleges implementáció zökkenőmentességét. Ezt a szabályt azonban bármikor megszegjük, amikor a könnyebb olvashatóság azt megkívánja (pl. halmazműveleteket is használunk a kódokban).

A jegyzetben használt pszeudokódok az alábbi részekből állnak össze:

- **Bemenet**: Az algoritmus bemenetét jelentő paraméterek felsorolása. A paraméterek neve mellett mindig szerepel a paraméterek típusa, esetleg egy rövid leírás.
- **Kimenet**: Az algoritmus kimenete, ami alapvetően két formában jelenhet meg:
  - Függvények esetében a visszatérési érték: A pszeudokódban megengedjük, hogy egy függvénynek több visszatérési értéke is legyen. Mivel a visszatérési értéknek nincs „neve" így itt általában azt a változót adjuk meg névként, aminek az értékét visszaadja a függvény futása végén (néha előfordul, hogy ilyen változó nincs, ilyenkor a név egy egyéb, a visszatérési értékre jellemző szó).
  - Függvények vagy eljárások esetében a címszerint átadott paraméterek: Az algoritmus futása közben ezeket megváltoztathatja, ami hatással van a paraméterként átadott változókra is, emiatt ezeket is kimenetnek tekintjük. Névként itt a paraméter nevét használjuk.
- Függvény vagy eljárás fejléce, illetve törzse: Ez tartalmazza magát az algoritmust.
- **Függvény hívása**: Rekurzív függvények esetében ez a szakasz mutatja meg a meghívás pontos módját.
- **Felhasznált változók és függvények**: Ez a terület tartalmazza azokat a pszeudokódban használt változókat, illetve meghívott egyéb függvényeket, amelyek leírása a bemenet/kimenet részből nem egyértelmű.

A függvények és eljárások pszeudokódja megfelel a szakirodalomban már megszokott formának, emiatt ennek részletes bemutatását itt mellőzzük. Alapvetően a strukturált programozás eszközeit használjuk itt, esetenként kiegészítve objektum-orientált jellemzőkkel (néha a rövidebb leírás kedvéért megszegjük ezeket a szabályokat, de csak akkor, ha az nem okoz félreértést).

Néhány további megjegyzés, értelmezési javaslat a pszeudokódokkal kapcsolatban:

- A változók esetében az olvashatóság érdekében nem adjuk meg külön azok deklarációját. Az egyes változók típusai sem szerepelnek abban az esetben, ha az egyértelmű (pl. tömbök címzésére használt változó nyilván egész lesz). Általában az alábbi elnevezéseket használjuk:
  - $i, j, k, ...$: egész számok, tipikusan tömbök címzésekor
  - $A, B, ...$: tömbök
  - $p, q, e, r, ...$: elemre való hivatkozást tartalmaznak (a pontos típus implementációfüggő)

  Amennyiben egy változó típusa nem egyértelmű, akkor az megtalálható a „Felhasznált változók és függvények" részben.
- Bizonyos esetekben szándékosan nem adunk meg pontos típust. Például az adatszerkezetekben egy **T** betűvel helyettesítjük az eltárolni kívánt tartalom típusát, ezzel is azt hangsúlyozva, hogy az tetszőleges lehet (szám, szöveg, stb.). Általában az alábbi elnevezéseket használjuk:

  - **T**: eltárolandó adat típusa (tetszőleges típus)
  - **K**: az eltárolandó adatokhoz tartozó kulcsok típusa (egyedi és általában összehasonlítható)
  - **M**: adatszerkezetek esetében egy elemre mutató hivatkozás típusa (tényleges implementációtól függően lehet mutató, referencia, index, stb.)

  Ezeknél a típusoknál néha élünk valamilyen megszorítással, pl. hogy a **T** összehasonlítható legyen. Tehát lehet bármilyen típus, aminél el tudjuk dönteni két elemről, hogy melyik kisebb.

- A tömbök indexelését (hacsak ezt külön nem jelezzük) mindig 1-től kezdjük.
- Egy A tömb i. elemét az alábbiak szerint érjük el: $A[i]$. Néha hasonló értelemben használjuk az A_i jelölést. Ez utóbbi alapvetően ugyanazt jelenti, de itt az A-t inkább mint tetszőleges adatszerkezetet tekintjük, nem ragaszkodunk ahhoz (vagy legalábbis nem akarjuk kihangsúlyzni), hogy az tömb legyen. Néha mindkét jelölés megjelenik egy kódban, ez általában nem okoz zavart, sőt, javítja az olvashatóságot, pl. $A[S[i]]$ helyett $A[S_i]$.
- Az $X.y$ formával jelöljük az X elem y tulajdonságát. Ez lehet akár egy tömb mérete, egy objektum mezője, egy szöveg első karaktere, stb.
- Az $f()$ vagy $X.f()$ formával jelöljük az f nevű függvény hívását, illetve az X elem f műveletének meghívását. Amennyiben nem nyilvánvaló, a „Felhasznált változók és függvények" rész tartalmazza, hogy mit csinál ez a függvény.
- Néhány speciális művelet, amiket gyakran használunk a kódokban:
  - $a \leftarrow b, a \leftrightarrow b$: értékadás, csere (hivatkozások esetében maguk a hivatkozások értéke változik, nem a hivatkozott elemé)
  - $a \wedge b, a \vee b, \neg a$: szokásos logikai operátorok (és, vagy, tagadás)
  - $a \in X, X \cup \{a\}$: szokásos halmazműveletek (eleme, unió)
  - $S \Leftarrow a, a \Leftarrow S$: az S sorba egy új elem elem elhelyezése, illetve kivétele
  - $V \Leftarrow a, a \Leftarrow V$: a V verembe egy új elem elem elhelyezése, illetve kivétele
  - $\emptyset, \otimes$: speciális konstansok, az üres illetve törölt tartalmat jelzik (pontos jelentésük az algoritmus leírásból derül ki)
  - $(a \leftarrow f\ ?\ b\ :\ c)$: a C nyelvekben megszokott művelet, amely szerint az a értéke legyen a b értéke akkor, ha az f igaz, illetve a c értéke egyébként

Végül egy fontos megjegyzés: néhány kivételtől eltekintve (nevezetes algoritmusok) a jegyzetben megjelenő pszeudokódokra csak mint egy lehetséges implementációra kell tekinteni. Tehát a cél nem az itt szereplő kódok betűről-betűre való megtanulása, sokkal inkább az egyes technikák elvi hátterének alapos megértése, amiben segítséget nyújthat a pontos pszeudokód illetve a példát bemutató ábra. Amennyiben valaki valóban, minden részletre kiterjedően érti a végrehajtandó műveletek pontos menetét, akkor az ez alapján bármikor tud rekonstruálni egy jól működő pszeudokódot.

Köszönetnyilvánítás

Köszönettel tartozom Dr. Csink Lászlónak, akivel a Programozás II. (illetve korábban Programozási Paradigmák és Technikák) nevű tárgy előadásait évek óta közösen tartjuk. Továbbá köszönettel tartozom Dr. Vámossy Zoltánnak, aki az évek folyamán számos értékes észrevétellel segítette a tárgy végső formájának kialakítását, majd pedig vállalta ennek a jegyzetnek a lektori feladatait is. Szintén köszönöm a sok segítséget Dr. Sergyán Szabolcsnak is, akivel mind a Programozás I. tárgy oktatójaként, mind pedig az Algoritmusok, adatszerkezetek I. jegyzet szerzőjeként is napi szinten egyeztettünk és remélhetőleg egyeztetünk a jövőben is.

Végül, de nem utolsó sorban szeretnék köszönetet mondani az Alkalmazott Informatikai Intézet munkatársai közül Cseri Orsolya Eszternek, Dr. Erdélyi Krisztinának, Nagy Tibor Istvánnak, Szabó Zsoltnak, Szántó Balázsnak és Urbán Andrásnak, akik a Programozás II. tárgy laborjait tartották. Az évek folyamán számos értékes észrevételt fogalmaztak meg, amelyek beépültek ebbe a jegyzetbe is.

1. fejezet

Visszalépéses keresés

1.1. Visszalépéses keresés alapgondolata

1.1.1. Alapelv

A *visszalépéses keresés* (backtrack) egy olyan problémamegoldási módszer, amely az alábbi esetekben használható a leghatékonyabban:

- A megoldandó feladat több, egymástól csak közvetve függő részfeladat megoldásából áll.
- Már a részfeladatok egy részéből is lehet arra következtetni, hogy az azokra adott részmegoldásokkal biztosan nem érhető el a teljes megoldás.

A visszalépéses keresés alapelve, hogy egyesével dolgozzuk fel a megoldandó részfeladatokat. Az első részfeladathoz választunk egy lehetséges részmegoldást, majd nézzük a következőt. Amennyiben találunk olyan részmegoldást, ami kielégíti a második részfeladatot, és nincs ellentmondásban az első részfeladathoz választottal, akkor ezt is rögzítjük, és továbblépünk a következő részfeladathoz. Amennyiben valamelyik részfeladatnál nem találunk egy, az előző feltételeknek megfelelő részmegoldást sem, akkor visszalépünk az előző részfeladathoz, és ott keresünk egy másik lehetőséget. Amennyiben a részfeladatok és a lehetséges megoldások száma véges, akkor véges számú lépést követően vagy találunk olyan részmegoldásokat, amelyek egyenként kielégítik az összes részfeladatot (tehát találtunk egy teljes megoldást), vagy előáll egy olyan helyzet, hogy az első részfeladatról is vissza kellene lépnünk, mivel az ott lévő minden egyes részmegoldásra beláttuk, hogy egyik se lehet egy teljes megoldás része (tehát a feladatnak nincs megoldása).

A visszalépéses keresést gyakran rekurzív algoritmussal valósítjuk meg, ahol a rekurzió alapelve:

- Visszavezetés: N részfeladat megoldását úgy keressük meg, hogy rögzítünk egy, az első részfeladatot kielégítő, és az előzőleg talált részmegoldásokat nem kizáró részmegoldást, majd újra meghívjuk a rekurzív eljárást a maradék N-1 darab részfeladatra. Amennyiben a rekurzió azzal az eredménnyel tért vissza, hogy nem sikerült végleges megoldást találnia, akkor az adott szinten keresünk egy másik lehetséges részmegoldást, és újrahívjuk a rekurziót. Amennyiben nem találunk több lehetőséget, akkor visszalépünk az előző rekurzió szintre.
- Triviális eset 1: A fenti hívás során ha már csak egy darab megoldandó részfeladat maradt, és arra találtunk a fenti feltételeknek megfelelő megoldást, akkor megoldottuk a feladatot.
- Triviális eset 2: Ha a legelső szinten nem találunk a fentieknek megfelelő részmegoldást, akkor nincs megoldása a feladatnak.

A módszer működésének ugyan nem előfeltétele, de csak akkor működik hatékonyan, ha már néhány részeredményből el tudjuk dönteni, hogy az nem vezethet jó teljes megoldáshoz. Ilyenkor ugyanis nem kell folytatnunk a még hátralévő részfeladatok megoldását, ehelyett azonnal megpróbálhatunk egy másik utat keresni az aktuális szinten. Ezzel ideális esetben jelentősen leszűkíthetjük a keresés során megvizsgált tér méretét, ami számottevő sebességnövekedést eredményezhet.

A mohó algoritmusokkal ellentétben egy megadott szinten a továbblépéskor soha nem tudhatjuk, hogy valóban jó lépést tettünk-e. A kizárási feltétel csak abban segít, hogy megadott szinten el tudjuk dönteni, hogy az előző eredmények kizárják-e egy részmegoldás kiválasztását. Azt azonban itt még nem tudjuk, hogy a választott megoldáshoz a még hátralévő részfeladatokhoz is találunk-e majd nem kizáró részmegoldásokat.

1.1.2. Példa visszalépéses keresésre

Példaként vegyük az alábbi feladatot: Egy építkezésen 6 különböző feladatra kell embert találnunk: "tervezés", "irányítás", "beszerzés", "ellenőrzés", "engedélyezés", "értékesítés", ahol az egyes feladatokra az alábbi személyek alkalmasak:

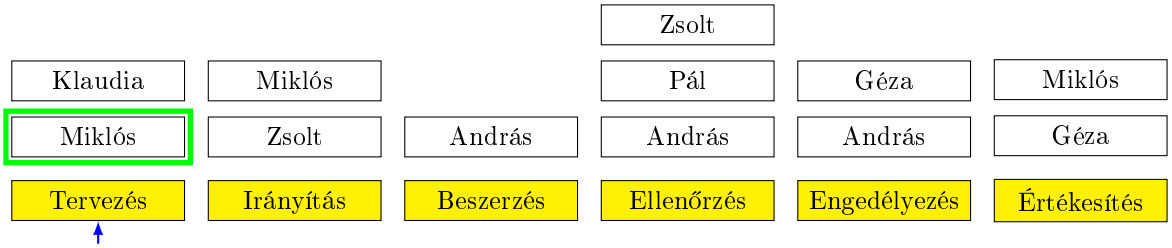
- "tervezés" feladatra alkalmas: "Miklós", "Klaudia",
- "irányítás" feladatra alkalmas: "Zsolt", "Miklós",
- "beszerzés" feladatra alkalmas: "András",
- "ellenőrzés" feladatra alkalmas: "András", "Pál", "Zsolt",
- "engedélyezés" feladatra alkalmas: "András", "Géza",
- "értékesítés" feladatra alkalmas: "Géza", "Miklós".

A feladatunk az, hogy rendeljünk az egyes feladatokhoz egy-egy személyt az arra alkalmasak közül, olyan módon, hogy egy személy csak egy feladatot vállalhat.

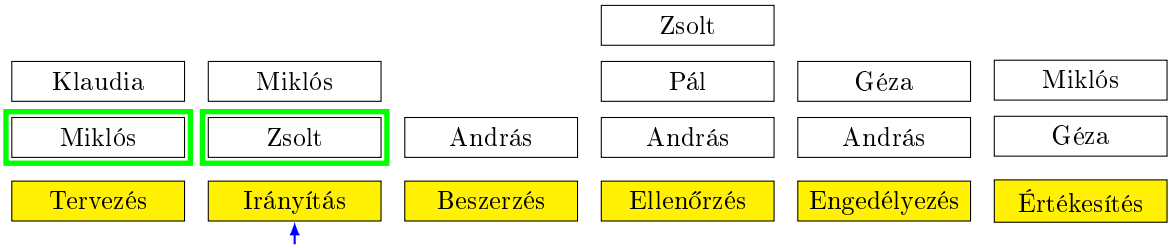
Az 1.1. ábra alapján érdemes végignézni, hogy pontosan milyen lépések követik egymást, ez alapján az alapelv már jól érthető. Az előző elvi leírásban is megjelenő lépések a itt is jól láthatóak, pl.:

- Részmegoldás rögzítése és továbblépés: Ez látható már az első lépéskor is (1.1a. ábra). Amennyiben az adott szinten találunk olyan részmegoldást, amelyet nem zárnak ki az előző eredmények, akkor feltételezzük, hogy ez része lehet egy teljes megoldásnak. Ezért ezt rögzítjük, és folytatjuk a keresést a következő szinten (szintek alatt a részfeladatokat értjük).
- Szinten belül új megoldás keresése: Az 1.1d. ábrán is látható, hogy az adott szinten az elsőként megvizsgált részeredmény nem választható, mivel azt már kizárta az egyik előző választásunk. Ilyenkor még ugyanezen a szinten maradva megpróbálunk más részmegoldást keresni.
- Visszalépés: Az első visszalépés az 1.1i. ábrán látható. Mindezt az okozza, hogy az adott szinten belül nem találtunk egy részmegoldást sem, amelyet ne zártak volna ki az előző választásaink, így nem tudtunk továbblépni a következő szintekre. Mivel az adott szinten belül se tudunk már új részmegoldást keresni, így nyilvánvaló, hogy az eddig követett út nem vezethet jó végeredményhez, emiatt visszalépünk egy szintet. Az pusztán véletlen egybeesés, hogy éppen az utolsó szintről fordultunk vissza, a példában számos eset látható, amikor már egy köztes szintről is vissza kellett lépnünk (pl. 1.1j. ábra, **??**. ábra).
- Teljes megoldás: Az 1.1u. ábrán látható, hogy találtunk megfelelő részmegoldást az utolsó szinten. Mivel ide csak úgy juthattunk, hogy már az összes előző szintet megoldottuk, így ez a teljes feladat megoldását is jelenti.

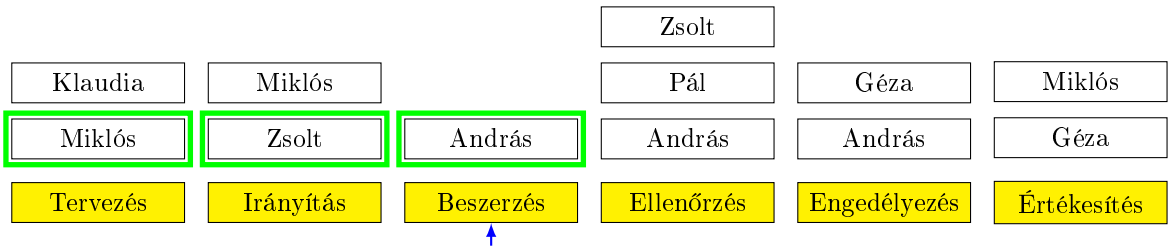
Egyedül arra nem láttunk példát, hogy mi történik akkor, ha nincs megoldása a feladatnak. De belátható, hogy amennyiben újabb visszalépések után ismét visszajutnánk a legelső szintre, akkor az azt jelentené, hogy sem a "Miklós", sem pedig a "Klaudia" nevű jelentkezővel nem sikerült a teljes feladatra megoldást találni. Más lehetőségünk pedig nincs, így nyilván nincs megoldása a feladatnak.



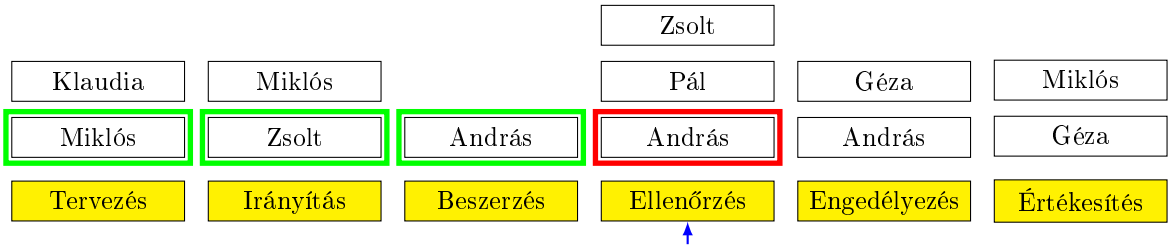
(a) Első szinten választjuk az első lehetséges részmegoldást, Miklóst. Mivel megfelel a feltételeknek, továbblépünk a következő szintre.



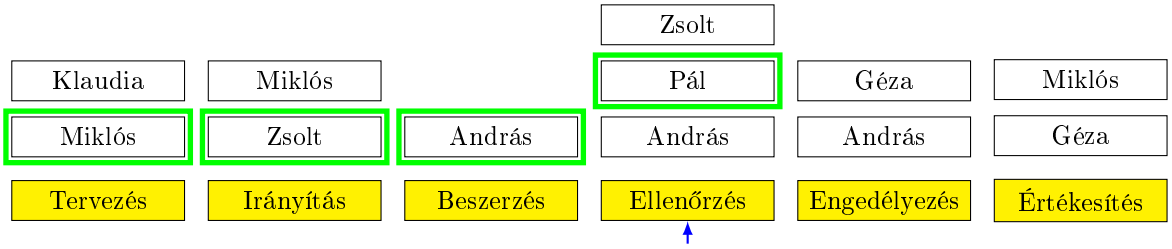
(b) Következő szinten választjuk az elsőt, Zsoltot. Miklóssal nem zárják ki egymást, így elfogadható. Továbblépünk a következő szintre.



(c) A harmadik szinten választhatjuk Andrást, mivel nem zárják ki az előző választások. Továbblépünk a következő szintre.

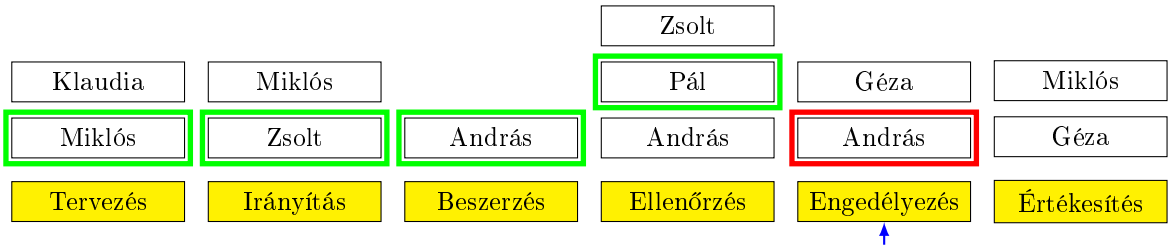


(d) A negyedik szinten András lenne az első lehetőség, de őt nem választhatjuk, mivel az előző szinten már adtunk neki munkát. Ezért keresünk újabb részmegoldást a szinten belül.

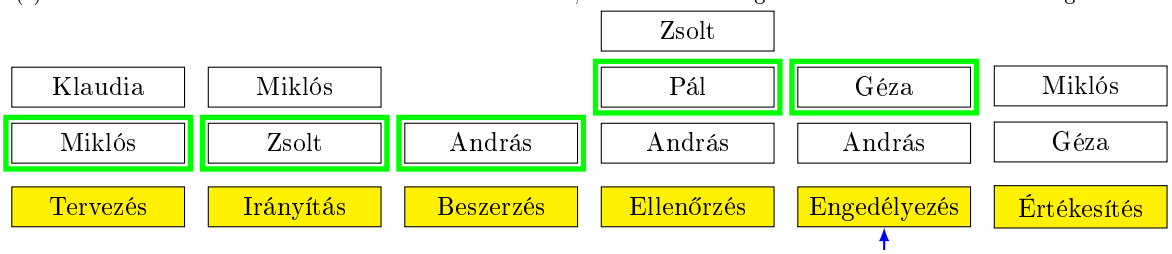


(e) Ugyanezen a szinten a következő lehetőség Pál, őt választhatjuk. Továbblépünk a következő szintre.

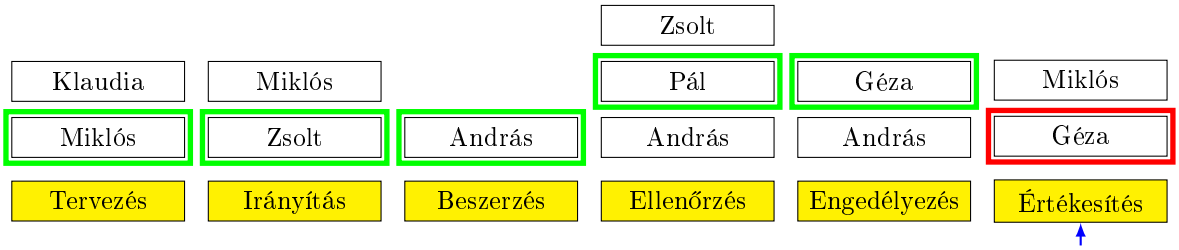
1.1. ábra. Visszalépéses keresés mintapélda (feladatok kiosztása).



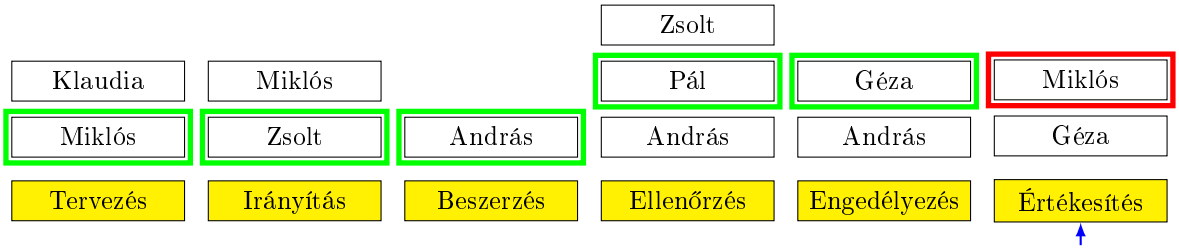
(f) Az ötödik szinten András ismét nem választható, hiszen ő már foglalt. Keresünk itt másik megoldást.



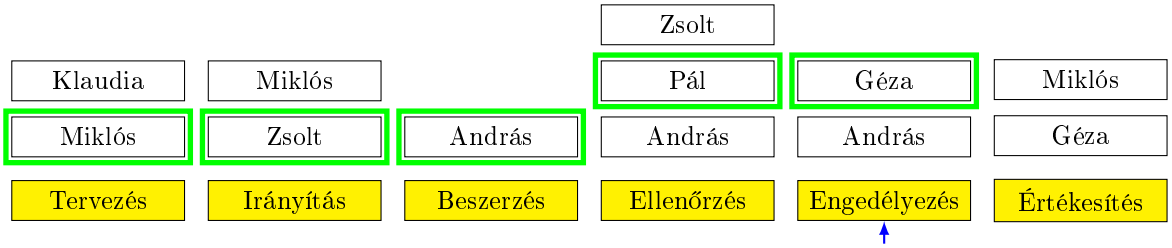
(g) A következő elem (Géza), azonban már jó, emiatt ezt rögzítjük, és folytatjuk a következő szinttel.



(h) Az utolsó szint első lehetséges részmegoldása nem megfelelő, hiszen az előző szinten is Gézát választottuk. Így a szinten belül keresünk tovább.

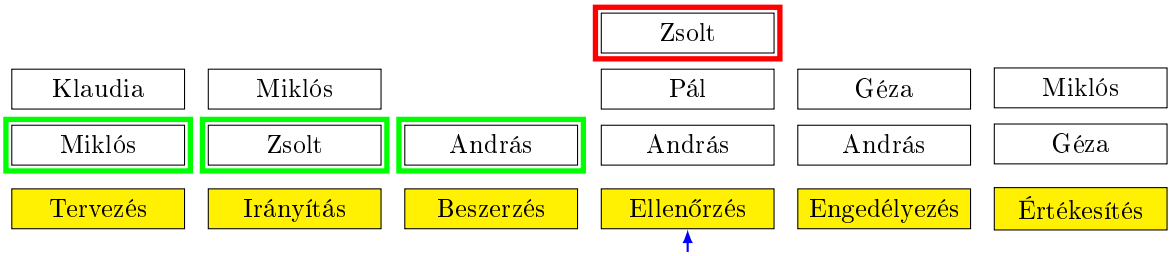


(i) Az utolsó szint második lehetséges részmegoldása sem megfelelő. Mivel így nem tudunk előrelépni, illetve a szinten belül sincs már további lehetőség, ezért egy szinttel visszalépünk.

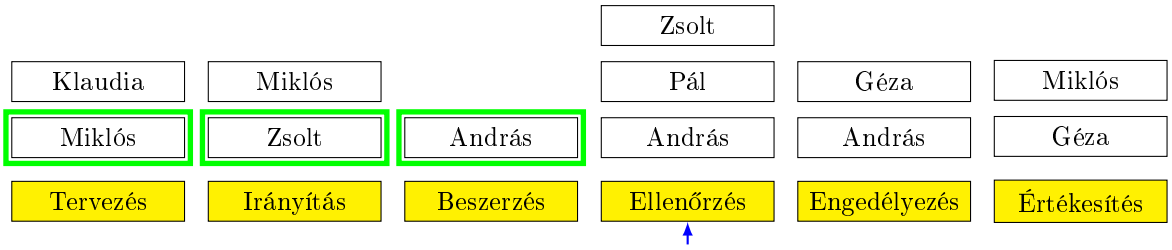


(j) Azonban már itt sincs további választási lehetőség, ezért itt is egy szinttel visszalépünk.

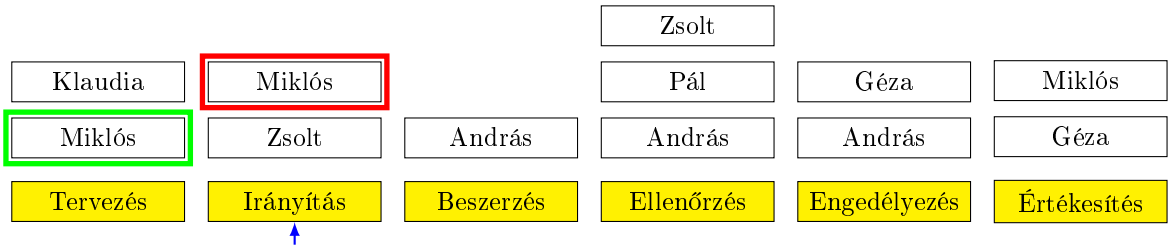
1.1. ábra. Visszalépéses keresés mintapélda (feladatok kiosztása). (folytatás)



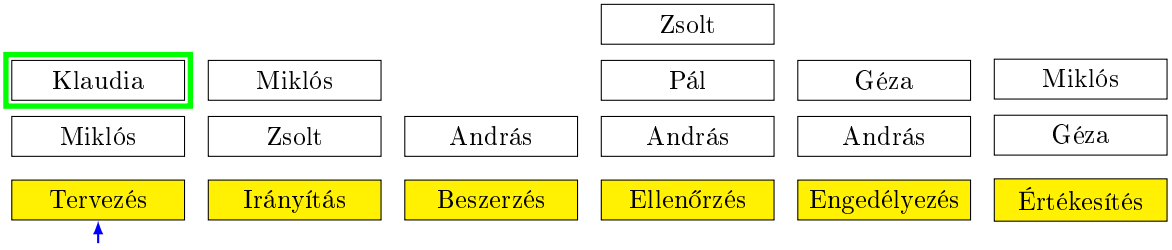
(k) A negyedik szintre visszatérve, itt még van egy újabb lehetőség. Zsoltot azonban már választottuk, így visszalépünk az előző szintre.



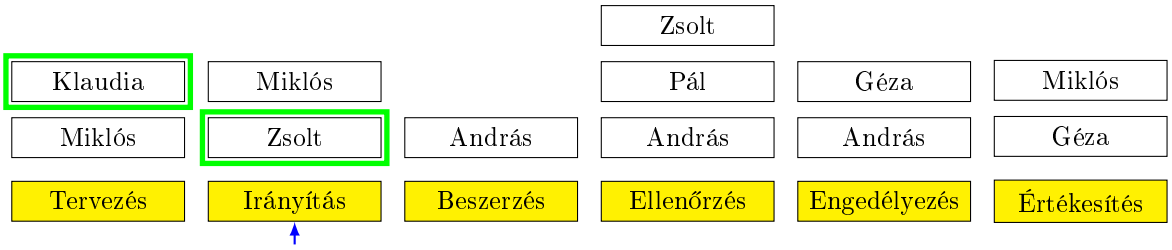
(l) Az előző szinten nincs más választási lehetőség, így innen is visszalépünk.



(m) A második szinten megpróbálhatnánk Miklóst választani, azonban az első szint aktuális részeredménye ezt kizárja. Így innen is visszalépünk.

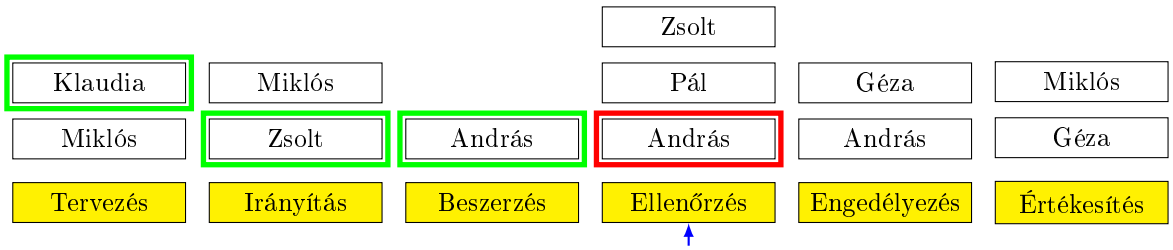


(n) A legelső szinten járunk ismét. Itt még van másik lehetőség, így itt megpróbáljuk Klaudiát választani.

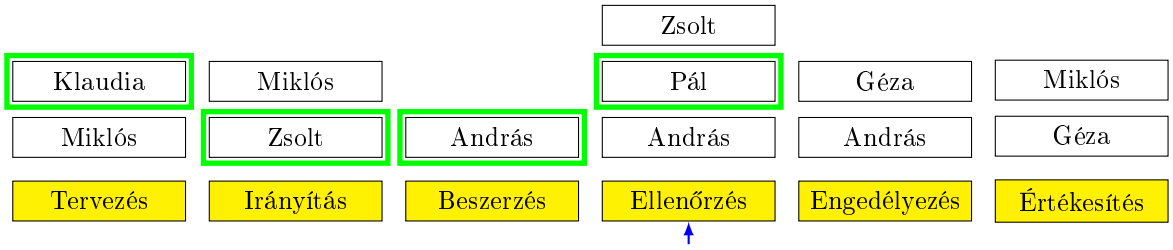


(o) A második feladatra Zsolt megfelelő lehet. Lépünk tovább.

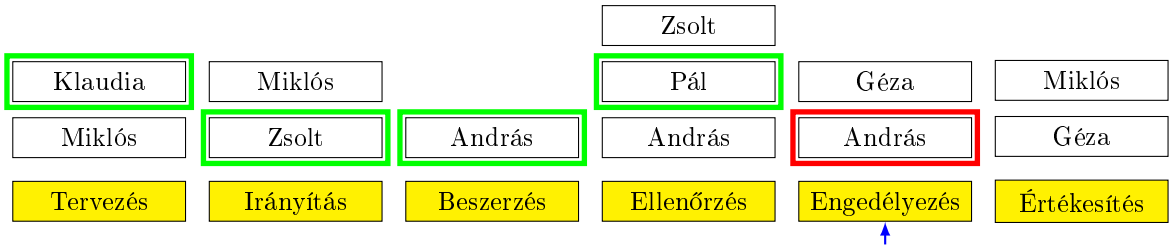
1.1. ábra. Visszalépéses keresés mintapélda (feladatok kiosztása). (folytatás)



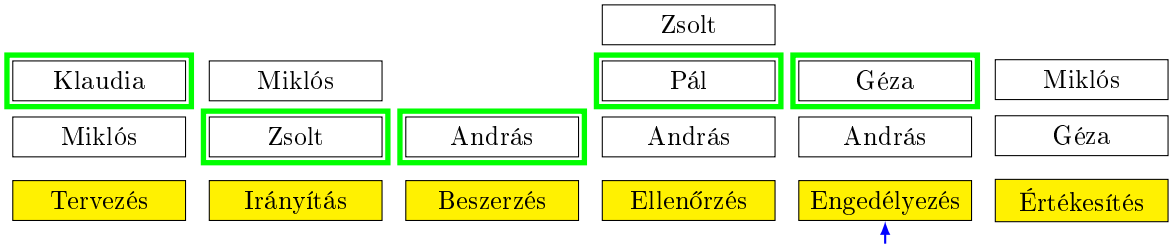
(p) Harmadik szinten értelemszerűen Andrást választjuk (ezt nem ábrázoltuk). A negyedik szinten András emiatt már nem választható. Így szinten belül keresünk mást.



(q) Pál viszont itt megfelelő, így léphetünk tovább.

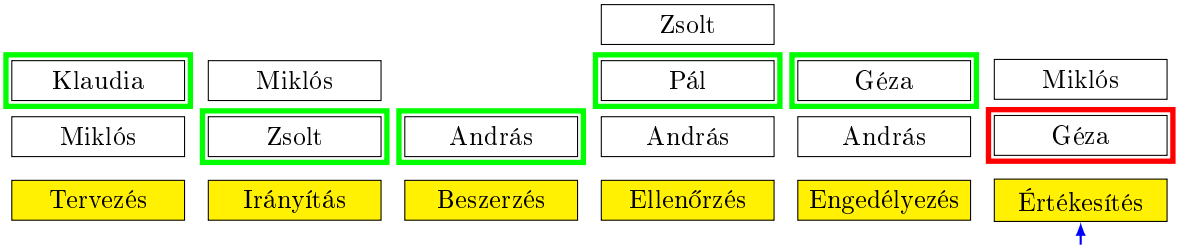


(r) András itt sem jó, így keresünk a szinten belül valaki mást.

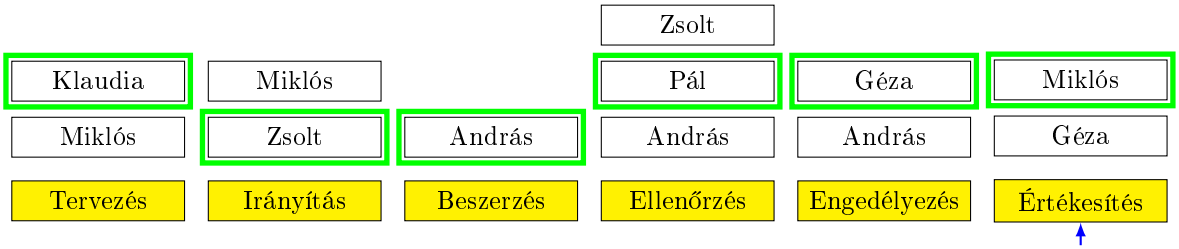


(s) Géza még szabad, így őt választhatjuk.

1.1. ábra. Visszalépéses keresés mintapélda (feladatok kiosztása). (folytatás)



(t) Az utolsó részfeladatra Géza így már nem választható. Így a szinten belül keresünk megoldást.



(u) Miklóst viszont nem zárják még ki az előző választások. Ezzel megoldást találtunk az utolsó részfeladatra is, így befejezhetjük a keresést, megvan egy lehetséges teljes megoldás.

1.1. ábra. Visszalépéses keresés mintapélda (feladatok kiosztása). (folytatás)

Az 1.1. ábrán látható lépésekkel lefutó algoritmus végeredménye az alábbi feladatkiosztás:

- "tervezés" feladatra: "Klaudia",
- "irányítás" feladatra: "Zsolt",
- "beszerzés" feladatra: "András",
- "ellenőrzés" feladatra: "Pál",
- "engedélyezés" feladatra: "Géza",
- "értékesítés" feladatra: "Miklós".

Az eredmény nem tartalmaz arra vonatkozó információt, hogy vannak-e még további megoldások (akár lehetnek is), az első helyes eredmény megtalálása után ugyanis leállt az algoritmus futása.

1.2. Visszalépéses keresés általános alakja

1.2.1. Bemenet és kimenet

Egy lehetséges definíció: „a visszalépéses keresés egy szisztematikus módszer, amely egyesével átvizsgálja a problématér összes lehetséges konfigurációját" [10]. Az alapelv tehát az, hogy folyamatosan építsünk egy utat a végső megoldás felé, menet közben elhagyva azokat a lehetséges leágazásokat, amelyek biztosan nem vezetnek jó eredményhez.

A példán láthattuk egy konkrét feladat megoldásának lépéseit, ez alapján próbáljuk meg leírni a módszer általános alakját. Ehhez első lépésként a bemutatott példában látható bemeneteket (feladatok, alkalmas személyek), és kimeneteket (van-e megoldás, és ha igen, akkor kit melyik munkához rendeltünk), illetve a végrehajtandó lépéseket próbáljuk meg általánosítani!

Egy visszalépéses keresési feladat bemenetét az alábbiakkal adhatjuk meg:

- N: A megoldandó részfeladatok száma.
- M_{szint}: A $szint$-edik részfeladat esetében rendelkezésre álló lehetséges részmegoldások száma (ahol $1 \leq szint \leq N$).
- $R_{szint,i}$: A $szint$-edik részfeladat i. lehetséges megoldása ($1 \leq szint \leq N$ és $1 \leq i \leq M_{szint}$).

A visszalépéses keresés kimenete:

- van: Logikai érték, ami azt mutatja, hogy találtunk-e teljes megoldást.
- E: Az eredményeket tartalmazó vektor, ahol E_i az i. részfeladat esetében az algoritmus által talált részmegoldás ($1 \leq i \leq N$).

Fontos észrevennünk, hogy maga az alapelv sokféle feladat megoldására alkalmassá tehető. Hasonló a helyzet, mint a már tanult lineáris keresésnél, ahol maga az algoritmus a tényleges keresési feltételtől függetlenül mindig ugyanaz volt, pusztán egy F paraméterrel jelöltük, hogy pontosan milyen feltételnek kell megfelelnie a keresett elemnek. A visszalépéses keresésnél is hasonló technikát alkalmazunk annyi különbséggel, hogy itt a keresési feltételt két különböző függvénnyel fogjuk meghatározni [12]:

- $F_t(szint, r)$: Egy függvény, ami azt határozza meg, hogy a $szint$-edik részfeladat esetében lehetséges megoldás-e az r?
- $F_k(szint, r, E)$: Azt határozza meg, hogy a $szint$-edik részfeladat esetében választhatjuk-e az r részmegoldást, amennyiben az előző szinteken az E vektorban található részmegoldásokat választottuk.

A két függvény közötti lényeges különbség tehát az, hogy az F_t az egyes részfeladatokat önmagukban kezeli, pusztán azt vizsgálja, hogy a többi megoldástól függetlenül egy részeredmény megoldhatja-e az adott részfeladatot. Az F_k függvény azonban már figyelembe veszi azt is, hogy a visszalépéses keresésnek több részfeladatra is megoldást kell találnia, így ezzel már azt vizsgálhatjuk meg, hogy a vizsgált részmegoldást választhatjuk-e az eddig választott részmegoldások ismeretében?

Megjegyzés

A visszalépéses keresés megvalósításától függően az F_t függvény sokszor el is hagyható. Amennyiben úgy választjuk meg a bemeneteket (M_{szint}, illetve $R_{szint,j}$), hogy eleve csak azokat a részmegoldásokat soroljuk fel, amelyek bármilyen esetben kiválaszthatók a hozzájuk tartozó részfeladat megoldásához, akkor az F_t azonosan igaz függvény lesz, és így értelemszerűen szükségtelen. Bizonyos feladatoknál azonban célszerű lehet egy külön függvénnyel vizsgálni ezt a kérdést, emiatt a mi megvalósításunkban megtartjuk az F_t függvényt.

1.1. Algoritmus Visszalépéses keresés általános esete

Bemenet: *szint* - **egész**, E - **T tömb**, *van* - **logikai**
Kimenet: E - **T tömb** (*az eredményt tartalmazza*), *van* - **logikai** (*van-e eredmény*)

```
1:  eljárás VisszalépésesKeresés(szint : egész, címszerint E : T tömb, címszerint van : logikai)
2:      i ← 0
3:      ciklus amíg ¬van ∧ i < M_szint
4:          i ← i + 1
5:          ha F_t(szint, R_szint,i) akkor
6:              ha F_k(szint, R_szint,i, E) akkor
7:                  E_szint ← R_szint,i
8:                  ha szint = N akkor
9:                      van ← igaz
10:                 különben
11:                     VisszalépésesKeresés(szint + 1, E, van)
12:                 elágazás vége
13:             elágazás vége
14:         elágazás vége
15:     ciklus vége
16: eljárás vége
```

Felhasznált változók és függvények

- *szint* - Az aktuális szint, ahol keresünk.
- N - A megoldandó részfeladatok száma.
- M_{szint} - A lehetséges részmegoldások száma a megadott szinten.
- $R_{szint,i}$ - Az i. részmegoldás a megadott szinten.
- E - Egy N elemű vektor, ami az eredményeket tárolja.
- *van* - Folyamatosan azt mutatja, hogy találtunk-e már teljes megoldást.
- F_t, F_k - A keresési feltételeket meghatározó függvények.

1.2.2. Tetszőleges megoldás keresése

A fenti bemeneteken dolgozó, visszalépéses keresésen alapuló általános megoldást mutatja be az 1.1. algoritmus.

Mint látható, maga az alapelv meglehetősen egyszerűen leírható. Rekurzív formában próbáljuk megoldani a problémát, ahol a rekurzió alapfeltételeit az előzőekben már ismertetett módon határoztuk meg. A rekurzió minden egyes végrehajtása a paraméterként megadott szintre próbál egy megoldást keresni.

A lokális i változó szerepe, hogy ez fogja végigvizsgálni a paraméterként átadott *szint* lehetséges megoldásait. A 2. sorban adunk ennek kezdőértéket, tehát az első lehetséges megoldással fog először próbálkozni.

Egy ciklus végignézni az egyes lehetséges részmegoldásokat. A 3. sorban látható ciklusfeltétel felelős azért, hogy a ciklus csak addig fusson, amíg:

- Nem találtunk-e egy teljes megoldást, hiszen ezt követően már felesleges lenne a további lehetőségek vizsgálata.
- Van-e még az adott szinten belül lehetséges részmegoldás. Hiszen ha nincs, akkor már nincs mit vizsgálni.

Amennyiben lefut a ciklusmag, akkor első körben növeljük az i változó értékét (4. sor), hogy az valóban a következő, megvizsgálandó részmegoldás sorszámát tartalmazza.

Ezt követően ellenőriznünk kell, hogy a vizsgált részmegoldás megfelelő lehet-e a megadott szinten. Ez két függvény hívását jelenti. Az 5. sorban látható az előzőleg bemutatott F_t függvényé, ami azt vizsgálja, hogy a megadott szinten (*szint*) az éppen vizsgált részmegoldás ($S_{szint,i}$) kiválasztható-e.

Amennyiben a válasz igen, akkor következik annak a vizsgálata, hogy a vizsgált megoldást nem zárja-e ki egy előzőleg már rögzített részmegoldás. Erre használható a már bemutatott F_k függvény, amelynek paraméterei a szint, amelyiken dolgozunk (*szint*), a részmegoldás, amit éppen vizsgálunk ($S_{szint,i}$), illetve az eddig rögzített eredmények sorozata (E).

Ha mindkét függvény igaz értékkel tért vissza, az azt jelenti, hogy az i. részmegoldás az adott szinten, az előző részmegoldások ismeretében jónak tűnik, ezért elmentjük ezt az értéket az E-be (7. sor).

Ezt követi egy vizsgálat (8. sor), hogy már az utolsó szinten járunk-e. Ha igen, akkor a *van* változó értékét igazra állítjuk, ezzel jelölve, hogy találtunk egy teljes megoldást (amit éppen az E tartalmaz). Érdemes megfigyelni, hogy a *van* változó új értékének hatására kilépünk az utolsó szinten lévő ciklusból, és ugyanez fog történni az előző rekurziós szinteken is, tehát az első teljes megoldás megtalálása után a lehető leggyorsabban kilépünk a visszalépéses keresésből.

Egy másik lehetőség, hogy találtunk ugyan egy részmegoldást, azonban a 8. sorban található vizsgálat alapján arra jutunk, hogy még nem oldottuk meg az egész feladatot. Ilyenkor folytatnunk kell a keresést a következő részfeladattal. A rekurzió szabályainak megfelelően ezt a 11. sorban látható rekurzív hívással tesszük meg. Itt az eljárás meghívja önmagát, paraméterként átadja a következő szint azonosítóját ($szint + 1$), illetve az eddig talált részeredményeket.

Az eljáráshívásból előbb-utóbb visszatér a vezérlés. Ilyenkor a *van* változó értékéből tudhatjuk meg, hogy mi volt ennek a visszatérésnek az oka. Ha a változó értéke *igaz*, az azt jelenti, hogy egy későbbi szinten találtunk egy teljes megoldást, tehát ezen a szinten is kiléphetünk. Ha azonban a változó értéke *hamis*, az csak annyit jelent, hogy a következő szinteken nem sikerült eljutni egy teljes megoldáshoz, tehát az itt feltételezett $S_{szint,i}$ eredményünk mégse lesz jó választás, az nem vezet végleges megoldáshoz. Ezért első körben próbálkozunk egy újabb részeredmény választással (amíg a ciklusfeltétel igaz), illetve ha ez nem vezet eredményre, akkor kilépünk az eljárásból, ezzel visszaadva a vezérlést az előző szintnek, hogy az tudjon más részmegoldást választani (a *van* változó alapértelmezetten *hamis*, tehát ilyenkor ezt külön nem kell beállítanunk).

A fenti leírás talán elsőre összetettnek tűnik, de mindez annak köszönhető, hogy egy meglehetősen általános formát választottunk. A későbbiekben látni fogjuk, hogy ugyanezt az algoritmust fogjuk felhasználni különféle, esetenként egymástól jelentősen különböző feladatok megoldásához. Az alapelv ugyanis végig ugyanaz marad, de a bemenő paraméterek, illetve az F_t és F_k függvények helyes megválasztásával teljesen eltérő feladatokat is meg tudunk oldani (feladat kiosztás, Sudoku feladat megoldás, 8 királynő elhelyezése sakktáblán, stb.).

Megjegyzés

Az algoritmus rugalmas voltát mutatja az is, hogy OOP környezetben egy absztrakt osztályként nagyon jól megvalósítható. Az ős absztrakt osztály tartalmazza a fent felsorolt mezőket, illetve magát a megoldó algoritmust. Az F_t és az F_k függvények célszerűen (virtuális) absztrakt metódusokként jelenhetnek meg, hiszen az általános alak még nem alkalmas tényleges feladatok megoldására. A későbbiekben egy konkrét feladat megoldása során elég egy leszármazottat készíteni, amely értéket ad a szükséges mezőknek, illetve a feladatnak megfelelően megvalósítja az F_t és F_k függvényeket.

1.2.3. Összes megoldás keresése

Az előzőleg megismert algoritmus mindig az első általa talált megoldást adja vissza. A gyakorlatban sokszor előfordul, hogy nem csak egy tetszőleges, hanem az összes megoldást keressük (ezt mutatja be az 1.2. algoritmus).

Az algoritmust csak minimálisan kellett megváltoztatni ahhoz, hogy az megtalálja az összes megoldást. Megjelent egy új paraméter, a $MIND$, ami egy halmaz, amiben folyamatosan gyűjtjük a menet közben megtalált megoldásokat (a rekurzió kezdő hívásakor egy üres halmazt kell átadni címszerinti paraméterként). Amennyiben a *van* változó értéke a futás végén *igaz*, akkor ebből a halmazból lehet kiolvasni a talált teljes megoldásokat.

Az alapesetet bemutató pszeudokódot két helyen kellett módosítani: a 10. sorban látható kiegészítés célja, hogy egy új teljes eredmény megtalálása esetében mentse el ezt a paraméterként átadott $MIND$ halmazba. Ezen túlmenően módosítanunk kellett a ciklust is, mivel az alapváltozatban az úgy működött, hogy az első megoldás után megpróbált minél gyorsabban kilépni a rekurzióból. Ha az összes megoldást keressük, akkor azonban erre nincs szükség. Sőt, egy teljes megoldás megtalálása nem befolyásolja a további vizsgálatokat, a keresésnek ugyanúgy kell folytatódnia, mint eddig. Ennek megfelelően változott a 3. sor.

1.2. Algoritmus Minden megoldást visszaadó visszalépéses keresés

Bemenet: $szint$ - **egész**, E - **T tömb**, van - **logikai**, $MIND$ - **T tömbök halmaza**
Kimenet: $MIND$ - **T tömbök halmaza** (*az eredményeket tárolja*), van - **logikai** (*van-e eredmény*)

```
1: eljárás VisszalépésesKeresés(szint : egész, címszerint E : T tömb, címszerint van : logikai, címszerint MIND : T tömb halmaz)
2:     i ← 0
3:     ciklus amíg i < M_szint
4:         i ← i + 1
5:         ha F_t(szint, R_szint,i) akkor
6:             ha F_k(szint, R_szint,i, E) akkor
7:                 E_szint ← R_szint,i
8:                 ha szint = N akkor
9:                     van ← igaz
10:                    MIND ← MIND ∪ E
11:                különben
12:                    VisszalépésesKeresés(szint + 1, E, van, MIND)
13:                elágazás vége
14:            elágazás vége
15:        elágazás vége
16:    ciklus vége
17: eljárás vége
```

Felhasznált változók és függvények

- $szint$ - Az aktuális szint, ahol keresünk.
- N - A megoldandó részfeladatok száma.
- M_{szint} - A lehetséges részmegoldások száma a megadott szinten.
- $R_{szint,i}$ - Az i. részmegoldás a megadott szinten.
- E - Egy N elemű vektor, ami az eredményeket tárolja.
- van - Folyamatosan azt mutatja, hogy találtunk-e már teljes megoldást.
- $MIND$ - A probléma megoldásait tartalmazó halmaz (abban az esetben, ha $van = igaz$).
- F_t, F_k - A keresési feltételeket meghatározó függvények.

1.2.4. Optimális megoldás keresése

Szintén gyakori módosítás, hogy nem az összes, hanem csak a legjobb megoldást keressük. A módosítás hasonló az előzőhöz, itt is el kell érnünk a ciklusfeltétel módosításával, hogy az első találat után ne álljon le a keresés, hanem lépjen tovább a következő lehetséges részmegoldáshoz. Ezt mutatja be az 1.3. algoritmus.

1.3. Algoritmus Optimális megoldást visszaadó visszalépéses keresés

Bemenet: *szint* - **egész**, E - **T tömb**, *van* - **logikai**, OPT - **T tömb**
Kimenet: OPT - **T tömb** (*az optimális eredményt tartalmazza*), *van* - **logikai** (*van-e eredmény*)

```
1: eljárás VisszalépésesKeresés(szint : egész, címszerint E : T tömb, címszerint van : logikai, címszerint OPT : T tömb)
2:     i ← 0
3:     ciklus amíg i < M_szint
4:         i ← i + 1
5:         ha F_t(szint, R_szint,i) akkor
6:             ha F_k(szint, R_szint,i, E) akkor
7:                 E_szint ← R_szint,i
8:                 ha szint = N akkor
9:                     ha ¬van ∨ Jóság(E) > Jóság(OPT) akkor
10:                        OPT ← E
11:                    elágazás vége
12:                    van ← igaz
13:                különben
14:                    VisszalépésesKeresés(szint + 1, E, van, OPT)
15:                elágazás vége
16:            elágazás vége
17:        elágazás vége
18:    ciklus vége
19: eljárás vége
```

Felhasznált változók és függvények

- *szint* - Az aktuális szint, ahol keresünk.
- N - A megoldandó részfeladatok száma.
- M_{szint} - A lehetséges részmegoldások száma a megadott szinten.
- $R_{szint,i}$ - Az i. részmegoldás a megadott szinten.
- E - Egy N elemű vektor, ami az eredményeket tárolja.
- *van* - Folyamatosan azt mutatja, hogy találtunk-e már teljes megoldást.
- *Jóság*(E) - Egy függvény, ami megadja a paraméterként átadott végeredmény értékét.
- OPT - A probléma optimális megoldása (abban az esetben, ha $van = igaz$).
- F_t, F_k - A keresési feltételeket meghatározó függvények.

A lényeges különbség pusztán annyi, hogy az új, teljes megoldások megtalálása esetén nem tároljuk el azokat egy halmazban, hanem minden új megoldás esetében ellenőrizzük, hogy az jobb-e mint az eddig talált legjobb. Ha igen, akkor mostantól ezt tekintjük a legjobbnak.

A változás tehát a 9. sornál látható. Amikor találunk egy új megoldást, akkor ellenőrizzük, hogy volt-e már megoldásunk (ezt egyszerűen megtehetjük, hiszen ezt mutatja a *van* változó értéke). Ha nem, akkor nyilvánvalóan ez az első lesz az eddigi legjobb. Ha pedig már volt, akkor ahhoz hasonlítjuk a most találtat, és csak akkor írjuk ezzel felül a régit, ha annál jobbat találtunk (10. sor). Ez utóbbi művelet azonban csak akkor hajtható végre, ha az egyes végeredmények összehasonlíthatóak. Emiatt bevezettünk egy kiegészítő *Jóság* nevű függvényt, aminek a paramétere egy végeredmény, visszatérési értéke pedig egy szám, ami két eredmény közül mindig annál lesz nagyobb, amelyiket jobbnak tekintjük.

Megjegyzés

Az algoritmust persze leírhatnánk a *Jóság* függvény nélkül is, egyszerűen $E > OPT$ műveletet írva. Ehhez csak azt kell megkövetelnünk, hogy az eredményként kapott tömbök összehasonlíthatóak legyenek.

1.2.5. Egymást kölcsönösen kizáró részfeladatok

Az előzőleg megismert algoritmus és változatai meglehetősen általánosnak tekinthetőek, azok implementációja nehézségekbe ütközhet. Érdemes lehet specializálni a megoldást, ennek megfelelően két fő csoportra bontjuk a visszalépéses kereséssel megoldható feladatokat, majd ezekre adunk egy-egy, már egyszerűbben implementálható megoldást.

Elsődlegesen, a már bemutatott F_k függvény formája alapján (tehát, hogy miként viszonyulnak egymáshoz az egyes részfeladatok) elkülöníthetjük egymástól az alábbi feladattípusokat:

- Kölcsönösen kizáró feltétel: Ebben az esetben egy újabb részfeladat vizsgálatakor annak eldöntéséhez, hogy egy ottani részmegoldást kizárnak-e előző részmegoldások, elég az előzőleg már kiválasztott részmegoldás jelöltekkel páronkénti összehasonlítást végezni. Tipikusan ilyen volt a példaként bemutatott feladat is, hiszen az i. részmegoldás vizsgálatához elég külön-külön megvizsgálni az előző $(i-1)$. részmegoldást, hogy a vizsgált személyt már hozzárendeltük-e valamelyik előző részfeladathoz.
- Összetett kizárási feltétel: Ebben az esetben nem elég a páronkénti összehasonlítás, mivel a részmegoldások egy nagyobb halmazáról tudjuk csak eldönteni, hogy azokkal folytatható-e a keresés vagy sem. Az elfogadási szabály tetszőleges lehet, pl. a hátizsákba pakolás feladatnál (lásd 1.3.5. alfejezet), ahol egy új tárgy vizsgálatakor nem elég az előzőleg választott tárgyakkal egyesével összehasonlítani, hanem ki kell számolni az előzőleg kiszemelt tárgyak össztömegét, és az alapján hozni meg a döntést. Az előzőleg megismert algoritmusok valójában ezt a módszert követik.

Az első esetben tudjuk, hogy a feltétel mindig csak két részfeladat vizsgálatát igényli, így magát az F_k függvényt is jóval egyszerűbben adhatjuk meg. Hiszen nincs szükség az összes eddig rögzített részeredmény átadására, elég paraméterként megadni két szintet, illetve az ott vizsgálandó két részeredményt, és ez alapján már el lehet dönteni, hogy ezek egymást kizárják-e vagy sem.

Ezt mutatja be az 1.4. algoritmus.

1.4. Algoritmus Visszalépéses keresés kölcsönösen kizáró esetben

Bemenet: *szint* - **egész**, E - **T tömb**, *van* - **logikai**
Kimenet: E - **T tömb** (*az eredményt tartalmazza*), *van* - **logikai** (*van-e eredmény*)

```
1:  eljárás VisszalépésesKeresés(szint : egész, címszerint E : T tömb, címszerint van : logikai)
2:      i ← 0
3:      ciklus amíg ¬van ∧ i < M_szint
4:          i ← i + 1
5:          ha F_t(szint, R_szint,i) akkor
6:              k ← 1
7:              ciklus amíg k < szint ∧ F_k(szint, R_szint,i, k, E_k)
8:                  k ← k + 1
9:              ciklus vége
10:             ha k = szint akkor
11:                 E_szint ← R_szint,i
12:                 ha szint = N akkor
13:                     van ← igaz
14:                 különben
15:                     VisszalépésesKeresés(szint + 1, E, van)
16:                 elágazás vége
17:             elágazás vége
18:         elágazás vége
19:     ciklus vége
20: eljárás vége
```

Felhasznált változók és függvények

- *szint* - Az aktuális szint, ahol keresünk.
- N - A megoldandó részfeladatok száma.
- M_{szint} - A lehetséges részmegoldások száma a megadott szinten.
- $R_{szint,i}$ - Az i. részmegoldás a megadott szinten.
- E - Egy N elemű vektor, ami az eredményeket tárolja.
- *van* - Folyamatosan azt mutatja, hogy találtunk-e már teljes megoldást.

Az alapváltozathoz képest két lényeges változás látható. Egyrészt némileg megváltozott az F_k függvény paraméterezése az alábbiak szerint:

- $F_k(szint_A, S_A, szint_B, S_B)$: Visszatérési értéke akkor igaz, ha a $szint_A$ szinten az S_A részmegoldás választása nem zárja ki a $szint_B$ szinten az S_B részmegoldás választását.

Mivel ez csak két szintet tud összehasonlítani, így ezt egy ciklusba kellett építeni, hogy az összes eddig választott részeredményt ellenőrizze az algoritmus.

A 7. sorban található ciklus megfelel a lineáris keresés programozási tételében megismert ciklusnak. A k változó segítségével végigszaladunk az előző szintek részmegoldásain (már ha van ilyen egyáltalán). Minden szinten ellenőrizzük, hogy az ott rögzített részmegoldás nem zárja-e ki azt, hogy az adott szinten az aktuálisan vizsgáltat válasszuk.

A lineáris kereséshez hasonlóan itt is ellenőriznünk kell, hogy miért léptünk ki a ciklusból (10. sor). Ha a k változó értéke kisebb, mint az előző szintek száma, akkor a ciklus futása során az F_k valahol hamisat adott vissza, tehát ez az eredmény nem lehet egy jó út része. Ellenkező esetben azonban megállapíthatjuk, hogy az előzőleg választott részeredmények nem zárják ki az aktuálisan vizsgáltat, tehát ezt rögzíthetjük.

Az összes, illetve az optimális megoldás keresése természetesen ebben a formában is megadható, ehhez ugyanazokat a módosításokat kell megtenni, mint amit az eredeti algoritmusnál is láthattunk.

1.3. Visszalépéses keresés példák

1.3.1. Megoldható feladatok osztályozása

Az előzőekben láttuk, hogy a visszalépéses kereséssel megoldható feladatokat célszerű lehet két fő csoportba osztani: az egyik esetben az egyes részfeladatok közötti kizárások pusztán a két részmegoldás vizsgálatával ellenőrizhetők, míg a másik esetben ehhez valamilyen komplex feltételt kell megadnunk. Ennek megfelelően két alapvető pszeudokódot írtunk fel, ezekre nézünk most külön-külön néhány gyakorlati példát.

A jobb áttekinthetőség kedvéért a két fő csoporton belül még további három osztályt határozhatunk meg, attól függően, hogy az egyes részfeladatok során milyen és hány darab részmegoldás közül választhatunk.

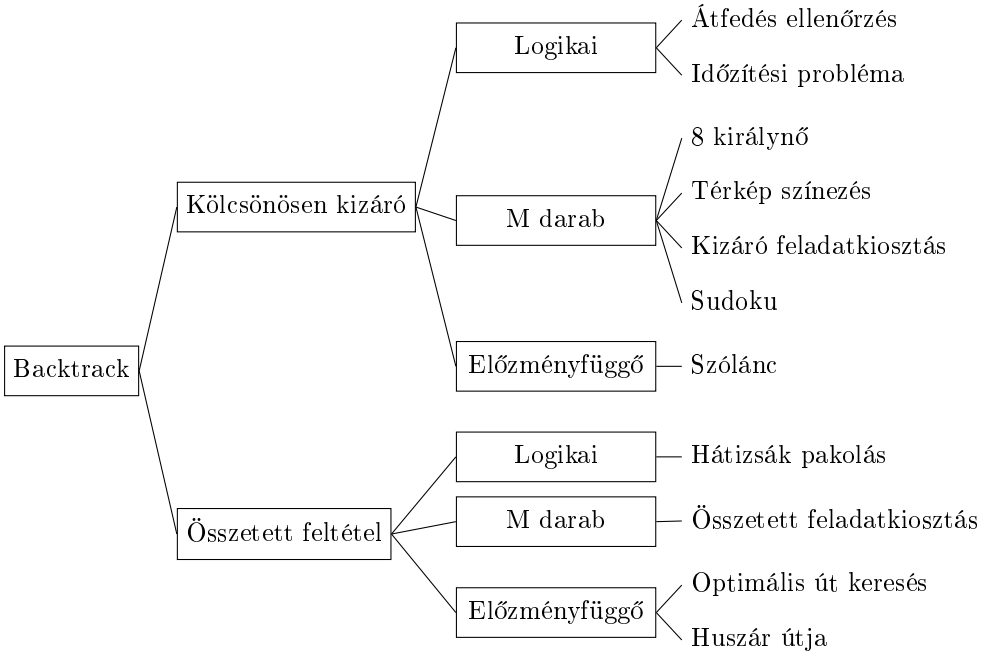
- Logikai érték: Ezek a legegyszerűbb feladatok, ahol minden részfeladatnál csak egy igaz/hamis érték közül választhatunk. Például ha több időpont halmazából kell kiválasztanunk egy olyan részhalmazt, ahol egyik időpont sem fedi át a másikat, akkor ez felfogható egy olyan visszalépéses keresésnek, ahol a szintek száma egyenlő a időpontok számával, és minden szinten a megoldás két érték közül választható: igaz, ha az időpont bekerül a részhalmazba, és hamis, ha nem. Ehhez hasonlóan a hátizsákba pakolásnál is minden tárgyról el kell dönteni, hogy berakjuk-e a zsákba vagy sem.
- M darab érték: Ebben az esetben minden részfeladat esetén már tetszőleges számú (tehát 1, 2, vagy több) részmegoldás közül kell kiválasztani egyet. Itt azonban még feltételezzük, hogy az egyes részhalmazok (és persze ezzel együtt azok mérete is) már a keresés kezdetekor is ismertek. Bizonyos feladatoknál minden szinten ugyanazok lehetnek ezek a részmegoldások (pl. térkép/gráf színezés [3] esetén minden szinten ugyanazon színek közül választhatunk), bizonyos esetben viszont szintenként különbözőek lehetnek (pl. a feladatok kiosztásakor láthattuk, hogy minden feladatnál külön-külön meg volt határozva, hogy azt kik tudják elvégezni).
- Előzményfüggő értékek: Általában ezek tekinthetők a legösszetettebb feladatoknak, itt minden részfeladat megoldásakor a lehetséges részmegoldások halmaza függ az előző részfeladatok megoldásaitól. Pl. a huszár útja a sakktáblán feladat (1.3.7. alfejezet) esetén az n. részfeladat azt jelenti, hogy az n. ugrásnál milyen irányba lépjünk tovább. Mivel a tábla szélén nem tudunk minden irányba továbbhaladni (és ez nem keverendő össze azzal a korláttal, hogy egy megadott helyen már jártunk), így az egyes lépések során a választható irányok száma változó, az a huszár aktuális pozíciójától függ. Az pedig, hogy az n. lépésnél pontosan hol vagyunk, az előző $(n-1)$ lépés alapján határozható meg.

Számtalan példát találhatunk a visszalépéses kereséssel megoldható feladatokra (kezdve a klasszikus példákkal: 8 királynő, hátizsák pakolási probléma, stb.). Mivel ezek szerkezetileg hasonlóak, célszerű őket a fenti osztályokba rendezni. Néhány példa látható erre az 1.2. ábrán.

Megjegyzés

Érdekes lehet még bevezetni egy harmadik osztályozást is, amely azt határozná meg, hogy az egyes részfeladatok száma (N) adott, vagy pedig a többi bemenő adattól függően változó. Például érdemes megvizsgálni a huszár útja a sakktáblán feladatot, illetve egy tetszőleges labirintusban útkereső feladatot. A kettő egymáshoz nagyon hasonlít (minden részfeladat egy irányt ad meg, hogy merre lépjünk tovább). Lényeges különbség azonban, hogy a huszár feladat esetén már a keresés indításakor is pontosan tudjuk, hogy összesen 63 darab lépést kell megtenni, míg egy útkeresés esetében ez előre nem ismert (esetleg egy maximális lépésszámot tudunk meghatározni a labirintus ismeretében, de lehet, hogy annál jóval kevesebb lépésből is meg tudjuk oldani a feladatot). A gyakorlatban ez az 1.1. algoritmus esetében csak kisebb módosítást igényel, mivel csak a megállási feltételt kell ennek megfelelően átalakítani (8. sor).

A példákkal is szeretnénk hangsúlyozni a visszalépéses keresés alapelvének általános voltát. A példák áttekintése során mindig érdemes észrevenni a tényt, hogy magát a megoldó algoritmust nem változtatjuk, pusztán a bemenetet, illetve az F_t és F_k függvényeket.



1.2. ábra. Visszalépéses kereséssel (backtrack) megoldható problémák osztályozása.

1.3.2. Logikai, egymást kizáró részmegoldás - Időzítési probléma

Ennek a feladat típusnak a jellegzetessége, hogy adott számos részfeladatunk, és mindegyik esetben egy igen/nem döntést kell hozni. Egy tipikus példa: egy tanácsteremre érkezett M darab foglalási igény (kezdő és befejező időpontpárok halmaza), ezek közül válogassunk ki egy olyan részhalmazt, amelyek között nincs átfedő időintervallum, a foglalási idők összege pedig maximális. A feladat a visszalépéses kereséssel egyszerűen megoldható, amennyiben az alábbi értékeket választjuk:

- N - Egyenlő a foglalási igények számával. Tehát minden szint megfelel egy kérésnek.
- M_{szint} - Értéke minden esetben 2 (ez természetesen így már be is helyettesíthető az algoritmusba).
- $R_{szint,i}$ - Értéke minden szinten $R_{szint,1} = igaz$, $R_{szint,2} = hamis$. Tehát minden szinten azt kell eldöntenünk, hogy a hozzá tartozó időpont foglalást teljesítjük-e vagy sem.
- $F_t(szint, r)$ megvalósítása: Azonosan igaz függvény, mivel az időpontok mindegyike választható és elhagyható is.
- $F_k(szint_A, r_A, szint_B, r_B)$ megvalósítása: A paraméterként átadott két részeredmény (jelen esetben idő intervallum) nem fedheti át egymást (az, hogy melyik szinten vannak, nem lényeges).

A függvények megvalósítását mutatja az 1.5. algoritmus. Mivel optimumot kell keresni, emiatt az esetlegesen megtalált eredményeket az ott leírt módon célszerű kezelni.

Hasonló alapelvek szerint nem csak egydimenziós esetben tudjuk az átfedéseket ellenőrizni, hanem az F_k függvény megfelelő megvalósításával akár síkbeli vagy térbeli elemekre is működik ez a megoldás (pl. válogassuk ki azokat a síkidomokat egy halmazból, amelyek között nincs átfedés, összterületük pedig maximális).

Megjegyzés

Az időzítési problémára létezik mohó algoritmus is, ami az itt megadottnál jóval hatékonyabban működik. A visszalépéses keresést azonban célszerű lehet megjegyezni, mivel a feltételrendszer változását a mohó algoritmus már nem biztos, hogy tudja követni, addig itt tetszőleges kizáró okokat tudunk kezelni (pl. több tárgyalónk van, prioritás az alkalmazottak között, stb.).

1.5. Algoritmus Időzítési probléma megoldása

Bemenet: *szint* - **egész**, *r* - **T**
Kimenet: *elfogadható* - **logikai** (*a részmegoldás elfogadható-e a megadott szinten*)
1: **függvény** $F_t(szint : \textbf{egész},\ r : \textbf{T})$
2: **vissza** ***igaz***
3: **függvény vége**

Bemenet: $szint_A$ - **egész**, r_A - **T**, $szint_B$ - **egész**, r_B - **T**
Kimenet: *elfogadható* - **logikai** (*a két részmegoldás a megadott szinteken nem zárja ki egymást*)
4: **függvény** $F_k(szint_A : \textbf{egész},\ r_A : \textbf{T},\ szint_B : \textbf{egész},\ r_B : \textbf{T})$
5: **vissza** $(r_A.kezdete > r_B.vége \vee r_A.vége < r_B.kezdete)$
6: **függvény vége**

1.3.3. M darab, egymást kizáró részmegoldás - Sudoku feladat

Ebben az esetben rendelkezésünkre áll egy 9×9 méretű tábla (9 darab 3×3 méretű blokkra osztva), ahol néhány szám már előre be van írva, a feladatunk pedig az, hogy az üresen maradt helyekre helyezzünk számokat 1..9 között a Sudoku játék szabályainak megfelelően (egy sorba, oszlopba, blokkba nem kerülhetnek azonos számok). Az üres mezőkbe a megfelelő szám beírását tekintjük részfeladatoknak, tehát a részfeladatok száma megegyezik az üresen hagyott mezők számával. Minden mezőbe 9 lehetséges részmegoldásunk van, tehát a feladat valóban ebbe az osztályba tartozik.

Elsőre nehéz lehet felismerni a visszalépéses keresés használatának lehetőségét, mivel a Sudoku tábla egy kétdimenziós adatszerkezet, az eddigi algoritmusaink azonban mindig csak egy egydimenziós feladatlistával működtek. Természetesen lehetne módosítani az algoritmust, de ennél egyszerűbb megoldás az, ha a feladatot próbáljuk meg átalakítani egydimenziós formára. Hozzunk létre egy sorozatot, és ebbe helyezzük el a táblázat üres celláit: legyen ez az U, ahol $U[i]$ visszaadja az i. üres cella koordinátáit (sor, oszlop). Ezt követően ennek a sorozatnak az elemei képviselik a megoldandó részfeladatokat, és máris a megszokott szinteket tudjuk használni.

Ebben az esetben a backtrack megvalósítás az alábbi:

- N - Egyenlő a kitöltendő cellák számával.
- M_{szint} - Értéke minden esetben 9 (minden helyre ennyiféle részmegoldás lehetséges).
- $R_{szint,i}$ - Értéke minden esetben az i, ahol $1 \leq i \leq 9$.
- $F_t(szint, r)$ megvalósítása: Akkor ad vissza igazat, hogy ha az előre beírt számok közül egyik sem akadályozza meg a megadott szinten rbeírását. Tehát vele egy sorban, oszlopban, vagy 3×3-as blokkban nincs ugyanilyen szám.
- $F_k(szint_A, r_A, szint_B, r_B)$ megvalósítása: Azt ellenőrzi, hogy a két szinten megadott két szám nem zárja-e ki egymást. Tehát csak akkor ad vissza igaz értéket, ha a két szintnek megfelelő mező azonos sorban, azonos oszlopban, vagy azonos 3×3-as blokkban van, és a paraméterként átadott két részeredmény (tehát a beírandó számok) azonosak.

A két függvény tényleges implementációját mutatja be az 1.6. algoritmus.

> Megjegyzés
>
> Ebbe a kategóriába tartozik a klasszikus „8 királynő elhelyezése a sakktáblán” feladat is [13], ahol úgy kell elhelyezni 8 királynőt egy szabványos sakktáblán, hogy azok ne üssék egymást. Mivel nyilvánvalóan minden királynőt más-más oszlopba kell helyezni, így már könnyen át tudjuk alakítani a feladatot a visszalépéses keresés által is használható formára. 8 részfeladatunk van, minden részfeladatban azt kell eldönteni, hogy adott oszlopon belül melyik sorba helyezzük a királynőt. Az F_t azonosan igaz függvény, az F_k pedig azt figyeli, hogy az újonnan vizsgált királynőt ne üsse egyik, az előző oszlopokba lerakott társa se.

1.6. Algoritmus Sudoku feladvány megoldása

Bemenet: $szint$ - **egész**, r - **T**
Kimenet: $elfogadható$ - **logikai** (*a részmegoldás elfogadható-e a megadott szinten*)

1: **függvény** $F_t(szint : \textbf{egész},\ r : \textbf{T})$
2: **vissza** $\nexists i \in \aleph, j \in \aleph | (1 \leq i \leq 9) \wedge (1 \leq j \leq 9) \wedge (S = P[i,j]) \wedge (i = U[szint].sor \vee j = U[szint].oszlop \vee (i \mod 3 = U[szint].sor \mod 3 \wedge j \mod 3 = U[szint].oszlop \mod 3))$
3: **függvény vége**

Bemenet: $szint_A$ - **egész**, r_A - **T**, $szint_B$ - **egész**, r_B - **T**
Kimenet: $elfogadható$ - **logikai** (*a két részmegoldás a megadott szinteken nem zárja ki egymást*)

4: **függvény** $F_k(szint_A : \textbf{egész},\ r_A : \textbf{T},\ szint_B : \textbf{egész},\ r_B : \textbf{T})$
5: **vissza** $(r_A \neq r_B) \vee \neg(U[szint_A].sor = U[szint_B].sor \vee U[szint_A].oszlop = U[szint_B].oszlop \vee (U[szint_A].sor \mod 3 = U[szint_B].sor \mod 3 \wedge U[szint_A].oszlop \mod 3 = U[szint_B].oszlop \mod 3))$
6: **függvény vége**

Felhasznált változók és függvények

- $P[i,j]$ - Megadja az i. sor, j. oszlopba előre beírt számot (ha van ilyen).
- $U[i]$ - Visszaadja az i. üres cella koordinátáit.

1.3.4. Előzményfüggő, egymást kizáró részmegoldás - Szólánc játék

Tegyük fel, hogy van egy szavakból álló készletünk, és ezeket úgy kell sorba rendeznünk, hogy (egy tetszőleges első szót követően) minden szó első betűje megegyezzen az előző szó utolsó betűjével. Több azonos kezdetű és végű szó esetén a lehetőségek száma meglehetősen nagy lehet, emiatt a visszalépéses keresés itt is jól használható.

Az egyes szavak választása nem független az előzőektől, emiatt itt feltételezzük, hogy az F_t hozzáfér az előző eredményekhez is (bár valójában csak az utolsóra van szüksége), ez az alap algoritmus kisebb módosítását igényli.

Ebben az esetben a backtrack megvalósítás az alábbi:

- N - A rendelkezésre álló szavak száma, mivel mindet fel akarjuk használni.
- M_{szint} - Értéke minden esetben egyenlő a szavak számával, mivel bármelyik szó szerepelhet bármelyik helyen.
- $R_{szint,i}$ - Értéke minden esetben az i. szó.
- $F_t(szint, r, E)$ megvalósítása: Azt vizsgálja, hogy minden szinten csak olyan szavakat lehessen választani, amelynek első karaktere megegyezik az előző részmegoldás utolsó karakterével (kivéve persze az első szót, ahol bármi szerepelhet). Látható, hogy paraméterként megkapja a E vektort, emiatt módosítani kell a kereső algoritmust, hogy ezt is adja át.
- $F_k(szint_A, r_A, szint_B, r_B)$ megvalósítása: Ebben az esetben ez csak egy egyszerű kizárást jelent, hogy ne lehessen ugyanazt a szót kétszer kiválasztani. Tegyük fel, hogy egy szó csak egyszer szerepel a bemenetben.

1.7. Algoritmus Szólánc játék

Bemenet: $szint$ - **egész**, r - **T**
Kimenet: $elfogadható$ - **logikai** (*a részmegoldás elfogadható-e a megadott szinten*)
1: **függvény** $F_t(szint : \textbf{egész},\ r : \textbf{T})$
2: **vissza** $(szint = 1 \vee E_{szint-1}.vége = S.eleje)$
3: **függvény vége**

Bemenet: $szint_A$ - **egész**, r_A - **T**, $szint_B$ - **egész**, r_B - **T**
Kimenet: $elfogadható$ - **logikai** (*a két részmegoldás a megadott szinteken nem zárja ki egymást*)
4: **függvény** $F_k(szint_A : \textbf{egész},\ r_A : \textbf{T},\ szint_B : \textbf{egész},\ r_B : \textbf{T})$
5: **vissza** $(r_A \neq r_B)$
6: **függvény vége**

Felhasznált változók és függvények
- $X.eleje$ - Az X szó első betűje.
- $X.vége$ - Az X szó utolsó betűje.

1.3.5. Logikai, nem kizáró részmegoldás - Hátizsák-pakolás

A hátizsák-pakolási feladatnál van N darab tárgyunk, illetve ismerjük azok a súlyát is (R_i legyen az i. tárgy súlya). Úgy kell kiválogatnunk közülük tetszőleges számút, hogy az összsúly ne haladja meg a $SÚLYKORLÁT$ konstans értéket. Az egyszerűség kedvéért most nem optimumot keresünk, mint a „0-1 hátizsák problémánál" [9], megelégszünk egy tetszőleges megoldással.

A feladat hasonló az átfedések ellenőrzéséhez, itt is célszerű úgy megválasztani a részfeladatokat, hogy minden esetben azt kelljen csak eldönteni, hogy egy megadott tárgyat berakunk-e a hátizsákba, vagy pedig sem. A lényeges különbség az, hogy míg az időpont átfedéseknél az egyes részfeladatok páronként kizárták egymást, addig itt az eddig rögzített részeredmények összességéből dönthető csak el, hogy azok kizárják-e az új megoldást.

A feladat megoldása a visszalépéses keresésre egyszerűen visszavezethető (1.8. algoritmus), amennyiben az alábbi bemenetet választjuk:

- N - Egyenlő a tárgyak számával. Minden részfeladat azt jelenti, hogy az adott tárgyat elhelyezzük-e a hátizsákba vagy sem.
- M_{szint} - Értéke minden esetben 2 (ez természetesen így már be is helyettesíthető az algoritmusba).
- $R_{szint,i}$ - Értéke minden szinten $R_{szint,1} = igaz$, $R_{szint,2} = hamis$. Tehát minden szinten azt kell eldöntenünk, hogy az adott tárgyat berakjuk-e a hátzsákba, vagy sem.
- $F_t(szint, r, E)$ megvalósítása: Azonosan igaz függvény, hiszen minden tárgy be is rakható, illetve ki is hagyható.
- $F_k(szint, r, E)$ megvalósítása: Az eljárás paraméterként megkapja az előzőleg rögzített részmegoldásokat (tehát mely tárgyakat raktuk be eddig a zsákba, és melyeket nem). Ez alapján ki tudja számolni az eddigi összes súlyt, és így már el tudja dönteni, hogy a $szint$-edik tárgy belefér-e a hátizsákba.

1.8. Algoritmus Hátizsákba pakolási probléma

Bemenet: *szint* - **egész**, r - **T**, E - **T tömb**
Kimenet: $elfogadható$ - **logikai** (*a részmegoldás elfogadható-e a megadott szinten*)

```
1: függvény F_t(szint : egész, r : T, E : T tömb)
2:     vissza igaz
3: függvény vége
```

Bemenet: *szint* - **egész**, r - **T**, E - **T tömb**
Kimenet: $elfogadható$ - **logikai** (*a megoldást nem zárja ki egy előző választás*)

```
4: függvény F_k(szint : egész, r : T, E : T tömb)
5:     ha r = hamis akkor vissza igaz
6:     SZUM ← R_szint
7:     i ← 1
8:     ciklus amíg i < szint ∧ SZUM ≤ SÚLYKORLÁT
9:         ha E_i = igaz akkor
10:            SZUM ← SZUM + R_i
11:        elágazás vége
12:        i ← i + 1
13:    ciklus vége
14:    vissza (SZUM ≤ SÚLYKORLÁT)
15: függvény vége
```

Felhasznált változók és függvények

- R_i - Megadja az i. tárgy súlyát.
- $SÚLYKORLÁT$ - A hátizsákban elférő tárgyak maximális összsúlya.

Megjegyzés

A gyakorlatban persze hasznosabb lenne egy olyan algoritmus, ami nem csak egy tetszőleges, hanem valamilyen szempont szerinti optimális megoldást ad vissza. Pl. legyen a hátizsák minél jobban kihasználva, vagy rendeljünk a tárgyakhoz egy értéket, és ezt próbáljuk maximalizálni. Bármi is a feltétel, a módosítás egyszerűen megoldható egy *Jóság* függvény bevezetésével, illetve az optimális eredményt kereső 1.3. algoritmus használatával.

1.3.6. M darab, nem kizáró részmegoldás - Feladatok kiosztása

Ebben az esetben adott számos részfeladat, és mindegyikre rendelkezünk különféle potenciális megoldásokkal, az egyes megoldások azonban *páronként* nem zárják ki egymást. A feladat ezzel nagyon hasonlít a fejezet elején bemutatott példához, azonban egy lényeges különbség, hogy nem zárjuk ki azt, hogy két feladatot ugyanaz a személy végezzen el. Tehát általánosabban fogalmazzuk meg a feladatot az alábbiak szerint: adott N darab feladat, és K darab személy. A J mátrix tartalmazza azt, hogy kik tudják megoldani az egyes feladatokat, pl. $J_{i,j}$ értéke igaz, ha az i. feladatot a j. személy meg tudja oldani, egyébként pedig hamis. Adjunk egy olyan feladatkiosztást, hogy minden feladathoz rendelünk valakit, de egy személy se vállalhat egyszerre Z darabnál több részfeladatot.

Ebben az esetben a visszalépéses keresés megvalósítása az alábbi (1.9. algoritmus):

- N - Egyenlő a feladatok számával.
- M_{szint} - Azt mutatja, hogy hányan jelentkeztek a $szint$-edik munkára.
- $R_{szint,i}$ - Értéke a $szint$-edik munkára i-edikként jelentkezett munkás.
- $F_t(szint, r, E)$ megvalósítása: A J mátrix ellenőrzését jelenti, hiszen minden feladatnál meg kell néznünk, hogy az adott személy alkalmas-e rá.

- $F_k(szint, r, E)$ megvalósítása: A függvény megszámolja, hogy a megadott személyt hány munkára jelöltük már. Amennyiben az aktuálissal együtt meghaladná a korlátot (Z), akkor *hamis* a visszatérési értéke, egyébként pedig *igaz*.

1.9. Algoritmus Feladat kiosztási probléma megoldása

Bemenet: $szint$ - **egész**, r - **T**, E - **T tömb**
Kimenet: $elfogadható$ - **logikai** (*a részmegoldás elfogadható-e a megadott szinten*)
1: **függvény** $F_t(szint : \textbf{egész},\ r : \textbf{T},\ E : \textbf{T tömb})$
2: **vissza** $J_{szint,r}$
3: **függvény vége**

Bemenet: $szint$ - **egész**, r - **T**, E - **T tömb**
Kimenet: $elfogadható$ - **logikai** (*a megoldást nem zárja ki egy előző választás*)
4: **függvény** $F_k(szint : \textbf{egész},\ r : \textbf{T},\ E : \textbf{T tömb})$
5: $db \leftarrow 0$
6: $i \leftarrow 1$
7: **ciklus amíg** $i < szint \wedge db < Z$
8: **ha** $E_i = r$ **akkor**
9: $db \leftarrow db + 1$
10: **elágazás vége**
11: $i \leftarrow i + 1$
12: **ciklus vége**
13: **vissza** $(db < Z)$
14: **függvény vége**

Felhasznált változók és függvények
- $J_{i,j}$ - Az j. személy jelentkezett-e az i. feladatra.
- Z - Egy személy egyidőben hány feladatot végezhet el.

1.3.7. Előzményfüggő, nem kizáró részmegoldás - Huszár útja a sakktáblán

A jól ismert feladatunk az, hogy egy szabványos sakktábla egyik sarkából elindulva egy huszárral a szabályos lépésekkel érinteni kell a tábla minden egyes mezőjét úgy, hogy minden mezőre pontosan egyszer lépjünk.

Ez egyben ismét egy jó példa arra is, hogy a kétdimenziós problémát hogyan tudjuk egydimenzióssá alakítani. Ugyanis ha zavar minket a sakktábla kétdimenziós volta, akkor a feladatot úgy is átfogalmazhatjuk, hogy 63-at kell lépnünk, ennek megfelelően van 63 részfeladatunk, ahol minden esetben a lehetséges lépések közül kell egyet kiválasztanunk.

A lehetséges részmegoldások száma azonban itt már előzményfüggő, mivel a huszár alapvetően 8 irányba tud lépni, de a tábla szélein nem biztos, hogy az összes irány választható. Azt pedig, hogy aktuálisan hol áll a huszár, csak az előző lépések ismeretében tudjuk meghatározni.

- N - 63, hiszen ennyi lépést kell tennünk.
- M_{szint} - Értéke minden esetben 8. Egy $LÉPÉS$ 8×2-es tömb tartalmazza a lehetséges huszár lépések relatív sor és oszlop koordinátáit.
- $R_{szint,i}$ - Értéke minden esetben az i, ahol $1 \leq i \leq 8$.
- $F_t(szint, r, E)$ megvalósítása: Azt vizsgálja, hogy megadott szinten a megadott irányú lépés a táblán belülre vezet, vagy sem. Mivel ez attól függ, hogy előzőleg milyen lépéseket hajtottunk végre, emiatt ehhez végignézzük az előző eredményeket, mintegy lejátszva az előző lépéseket.
- $F_k(szint, r, E)$ megvalósítása: Az előző alapján már érthető ennek a működése is. A kérdés az, hogy az új lépés olyan helyre vezet-e, ahol még nem jártunk. Ehhez végigkövetjük az előző lépéseket, és ha azok egyike se vezetett a vizsgálandó pozícióba, akkor a visszatérési értéke *igaz*, egyébként pedig *hamis* lesz.

A függvények megvalósítását mutatja az 1.10. algoritmus.

1.10. Algoritmus Huszár útja a sakktáblán

Bemenet: *szint* - **egész**, r - **T**, E - **T tömb**
Kimenet: $elfogadható$ - **logikai** (*a részmegoldás elfogadható-e a megadott szinten*)

```
1: függvény F_t(szint : egész, r : T, E : T tömb)
2:     sor ← 1; oszlop ← 1
3:     ciklus k ← 1-től (szint − 1)-ig
4:         sor ← sor + LÉPÉS[E_k].sor
5:         oszlop ← oszlop + LÉPÉS[E_k].oszlop
6:     ciklus vége
7:     sor ← sor + LÉPÉS[S].sor
8:     oszlop ← oszlop + LÉPÉS[S].oszlop
9:     vissza (1 ≤ sor ≤ 8 ∧ 1 ≤ oszlop ≤ 8)
10: függvény vége
```

Bemenet: *szint* - **egész**, r - **T**, E - **T tömb**
Kimenet: $elfogadható$ - **logikai** (*a megoldást nem zárja ki egy előző választás*)

```
11: függvény F_k(szint : egész, r : T, E : T tömb)
12:     posA_sor ← 1; posA_oszlop ← 1; posB_sor ← 1; posB_oszlop ← 1
13:     ciklus k ← 1-től (szint_A − 1)-ig
14:         posA.sor ← posA.sor + LÉPÉS[E_k].sor
15:         posA.oszlop ← posA.oszlop + LÉPÉS[E_k].oszlop
16:         ha k ≤ szint_B akkor
17:             posB.sor ← posB.sor + LÉPÉS[E_k].sor
18:             posB.oszlop ← posB.oszlop + LÉPÉS[E_k].oszlop
19:         elágazás vége
20:     ciklus vége
21:     posA.sor ← posA.sor + LÉPÉS[S].sor
22:     posA.oszlop ← posA.oszlop + LÉPÉS[S].oszlop
23:     vissza (posA.sor ≠ posB.sor ∨ posA.oszlop ≠ posB.oszlop)
24: függvény vége
```

Felhasznált változók és függvények

- *sor* - Az egyes pozíciókat tartalmazó értékek sor komponense.
- *oszlop* - Az egyes pozíciókat tartalmazó értékek oszlop komponense.
- *LÉPÉS* - Egy 8 elemű tömb, amely tárolja a sakk szabályainak megfelelő huszár lépések relatív koordinátáit (sor, illetve oszlop formában).

Megjegyzés

A bemutatott mintapéldák jelentős része megoldható egyszerűbben, illetve gyakran jóval hatékonyabban is (pl. a huszár útja feladatnál egy köztes tömb segítségével eltárolhatjuk azokat a pozíciókat, ahol már jártunk, így nem kell minden vizsgálatnál újrajátszani az előzményeket). A példák célja azonban az egyes feladatok leghatékonyabb megoldása helyett jelen esetben az volt, hogy bemutassuk, hogy az eredeti visszalépéses keresés módosítása nélkül, pusztán az F_k és F_t függvények helyes megválasztásával is meg lehet oldani egészen különböző feladatokat.

2. fejezet

Optimalizációs stratégiák

2.1. Alapfogalmak

2.1.1. Optimalizáció

Számos módszert ismerünk már különféle jellegű feladatok megoldására. Eddig általában megelégedtünk azzal, hogy egy olyan algoritmust készítsünk, ami megoldást ad egy megadott problémára (pl. megkeresi egy tömb legkisebb elemét, stb.). Az optimalizációs feladatok esetében azonban feltételezhetjük, hogy több megoldás is létezik (persze az is lehet, hogy csak egy, vagy akár egy sincs), és ezek közül kell kiválasztanunk egy optimálisat. Ezzel kapcsolatban néhány megjegyzés:

- Feladattól függően nekünk kell meghatározni, hogy mit értünk optimális alatt (pl. tárgyak közüli választás során célunk lehet a minél olcsóbb megoldás keresése, de akár a minél jobb, minél könnyebb, stb. is). Továbbra is szeretnénk minél általánosabb algoritmusokat készíteni, így az optimális fogalmát nem szeretnénk belekódolni az algoritmusokba, ezt inkább egy külső jóság/fitnessz függvény segítségével határozzuk meg. Ez a függvény minden lehetséges megoldáshoz hozzárendel egy szám értéket, ami minél nagyobb, annál jobb a megoldás.
- Bizonyos feladatoknál nem jóság, hanem költség függvényt határozunk meg. Ebben az esetben a cél a minél kisebb költségű megoldás megkeresése, ez általában csak apró módosításokat igényel az algoritmusokban.
- Szándékosan nem „az optimálisról" beszélünk, hiszen gyakori, hogy a sok lehetséges megoldás közül nem emelhető ki egyetlen darab legjobb, hanem akár több megoldás is található, amelyek jósága maximális.

2.1.2. A 0-1 hátizsák probléma

A fejezetben többféle módszerrel próbáljuk megoldani ugyanazt a feladatot. Ez az ún. hátizsák probléma (knapsack) lesz, amelynek bemenete:

- N darab tárgy, amelyek jellemzői:
  - w_i - Az i. tárgy súlya ($1 \leq i \leq N$).
  - p_i - Az i. tárgy értéke ($1 \leq i \leq N$).
- Egy hátizsák, aminek maximális kapacitása W_{max}, tehát a benne elhelyezett tárgyak összsúlya nem lehet ennél több.

Pakolásnak nevezzük annak meghatározását, hogy mely tárgyak kerülnek a hátizsákba, és melyek nem. Konkrét programozási nyelven a pakolást többféle adatszerkezet reprezentálhatja, pl. az elhelyezett tárgyak halmaza, stb. Mi egy N elemű logikai tömböt fogunk használni (OPT), amelynek i. eleme ($1 \leq i \leq N$) azt mutatja, hogy az i. elem a megadott pakolás szerint bekerült-e a zsákba vagy sem (igaz érték esetén bekerült, hamis érték esetén nem).

Egy pakolás összsúlya az általa a zsákba helyezett tárgyak súlyának az összege. Hasonlóan egy pakolás összértéke az általa zsákba helyezett tárgyak értékének az összege.

Érvényes pakolásnak nevezzük azokat a pakolásokat, amelyek megfelelnek a maximális kapacitás feltételnek. Tehát a pakolás összsúlya kevesebb, mint a hátizsák kapacitása (W_{max}).

Optimális pakolásnak nevezzük azokat az érvényes pakolásokat, amelyeknél nagyobb összértékkel bíró érvényes pakolás nem állítható elő.

Feladatunk annak meghatározása, hogy mekkora egy optimális pakolásnak az összértéke (P_{opt}). Tehát a pakolás által a zsákba helyezendő tárgyak értékének az összege. Ez értelemszerűen minden optimális pakolásnál ugyanannyi.

Megjegyzés

Tehát most nem ragaszkodunk ahhoz, hogy az elhelyezendő tárgyak listáját is megkapjuk eredményként. Ez persze bizonyos esetekben szükséges lehet, de az egyszerűség kedvéért mi csak az optimális összértékre összpontosítunk.

A hátizsák problémának két alváltozatával foglalkozunk majd:

- 0-1 hátizsák probléma: A tárgyakat itt atominak tekintjük (tipikusan pl. pohár, villa, stb.). Minden tárgyról azt kell tehát eldöntenünk, hogy az bekerül-e a zsákba vagy sem.
- Töredékes (fractional) hátizsák probléma: Ebben az esetben arra is van lehetőségünk, hogy az egyes tárgyaknak csak egy részét helyezzük a hátizsákba (tipikusan pl. víz, liszt, stb.).

Mindkét feladat típusnak megvan a maga előnye és hátránya, mi alapvetően a 0-1 hátizsák problémát szeretnénk megoldani.

Két segédműveletet bevezetünk már a feladat megoldása előtt, ezek kiszámítják egy pakolás összértékét (2.1. algoritmus), illetve összsúlyát (2.2. algoritmus).

2.1. Algoritmus Egy pakolás összértékének kiszámítása

Bemenet: X - **logikai tömb**
Kimenet: S - **egész** (*a pakolás összértéke*)

```
1: függvény ÖsszÉrték(X : logikai tömb)
2:     S ← 0
3:     ciklus i ← 1-től N-ig
4:         ha X[i] = igaz akkor
5:             S ← S + p_i
6:         elágazás vége
7:     ciklus vége
8:     vissza S
9: függvény vége
```

Felhasznált változók és függvények

- N - A hátizsákba pakolható tárgyak száma.
- p_i - A hátizsákba pakolható i. tárgy értéke ($1 \leq i \leq N$).
- X - Az aktuálisan vizsgált pakolás adatai (logikai tömb, ahol $X[i]$ azt mutatja, hogy az i. tárgy benne van-e a zsákban ($1 \leq i \leq N$)).
- S - A pakolás összértéke.

2.2. Algoritmus Egy pakolás összsúlyának kiszámítása

Bemenet: X - **logikai tömb**
Kimenet: S - **egész** (*a pakolás összsúlya*)

```
1: függvény ÖsszSúly(X : logikai tömb)
2:     S ← 0
3:     ciklus i ← 1-től N-ig
4:         ha X[i] = igaz akkor
5:             S ← S + w_i
6:         elágazás vége
7:     ciklus vége
8:     vissza S
9: függvény vége
```

Felhasznált változók és függvények

- N - A hátizsákba pakolható tárgyak száma.
- w_i - A hátizsákba pakolható i. tárgy súlya ($1 \leq i \leq N$).
- X - Az aktuálisan vizsgált pakolás adatai (logikai tömb, ahol $X[i]$ azt mutatja, hogy az i. tárgy benne van-e a zsákban ($1 \leq i \leq N$)).
- S - A pakolás összsúlya.

2.2. Nyers erő módszere

A módszer bemutatása

A nyers erő módszere (brute force) számos egyéb néven ismert: „teljes kipróbálás módszere", „generálj és tesztelj", „leszámlálási eljárás" stb. Ezek a nevek mind jól jellemzik a módszert, ami a maga tökéletlenségében nagyon könnyen alkalmazható, bizonyos feladat típusoknál talán egyedülálló módon, azonban az esetek túlnyomó többségében drasztikus erőforrásigénnyel bír.

A módszer használatának tulajdonképpen csak egy alapkövetelménye van: a lehetséges megoldásjelöltek (candidate solution) száma legyen véges. Maga az alapelv pedig ez alapján már egyszerű: szisztematikusan próbálgassuk végig az összes megoldásjelöltet. Amennyiben csak egy megoldásra van szükségünk, akkor annak megtalálásáig, amikor viszont optimalizálni szeretnénk, akkor mindet (így már érthető, hogy miért ragaszkodunk ahhoz, hogy ezek száma véges legyen).

A módszer általában egy ciklust feltételez, ami az alábbi lépéseket hajtja végre:

1. Következő megoldásjelölt generálása
2. A megoldásjelölt vizsgálata
3. Amennyiben ez valódi megoldása a problémának, akkor
   - Keresés esetén kilépünk a ciklusból, hiszen megvan a keresett megoldás
   - Optimalizáció esetén ellenőrizzük, hogy ennek a jósága nagyobb-e, mint az eddig talált legjobb megoldás. Ha igen, (vagy ez volt az első megoldás) akkor ezt követően ezt tekintjük legjobbnak.
4. Amennyiben van még meg nem vizsgált megoldásjelölt, akkor folytatjuk a ciklust

A 0-1 hátizsák probléma megoldása

Az alapelv megismerését követően már egyszerűen megírható a hátizsák problémát megoldó algoritmus (2.3. algoritmus). Miként a fejezet többi megoldásánál is, a feladat bemenetét (N,w,p,W_{max} értékek) ismertnek tekintjük, a könnyebb áttekinthetőség miatt ezeket nem tüntetjük fel a paraméterlistában. A függvény visszatérési értéke egy optimális pakolás összértéke.

A függvény első sorában inicializáljuk az OPT változót. Ez nem minden optimalizációnál lehetséges, de itt egyszerűen, bármiféle keresés nélkül tudunk adni egy lehetséges megoldást: ha nem rakunk semmit a zsákba, akkor az biztosan érvényes pakolás lesz. Az ezt követő ciklusban arra törekszünk, hogy találjunk egy ennél jobbat.

Tudjuk, hogy N darab független tárgyról kell igen vagy nem döntést hoznunk, ennek megfelelően összesen 2^N darab lehetséges megoldásjelöltünk van. Ezek egy része lesz csak érvényes pakolás, és ezek közül is csak néhány lesz optimális.

A megoldásjelöltek generálását a 4. sorban kezdődő ciklus végzi. A 0-tól 2^N-1-ig tartományban minden egész számhoz egy megoldásjelöltet rendelünk. A megoldásjelölt generálása azon alapul, hogy megvizsgáljuk az adott egész szám kettes számrendszerbeli alakját: minden bithez egy tárgyat rendelünk, és ha az adott szám j. bitjének értéke 1, akkor a j. tárgyat elhelyezzük a pakolásba, ha a bit értéke 0, akkor pedig nem. A j. bit értékét az egész szám 2^j-nel való osztásának maradékából tudjuk meghatározni. Az aktuálisan vizsgált pakolást a K logikai tömb tartalmazza.

Miután megvan a következő megoldásjelölt, ezt ellenőriznünk kell. A 7. sorban látható feltétel első tagja azt figyeli, hogy egy érvényes pakolásról van-e szó (összsúlya nem nagyobb, mint a hátizsák kapacitása). Amennyiben igen, csak akkor értékelődik ki a második feltétel, ami azt vizsgálja, hogy az aktuálisan vizsgált megoldás jósága nagyobb-e, mint az eddig talált legjobb megoldásé. Ha igen, akkor ezt követően ezt tekintjük optimálisnak.

A fenti ciklust tehát a 1-től 2^N-1-ig tartó tartomány minden egész számát végigjárja (a 0-t nem kell, hiszen az pont a nincs a zsákban semmi állapotot képviseli, ami az OPT kezdeti értéke). A megoldás tulajdonképpen megfelel a már tanult maximumkiválasztás tételnek, ennek megfelelően a visszatérési érték az optimális összértéket tartalmazza. Amennyiben szükségünk van magára a pakolás részletes adataira, akkor azt az OPT tömbből kiolvashatjuk az eljárás végén.

2.3. Algoritmus A 0-1 hátizsákprobléma megoldása „nyers erő” algoritmussal

Kimenet: P_{opt} - **egész** (*egy optimális pakolás összértéke*)

```
1:  függvény HátizsákBF
2:      OPT ← [hamis, hamis, ..., hamis]
3:      ciklus i ← 1-től 2^N − 1-ig
4:          ciklus j ← 0-től N-1-ig
5:              K[j] ← ⌊i/2^j⌋ mod 2 = 1
6:          ciklus vége
7:          ha (ÖsszSúly(K) ≤ W_max) ∧ (ÖsszÉrték(K) > ÖsszÉrték(OPT) akkor
8:              OPT ← K
9:          elágazás vége
10:     ciklus vége
11:     vissza ÖsszÉrték(OPT)
12: függvény vége
```

Felhasznált változók és függvények

- N - A hátizsákba pakolható tárgyak száma.
- w_i - A hátizsákba pakolható i. tárgy súlya ($1 \leq i \leq N$).
- p_i - A hátizsákba pakolható i. tárgy értéke ($1 \leq i \leq N$).
- W_{max} - A hátizsák mérete (a belepakolható tárgyak maximális összsúlya).
- P_{opt} - Egy optimális pakolás összértéke.
- K - Az aktuálisan vizsgált pakolás adatai (logikai tömb, ahol $K[i]$ azt mutatja, hogy az i. tárgy benne van-e a zsákban ($1 \leq i \leq N$).
- OPT - Az eddig talált legjobb pakolás adatai (logikai tömb, ahol $OPT[i]$ azt mutatja, hogy az i. tárgy benne van-e a zsákban ($1 \leq i \leq N$)).
- ÖsszÉrték(X) - Megadja az X pakolás összértékét (2.1. algoritmus).
- ÖsszSúly(X) - Megadja az X pakolás összsúlyát (2.2. algoritmus).

A módszer értékelése

Látható, hogy a módszer tárhelyigénye nem jelentős, független az N értékétől. A futásidő azonban extrém magas, a segédműveletektől eltekintve is látható, hogy a külső ciklus $2^N - 1$ iterációt igényel. Ez talán a legrosszabb futásidő amivel eddig találkoztunk.

A módszer előnyei:

- Egyszerűen megérthető és nagyon gyorsan implementálható.
- Elvileg minden esetben használható, ahol véges a lehetséges megoldások száma. Garantáltan megtalálja az optimális megoldás(oka)t.
- Amennyiben nincs semmilyen támpontunk, akkor gyakran csak ez használható (pl. jelszavak feltörése, stb.)
- Tárhelyigénye minimális.

A módszer hátrányai:

- Futásideje nagyon magas. Emiatt nagyobb problémák megoldása során a gyakorlatban nem használható.

2.3. Oszd meg és uralkodj módszer

A módszer bemutatása

A módszer remélhetőleg már ismert [9], így annak részletes elemzésétől eltekintünk. Ismétlésként a fő lépései:

1. Amennyiben a megoldandó probléma egyszerű, akkor
   (a) Megoldás.
2. Amennyiben a megoldandó probléma ehhez túl bonyolult, akkor
   (a) Felosztás több kisebb részproblémára.
   (b) Az így kapott kisebb részproblémák megoldása (tipikusan rekurzív hívások segítségével).
   (c) Az így kapott részeredmények egyesítése, azok alapján a megoldandó probléma eredményének előállítása.

Az alapelv jól használható számos hagyományos algoritmusnál (pl. gyorsrendezés, k-adik legkisebb tömbbeli elem kiválasztása, stb.), és ugyanígy használható optimalizációs feladatok esetében is. Az alapelv hasonló: ha az optimális megoldás meghatározása túl bonyolult, akkor próbáljuk meg visszavezetni egyszerűbb részfeladatokra. Gyakran használható az a technika, hogy a keresési teret több kisebb részre bontjuk, és ezeken végezzük el az optimalizációt. Majd ezen részeredmények egyesítéséből próbáljuk meg meghatározni a teljes optimumot.

A 0-1 hátizsák probléma megoldása

Érdemes megjegyezni, hogy nem minden optimalizálási feladat oldható meg ezzel a módszerrel, a 0-1 hátizsák probléma azonban szerencsére igen. A részproblémákra bontást elvégezhetjük úgy, hogy nem az összes tárgyra próbáljuk meg egyszerre keresni az optimális állapotot, hanem egyszerre csak eggyel foglalkozunk. Megvizsgáljuk azt az esetet, amikor berakjuk őt a zsákba, és azt, amikor nem, és a kettő közül a jobbik eset lesz az optimális. Természetesen mindkét esetben meg kell vizsgálnunk a többi tárgyat is, viszont ez már két, kicsit egyszerűbb (hiszen eggyel kevesebb tárgyra, és az első esetben kicsit kevesebb szabad helyre kell optimumot keresni) rekurzív hívással megtehetjük.

A fenti fő lépésekre támaszkodva az alábbiak alapján képzelhetjük el az így előálló részproblémák megoldását. Minden ilyen részprobléma esetén adott a még vizsgálandó tárgyak listája, illetve az, hogy még mennyi hely van a zsákban (ez nem azonos a zsák teljes kapacitását jelző W_{max} értékkel, lehet annál kisebb is):

1. A triviális esetek
   (a) Triviális eset 1: egyértelműnek tekinthetjük azt az állapotot, amikor a még vizsgálandó tárgyak listája üres (2.1. egyenlet első feltétele). Ilyenkor nyilván nem tudunk berakni semmit a hátizsákba, így az optimális pakolás összértéke biztosan 0 lesz.
   (b) Triviális eset 2: szintén egyértelmű az az állapot, amikor a hátizsákban lévő szabad hely 0 (2.1. egyenlet második feltétele). Ilyenkor szintén nem tudunk semmit elhelyezni a zsákban, az optimális pakolás összértéke tehát ilyenkor is 0.
2. Nem triviális eset: amennyiben van még egy vagy több vizsgálandó tárgyunk, akkor az alábbiak szerint járunk el:
   (a) Amennyiben az utolsó még nem vizsgált tárgy nem fér bele a zsákba, akkor számoljuk ki, hogy a maradék elemek milyen optimális összértékkel helyezhetők el a meglévő szabad helyre. Ez célszerűen egy rekurzív hívás lesz, ahol már csak az utolsótól különböző tárgyakat adjuk tovább változatlan szabad hely mellett (2.1. egyenlet harmadik feltétele).
   (b) Amennyiben az utolsó még nem vizsgált tárgy befér a zsákba, akkor vizsgáljuk meg azt is, hogy vele együtt milyen optimális érték érhető el. Ez egy újabb rekurzív hívás lesz, ahol újra csak az utolsó előtti tárgyakat adjuk tovább, viszont itt már a megmaradt szabad helyet is

csökkenteni kell az utolsó tárgy súlyával (hiszen ennyivel kevesebb hely marad a többinek). A rekurzív függvényhívás visszatérési értékét ilyenkor még növelnünk kell az utolsó tárgy értékével, hiszen az is bekerült a zsákba.
Persze attól, hogy befér, még nem kötelező beraknunk a zsákba, tehát ebben az esetben az optimális érték az előzőleg kiszámolt vele vagy nélküle számított értékek maximuma (2.1. egyenlet negyedik feltétele).

Tehát a feladatot megoldó rekurzió:

$$F_{t,h} = \begin{cases} 0, & \text{ha } h = 0 \\ 0, & \text{ha } t = 0 \\ F_{t-1,h}, & \text{ha } h > 0 \wedge t > 0 \wedge h < w_t \\ max\{F_{t-1,h},\ F_{t-1,h-w_t} + p_t\}, & \text{ha } h > 0 \wedge t > 0 \wedge h \geq w_t \end{cases} \tag{2.1}$$

Ezt a működést valósítja meg a 2.4. algoritmus.

2.4. Algoritmus A 0-1 hátizsákprobléma megoldása az „oszd meg és uralkodj" elv segítségével

Bemenet: t - **egész**, h - **egész**
Kimenet: P_{opt} - **egész** (*optimális pakoláshoz tartozó összérték*)

```
1: függvény LegjobbRészMegoldás(t : egész, h : egész)
2:     ha (t = 0) ∨ (h = 0) akkor
3:         vissza 0
4:     különben
5:         P_nem ← LegjobbRészMegoldás(t − 1, h)
6:         ha h ≥ w_t akkor
7:             P_igen ← p_t + LegjobbRészMegoldás(t − 1, h − w_t)
8:             P_opt ← max(P_igen, P_nem)
9:         különben
10:            P_opt ← P_nem
11:        elágazás vége
12:        vissza P_opt
13:    elágazás vége
14: függvény vége
```

Kimenet: P_{opt} - **egész** (*egy optimális pakolás összértéke*)

```
15: függvény HátizsákDnC
16:     vissza LegjobbRészMegoldás(N, W_max)
17: függvény vége
```

Felhasznált változók és függvények

- N - A hátizsákba pakolható tárgyak száma.
- w_i - A hátizsákba pakolható i. tárgy súlya ($1 \leq i \leq N$).
- p_i - A hátizsákba pakolható i. tárgy hasznossága ($1 \leq i \leq N$).
- W_{max} - A hátizsák mérete (a belepakolható tárgyak maximális összsúlya).
- P_{opt} - Egy optimális pakolás összértéke.
- t - A részmegoldás kereséskor a tárgyak darabszáma.
- h - A részmegoldás kereséskor a rendelkezésre álló szabad hely.

A rekurzív függvény első paramétere, a t változó azt mutatja, hogy hány tárgyat vizsgálunk (az elsőtől a t.-ig). Az ezt követő tárgyak állapotát a rekurzió előző szintjei már beállították, így velük már nem foglalkozunk. Valójában a még vizsgálandó első t darab tárgyból is csak az utolsót vizsgáljuk közvetlenül. Az ezt megelőző tárgyak állapotát pedig majd a következő rekurziós szintek fogják vizsgálni.

A második h nevű paraméter pedig azt mutatja, hogy a még elhelyezendő tárgyak (tehát a t. és az ez előttiek) számára mekkora szabad hely maradt. Ez az érték mindig kisebb vagy egyenlő, mint W_{max}, értéke attól függ, hogy az előző rekurziós szinteken elhelyezett tárgyak mennyivel csökkentették a még rendelkezésre álló szabad helyet.

A rekurzív függvény megvalósításában jól elkülöníthetők a fent leírt ágak. Az első sor végzi el a triviális esetek ellenőrzését. Amennyiben a t értéke nullával egyenlő (tehát nincs több vizsgálandó tárgyunk), vagy a h értéke nulla (tehát nincs szabad hely), akkor az optimum nyilvánvalóan az üres zsák, aminek összértéke 0 (3. sor).

Amennyiben vannak még elemek, akkor elkezdjük megvizsgálni a t.-et. Nézzük meg, hogy mennyi lenne a vizsgálandó tárgyakhoz tartozó optimum a maradék szabad hely mellett, ha nem rakjuk bele ezt az elemet a zsákba. A megoldás ilyenkor egy egyszerű rekurziós hívással megadható (5. sor), hiszen ennek az értéknek pont ugyanannyinak kell lennie, mint az előtte lévő tárgyakra vonatkozó optimum változatlan szabad hely mellett.

Ezt követően nézzük meg, hogy mi az optimum akkor, ha berakjuk a t. tárgyat a zsákba. Először ellenőrizzük, hogy egyáltalán belefér-e (6. sor). Ha igen, akkor kiszámoljuk az őt megelőző elemek által elérhető optimális összértéket (7. sor). Ez hasonló az előző híváshoz, azonban fontos változás, hogy a maradék elemek számára rendelkezésre álló szabad helyet csökkentettük a t. elem súlyával, hiszen már az is benne van a zsákban. Ennek megfelelően a rekurzió által visszaadott értéket növeljük a p_t értékkel, hiszen a t. elem értékével növelnünk kell részmegoldás értékét.

Amennyiben a vizsgált elem befért a zsákba, akkor megvizsgáljuk, hogy vele, vagy nélküle kaptunk jobb lokális optimumot (8. sor). Ha nem fért be, akkor nyilván a nélküle értéket kell figyelembe vennünk (10. sor). Bármelyik eset is lesz igaz, a P_{opt} változó az optimális részeredményt fogja mutatni ami elérhető a megadott paraméterek alapján. Ez a függvény visszatérési értéke (12. sor).

A fentiek ismeretében a teljes pakolást optimalizáló függvény már könnyen érthető. Egyszerűen meghívja az előbb bemutatott rekurziót N, illetve W_{max} értékekkel. Tehát a rekurzív függvény első hívásakor az utolsó tárgyat kezdi el vizsgálni, és még rendelkezésére áll a hátizsák teljes kapacitása.

A módszer értékelése

A módszer tárhelyigénye itt sem jelentős, a rekurzív hívások miatt ugyan több, mint a nyers erő alkalmazásánál, azonban valószínűleg így sem okoz problémákat. A futásidőt itt már nem tudjuk olyan egyszerűen meghatározni, ugyanis az már függ a bemenő adatoktól. Érdemes észrevenni, hogy önmagában a rekurzív megvalósítás nemhogy javítana, inkább csak ront a helyzeten, hiszen a függvény több úton is eljuthat ugyanazokhoz a részproblémákhoz, és ilyenkor minden esetben újra és újra megoldja azokat.

Ami javít a helyzeten az az, hogy maga az eljárás jóval körültekintőbben működik, mint a nyers erő egyszerű próbálgatása, hiszen a 6. sorban található feltétel miatt az eleve esélytelen részproblémák megoldásaival nem foglalkozik (pl. ha már az első tárgy olyan nagy, hogy nem fér bele a zsákba, akkor nem kezdi el feleslegesen vizsgálgatni az ebből adódóan eleve reménytelen 2^{N-1} darab részmegoldást).

A módszer előnyei:

- Van lehetőség valamilyen szinten irányítani a keresést, hogy ne vizsgáljon garantáltan rossz útvonalakat.
- Ennek megfelelően lépésszáma alacsonyabb lehet, mint a nyers erő technikánál látott.

A módszer hátrányai:

- Csak bizonyos szerkezetű megoldások esetén használható hatékonyan a technika.
- Nagyon sok felesleges számítást végez, amikor ugyanahhoz a részproblémához több ágon is eljut.

2.4. Feljegyzéses módszer

A módszer bemutatása

Az „oszd meg és uralkodj” módszer működése során gyakran lehet belefutni abba a hibába, hogy az algoritmus a részproblémákra bontás során ugyanazt a részproblémát többször is előállítja, és ennek megfelelően többször meg is oldja. Néhány példa:

- Fibonacci számok: klasszikus példa a rekurzióra, amikor az i. Fibonacci számot próbáljuk meg egy rekurzív képlettel kiszámítani. Érdemes egyszer lerajzolni a rekurzió által előállított részproblémákat, és jól látszik, hogy nagyobb kezdőparaméterek esetén a többszörösen kiszámított részproblémák száma nagyon magas.
- 0-1 hátizsák probléma: hasonlót tapasztalhatunk a 2.3 fejezetben megismert algoritmus esetében is. Érdemes megvizsgálni az implementált program futását, és itt is jól látható, hogy a rekurziós függvény általában többször is meghívódik ugyanazokkal a paraméterekkel, és ilyenkor értelemszerűen többször is kiszámítja ugyanazokat az értékeket.

Ez a probléma sokkal súlyosabb, mint ahogy elsőre gondolnánk. A rosszul megírt „oszd meg és uralkodj” elvű algoritmusok futásidejének szinte elhanyagolható része az, ami tényleges értékes munkát végez, a számítási kapacitás túlnyomó része feleslegesen számol újra már ismert részeredményeket. Ez a probléma bizonyos feladatok esetében az egész módszer gyakorlati használhatóságát kérdőjelezi meg.

Amennyiben szeretnénk mégis ezt az alapelvet használni a megoldás során, akkor is van lehetőségünk a fenti probléma (teljes vagy részleges) kiküszöbölésére. A fentiek alapján nyilvánvalóan adja magát az ötlet, hogy minden egyes részprobléma kiszámítása után tároljuk el ezt az eredményt valahol, és ha ezt követően a függvény ismét ugyanolyan paraméterekkel hívódna meg, akkor elég ebből a tárolóból elővenni a már kiszámított részeredményt.

Ezt nevezzük „feljegyzéses módszernek” (memoization). Maga a tároló tetszőleges típus lehet, egy halmaz, tömb, esetleg hasító táblázat (3). A tárolás során kulcsként célszerű a rekurzív függvény által kapott paramétereket használni (ez persze csak akkor helyes választás, ha a részeredmény csak ezektől a paraméterektől függ). Így minden függvényhíváskor először azt ellenőrizzük, hogy ilyen paraméterekkel van-e már részeredményünk. Ha igen, akkor ezt visszaadjuk. Ha még nincs, akkor a hagyományos módon kiszámítjuk a részeredményt, és eltároljuk a tárolóban.

A 0-1 hátizsák probléma megoldása

A fenti elveknek megfelelő megoldást mutatja a 2.5. algoritmus. Az algoritmus alapvető működése szándékoltan azonos az előző fejezetben megismert 2.4. algoritmuséval. Emiatt a feladat megoldásának részleteire nem is térnénk ki, csak a feljegyzéses módszer jellegzetességeire.

Az implementáció csak két helyen tér el az előző fejezetben megismerttől:

- Részeredmény feljegyzése: alapesetben az algoritmus pontosan ugyanazzal a módszerrel számolja ki a részeredmény értékét, mint a 2.4. algoritmus. Az egyik fontos különbség azonban az, hogy miután megvan a részeredmény pontos értéke, még annak visszaadása (és a függvényből való kilépés) előtt azt eltároljuk. A pszeudokódban ezt a RészMegoldásTárolóbaFeljegyez eljárás meghívásával tettük meg (15. sor). Ennek első paramétere a részfeladat azonosítása (ez jelen esetben nem más, mint a rekurzív függvény hívási paramétereiből alkotott számpár), második paramétere pedig az ehhez tartozó részmegoldás.
- Részeredmény keresése: már a rekurzív függvény elején (5. sor), mielőtt bármit is számolnánk, először megvizsgáljuk, hogy a kiszámítandó részprobléma megoldása szerepel-e már a tárolóban. Ehhez használjuk a RészMegoldásTárolóbanKeres függvényt. Ennek egyetlen paramétere a részprobléma azonosítója (jelen esetben ez a tároláshoz hasonlóan a t és a h egészek által alkotott számpár). Amennyiben van már ilyen részprobléma a tárolóban, akkor az ahhoz tartozó eredményt azonnal vissza is adja a függvény, így nem végez felesleges számításokat. Amennyiben nincs (a függvény visszatérési értéke ilyenkor $\emptyset$), akkor pedig megindul a már előzőleg megismert kiszámítási folyamat (a végén a tárolással).

2.5. Algoritmus A 0-1 hátizsákprobléma megoldása a „feljegyzéses módszer” segítségével

Bemenet: t - **egész**, h - **egész**
Kimenet: P_{opt} - **egész** (*optimális pakoláshoz tartozó összérték*)

```
1:  függvény LegjobbRészMegoldás(t : egész, h : egész)
2:      ha (t = 0) ∨ (h = 0) akkor
3:          vissza 0
4:      különben
5:          ha RészMegoldásTárolóbanKeres([t, h]) ≠ ø akkor
6:              vissza RészMegoldásTárolóbanKeres([t, h])
7:          különben
8:              P_nem ← LegjobbRészMegoldás(t − 1, h)
9:              ha h ≥ w_n akkor
10:                 P_igen ← p_n + LegjobbRészMegoldás(t − 1, h − w_t)
11:                 P_opt ← max(P_igen, P_nem)
12:             különben
13:                 P_opt ← P_nem
14:             elágazás vége
15:             RészMegoldásTárolóbaFeljegyez([t, h], P_opt)
16:             vissza P_opt
17:         elágazás vége
18:     elágazás vége
19: függvény vége

Kimenet: P_opt - egész (egy optimális pakolás összértéke)
20: függvény HátizsákMemo
21:     vissza LegjobbRészMegoldás(N, W_max)
22: függvény vége
```

Felhasznált változók és függvények

- N - A hátizsákba pakolható tárgyak száma.
- w_i - A hátizsákba pakolható i. tárgy súlya ($1 \leq i \leq N$).
- p_i - A hátizsákba pakolható i. tárgy hasznossága ($1 \leq i \leq N$).
- W_{max} - A hátizsák mérete (a belepakolható tárgyak maximális összsúlya).
- P_{opt} - Egy optimális pakolás összértéke.
- t - A részmegoldás kereséskor a tárgyak darabszáma.
- h - A részmegoldás kereséskor a rendelkezésre álló szabad hely.
- RészMegoldásTárolóbaBeszúr(*kulcs*, *érték*) - Egy tetszőleges adatszerkezetbe (pl. hasító táblázatba) eltárolja a megadott kulcshoz (amely jelen esetben a t és h változók párosa), tartozó egész szám értéket (ami a részmegoldás-keresés eredménye).
- RészMegoldásTárolóbaFeljegyez(*kulcs*) - Visszaadja a paraméterként átadott kulcshoz (amely jelen esetben a t és h változók párosa) tartozó egész szám értéket (ami egy előzőleg eltárolt részmegoldás-keresés eredménye). Amennyiben ilyen nincs, akkor visszatérési értéke **ø**.

A módszer értékelése

A módszer jelentősen növelte az algoritmus tárigényét, hiszen ezt követően minden vizsgálandó részprobléma azonosítóját és eredményét el kell tárolni (a rekurzió miatt ez persze a részproblémák részproblémáira is igaz). Klasszikus példája ez annak a kódoptimalizációnak, amikor a futásidő javításáért cserébe gyengébb tárhelykihasználással fizetünk. Mindazonáltal a tárhely igény így se lesz vészes, könnyen belátható, hogy a részproblémák száma nem lehet több $N \times W_{max}$-nál. Hiszen a függvény t paramétere csak $1..N$ közötti értékeket, a h paramétere pedig csak $1..W_{max}$ értékeket vehet fel.

A módszer előnyei:

- Jelentősen tudja csökkenteni az egyébként nagy futásidejű algoritmusok lépésszámát.
- Nagyon egyszerűen implementálható. Ha be tudjuk látni, hogy a gyorsítandó függvény kimenete csak a bemenő paraméterektől függ, akkor tetszőleges függvénybe beépíthető ez a gyorsítótár, annak működésének pontos ismerete nélkül.

A módszer hátrányai:

- Csak bizonyos szerkezetű megoldások esetén használható hatékonyan a technika (ha sokszor kerül ugyanannak a részfeladatnak a megoldására a vezérlés).
- Jelentős többlet tárhelyigényt jelent.

Érdemes megjegyezni, hogy bár mi a „feljegyzéses módszert” az „oszd meg és uralkodj” technika kiegészítéseként vizsgáltunk, de természetesen attól függetlenül is egy értékes alapelvről van szó, amivel tetszőleges algoritmus gyorsítása megvalósítható.

2.5. Dinamikus programozás

Egy bevezető példa

Az előző alfejezetekben láttunk példát az „oszd meg és uralkodj” elvű optimalizációra, illetve ennek gyorsítására, a „feljegyzéses módszerre”. Most megnézünk egy újabb gyorsítási lehetőséget. Hogy egy könnyen érthető példán lássuk ennek működési mechanizmusát, egy pillanatra térjünk vissza a már jól ismert Fibonacci számokat kiszámító példához.

A feladat ismert [?], az N-edik Fibonacci számot keressük, ami az alábbiak szerint definiálható:

$$F_N = \begin{cases} 1, & \text{ha } N \leq 1 \\ F_{N-2} + F_{N-1}, & \text{ha } N \geq 2 \end{cases} \tag{2.2}$$

Ez egy klasszikus „oszd meg és uralkodj” algoritmus, egyszerűen implementálható (2.6. algoritmus).

2.6. Algoritmus A Fibonacci sorozat N-edik elemének előállítása

Bemenet: N - **egész**
Kimenet: *szám* - **egész** (*az N-edik Fibonacci szám*)

```
1: függvény FibonacciDnC(N : egész)
2:     ha N ≤ 1 akkor
3:         vissza 1
4:     különben
5:         vissza FibonacciDnC(N − 1) + FibonacciDnC(N − 2)
6:     elágazás vége
7: függvény vége
```

Felhasznált változók és függvények

- N - Nemnegatív egész, a függvény kimenete az N-edik Fibonacci szám lesz.

Azonban tudjuk, hogy működése kimondottan pazarló, hiszen ugyanazokat a részeredményeket többször is kiszámolja. Érdemes egyszer átgondolni, hogy a rekurzív hívások miként építenek fel a megoldandó részfeladatokból egy hatalmas fát, amelynek sok azonos részfája van. Ezért alkalmaztuk a „feljegyzéses módszer” technikát, ami azon alapult, hogy eltároltuk a már egyszer kiszámolt részeredményeket. Így azokat nem kellett mindig újra kiszámítani, ez pedig jelentősen csökkentette a futásidőt, cserébe egy kicsivel több tárigénnyel járt. Ezt a megoldást mutatja a 2.7. algoritmus (az egyszerűség kedvéért az átmeneti tárolóba való mentést és keresést nem írtuk külön függvényekbe).

Az előző fejezetben már elemeztük ennek a technikának az előnyeit és a hátrányait, látjuk, hogy jóval hatékonyabb megoldást kaptunk a segítségével. Itt is érdemes egyszer végiggondolni, hogy miként hívja magát a függvény. Tulajdonképpen ugyanazt a fát kezdi el építeni, mint az előző algoritmus, a lényeges különbség azonban az, hogy nincsenek benne azonos részfeladatot megoldó részfák, azokat azonnal a már egyszer kiszámított eredménnyel helyettesítjük (ezt nevezik vágásnak).

Ha a feldolgozást, mint dinamikus folyamat tekintjük, akkor az is jól látszik, hogy a rekurzív függvény először csak hívogatja magát egészen addig, amíg el nem jut a már előre megadott triviális megoldásokig. Ezt követően kiszámolja a $REK[2]$ értéket, majd egy szinttel visszalép a rekurzióban, és nekiáll kiszámolni a $REK[3]$ értéket (ez már gyorsan megy, hiszen addigra a 2. és az 1. elem is ismert), ezt követi a $REK[4]$ számítása, és így tovább... Tehát kezdve azzal, hogy az inicializáláskor mi magunk adtunk értéket a $REK[0]$ és $REK[1]$ változóknak, jól látható, hogy az algoritmus tulajdonképpen egyesével végigszámolgatja a REK tömb elemeit 0-tól N-ig.

Ez adja a nagyon egyszerű ötletet, hogy hagyjuk el az egész rekurziót, és csak erre a tömb feltöltésre összpontosítsunk.

A végeremény (2.8. algoritmus) megdöbbentően egyszerű az előzőekhez képest. Alapvetően azonos lépéseket végez, mint az előző megoldás, és a tárhelyigénye is ugyanaz. Azonban itt már nincs szükség felesleges rekurziós lépésekre, illetve maga a rekurzió elhagyása is javíthatja a teljesítményt.

2.7. Algoritmus A Fibonacci sorozat N-edik elemének előállítása feljegyzéses módszerrel

Bemenet: N - **egész**, RET - **egész tömb**
Kimenet: *szám* - **egész** (*az N-edik Fibonacci szám*)

```
1: függvény FibonacciMemoRek(N : egész, RET : egész tömb)
2:     ha RET[N] = ø akkor
3:         RET[N] ← FibonacciMemoRek(N − 1, RET) + FibonacciMemoRek(N − 2, RET)
4:     elágazás vége
5:     vissza RET[N]
6: függvény vége

Bemenet: N - egész
Kimenet: szám - egész (az N-edik Fibonacci szám)
7: függvény FibonacciMemo(N : egész)
8:     RET[0..1] ← 1
9:     RET[2..N] ← ø
10:    vissza FibonacciMemoRek(N, RET)
11: függvény vége
```

Felhasznált változók és függvények

- N - Nemnegatív egész, a függvény kimenete az N-edik Fibonacci szám lesz.
- RET - Részeredmény tároló. N+1 elemű, egész szám típusú tömb (kivételesen 0-tól kezdjük az indexelést).

2.8. Algoritmus A Fibonacci sorozat N-edik elemének előállítása dinamikus programozással

Bemenet: N - **egész**
Kimenet: *szám* - **egész** (*az N-edik Fibonacci szám*)

```
1: függvény FibonacciMemo(N : egész)
2:     RET[0..1] ← [1, 1]
3:     ciklus i ← 2-től N-ig
4:         RET[i] ← RET[i − 1] + RET[i − 2]
5:     ciklus vége
6:     vissza RET[N]
7: függvény vége
```

Felhasznált változók és függvények

- N - Nemnegatív egész, a függvény kimenete az N-edik Fibonacci szám lesz.
- RET - Részeredmény tároló. N+1 elemű, egész szám típusú tömb (kivételesen 0-tól kezdjük az indexelést).

Megjegyzés

Vegyük észre, hogy a REK tömb következő elemének kiszámítása mindig csak az előző két elem kiszámítását igényli. Tehát a teljes $N + 1$ elemű tömb helyett elég lenne egy összesen 2 elemű, ahol mindig csak az utolsó két értéket tároljuk el. Ezzel a tárhelyigény máris konstans értékre csökkenthető.

Vegyük észre, hogy nem egy teljesen új megoldást alkottunk, hanem a kezdetben megadott „oszd meg és uralkodj" algoritmust alakítottuk át két lépésben erre a nagyon egyszerű és hatékony formára a tényleges rekurziós hívások kiküszöbölésével. A szemléletbeli különbséget talán azzal tudnánk a legegyszerűbben szemléltetni, ha rámutatunk, hogy az „oszd meg és uralkodj" jellegű módszerek mindig *felülről lefelé* építkeznek, tehát az összetett problémából indulnak ki, és próbálják azt több egyszerűbb részproblémává alakítani. Addig az utolsó megoldásunk éppen fordítva működött, ez *alulról felfelé* építkezik, tehát egyből a kis részproblémák megoldásával kezd, és ezek összeépítésével fokozatosan halad a bonyolultabb, majd végül a végső probléma megoldása felé. Ez vezet a „dinamikus programozás" (dynamic programming) alapelvéhez.

A módszer bemutatása

Elsőként vizsgáljuk meg, hogy mikor lehet/érdemes használni a dinamikus programozást? Alapvetően két követelményt határozhatunk meg (később, a konkrét példánál ezekre még visszatérünk):

- A feladat legyen *optimális részstruktúrájú*: a feladat optimális megoldása önmagán belül a részfeladatok optimális megoldásait is tartalmazza.
- Legyenek a *részfeladatok átfedőek*: a részfeladatokra bontás során többször is merüljön fel ugyanannak a részfeladatnak a megoldása.

Az első feltétel csak azt határozza meg, hogy maga az optimalizálási feladat a kisebb részfeladatok optimumából épüljön fel. A második feltétel is könnyen érthető, ezt már a feljegyzéses módszernél is láttuk. Amennyiben ugyanazt a részfeladatot sokszor kell megoldani, akkor az egy jelentős gyorsítási lehetőséget ad a kezünkbe, ha meg tudjuk oldani, hogy ezeket csak egyszer kelljen kiszámolni (miként erre az imént láttunk két példát is, felülről lefelé és alulról felfelé építkezve is).

A módszer használata általában az alábbi lépéseket igényli:

1. Az optimális megoldás szerkezetének jellemzése.
2. A megoldás értékének (jóságának) rekurzív módon való definiálása.
3. Az optimális megoldás értékének (jóságának) kiszámítása alulról felfelé történő módon.
4. Amennyiben szükséges, magának az optimális megoldásnak a magadása.

Fontos megkülönböztetünk a „megoldást" és a „megoldás értékét". A hátizsák pakolási probléma esetében a megoldást egy konkrét pakolás jelenti (tehát melyik tárgyakat rakjuk a hátizsákba és melyiket nem). A megoldás értéke pedig csak a pakolás értéke (tehát a pakolás által tartalmazott tárgyak értékének összege). A dinamikus programozás gyakran csak az értéket adja meg számunkra, és sokszor ez is elég (miként azt a fejezet elején tisztáztuk, most mi is csak ezt keressük). Persze gyakran szükség van magára a megoldásra is, ezt általában valamilyen kiegészítő technikával tudjuk előállítani a dinamikus programozás által előállított segéd adatokból (ez a 4. lépés).

A 0-1 hátizsák probléma megoldása (optimális érték)

A fentiek szerint próbáljuk megoldani a már ismert hátizsák problémánkat dinamikus programozás segítségével. Elsőként vizsgáljuk meg, hogy a feladat megfelel-e a szükséges követelményeknek:

- A feladat optimális részstruktúrájú. A rekurzív hívásokból is jól látható, hogy a teljes hátizsákra és összes tárgyra vonatkozó optimumot visszavezettük az egyes kevesebb tárgyat és esetenként valamivel kevesebb szabad helyet tartalmazó részproblémák optimális megoldásainak a megkeresésére.

- A részfeladatok gyakran átfedőek. Épp ez jelentette a problémát az „oszd meg és uralkodj” módszer esetén, amit jelentősen tudtunk enyhíteni a „feljegyzéses technika” segítségével. Most egy másik módszerrel próbáljuk ezt kezelni.

Ennek megfelelően a megoldás menete:

1. Az optimális megoldást az eddigiekhez hasonlóan egy N elemű logikai tömbként képzeljük el, aminek az i. eleme azt mutatja, hogy egy optimális pakolás esetén az i. tárgy bekerül-e a zsákba vagy sem. A jóságot pedig az így bekerült tárgyak összértékének tekintjük.

2. A rekurzív módon való definiáláshoz ugyanazt az elvet használjuk, amit már leírtunk az „oszd meg és uralkodj” módszernél. Tehát a teljes feladat megoldását visszavezetjük arra, hogy megvizsgáljuk, hogy az utolsó elemmel vagy anélkül kapunk-e jobb eredményt. Ehhez pedig mindkét esetben (már ha egyáltalán elfér az utolsó tárgy) egy rekurzív hívással számítjuk ki a többi tárgy optimális pakolásának értékét a részükre maradt hely alapján. A leírás tehát teljesen azonos:

$$F_{t,h} = \begin{cases} 0, & \text{ha } h = 0 \\ 0, & \text{ha } t = 0 \\ F_{t-1,h}, & \text{ha } h > 0 \wedge t > 0 \wedge h < w_t \\ max\{F_{t-1,h},\ F_{t-1,h-w_t} + p_t\}, & \text{ha } h > 0 \wedge t > 0 \wedge h \geq w_t \end{cases} \tag{2.3}$$

3. Az így megadott rekurzív megoldást megpróbáljuk kiszámítani alulról felfelé történő módon. Célszerű újra rátekinteni a Fibonacci számoknál alkalmazott módszerre, ahogy a rekurziót egy egyszerű táblázat kitöltéssel tudtuk helyettesíteni. Itt valójában ugyanez történik, csak a táblázat nem egy, hanem két dimenziós.

4. Az előző lépések még csak az optimális megoldás értékét adják meg. A megoldás előállítását a következő alfejezet (44. oldal) tartalmazza.

Ha csak az optimális megoldás értékére vagyunk kíváncsiak, akkor a 3. lépésben megadott táblázat kitöltést kell csak implementálnunk. Ezt mutatja a 2.9. algoritmus.

Az algoritmus tehát egy F táblázatot tölt fel. A táblázat t. sorában és h. oszlopában lévő érték ($F[t,h]$) mindig az $F_{t,h}$ függvény értéket fogja tárolni annak első kiszámítását követően. Tehát annak az optimális pakolásnak az összértékét, amely az első t darab tárgyat próbálja elhelyezni h szabad helyen. Mivel a tárgyak száma és a szabad hely is korlátos ($0 \leq t \leq N, 0 \leq h \leq W_{max}$), így ezt a tömböt már előre létrehozhatjuk. Mérete $(N+1) \times (W_{max}+1)$ (hogy jobban igazodjunk a rekurzív definícióhoz, a tömböt most kivételesen 0-tól indexeljük). Kezdetben a teljes táblázat üres.

Első lépésként beírjuk a táblázatba a triviális megoldások eredményeit, hiszen ezek nem igényelnek további számításokat. A program első ciklusa (2. sor) feltölti a 0 indexű oszlopot nullákkal. Ez tulajdonképpen azt jelzi, hogy a tárgyak számától és adataitól függetlenül, ha nincs már a zsákban szabad hely, akkor az optimális pakolás értéke csak 0 lehet. A következő ciklus (5. sor) feltölti a 0 indexű sort is nullákkal. Ez pedig azt a triviális megoldást képviseli, hogy a szabad helytől függetlenül, ha nincsenek tárgyaink, akkor az optimális pakolás értéke szintén csak 0 lehet. (2.1a. ábra).

Ezt követően megkezdhetjük a táblázat belső elemeinek a kitöltését. A sorrend természetesen fontos, hiszen csak azokat az értékeket tudjuk kiszámolni, amelyek már ismert részeredményeket használnak fel. A rekurzív képletből jól látható, hogy a táblázat t. sorában lévő értékekhez mindig csak a $t-1$. sor adataira van szükség. Ugyanígy belátható, hogy a táblázat bármelyik h. oszlopában lévő értékhez csak a h-nál kisebb cellákban lévő adatokra lehet szükség. Tehát az algoritmusban megadott ciklusokat használva a feltöltés mindig csak a már előzőleg kiszámolt adatokon fog alapulni.

Az egyes cellák adatait kiszámoló 10–14. sorok a fentiek alapján már nem igényelnek különösebb magyarázatot. Jól látható, hogy teljes egészében a már részletesen megvizsgált 2.3. egyenlet lett implementálva. Működése tehát annak megfelelő.

Miután a ciklus végzett, kivesszük a táblázat $F[N, W_{max}]$ cellájában lévő értéket. A fentiek szerint itt az N darab tárgy W_{max} szabad helyen való optimális elrendezésének értékét tároljuk. Ez pedig nem más, mint a teljes feladat megoldása, ezt keressük. Emiatt ez a függvény visszatérési értéke (17. sor).

Egy gyakorlati példán jól látható, hogy mennyire egyszerűen működik a fenti algoritmus. Feltételezzük, hogy van 6 darab tárgyunk a a 2.1. táblázatban látható súly (w_i) és érték (p_i) jellemzőkkel. Továbbá azt, hogy a hátizsák kapacitása (W_{max}) egyenlő 4-el.

2.9. Algoritmus A 0-1 hátizsákprobléma megoldása „dinamikus programozás” technikával

Kimenet: P_{opt} - **egész** (*egy optimális pakolás összértéke*)

```
1: függvény HÁTIZSÁKDP
2:     ciklus t ← 0-től N-ig
3:         F[t, 0] ← 0
4:     ciklus vége
5:     ciklus h ← 1-től W_max-ig
6:         F[0, h] ← 0
7:     ciklus vége
8:     ciklus t ← 1-től N-ig
9:         ciklus h ← 1-től W_max-ig
10:            ha h ≥ w_t akkor
11:                F[t, h] ← max(F[t − 1, h], F[t − 1, h − w_t] + p_t)
12:            különben
13:                F[t, h] ← F[t − 1, h]
14:            elágazás vége
15:        ciklus vége
16:    ciklus vége
17:    vissza F[N, W_max]
18: függvény vége
```

Felhasznált változók és függvények

- N - A hátizsákba pakolható tárgyak száma.
- w_i - A hátizsákba pakolható i. tárgy súlya ($1 \leq i \leq N$).
- p_i - A hátizsákba pakolható i. tárgy hasznossága ($1 \leq i \leq N$).
- W_{max} - A hátizsák mérete (a belepakolható tárgyak maximális összsúlya).
- P_{opt} - Egy optimális pakolás összértéke.
- F - $(N + 1) \times (W_{max} + 1)$ méretű, egész számokat tartalmazó tömb. Kivételesen 0-tól kezdjük a tömb indexelését mindkét dimenzióban.

2.1. táblázat. Dinamikus programozás példában vizsgált tárgyak súlyai és értékei

i	1	2	3	4	5	6
w_i	2	1	1	1	3	2
p_i	4	3	2	8	7	5

Az algoritmus első lépése a táblázat feltöltése a triviális értékekkel (2.1a. ábra). Tehát a 0. sort és a 0. oszlopot feltöltjük 0 értékekkel.

Ezt követően soronként (1-től kezdődő növekvő indexeléssel), és azon belül oszloponként (1-től kezdődő növekvő indexeléssel) megkezdjük a táblázat alulról-felfelé való feltöltését. Maga a kitöltés nagyon egyszerű, mivel csak a 2.3. képletet kell használnunk minden esetben. A kitöltés irányából adódik, hogy a szükséges részeredmények már mind rendelkezésre fognak állni a szükséges pillanatban.

Egy-egy konkrét példát érdemes kiemelni az alábbi esetekre:

- Az éppen vizsgált tárgy nem fér el a rendelkezésre álló helyen. Erre mutat példát a 2.1b. ábra, vagy egy későbbi lépés esetén pl. a 2.1v. ábra. Ilyenkor nincs mérlegelési lehetőségünk, értelemszerűen csak a változatlan szabad hely mellett a vizsgált tárgy nélküli optimális pakolás értékét tudjuk figyelembevenni (és ez lesz az optimum ebben az esetben is). Ezt nem kell újra kiszámolnunk, hiszen ezt az értéket mutatja az $F[t-1,h]$ érték.

- Az éppen vizsgált tárgy elfér a zsákban és érdemes is belerakni. Erre mutat példát a 2.1q. ábra. Ebben az esetben a t értéke 4, tehát összesen négy tárgyunk van csak, és ezek közül a negyediket vizsgáljuk. A h értéke 4, tehát feltételezzük, hogy a szabad hely mérete 4. A w_t értéke 1, tehát a negyedik tárgy súlya kevesebb, mint a szabad hely, így elvileg elhelyezhetjük a zsákban. Ilyenkor célszerű megvizsgálni az alábbiakat:

  - Ha azt feltételezzük, hogy berakjuk a zsákba, akkor egyrészt értékként megkapjuk a t. tárgy értékét (p_t), tehát 8-at. Ha berakjuk a 4 méretű zsákba az 1 méretű tárgyat, akkor még marad is 3 szabad helyünk ($h - w_t$). Tudnunk kellene, hogy erre a 3 maradék helyre mi a maradék tárgyak optimális elrendezése, és szerencsére ezt tudjuk, hiszen már kiszámítottuk. Az $F[3,3]$ azt mutatja, hogy csak az első 3 tárgy figyelembevételével 3 szabad helyet feltételezve mennyi az elérhető optimális pakolás összértéke. Ez pedig (7) pontosan az, amit keresünk. Összefoglalva: ha a negyedik tárgyat elhelyezzük a zsákban, akkor az optimális érték: $F[t, h - w_t] + p_t = 7 + 8 = 15$
  - Ha azt feltételezzük, hogy nem rakjuk be a zsákba, akkor a tárgy nem fogja növelni a pakolás értékét, de cserébe a szabad helyet se csökkenti. Ilyenkor az elérhető optimum tehát nem más, mint az azonos szabad hely mellett, de eggyel kevesebb tárgy pakolásánál elérhető érték. Ezt pedig az $F[t-1,h]$ mutatja, ami a kitöltés sorrendje miatt már szintén ismert (9).

  Attól függően tehát, hogy a tárgyat berakjuk-e a zsákba vagy sem, 15 illetve 9 jóság lenne elérhető. Ennek megfelelően a kettő közül a nagyobbik lesz az $F[t,h]$ értéke.

- A harmadik lehetőség pedig az, amikor befér ugyan a következő tárgy a zsákba, azonban nem jutunk vele jobb eredményhez. Erre mutat példát a 2.1l. ábra. Ebben az esetben a t értéke 3, tehát az első három tárggyal dolgozunk, és a harmadikat vizsgáljuk. A h értéke 3, tehát feltételezzük, hogy a szabad hely mérete 3.

  - A harmadik tárgy mérete 1, tehát nyilvánvaló, hogy önmagában elfér a zsákban ($h \geq w_t$). Sőt, még marad is 2 szabad hely ($w_t - h$), tehát tudnunk kell, hogy erre a maradék helyre a maradék tárgyak milyen optimális pakolási értéket tudnak elérni. Ez nem más, mint a táblázat $F[t-1, h-w_t]$ elemének az értéke, ami már ismert, jelen esetben 4. Ebben az esetben ehhez még hozzáadjuk a harmadik tárgy értékét (p_t), így összesen 6-os összértéket érhetünk el.
  - Vizsgáljuk meg azt az esetet is, ha nem rakjuk bele a tárgyat a zsákba. Ilyenkor a nélküle elérhető optimális érték $F[t-1,h]$, ami már szintén ismert, hiszen előzőleg kiszámítottuk: 7.

  Jól látható, hogy ugyan a tárgy belefér a zsákba, de nem érdemes belerakni. Nyerünk ugyan vele p_t értéket, de az általa elfoglalt terület miatt a többi tárgy által elérhető érték ennél többel csökkenne. Emiatt az optimális pakolás értéke (és ezzel az $F[3,3]$ értéke) 7 lesz.

Ugyanazokat a lépéseket követjük végig az algoritmus futása során, beleérve a legutolsó $F[N, W_{max}]$ érték kiszámítását is. A példában látható, hogy még az utolsó lépésben is sikerült egy jobb megoldást találnunk (2.1y. ábra). Itt már nagyon nehéz lenne fejben visszakövetni, hogy pontosan hogyan is jöttek ki az előző sorok értékei, de elég azt tudnunk, hogy az algoritmus végig jól működött, tehát az itt felhasznált $F[5,2]$ és $F[5,4]$ értékek valóban a megadott paraméterekkel elérhető optimális eredményt tartalmazzák, és ezekre építve már egyszerűen kiszámolható a végeredményt.

A teljes feladat megoldát tehát a táblázat $F[N, W_{max}]$ cellája tartalmazza a program lefutását követően (2.1z. ábra).

A 0-1 hátizsák probléma megoldása (megoldás előállítása)

Az optimalizálási feladat kiírásakor nagyvonalúan csak az optimális pakolás értékének keresését igényeltük, ezt teljesítettük is. A mintapéldában a dinamikus algoritmussal megtudtuk, hogy egy optimális megoldás értéke 16. A gyakorlatban azonban sokszor szükség van magára a tényleges megoldásra is, erre viszont nem kaptunk még választ, az algoritmus nem adta meg az optimális pakolást (tehát azt, hogy konkrétan melyik tárgyakat kell belerakni a zsákba és melyeket nem).

Önmagában a dinamikus programunk erre nem tud választ adni, egy kis kiegészítéssel azonban az általa felépített táblázatból ez is kiolvasható. A felépített F táblázat alapján tehát az optimális pakolás adatait keressük.

2.10. Algoritmus A 0-1 hátizsákprobléma egyik optimális pakolásának előállítása

Bemenet: F **- egész tömb**
Kimenet: OPT **- egész** (*egy optimális pakolás adatai*)

```
1:  függvény HátizsákDPEredmény(F : egész tömb)
2:      OPT ← [hamis, hamis, ..., hamis]
3:      t ← N
4:      h ← W_max
5:      ciklus amíg (t > 0) ∧ (h > 0)
6:          ha F[t, h] ≠ F[t − 1, h] akkor
7:              OPT[t] ← igaz
8:              h ← h − w_j
9:          elágazás vége
10:         t ← t − 1
11:     ciklus vége
12:     vissza OPT
13: függvény vége
```

Felhasznált változók és függvények

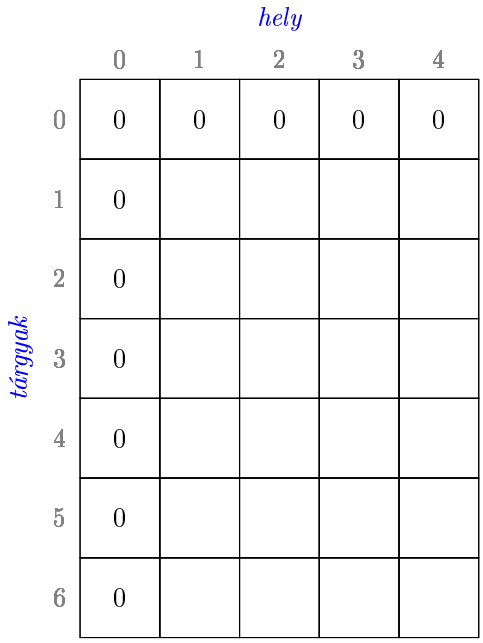
- F - $(N+1)\times(W_{max}+1)$ méretű, egész számokat tartalmazó tömb, amit kivételesen 0-tól indexelünk. Az előzőleg megismert HátizsákDP állítja elő (2.9. algoritmus).
- N - A hátizsákba pakolható tárgyak száma.
- w_t - A hátizsákba pakolható i. tárgy súlya ($1 \leq t \leq N$).
- W_{max} - A hátizsák mérete (a belepakolható tárgyak maximális összsúlya).
- OPT - Az eddig talált legjobb pakolás adatai (logikai tömb, ahol $OPT[t]$ azt mutatja, hogy a t. tárgy benne van-e a zsákban ($1 \leq t \leq N$)).

A 2.2. ábra a teljes megoldás visszafejtését mutatja be. Az első ábra alapján már könnyen megérthető ennek menete. Egy ciklussal elindulunk a táblázat $F[N, W_{max}]$ elemétől. Ennek értéke a példában 16. Tehát az ideális pakolás 6 tárgy és 4 méretű hátizsák esetén 16 értékkel bír. Elsőként vizsgáljuk meg azt, hogy a hatodik tárgy része lehet-e ennek a pakolásnak (2.2a. ábra).

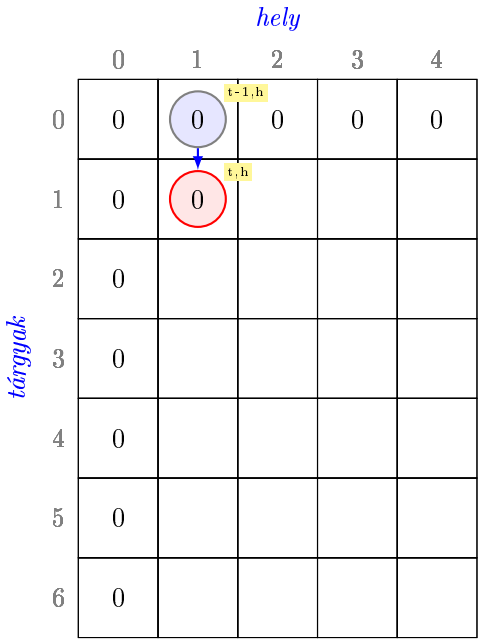
Elsőként tegyük fel, hogy nem része. Ekkor a 6. tárgy által adott 16 összértéket kell adnia az első 5 tárgynak is ugyanilyen hátizsákméret mellett. Ezért ellenőrizzük az $F[t-1, h]$ értéket. Mivel ez nem 16, ez azt mutatja, hogy a 6. tárgy nélkül nem lehet ugyanezt az eredményt elérni. Ezért biztos, hogy a 6. tárgy része az optimális pakolásnak, ezt jelöljük az OPT tömb utolsó elemének igazra állításával.

Tehát már tudjuk, hogy a hatodik része a pakolásnak, akkor vizsgáljuk meg a többit. Ha összesen 16 volt az optimális összérték, akkor nyilvánvaló, hogy a 6. elem nélkül a többi elemmel is előállítható egy 11 $(16-5)$ összértékű pakolás. A kérdés csak az, hogy miként. Ez a lépés még szintén a 2.2a. ábra alapján látható, átlépünk a 5. sor 2. oszlopába.

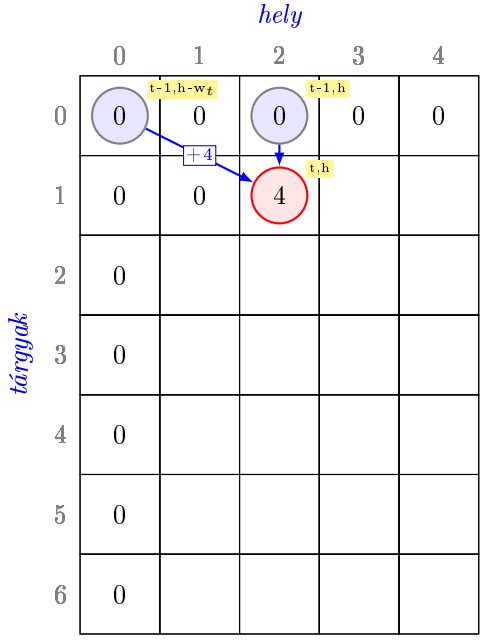
A 2.2b. ábra a következő tárgy meghatározását mutatja, ami egy másik féle lépésre mutat példát. Az $F[5, 2]$ értéke 11, tehát 5 tárggyal 2 szabad helyet feltételezve legfeljebb 11-es összérték érhető el. Látszik, hogy az $F[4, 2]$ értéke is ugyanez, tehát ugyanekkora szabad helyen csak az első 4 tárggyal is



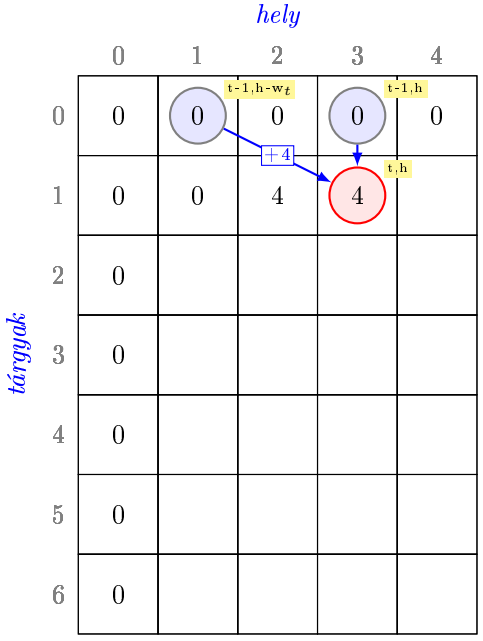
(a) Az inicializálás utáni üres táblázat. 6 tárgyunk van, és a zsák kapacitása 4. A 0. indexű oszlopot feltöltöttük 0-val (tehát ha nincs hely, akkor ez az optimális érték). A 0. indexű sort is feltöltöttük (ha nincsenek tárgyaink, akkor is ez az optimális érték).



(b) $t = 1, h = 1, w_t = 2, p_t = 4$
Mivel $h < w_t$, ezért a felette lévő mezőt másoljuk át (mivel a 2 súlyú tárgy nem kerülhet bele az 1 méretű zsákba).

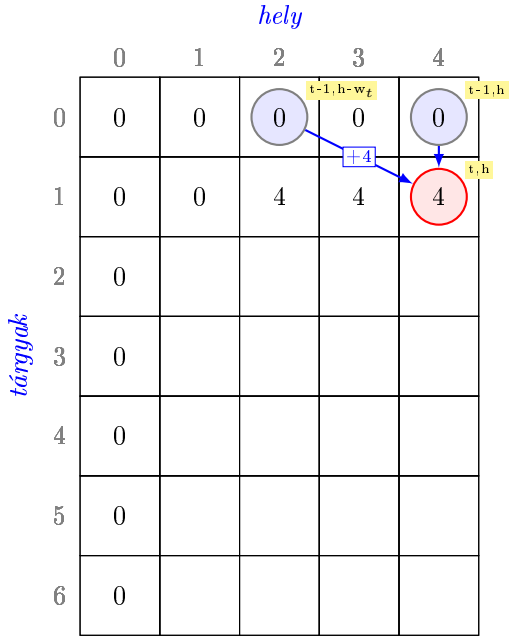


(c) $t = 1, h = 2, w_t = 2, p_t = 4$
Mivel $h \geq w_t$, ezért megvizsgáljuk az optimumot vele és nélküle is. Az F[t-1,h-2]+4 a legmagasabb elérhető érték (tehát az 1. tárgy bekerül a zsákba).



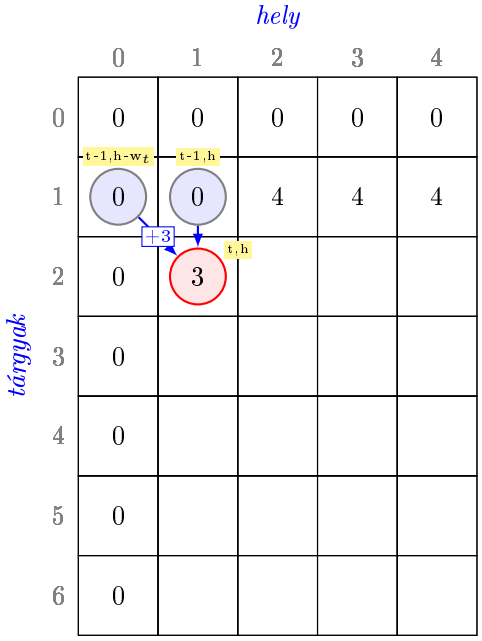
(d) $t = 1, h = 3, w_t = 2, p_t = 4$
Ugyanaz a helyzet mint az előző esetben. Itt is érdemes berakni a tárgyat a zsákba (és persze bele is fér).

2.1. ábra. A 0-1 hátizsák probléma megoldása dinamikus programozással.



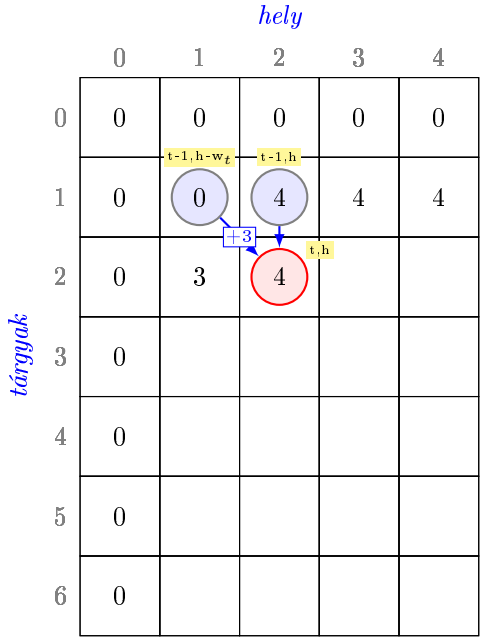
(e) $t = 1, h = 4, w_t = 2, p_t = 4$

Értelemszerűen az eredmény itt is ugyanaz mint az előzőekben. Hiszen egyetlen tárgyat próbálgatunk egyre nagyobb szabad helyre.



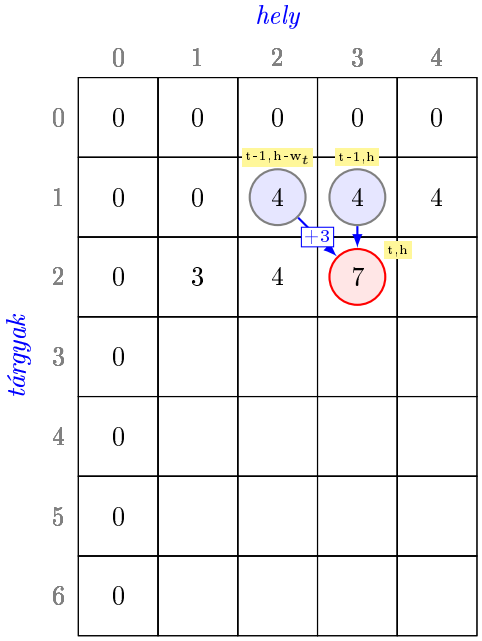
(f) $t = 2, h = 1, w_t = 1, p_t = 3$

Eggyel több tárgyat vizsgálunk a következő sorban. A második tárgy súlya 1, tehát 1 szabad hely esetén elfér ($h \geq w_t$). Ezért megvizsgáljuk a vele és nélküle elérhető optimumot, ezek közül az előbbit választjuk.



(g) $t = 2, h = 2, w_t = 1, p_t = 3$

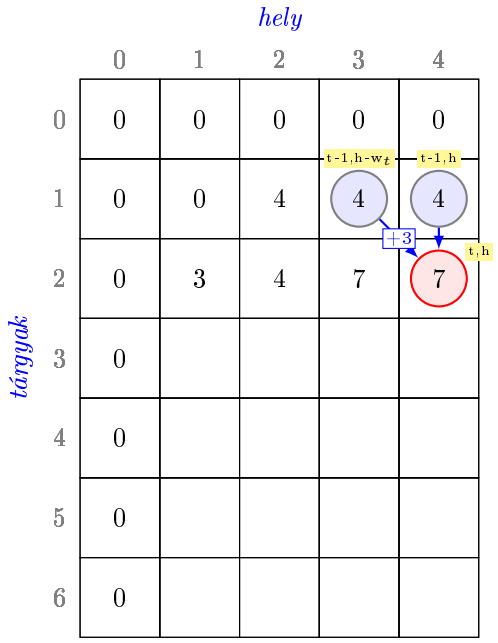
A következő lépésben ugyanazt a tárgyat vizsgáljuk eggyel nagyobb szabad helyen. Ha felvesszük a zsákba, akkor nem fér mellé semmi, így 3 az optimális érték. Viszont ha nem vesszük fel a zsákba, akkor nélküle 4 is elérhető, így ezt választjuk.



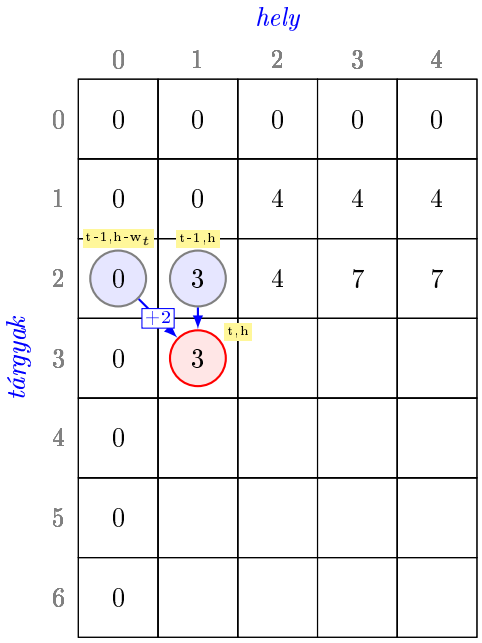
(h) $t = 2, h = 3, w_t = 1, p_t = 3$

Két tárgyat vizsgálunk 3 szabad hely esetén. Itt már látszik, hogy a második tárgy bevétele esetén érjük el az optimális pakolási összértéket.

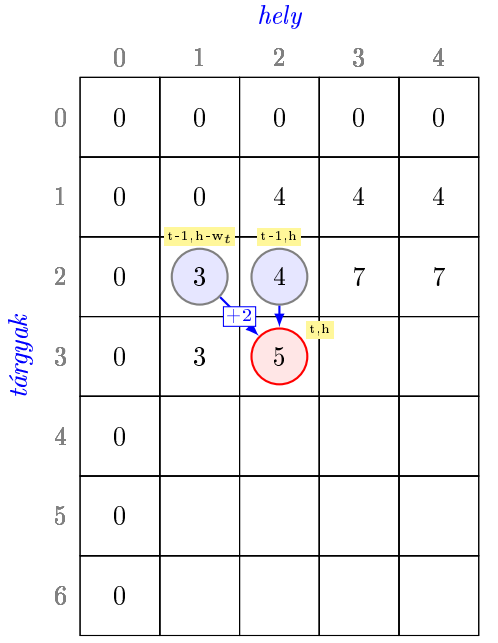
2.1. ábra. A 0-1 hátizsák probléma megoldása dinamikus programozással. (folytatás)



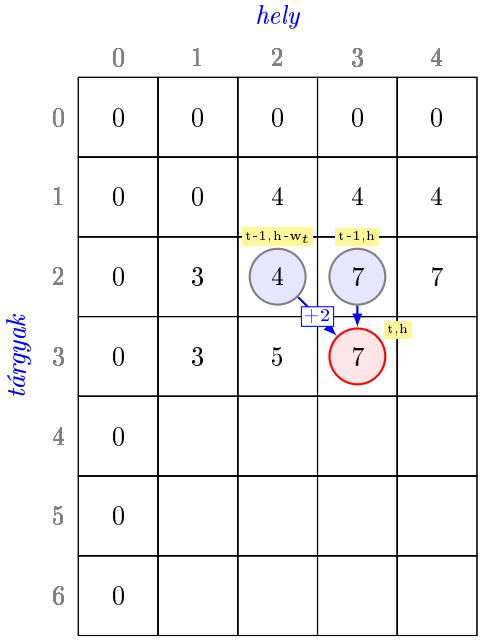
(i) $t = 2, h = 4, w_t = 1, p_t = 3$
Eggyel nagyobb a hely mint az előző esetben, de az optimális érték ugyanennyi (mivel már előzőleg is elfért minden tárgy a zsákban).



(j) $t = 3, h = 1, w_t = 1, p_t = 2$
Most már három tárgyat vizsgálunk. A harmadik elfér a rendelkezésre álló helyre, viszont nélküle jobb eredmény érhető el, így az lesz az optimum.

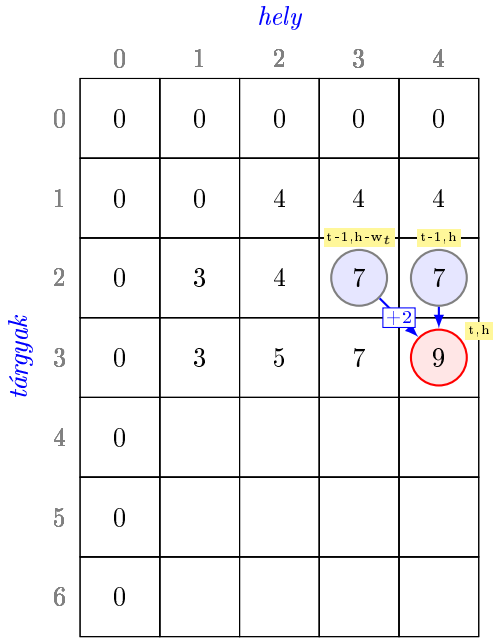


(k) $t = 3, h = 2, w_t = 1, p_t = 2$
Eggyel több szabad hely esetén már érdemes berakni a zsákba, mivel így érjük el a legjobb eredményt.

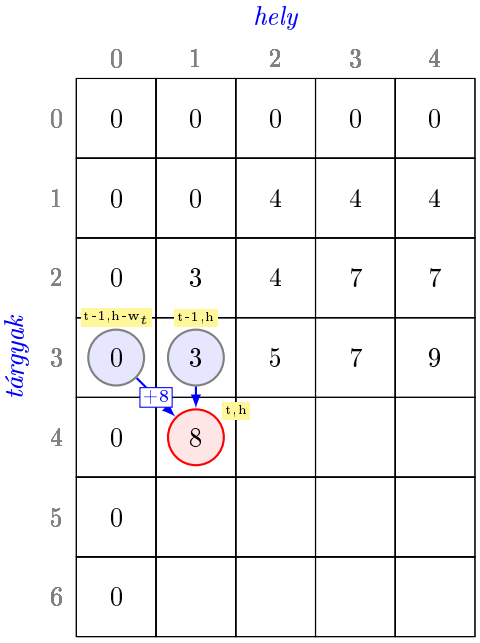


(l) $t = 3, h = 3, w_t = 1, p_t = 2$
Itt már egyre nehezebb fejben áttekinteni az eredményt. De a táblázat felépítéséből adódik, hogy az új tárgy felvételével, és az így csökkentett szabad hely optimális kitöltésével rosszabb eredményt kapnánk, mintha nélküle a teljes szabad helyre keresnénk optimális pakolást.

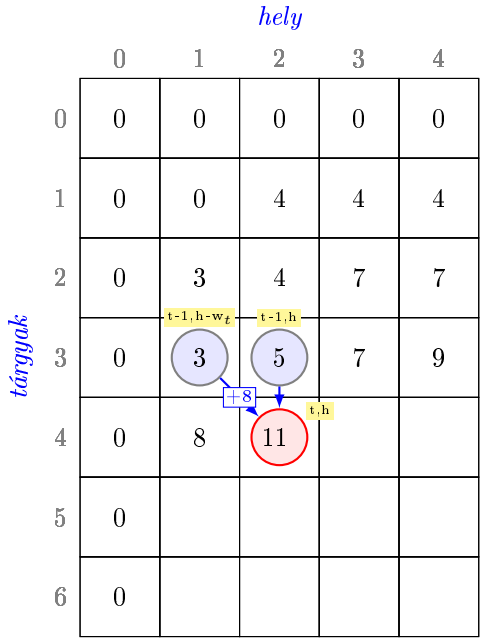
2.1. ábra. A 0-1 hátizsák probléma megoldása dinamikus programozással. (folytatás)



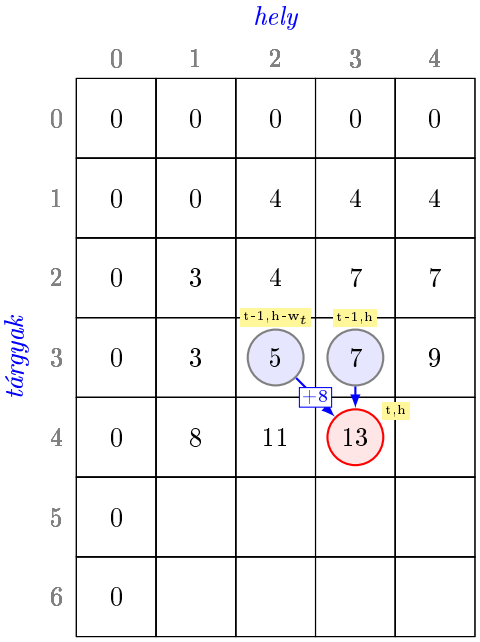
(m) $t = 3, h = 4, w_t = 1, p_t = 2$
Az optimális eredményt tehát úgy érjük el, ha berakjuk a zsákba.



(n) $t = 4, h = 1, w_t = 1, p_t = 8$
Vizsgáljuk a következő tárgyat. Elfér 1 helyen és jobb értéket ad mint az előzőleg elért optimumok, tehát beválasztjuk.

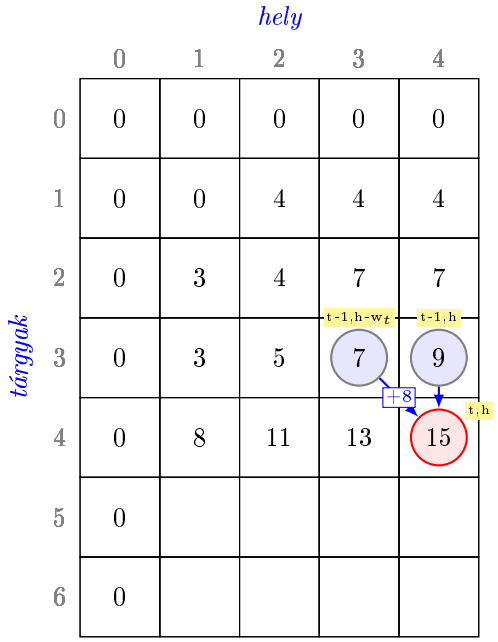


(o) $t = 4, h = 2, w_t = 1, p_t = 8$
Követjük a már megismert algoritmust.

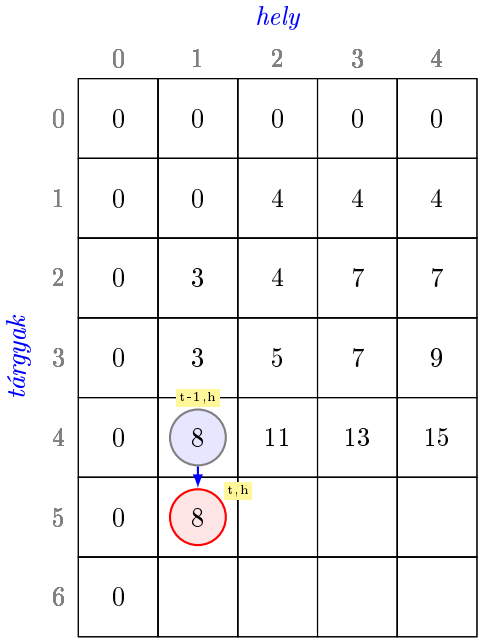


(p) $t = 4, h = 3, w_t = 1, p_t = 8$
Követjük a már megismert algoritmust.

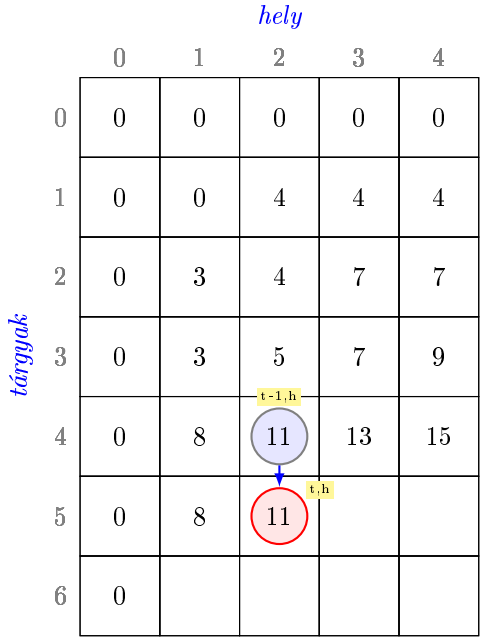
2.1. ábra. A 0-1 hátizsák probléma megoldása dinamikus programozással. (folytatás)



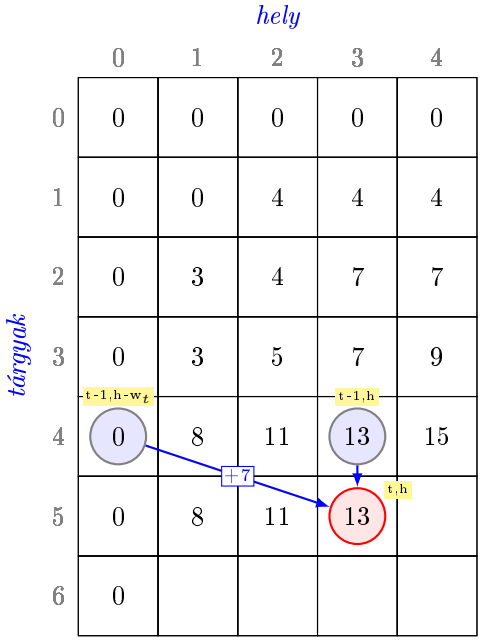
(q) $t = 4, h = 4, w_t = 1, p_t = 8$
Követjük a már megismert algoritmust.



(r) $t = 5, h = 1, w_t = 3, p_t = 7$
A vizsgált tárgy nem fér el a rendelkezésre álló szabad helyen, így nincs más választásunk, másoljuk a felette lévő értéket.

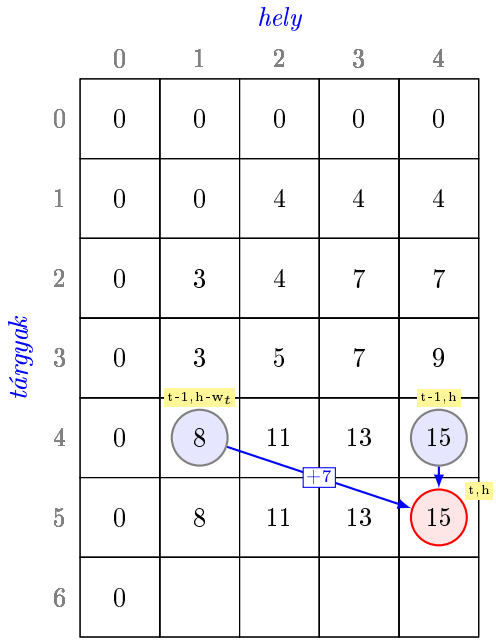


(s) $t = 5, h = 2, w_t = 3, p_t = 7$
A tárgy nem fér el, így nincs más választásunk, másoljuk a felette lévő értéket.

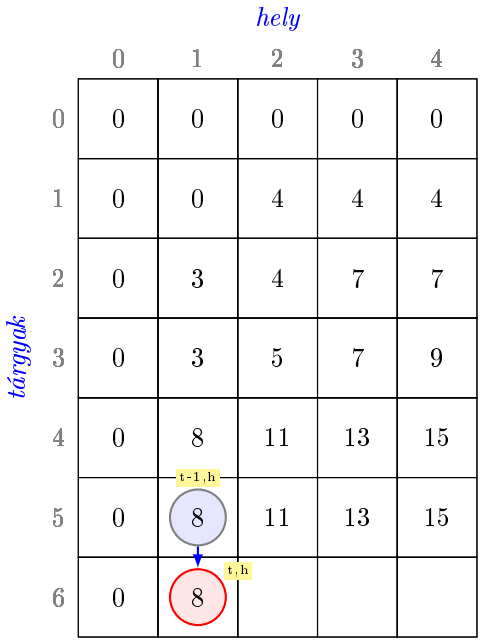


(t) $t = 5, h = 3, w_t = 3, p_t = 7$
A tárgy itt már elfér, de mellé más nem. Így pedig nem kapunk optimumot (a tárgy értéke kevesebb mint a nélküle elérhető optimális érték).

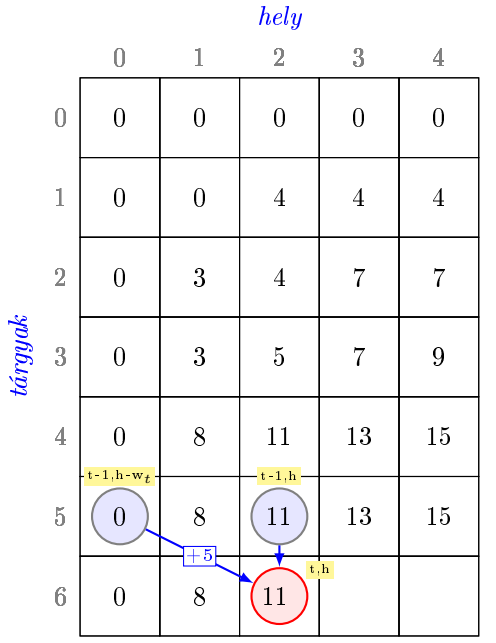
2.1. ábra. A 0-1 hátizsák probléma megoldása dinamikus programozással. (folytatás)



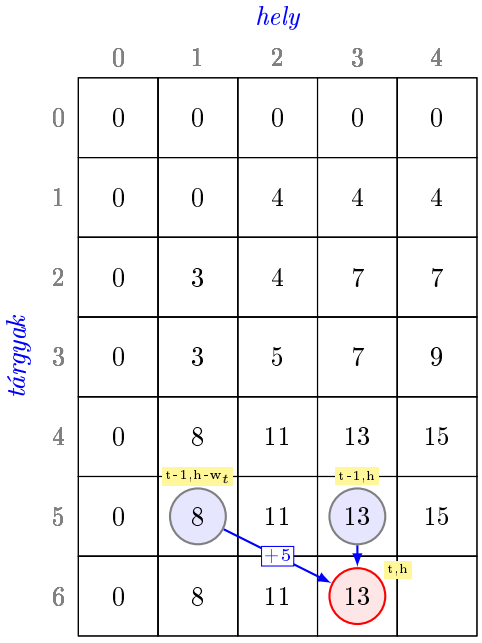
(u) $t = 5, h = 4, w_t = 3, p_t = 7$
Ugyanazt az eredményt érjük el vele, mint nélküle. Az optimum így egyszerűen megadható.



(v) $t = 6, h = 1, w_t = 2, p_t = 5$
A tárgy nem fér el a rendelkezésre álló 1 helyen, így nincs más választásunk, másoljuk a felette lévő értéket.

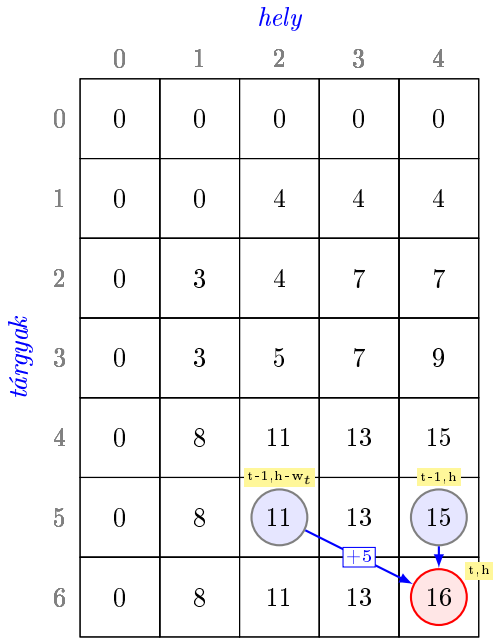


(w) $t = 6, h = 2, w_t = 2, p_t = 5$
A tárgy itt már elfér, de nem kapunk vele jobb megoldást, mint amikor nem vettük figyelembe.

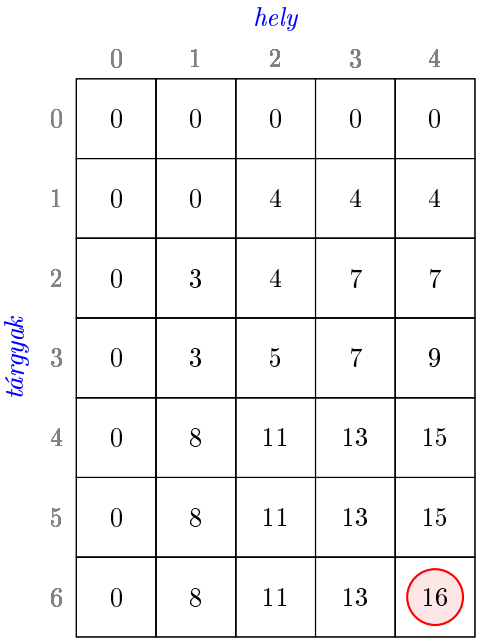


(x) $t = 6, h = 3, w_t = 2, p_t = 5$
A helyzet hasonló, mint az előző lépésben.

2.1. ábra. A 0-1 hátizsák probléma megoldása dinamikus programozással. (folytatás)



(y) $t = 6, h = 4, w_t = 2, p_t = 5$
A legutolsó cella vizsgálatakor még találunk egy még jobb eredményt. Ha berakjuk a zsákba a 6. elemet is, akkor egy kicsivel jobb eredményhez jutunk.



(z)
Ezzel megkaptuk a végeredményt. A táblázat utolsó cellája mutatja a optimális pakolás értékét.

2.1. ábra. A 0-1 hátizsák probléma megoldása dinamikus programozással. (folytatás)

elérhető ez a 11-es összérték. Tehát az 5. elem nem része az optimális pakolásnak, itt hagyjuk az OPT tömb értékét hamisnak.

Könnyen belátható, hogy minden esetben a fenti két lépés egyikét kell alkalmaznunk, és így sorban kideríthetjük, hogy a t. elem része-e egy megadott optimális megoldásnak, vagy sem. Mindezt addig folytatjuk, amíg a t vagy a h értéke 0 lesz. Ebben az esetben leállítjuk a ciklust, és az OPT aktuális értéke egy optimális pakolást fog mutatni.

A fentieket általánosítva adhatjuk a 2.10. algoritmust: első körben inicializáljuk az segédváltozókat (2–4. sorok), ehhez az OPT tömböt feltöltjük hamis értékekkel, majd a $t = N, h = W_{max}$ pontból kiindulva az alábbi ciklust követjük:

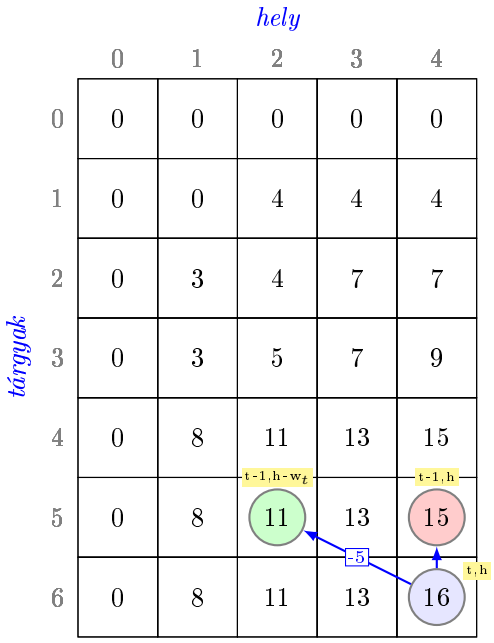
- Ha $t = 0$ vagy $h = 0$, akkor az algoritmust leállítjuk (5. sor).
- Ha $F[t, h] = F[t-1, h]$, akkor a t. tárgy nem része az optimális pakolásnak. Tehát az OPT tömböt nem módosítjuk, a t értékét eggyel csökkentjük (10. sor).
- Ha $F[t, h] \neq F[t-1, h]$ akkor a t. tárgy része az optimális pakolásnak. Tehát az OPT tömb t. elemét igazra állítjuk (7. sor). Csökkentjük eggyel a t értékét (7. sor), és w_t-vel a h értékét (8. sor).

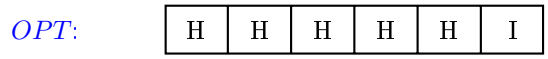
A függvény visszatérési értéke az OPT tömb, ami egy optimális pakolás adatait tartalmazza.

A módszer értékelése

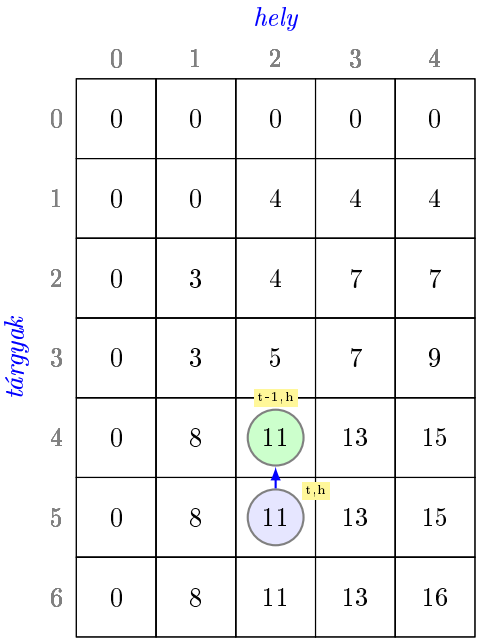
A módszer tehát pontosan ugyanazt az eredményt adja, mint az előzőek, ami nem meglepő, hiszen valójában pontosan ugyanazokat a részeredményeket számolja ki, mint az „oszd meg és uralkodj” illetve a „feljegyzéses módszer”. A lényeges különbség csak az, hogy ebben az esetben a feldolgozás nem felülről lefelé, hanem alulról felfelé halad. Tehát először a kisebb részproblémákat oldjuk meg, ezeket valamilyen formában eltároljuk, majd ezek alapján építjük fel a komplexebb (rész)problémák megoldását.

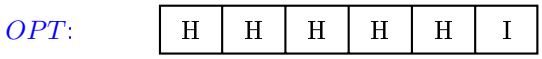
A módszer előnyei



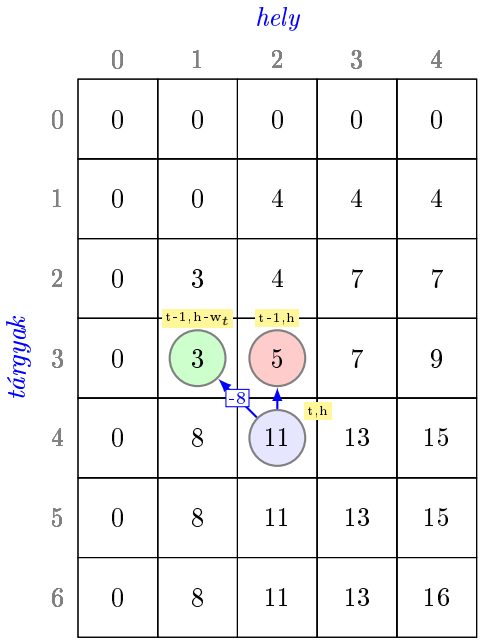


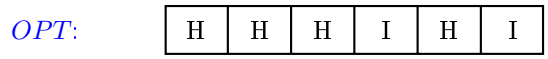
(a) $t = 6, h = 4, w_t = 2, p_t = 5$
Ezt az optimális eredményt csak a 6. tárgy zsákba helyezésével érhettük el (mivel $F[t,h] \neq F[t-1,h]$). Ezért eltároljuk, hogy a 6. elem a zsákban van, és átlépünk az $F[t-1, h-w_t]$ cellába.



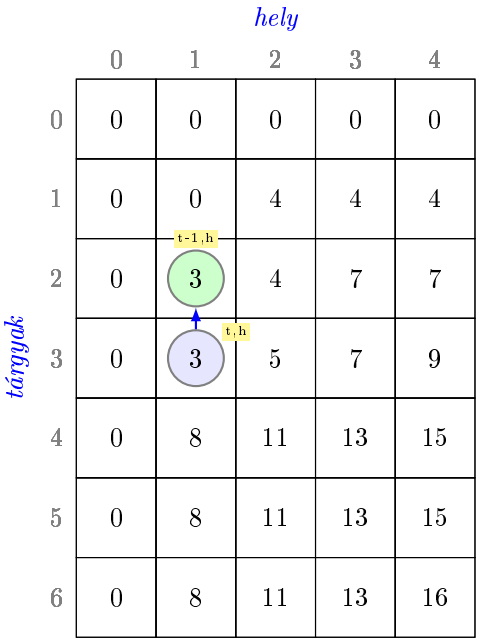


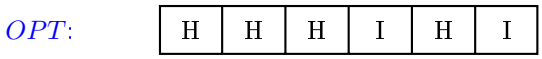
(b) $t = 5, h = 2, w_t = 3, p_t = 7$
Mivel $F[t,h] = F[t-1,h]$), ezért az 5. tárgy nélkül kaptuk meg az optimális megoldás értékét. Ezért ezt az utat követjük. Az OPT[5] emiatt hamis marad.





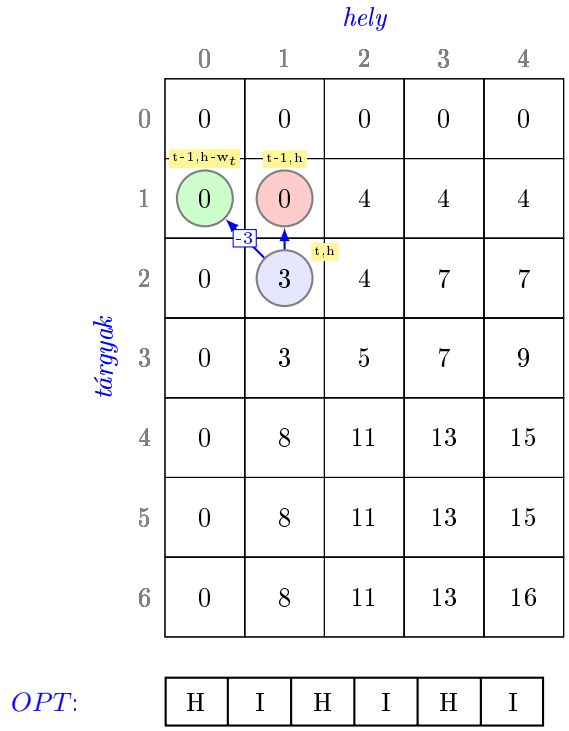
(c) $t = 4, h = 2, w_t = 1, p_t = 8$
Mivel $F(t,h) \neq F(t-1,h)$, ezért a 4. tárgy is benne van a zsákban.



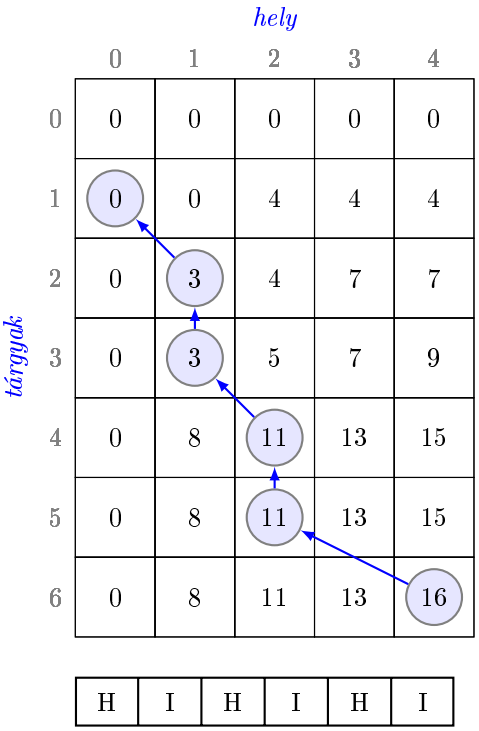


(d) $t = 3, h = 1, w_t = 1, p_t = 2$
Mivel $F[t,h] = F[t-1,h]$, ezért a 3. tárgy nélkül kaptuk meg az optimális megoldás értékét. Ezért ezt az utat követjük. Az OPT[3] emiatt hamis marad.

2.2. ábra. A 0-1 hátizsák probléma megoldásának előállítása dinamikus programozással.



(a) $t = 2, h = 1, w_t = 1, p_t = 3$
Most $F[t,h] \neq F[t-1,h]$, ezért a 2. tárgyat is berakjuk a hátizsákba. Átlépünk a következő helyre.



(b)
Mivel h értéke 0 lett, így nem kell tovább vizsgálnunk a táblázatot. Az OPT tömb tartalmazza a tárgyak adatait. Az ábra a teljes bejárt útvonalat mutatja.

- A „nyers erő” és a rekurzív módszerekkel összehasonlítva a futásidő általában jóval kedvezőbb. Jó tervezés esetén nincs szükség egy részprobléma többszöri megoldására.
- A megoldás nem tartalmaz rekurziót. Ez növeli a teljesítményt, illetve az algoritmust is egyszerűbbé teszi.
- Bár elsőre talán mágikusnak tűnik, hogy miként működnek a dinamikus programozás alapú megoldások, valójában nagyon egyszerűek. Ha valaki megérti a módszer alapelvét, akkor szinte tetszőleges (a bevezetőben említett feltételeknek megfelelő) rekurzív algoritmust át tud alakítani erre a formára.

A módszer hátrányai

- A „feljegyzéses módszerhez” viszonyítva bizonyos feladatoknál magasabb lehet a lépésszáma. Itt ugyanis az alulról felfelé való építkezés miatt nem mindig tudjuk finomhangolni, hogy mely részproblémák kiszámítására van szükség és melyekre nincs.
- A többi megoldáshoz viszonyítva nagyobb tárterületet igényel. Egy nagyobb feladat megoldása során a részmegoldások száma meglehetősen nagy lehet, ezeket valahol mind el kell tárolnunk. Leegyszerűsítve azt is mondhatjuk, hogy a dinamikus programozás során általában tárhelyet adunk hatékonyabb futásidőért cserébe.

2.6. Mohó algoritmusok

Egy bevezető példa

A „mohó algoritmusok” (greedy algorithm) bemutatására nézzünk egy nevezetes problémát, ami egy megadott összeg felbontása a lehető legkevesebb adott címletű bankjegy segítségével. Tehát előre meghatározott címletű pénzjegyeink vannak (pl. 1,5,10,20,50,100,200,500), és a feladat annak megállapítása, hogy melyek felhasználásával tudunk a lehető legkevesebb pénzjegyből kifizetni egy megadott összeget.

A megoldás hétköznapi ismereteink alapján már ismert: a kifizetendő összegnél nem nagyobb címletek közül a legnagyobból vegyünk el annyit, amennyi szükséges, majd a maradék összeget fizessük a nála kisebb pénzjegyekkel (ezt megtehetjük egy rekurzív hívással, vagy akár egy ciklussal is).

2.11. Algoritmus Pénzváltás legkevesebb címlettel

Bemenet: a - **egész**, N - **egész**, C - **egész tömb**
Kimenet: OPT - **egész tömb**a szükséges címletek darabszáma

```
1: függvény PénzVáltás(a : egész, N : egész, C : egész tömb)
2:     ciklus i ← N-től 1-ig
3:         OPT[i] ← ⌊a/C_i⌋
4:         a ← a mod C_i
5:     ciklus vége
6:     vissza OPT
7: függvény vége
```

Felhasznált változók és függvények

- a - Nemnegatív egész, a kifizetendő összeg.
- N - A lehetséges címletek száma.
- C - A lehetséges címleteket tartalmazó növekvő sorrendben rendezett tömb.
- OPT - Az eljárás eredménye, az egyes címletek szükséges darabszámát tartalmazza.

Látható, hogy a fenti címletek használatával a program jól működik, pl. 345 Ft felbontására a következő eredményt adja: $[0, 1, 0, 2, 0, 1, 1, 0]$.

Azonban mielőtt elbíznánk magunkat, nézzük egy másik bemenetet, ahol 10 Ft-ot kell felbontani, és a címletek $[1, 5, 7]$. Az algoritmus a 7 Ft-os címlethez 1-et rendel, majd a maradék 3 Ft-ot próbálja tovább osztani, azonban ez már eleve kisebb, mint a hátralévő 5Ft-os címlet, így három darab 1Ft-ost fog keresni, így az eredmény $[3, 0, 1]$ lesz. Ez nyilvánvalóan rossz, hiszen azonnal tudunk mondani egy kevesebb címletet igénylő megoldást: $[0, 2, 0]$ az optimális megoldás.

A fenti példa nagyon jól jellemzi a mohó algoritmusokat:

- Ezek általában nagyon egyszerű algoritmusok, amelyek mindig az aktuálisan legjobbnak tűnő irányba haladnak.
- Ezek gyakran nem találják meg a tényleges optimumot, csak egy közelítő értéket. Ez persze nem a véletlenen múlik, egy megfelelő szabályrendszerrel meg lehet határozni, hogy mely esetekben fog tökéletesen működni az algoritmusunk (pl. az előző feladatnál akkor, ha $2 * C_i \leq C_{i+1}$).

> **Megjegyzés**
>
> A második pontban található szabályrendszer kidolgozása, majd annak bizonyítása, hogy azon belül tényleg hibátlan a mohó algoritmus már koránt sem ilyen egyszerű. De ismert nevezetes algoritmusoknál ez már szerencsére nem ránk vár.

A módszer bemutatása

Azokat az algoritmusokat nevezzük mohónak, amelyek a feladat megoldása során felmerülő részproblémák megoldásakor mindig az aktuálisan legjobbnak tűnő részeredményt választják. Legjobbnak tűnő alatt azt értjük, hogy valamilyen egyszerű, gyorsan kiszámítható kritérium alapján kell dönteni.

Ez természetesen nagyon kedvező futásidőhöz vezet (nincs szétbontás, rekurzió, visszalépés, stb.), azonban a problémák csak szűk körénél adja vissza a teljes probléma optimális megoldását. Érdemes azonban megjegyezni, hogy azoknál a problémáknál, ahol nincs optimális megoldást visszaadó mohó algoritmus, gyakran létezik valamilyen közel optimális megoldást adó változat. Amennyiben ez is megfelelő, akkor tudjuk használni ezt a stratégiát.

A mohó stratégia általában az alábbi két tulajdonságot igényli:

- Optimális részproblémák: a dinamikus programozáshoz hasonlóan ez azt jelenti, hogy az optimális megoldás felépíthető részproblémák optimális megoldásából.
- Mohó választás: a globális optimális megoldás elérhető lokális optimumok választásán keresztül.

A mohó stratégia sok szempontból hasonlít a dinamikus programozásra. A leglátványosabb különbség az, hogy míg a dinamikus programozás alulról felfelé haladva építi fel a teljes megoldást, addig a mohó stratégia felülről lefelé haladva ad optimális megoldást.

A 0-1 hátizsák probléma megoldása

A megvalósítás meglehetősen hasonló lesz a pénzváltási feladathoz. A cél itt is az lenne, hogy a rendelkezésre álló tárgyakat valamilyen formában sorbarakjuk, majd ebben a sorrendben próbáljuk őket berakni a hátizsákba. Amely elemek befférnek, azokat berakjuk, ami nem, az pedig kimarad.

Megjegyzés

Előrebocsátjuk, hogy a 0-1 hátizsák probléma maradéktalan megoldására nem létezik mohó algoritmus, ami visszaadná az optimális pakolás értékét. Olyan azonban van, ami egy közel optimális megoldást ad, ezért a továbbiakban ezt vizsgáljuk (ezzel megtartva azt az előnyt, hogy minden módszerrel ugyanazt a feladatot próbáljuk megoldani).

A megoldás kulcsa persze az, hogy milyen módon határozzuk meg ezt a sorrendet. Néhány ötlet:

- Rendezhetjük a tárgyakat érték szerint csökkenő sorrendben. Így tehát először a legnagyobb értékű tárgy kerül a zsákba (már persze ha belefér), és ha van még hely, akkor sorba próbálgatjuk az egyre kisebbeket. Mivel a nagy értékű tárgyakat rakjuk a zsákba, így azt várhatjuk, hogy az összérték is magas lesz.
- Rendezhetjük a tárgyakat méret szerint növekvő sorrendben. Tehát először a legkisebb tárgyat próbáljuk berakni, majd a második legkisebbet, stb. Ettől a megoldástól azt várjuk, hogy sok tárgy fog beleférni a zsákba, hiszen a kisebbekkel kezdünk, és ennek hatására ezek összértéke is magas lesz.
- Rendezhetjük a tárgyakat az „érték/súly” arányuk alapján is csökkenő sorrendbe. Ez a fenti két megoldás ötvözeteként is felfogható, hiszen a nagyobb értékű de mégis kis helyet igénylő tárgyakat választjuk először. Így megintcsak azt várhatjuk, hogy az eredmény lehetőleg magas lesz.

A fentiek közül bármelyik módszert is választjuk, a hátizsákba pakoló algoritmus ugyanaz lesz (2.12. algoritmus). A gyakorlatban egyik megoldás se ad garantáltan optimális eredményt, mindegyik valamelyik szempont szerint ad egy közel optimális kimenetet.

Az algoritmus első sora a tárgyak rendezését végzi el a fent megadott valamelyik stratégia szerint. Amennyiben vannak részletesebb információink a tárgyakról, akkor a rendezés lehet persze ezeknél is összetettebb. Minél jobb sorrendet tudunk felállítani, annál jobban fogja közelíteni a kimenet az optimálisat.

Az algoritmus alapelve tehát hasonló a pénzváltási feladatéhoz. Mivel itt nem biztos, hogy minden tárgyat meg fogunk vizsgálni, ezért előre kitöltjük az OPT vektor mezőit hamis értékekkel (3. sor), majd az t változó beállítása után elindul a belső ciklus.

A ciklusfeltétel (5. sor) azt vizsgálja, hogy

- Nincs még teli a zsák. Hiszen csak ebben az esetben van értelme további vizsgálatoknak.
- Vannak még meg nem vizsgált tárgyak. Ellenkező esetben ki lehet lépni a ciklusból.

2.12. Algoritmus A 0-1 hátizsákprobléma közelítő megoldása „mohó algoritmussal”

Kimenet: P_{opt} - **egész** (*egy közel optimális pakolás összértéke*)

```
1:  függvény HátizsákMohó
2:      TárgyakRendezése(w, p)
3:      OPT ← [hamis, hamis, ..., hamis]
4:      t ← 1
5:      ciklus amíg (ÖsszSúly(OPT) < W_max) ∧ (t ≤ N)
6:          ha ÖsszSúly(OPT) + w_t ≤ W_max akkor
7:              OPT[t] ← igaz
8:          elágazás vége
9:          t ← t + 1
10:     ciklus vége
11:     vissza ÖsszÉrték(OPT)
12: függvény vége
```

Felhasznált változók és függvények

- N - A hátizsákba pakolható tárgyak száma.
- w_i - A hátizsákba pakolható i. tárgy súlya ($1 \leq i \leq N$).
- p_i - A hátizsákba pakolható i. tárgy hasznossága ($1 \leq i \leq N$).
- W_{max} - A hátizsák mérete (a belepakolható tárgyak maximális összsúlya).
- P_{opt} - Egy közel optimális pakolás összértéke.
- TárgyakRendezése(w, p) - A megadott három módszer közül valamelyikkel rendezi a hátizsákba pakolandó tárgyakat. Ennek megfelelően módosítja a paraméterként átadott súly és érték listákat.

Amennyiben belépünk a ciklusba, akkor megvizsgáljuk a t. tárgyat. A hasznossággal itt nem is foglalkozunk, hiszen már a rendezéskor igyekeztünk az elemeket olyan sorrendbe rakni, hogy a minél előbb lévő elemeket próbáljuk berakni a zsákba. Itt már csak azt nézzük, hogy a t. tárgy a már beválasztottak mellett befér-e a zsákba (6. sor). Ha igen, akkor őt is berakjuk (7. sor), ha nem, akkor kihagyjuk.

Miután a ciklus végetért, az OPT változó tartalmazza az általunk talált legjobb pakolás adatait. Ennek összértéke lesz a függvény visszatérési értéke (7. sor).

Érdemes néhány teszt bemenettel kipróbálni az algoritmust, és tudatosan keresni olyan értékeket, amelyekre megtalálja az optimumot, és olyat is, amire csak egy közelítő értéket tud adni.

> Megjegyzés
>
> A 0-1 hátizsák problémát ugyan nem tudta tökéletes megoldani az algoritmus, de egy kis módosítással a töredékes hátizsák problémára maradéktalanul alkalmazható. Ez utóbbi egy jó példa arra, amikor a mohó algoritmus hatékonyan és hibátlanul megoldja a feladatot.

A módszer értékelése

A mohó stratégia tehát azon alapult, hogy minden lépésben csak az ott ismert információk alapján próbálunk döntést hozni a továbblépés irányáról. Ez kétségtelenül gyors, azonban miként az láttuk, nem vezet mindig optimális megoldáshoz.

A módszer előnyei

- Egyértelműen az erőforrásigényt jelölhetjük meg első sorban. Mind futásidő, mind pedig tárhely szempontjából meglehetősen hatékony megoldásokat tud adni a technika.
- Ha nem is adja vissza az optimális megoldást, gyakran ad egy jól használható közelítést. Ez kiindulási alapja lehet egyéb kiegészítő módszereknek.

A módszer hátrányai

- Csak a feladatok egy nagyon szűk körén alkalmazható.

Számos példát lehet hozni arra, amikor a mohó algoritmus bizonyíthatóan optimális megoldást ad vissza. Ezek közül néhánnyal még később foglalkozunk, tipikusan a különféle gráf algoritmusok vizsgálata során (7.4. fejezet, 7.5.1. fejezet, 7.5.2. fejezet).

2.7. Visszalépéses keresés

A módszer bemutatása

A visszalépéses keresésnek egy teljes fejezetet szenteltünk (1. fejezet), így annak részletes bemutatása ott megtalálható. A módszer alap gondolata az volt, hogy magát a teljes feladatot szétbonja kisebb részfeladatokra, majd ezekre próbál megoldást keresni. Alapelve, hogy egyszerre csak egy részfeladatot próbál megoldani. Elindul az elsőnél, ha olyan részmegoldást talál, ami része lehet egy teljes megoldásnak, akkor továbblép a következőre, és így halad tovább. Ha nem talál ilyen részmegoldást, akkor pedig egy szinttel visszalép, és az előző részfeladatnak próbál egy másik részmegoldást keresni. Az eljárás két esetben érhet véget: a) ha az összes részfeladatra talált részmegoldást, és ezzel megoldotta a teljes feladatot b) ha a visszalépések során az első feladatra már nem talál több alternatív megoldást, ilyenkor nincs megoldása a feladatnak.

Láttuk, hogy a visszalépéses keresés jól használható optimalizációra is (1.2.4. fejezet). Az előzővel ellentétben ilyenkor nem áll le egy teljes megoldás során, hanem csak ellenőrzi, hogy ez a megoldás jobb-e, mint az eddig talált legjobb (ha még nincs ilyen, akkor azt igennek tekinti), és ha igen, akkor mostantól az aktuálisat tekinti optimális megoldásnak. De ezt követően fut tovább a keresés.

A 0-1 hátizsák probléma megoldása

A fentieknek megfelelően a hátizsák problémát is egyszerűen megoldhatjuk a visszalépéses kereséssel. Részfeladatnak itt azt tekinthetjük, hogy az egyes tárgyak bekerülnek-e a zsákba vagy sem. N darab tárgy esetén tehát lesz N darab részfeladat, mindegyik lehetséges részmegoldásai az igaz vagy hamis. A 2.13. algoritmus bemutatja a feladat megoldását visszalépéses kereséssel. A megoldó algoritmus némileg eltér az 1. fejezetben bemutatott általános formától, de itt a cél nem a módszer általánosságának bemutatása, hanem az előző problémamegoldó technikákkal való összehasonlítás.

Egyszerűsítésnek tekinthető, hogy nincs külön lehetséges részmegoldások mátrixunk, hanem minden szinten két lehetőséget vizsgál meg az algoritmus (5. sor). Amennyiben az i értéke 0, akkor úgy tekinti, hogy a *szint*-edik tárgyat berakjuk a hátizsákba, amennyiben az i értéke 1, akkor pedig nem. A következő sorban ennek megfelelően tölti ki az E vektor megfelelő elemét.

Az F_k feltétel ellenőrzése jelen esetben a hátizsákba eddig bepakolandó elemek összsúlyának vizsgálatát jelenti. Amennyiben ez kevesebb, mint a rendelkezésre álló teljes kapacitás (W_{max}), akkor a részmegoldást jónak tekinti. Amennyiben nem, akkor pedig nem. Ez utóbbi esetben próbálkozik az i ciklus következő lehetséges értékével, illetve ha már mindkét változatot megnézte, akkor engedi visszalépni a rekurziót.

Amennyiben találtunk egy teljes megoldást (8. sor), akkor megvizsgáljuk, hogy ez jobb-e, mint az eddig talált legjobb (9. sor). Ha igen, akkor ezt követően ezt tekinti optimális megoldásnak.

Amennyiben még csak egy részmegoldást rögzítettünk, akkor a tanult visszalépéses keresés elvét követve meghívja önmagát (13. sor), és egy szinttel feljebb próbál megoldást találni.

Megjegyzés

Szintén egyszerűsítés, hogy nincs VAN változó az algoritmusban. Bár az eredeti visszalépéses keresésben a változónak több szerepe is volt, itt egyikre sincs szükség. Az algoritmust nem kell leállítanunk az első találat esetén, tehát emiatt nem kell. Azt se kell vizsgálnunk, hogy találtunk-e legalább egy elfogadható megoldást, hiszen tudjuk, hogy ilyen biztosan van (a teljesen üres hátizsák). Ennek köszönhetően az új eredmény vizsgálatakor (hogy az jobb-e, mint az eddig megtalált) sem kell külön foglalkoznunk azzal a kérdéssel, hogy mi van akkor, ha még nem is találtunk egy jó megoldást sem.

2.13. Algoritmus A 0-1 hátizsákprobléma megoldása „visszalépéses keresés” segítségével

Bemenet: $szint$ **- egész**, E **- logikai tömb**
Kimenet: $elfogadható$ **- logikai**

```
1: függvény F_k(szint : egész, E : logikai tömb)
2:     vissza ÖsszSúly(E) ≤ W_max
3: függvény vége
```

Bemenet: $szint$ **- egész**, E **- logikai tömb**, OPT **- logikai tömb**
Kimenet: E **- logikai tömb**, OPT **- logikai tömb**

```
4: eljárás Backtrack(szint : egész, címszerint E : logikai tömb, címszerint OPT : logikai tömb)
5:     ciklus i ← 0-től 1-ig
6:         E[szint] ← (i = 0)                    ▷ ha i=0 akkor legyen igaz, különben hamis
7:         ha F_k(szint, E) akkor
8:             ha szint = N akkor
9:                 ha ÖsszÉrték(E) > ÖsszÉrték(OPT) akkor
10:                    OPT ← E
11:                elágazás vége
12:            különben
13:                Backtrack(szint + 1, E, OPT)
14:            elágazás vége
15:        elágazás vége
16:    ciklus vége
17: eljárás vége                    ▷ visszalépéskor E[szint] értéke mindig hamis
```

Kimenet: P_{opt} **- egész** (*egy optimális pakolás összértéke*)

```
18: függvény HátizsákBT
19:     E ← [hamis, hamis, ..., hamis]
20:     OPT ← [hamis, hamis, ..., hamis]
21:     Backtrack(1, E, OPT)
22:     vissza ÖsszÉrték(OPT)
23: függvény vége
```

Felhasznált változók és függvények

- N - A hátizsákba pakolható tárgyak száma.
- w_i - A hátizsákba pakolható i. tárgy súlya ($1 \leq i \leq N$).
- p_i - A hátizsákba pakolható i. tárgy hasznossága ($1 \leq i \leq N$).
- W_{max} - A hátizsák mérete (a belepakolható tárgyak maximális összsúlya).
- P_{opt} - Egy optimális pakolás összértéke.
- E - Az aktuálisan vizsgált pakolás adatai (logikai tömb, ahol $E[i]$ azt mutatja, hogy az i. tárgy benne van-e a zsákban ($1 \leq i \leq N$)).
- OPT - Az eddig talált legjobb pakolás adatai (logikai tömb, ahol $OPT[i]$ azt mutatja, hogy az i. tárgy benne van-e a zsákban ($1 \leq i \leq N$)).

Megjegyzés

Ezt a pszeudokódnál is kiemeltük, hogy lényeges körülmény, hogy a belső i ciklus először az *igaz* és utána a *hamis* értéket vizsgálja. Ez azért fontos, mert így az eljárásból való kilépésnél az $E[szint]$ értékében mindig *hamis* fog maradni. Tehát egy előrelépkedés utáni visszalépkedés után az E tömb *szint*-edik utáni elemei mindig *hamis* értéket fognak tartalmazni. Ez azért fontos, mert különben az ÖsszÉrték függvény hívása hibás eredményt adna.

A módszer értékelése

A visszalépéses kereséssel részletesen foglalkozik az 1. fejezet, így itt csak a konkrét feladat megoldásával kapcsolat tapasztalatainkkal foglalkozunk.

A módszer előnyei

- Jól áttekinthető, általános használható módszer.
- Az „oszd meg és uralkodj” módszerhez hasonlóan a nyilvánvalóan felesleges részfeladatok vizsgálatát elkerüli.

A módszer hátrányai

- A rekurzív hívások miatt erőforrásigénye nagy lehet.
- Bizonyos feladatokra találhatunk hatékonyabb megoldásokat is.

2.8. Szétválasztás és korlátozás

A módszer bemutatása

A „szétválasztás és korlátozás” (branch and bound) technikája sok szempontból hasonlít az „oszd meg és uralkodj” és a „visszalépéses keresés” módszerére, akár azok továbbfejlesztésének is tekinthetjük. A „nyers erő” módszerhez képest az említett kettő előnye az volt, hogy bár próbálkozással haladnak a végső megoldás felé, a részfeladatok megoldásából is próbálják meghatározni, hogy melyik utakon érdemes tovább haladni és melyiken nem. Azonban mindig csak az előző választásaikat veszik figyelembe, és csak akkor fordulnak vissza, ha azok alapján már nem juthatunk elfogadható eredményhez (így optimálishoz sem).

A „szétválasztás és korlátozás” általában annyival kifinomultabb, hogy ez nem csak azt nézi, hogy megadott úton eljuthatunk-e egy elfogadható megoldáshoz, hanem azt is próbálja megbecsülni, hogy ez mennyire lehet optimális. Már az is elég ha egy megfelelően pontos felső becslést tud adni, hiszen már ez alapján is gyakran kerülhetünk olyan helyzetbe, hogy egy megadott úton lehetne ugyan tovább folytatni a keresést, de az ottani legjobb eredmény felső becslése azt mutatja, hogy nem érdemes, mert a már eddig talált legjobbnál úgyse foguk jobbat találni. Ezzel nagyon hatékonyan lehet tovább gyorsítani a kereső algoritmusokat.

Ahhoz, hogy a technikát jól használhassuk, az alábbi két függvényre van szükségünk:

- *Szétválasztási függvény*: ennek szerepe, hogy a meglévő (rész)feladatot további részfeladatokra bontsa (branch). Ez már ismerős lehet a többi megismert technikából („oszd meg és uralkodj”).
- *Korlátozó függvény*: ez egy korlátot ad egy részfeladat optimális megoldására. Ennek segítségével tudjuk eldönteni, hogy érdemes-e továbblépni egy úton, vagy akár vissza is fordulhatunk, hiszen nem fogunk a már meglévőnél jobb megoldást találni. Ezzel tulajdonképpen levágjuk (bound) a keresési fának azokat az ágait, amit nem érdemes vizsgálni.

A fentiek alapján már könnyen elképzelhető a megoldó algoritmus. Az eredeti problémát a szétválasztási függvénnyel szétbontjuk kisebb részproblémákra. A korlátozó függvény segítségével megvizsgáljuk, hogy az egyes részproblémák esetében mi az optimális megoldás értékének felső korlátja (persze nem biztos, hogy tényleg lesz is ilyen optimális eredményünk, csak egy felső becslést adunk). Amennyiben ez alapján érdemesnek tűnik, akkor rekurzív módon megvizsgáljuk az egyes részproblémákat (azokat is szétbontjuk, stb.).

A 0-1 hátizsák probléma megoldása

Vizsgáljuk meg a megadott módszert a már jól ismert feladat megoldásával. Elsőként nézzük meg a szétválasztás és a korlátozás megválasztását:

- Szétválasztás: a teljes feladatot a többi rekurzív megoldásnál megszokott módon bontjuk szét kisebb részfeladatokra. N darab tárgy elhelyezését visszavezetjük két kisebb részfeladat megoldására: feltételezzük, hogy az utolsó elem benne van a zsákban és ez alapján számolunk optimumot a maradékra, illetve feltételezzük, hogy az utolsó nincs benne a zsákban és ez alapján számolunk optimumot a maradékra.
- Korlátozás: a korlátozás során a még vizsgálatra váró részprobléma optimális pakolási értékére adunk egy felső becslést. Ezt mutatja be az F_b függvény. Végignézzük az összes még hátralévő tárgyat, amennyiben a tárgy önmagában befér a zsákban maradt szabad helyre, akkor annak értékét hozzáadjuk egy segédváltozóhoz. A segédváltozó értéke tehát azt mutatja, hogy milyen optimális pakolási értéket érhetünk el, ha az összes önmagában beférő tárgyat bele fogjuk tudni rakni a zsákba. Ez nyilvánvalóan egy felső becslés, hiszen valószínűleg nem fog minden beleférni, de számunkra most csak a fontos, hogy ennél jobb eredményt biztosan nem kaphatunk.

A pszeudokódnál (2.14. algoritmus) szándékoltan megtartottuk az előző visszalépéses keresésnél már megismert alapvető felépítést. Az algoritmus működése szinte teljesen ugyanaz, egy pontban történt változtatás, a 22. sorban. Itt látható, hogy a következő rekurziós hívás előtt ellenőrizzük, hogy az arra az ágra vonatkozó felső becslést figyelembe véve kaphatunk-e jobb eredményt az eddig elérnél. Ha igen, akkor a függvény meghívja önmagát, ha nem, akkor pedig nem történik rekurzív hívás.

2.14. Algoritmus A 0-1 hátizsákprobléma megoldása „szétválasztás és korlátozás” technikával

Bemenet: *szint* - **egész**, E - **logikai tömb**
Kimenet: $elfogadható$ - **logikai**
1: **függvény** F_k(*szint* : **egész**, E : **logikai tömb**)
2: **vissza** ÖSSZSÚLY(E) $\leq W_{max}$
3: **függvény vége**

Bemenet: *szint* - **egész**, E - **logikai tömb**
Kimenet: pfk - **egész** (*felső becslés a várható optimumra*)
4: **függvény** F_b(*szint* : **egész**, E : **logikai tömb**)
5: $pfk \leftarrow 0$
6: **ciklus** $i \leftarrow szint + 1$-től N-ig
7: **ha** ÖSSZSÚLY(E) $+ w_i \leq W_{max}$ **akkor**
8: $pfk \leftarrow pfk + p_i$
9: **elágazás vége**
10: **ciklus vége**
11: **vissza** pfk
12: **függvény vége**

Bemenet: *szint* - **egész**, E - **logikai tömb**, OPT - **logikai tömb**
Kimenet: E - **logikai tömb**, OPT - **logikai tömb**
13: **eljárás** BACKTRACK(*szint* : **egész**, **címszerint** E : **logikai tömb**, **címszerint** OPT : **logikai tömb**)
14: **ciklus** i ← 0-től 1-ig
15: $E[szint] \leftarrow (i = 0)$ ▷ ha i=0 akkor legyen igaz, különben hamis
16: **ha** $F_k(szint, E)$ **akkor**
17: **ha** $(szint = N)$ **akkor**
18: **ha** ÖSSZÉRTÉK(E) > ÖSSZÉRTÉK(OPT) **akkor**
19: $OPT \leftarrow E$
20: **elágazás vége**
21: **különben**
22: **ha** ÖSSZÉRTÉK(E) $+ F_b(szint, E)$ > ÖSSZÉRTÉK(OPT) **akkor**
23: BACKTRACK($szint + 1$, E, OPT)
24: **elágazás vége**
25: **elágazás vége**
26: **elágazás vége**
27: **ciklus vége**
28: **eljárás vége** ▷ visszalépéskor E[szint] értéke mindig hamis

Kimenet: P_{opt} - **egész** (*egy optimális pakolás összértéke*)
29: **függvény** HÁTIZSÁKBNB
30: $E \leftarrow [\textbf{\textit{hamis}}, \textbf{\textit{hamis}}, ..., \textbf{\textit{hamis}}]$
31: $OPT \leftarrow [\textbf{\textit{hamis}}, \textbf{\textit{hamis}}, ..., \textbf{\textit{hamis}}]$
32: BACKTRACK(1, E, OPT)
33: **vissza** ÖSSZÉRTÉK(OPT)
34: **függvény vége**

Felhasznált változók és függvények

- N - A hátizsákba pakolható tárgyak száma.
- w_i - A hátizsákba pakolható i. tárgy súlya ($1 \leq i \leq N$).
- p_i - A hátizsákba pakolható i. tárgy hasznossága ($1 \leq i \leq N$).
- W_{max} - A hátizsák mérete (a belepakolható tárgyak maximális összsúlya).
- P_{opt} - Egy optimális pakolás összértéke.
- E - Az aktuálisan vizsgált pakolás adatai (logikai tömb, ahol $E[i]$ azt mutatja, hogy az i. tárgy benne van-e a zsákban ($1 \leq i \leq N$)).
- OPT - Az eddig talált legjobb pakolás adatai (logikai tömb, ahol $OPT[i]$ azt mutatja, hogy az i. tárgy benne van-e a zsákban ($1 \leq i \leq N$)).

A felső becslést az F_b függvény végzi, aminek két paramétere van (*szint* és E). Ezek alapján a visszatérési értéke egy felső becslés arra vonatkozóan, hogy az E által aktuálisan tárolt (tehát 1-től *szint*-ig már ismert) részeredményeket figyelembe véve, a hátralévő (tehát *szint* utáni) tárgyak felhasználásával milyen optimális pakolást lehet elérni.

A módszer értékelése

Általában elmondható, hogy az itt megismert technika túlmutat az eddigi hasonló módszerek lehetőségein. Mindegyik hasonló módszer többé-kevésbé próbálkozással kereste meg az optimális eredményt, közülük néhány volt annyira hatékony, hogy nem folytatott olyan útvonalakat, ahol eleve nem fog elfogadható eredményt (érvényes pakolást) találni. A most megismert módszer pedig ezt annyiból továbbfejleszti, hogy próbálja kiszűrni azokat az ágakat is, ahol ugyan lehet elfogadható eredmény (érvényes pakolás), de az várhatóan nem lesz jobb, mint az eddig talált legjobb (optimális pakolás).

A módszer előnyei

- A fentieknek megfelelően egy kellőképpen finomhangolt algoritmus nagyon sok felesleges lépést el tud kerülni.
- Nem csak a már rögzített részeredményeken alapul a továbblépés, hanem a még nem feldolgozott részproblémát is figyelembe veszi.

A módszer hátrányai

- Mindezek az előnyök csak akkor használhatók ki, ha kellőképpen jó szétválasztó és korlátozó függvényeket írunk. Ezek kidolgozása gyakran meglehetősen bonyolult lehet.

2.9. A módszerek összehasonlítása

Láthattuk, hogy az egyes módszerek sok alapelvben osztoznak egymással, így a működésük is hasonló, nem is mindig lehet egyértelmű határvonalat húzni közöttük. Egy konkrét feladat megoldásakor ezeket az alapelveket célszerű figyelembe venni, és ezek alapján akár egy saját, a fentieken alapuló módszert kidolgozni.

Nem lehetséges az egyes módszerek közül egyet vagy többet kiemelni mint legjobbakat, sem pedig sorrendet készíteni, hogy melyik jobb a másiknál. Az egyes módszerek közötti választásnak ugyanis mindig a megadott feladathoz kell alkalmazkodnia, ebben próbál segíteni az egyes módszereknél megadott rövid értékelés.

Érdemes lehet az összes fenti módszert megvalósítani, és a bemeneti adatokkal kicsit „játszani". Még azonos probléma mellett is gyakran elő lehet állítani olyan bemenő adatsorokat, amelyet az egyik vagy éppen a másik módszer fog a leghatékonyabban megoldani.

Néhány általános gondolat azért megállapítható:

- A „nyers erő" módszere fogja általában a legnagyobb futásidőt igényelni. Használata főként akkor javasolt, amikor egyszerűen nincs jobb ötletünk (pl. jelszó feltörése, ahol semmilyen további támpontunk nincs a megoldásra vonatkozóan).

- Az „oszd meg és uralkodj" módszer általában nagyon könnyen értelmezhető és implementálható. Előnye akkor jelenik csak meg, ha a részfeladatok generálása során már ki tudjuk küszöbölni azokat, amelyeket felesleges vizsgálni. Ha ilyen lehetőség nincs, akkor ugyanúgy végig kell néznie az összes megoldást, csak a rekurzió miatt még lassabb is. Azt is láttuk, hogy ha sok átfedő részfeladatunk van, akkor célszerű valamelyik javítással próbálkozni.

- A „feljegyzéses" módszer nagyszerűen kiküszöböli a sok átfedő részfeladat által okozott hátrányokat. Túl nagy problématér esetén a tárigény problémás lehet (hiszen el kell tárolnunk a sok részfeladathoz tartozó optimális részmegoldást), de ettől függetlenül érdemes vele foglalkozni. Egyszerűsége ellenére meglepően hatékony futásidőt tud adni.

- A „dinamikus programozás" sok szempontból kilóg a többi közül. Érdekessége, hogy pl. a hátizsák probléma esetén lépésszáma és tárhelyigénye csak a tárgyak számától és a rendelkezésre álló helytől függ, de a súlyoktól és értékektől már nem (mint a többinél). Ez egyrészt praktikus, de mindig fontos figyelembe venni, hogy ezekre az adatokra nagyon érzékeny. Pl. ha csak 2 tárgyat akarunk berakni egy 2000 méretű zsákba, akkor is ki fogja tölteni a teljes táblázatot. Az alulról felfelé építkezés hátránya, hogy itt nem tudunk a többi módszernél jól használható vágásokkal élni.

- A „mohó algoritmusokkal" kapcsolatban egyszerű a dolgunk. Általában igaz, hogy ha van valamire mohó stratégiát alkalmazó módszer, akkor célszerű azt használni. A hangsúly sajnos itt a „ha van"-on van, ugyanis a legtöbb probléma nem oldható meg ilyen algoritmusokkal. Pl. a 0-1 hátizsákproblémára sincs jelenleg ismert mohó megoldás, csak olyan, ami közel optimális eredményt ad.

- A „visszalépéses keresés" is az „oszd meg és uralkodj" módszerhez hasonlóan egy alapvető módszer, ami gyakran további módosításokat és finomhangolást igényel a hatékony működéshez (érdemes átgondolni, hogy miben is különbözik egymástól a két módszer stratégiája). Ennek is az a problémája, hogy ha nem tudunk megadni korlátokat, akkor a teljes problémateret át kell vizsgálnia.

- A „szétválasztás és korlátozás" módszer a fentiekhez képest sokkal hatékonyabb tud lenni, de csak akkor, ha ki tudja használni az előre tekintésből adódó előnyét. Pl. ha a hátizsákban sok kis tárgy van, akkor a többi rekurzív módszer kénytelen egészen sok tárgyat vizsgálni mire rájön, hogy vissza kell fordulni. A korlátozás miatt ez a módszer azonban ilyenkor is gyorsabban vissza tud lépni, ha már előre látja hogy a hátralévő sok tárgyból úgyse lesz optimális megoldás.

3. fejezet

Hasító táblázat

3.1. Alternatívák elemzése

3.1.1. Tömbök felépítése

Az adatszerkezetek közül mindenki (legalábbis a strukturált programozást tanulók) a tömbbel ismerkednek meg először. A *tömb* egy meglehetősen egyszerű adatszerkezet, az egyes elemeket egy egész szám index segítségével érjük el, a tárolási mód pedig ehhez igazodik, az egyes elemek közvetlenül egymás után helyezkednek el a memóriában (*celláknak*, vagy *réseknek* fogjuk nevezni az elemek tárolására szolgáló memóriaterületeket). Ez számos előnnyel jár:

- Egyszerű: Az adatszerkezet megvalósítása, illetve használata is egyszerű.
- Ideális helyfoglalás: A tömb mindig pontosan annyi helyet foglal, amennyit a benne található elemek igényelnek. Tehát a helyfoglalás egyenlő az elemméret és az elemszám szorzatával.
- Gyors véletlen elérés: Nagyon gyors véletlen elérést biztosít, egy lépéssel ki tudjuk olvasni a tömb megadott indexű elemét (a későbbiekben látni fogjuk, hogy ez a többi adatszerkezetnél nem lesz ennyire nyilvánvaló).

> Megjegyzés
>
> Ez utóbbi előny a tárolás módjából adódik, hiszen tömbök esetében egy elem helyét a memóriában egyszerűen ki tudjuk számolni az alábbiak szerint:
> *elem_cím = kezdő_cím + index * elem_méret*
> Ahol
>
> - *elem_cím*: A keresendő elem memóriacíme.
> - *kezdő_cím*: A tömb elejének (első elemnének) a memóriacíme.
> - *index*: Az keresendő elem indexe (a fenti képlet alapján érthető, hogy miért terjedt el a 0-val kezdődő indexelés a C jellegű nyelveknél).
> - *elem_méret*: Egy elem mérete, ez nyilván az elem típusától függ (egész szám, karakter, stb.).

Viszont a tömb számos hátránnyal is bír:

- Fix méretű: A programozási nyelvekben a tömbök mérete általában nem változtatható (de ha igen, akkor is meglehetősen erőforrásigényes műveletek árán). Ez egyben azt is jelenti, hogy már a tömb létrehozásakor tudnunk kell, hogy maximálisan hány elemet szeretnénk elhelyezni benne, ami általában jelentős felülbecslést igényel.

- Nehézkes beszúrás/törlés: Tömbök végére általában könnyen tudunk új elemet felvenni, azonban egy új elem beszúrása a tömb közepébe már kimondottan erőforrásigényes lehet, hiszen az összes mögötte lévő elemet hátrébb kell tolni. Törlésnél ugyanez a probléma, csak fordított irányba.
- Lassú keresés: Rendezetlen tömbben csak a lineáris keresést használhatjuk, ami meglehetősen lassú. Rendezett tömbökben a logaritmikus keresés már jóval hatékonyabb, de pl. a hasító táblázatoknál látunk erre jobb megoldásokat is.

Megjegyzés

A későbbiekben megismerünk különböző adatszerkezeteket, de előre érdemes leszögezni, hogy egyik se lesz közülük olyan, amelyik minden szempontból túlszárnyalná a többit. Általában kompromisszumot kell kötnünk, hogy gyors keresést szeretnénk, vagy inkább gyors módosítási lehetőséget. Hasonlóan választanunk kell a kis tárigény, vagy pedig gyors működés között. A tömb csak egy lehetőség az adatszerkezetek közül, a maga előnyeivel és hátrányaival.

3.1.2. Közvetlen címzés

Az adatszerkezetekben való tárolás érdekében célszerű bevezetni a *kulcsok* fogalmát. Ez egy tetszőleges típusú érték, ami arra szolgál, hogy ez alapján azonosíthatjuk egyértelműen az adatelemünket, ez alapján tudjuk eltárolni, illetve a későbbiekben visszakeresni. A keresés miatt azt is megköveteljük, hogy ez a kulcs legyen egyedi, tehát két különböző elem ne rendelkezhessen ugyanazzal a kulccsal.

Ideális esetben a már egyébként is meglévő mezők közül valamelyiket közvetlenül használhatjuk kulcsként (pl. cégek neveit, stb.). Gyakran ez nem csak egy mezőt, hanem többet jelent, ilyenkor beszélünk *összetett kulcsokról* (pl. a gyakorlatban a név+anyja neve+születési hely és dátum egészen jól bevált a személyek azonosítására). Ez utóbbi esetben az összes értéket meg kell adnunk az egyértelmű azonosításhoz, de ez inkább csak implementációs problémát jelent, a későbbiekben ezzel nem foglalkozunk.

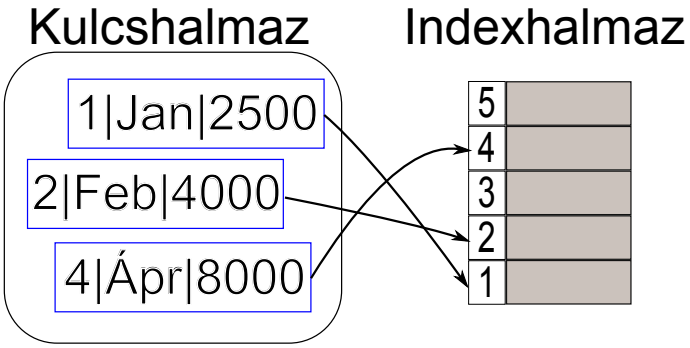
Néha azonban a meglévő mezők nem alkalmasak az egyértelmű megkülönböztetésre, ilyenkor bevezethetünk *technikai kulcsokat* (pl. személyi szám, memóriabeli cím, stb.), amelyek már nyilvánvalóan biztosítani tudják a kívánt egyediséget.

Az adatszerkezetekben a kulcsok alapján szoktuk elvégezni a kereséseket (pl. keressük a "998877AB" azonosítójú személy adatait), ami azonnal elvezet a tömbök egyik hátrányához. Tömböknél ugyanis a hozzáférés alapja az indexelés, ami az elemek memóriabeli elhelyezkedésén alapul, az pedig a kulcsoktól alapvetően független. A legtöbb amit tehetünk, hogy magát a tömböt a kulcsok alapján rendezzük, így a kulcs szerinti kereséskor (pl. adjuk vissza a megadott személyi számmal rendelkező személyt) a logaritmikus keresést használjuk, ami n darab elem esetében várhatóan $\mathcal{O}(log_2 n)$ lépésből megtalálja a keresett kulcsú elemet. Később látni fogjuk, hogy ez alapvetően nem rossz, de bizonyos feltételek megléte esetében ennél sokkal hatékonyabb megoldásokat is találhatunk: az elemeket közvetlenül, pontosan egy darab lépéssel is ki tudnánk keresni a tömbből. Ezt valósítja meg a *közvetlen címzés*. Ennek az alapja az, hogy a kulcsokat közvetlenül a tömb címzésére használjuk, aminek persze meglehetősen szigorú előfeltételei vannak:

- A kulcsoknak természetes számoknak kell lenniük (mivel tömböt csak ezekkel tudunk indexelni).
- A kulcsoknak egy megadott tartományon belül kell maradniuk (hiszen a tömb méretet előre meg kell adnunk). Tehát n darab elem esetében a kulcsoknak a $0..(n-1)$ tartományon belül kell lenniük.
- A kulcsoknak mindenképpen egyedinek kell lenniük.

Ha teljesülnek a fenti feltételek, akkor a különböző műveletek már nagyon egyszerűen implementálhatóak.

- Beszúrás: Egy elem elhelyezése a tömbben csak annyit igényel, hogy kiolvassuk az elem k kulcsát, majd elhelyezzük az elemet a tömb k indexű helyére.
- Keresés: A keresés hasonlóan egyszerű, ha a k kulcsú elemet keressük, akkor egyszerűen megvizsgáljuk a tömb k indexű elemét. Ha van ott valami, akkor azt kerestük, ha nincs, akkor pedig nincs ilyen kulcsú elemünk.



3.1. ábra. Példa a közvetlen címzésre. A címzés során a hónap sorszámát használjuk kulcsként, tehát beszúráskor a januári érték az 1. cellába kerül, stb. Kereséskor ehhez hasonlóan, ha pl. az áprilisi adatra van szükség, akkor azt a 4. cellából tudjuk kivenni.

- Törlés: Az előzőleg megismert kereséssel eldöntjük, hogy van-e k kulcsú elem, és ha igen, akkor töröltnek jelöljük a tömb k indexű elemét.

Ezzel sikerült is megvalósítanunk egy olyan adatszerkezetet, ami $\mathcal{O}(1)$ darab lépéssel tudja megvalósítani az alapműveleteket (sőt, valójában az $\mathcal{O}$ is szükségtelen, hiszen konkrétan egy lépést igényel csak). Ez valóban igaz, de sajnos ez is számos hátránnyal bír:

- A kulcsokra nagyon szigorú feltétel vonatkozik, ez a legtöbb gyakorlati esetben nem kivitelezhető (mert a kulcsok pl. szövegek).
- A használhatóságot nagyban befolyásolja a lehetséges kulcsok számának, illetve a ténylegesen eltárolni kívánt elemek számának aránya. Ha kulcsként pl. születési dátumot szeretnénk használni (az egyszerűség kedvéért év utolsó két számjegyétől kezdve, tehát pl. 781218,120228, 931211, stb.) akkor egy 991231 méretű tömböt kell létrehoznunk. Amennyiben az eltárolandó személyek száma csak néhány száz, akkor jól látható, hogy tárterület szempontjából nagyon pazarlóan működik az adatszerkezetünk (és a születési dátumot még nem is tekinthetjük egyedinek, a tökéletességre törekedve jobb lenne a 11 jegyű személyi szám).

A fenti hátrányoktól eltekintve a közvetlen címzés bizonyos esetekben nagyon jól használható, főleg olyankor, amikor a kulcshalmaz meglehetősen kicsi (pl. a havi fizetéseket akarjuk eltárolni közvetlen címzéssel, akkor elég egy 12 méretű tömböt létrehozni), vagy legalább jól körülhatárolt (pl. mi magunk adunk olyan technikai kulcsokat, hogy azok egy kisebb tartományon belül maradjanak). Közvetlen címzésre mutat egy egyszerű példát a 3.1. ábra.

A használhatóság mindig az egyedi feladatok esetében dönthető el, a tárolt adatok mennyisége és mérete, a kulcsok milyensége és a várható keresések számának függvényében.

3.2. Hasító táblázatok felépítése

A közvetlen címzés gondolatából kiindulva egy olyan adatszerkezetet készítünk, ami megpróbálja kiküszöbölni a 3.1.2. alfejezetben megfogalmazott problémákat. A közvetlen címzés legnagyobb gyengesége éppen annak "közvetlen" volta, tehát, hogy a kulcsot közvetlenül akarjuk felhasználni a tömb címzésére, ami gyakran problémákat vet fel (hiszen a kulcsok típusát gyakran nem mi határozzuk meg, hanem az természetszerűleg adódik). Érdemes tehát mindezt közvetett módon megvalósítani, tehát legyen lehetőség a kulcsokra először valamilyen transzformációt végezni, mielőtt azokat címzésre használnánk. Ezt valósítják meg a hasító táblázatok. Ebben az esetben bevezetünk egy függvényt (az úgynevezett *hasítófüggvényt*), aminek az a szerepe, hogy a beszúrás/keresés/törlés során nem a kulcsot használjuk közvetlenül a címzésre, hanem először a kulcsra alkalmazzuk ezt a függvényt, és annak a visszatérési értékével fogunk indexelni. Az adatok tárolását a háttérben általában egy egyszerű tömb végzi, amit A-val jelölünk. A hasító táblázatra mutat egy egyszerű példát a 3.2. ábra.

Ez nagyrészt megoldja a közvetlen címzés problémáit, mivel:

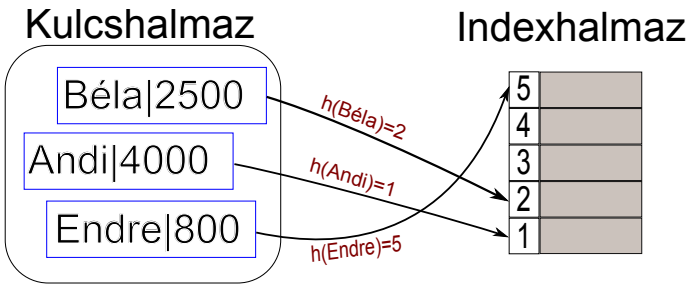
- A kulcsok lehetnek tetszőleges típusúak, hiszen végül nem azokkal fogunk címezni. Lehet a kulcs akár szöveg is, csak akkor olyan hasítófüggvényt kell találnunk, ami a szöveges bemenethez egy egész szám kimenetet rendel.
- A kulcsoknak nem kell egy megadott tartományon belül maradniuk, elég ha úgy választjuk meg a hasítófüggvényt, hogy a visszatérési értéke egy megfelelő tartományon belül maradjon.

Néhány alapvető fogalom:

- Kulcshalmaz (K): A lehetséges kulcsok halmaza. Ez lehet tetszőleges típusú és méretű. Mérete legyen $n =\mid K \mid$ (tehát a K halmaz eleminek a száma).
- Indexhalmas (I): A lehetséges indexek halmaza. Mivel a hasító táblázatok adatait általában egy tömbben tároljuk, így az indexhalmaz mérete ($m =\mid I \mid$) összhangban kell legyen a tömb méretével. A tömb lehetséges indexei: $0..(m-1)$.
- Hasítófüggvény ($h : K \to I$): Egy leképezés, amelyik a kulcs alapján megadja a keresett indexet (másnéven *kulcstranszformációs-függvény*).

> **Megjegyzés**
>
> Bár a pszeudokódokban általában 1-től kezdjük a tömbök címzését, itt kivételesen 0-tól kezdődő indexelést használunk. Természetesen egy +1 hozzáadással bármikor áttérhetünk a másik indexelésre, de az egyszerűbb leírás kedvéért ezt nem alkalmazzuk.

A hasító táblázatok legszembetűnőbb előnye a közvetlen címzéshez képest, hogy ezzel lehetőségünk nyílik arra, hogy egy meglehetősen nagy kulcshalmaz esetén (pl. ha személyi számot tekintünk kulcsnak, akkor a lehetséges kulcsok halmaza meglehetősen nagy) is elég lehet egy nagyságrendekkel kisebb



3.2. ábra. Példa hasító táblázatra. A lényeges különbség a direkt címzéshez képest az, hogy a kulcsmezőt (ami jelen esetben a név) nem közvetlenül használjuk indexelésre, hanem előtte meghívjuk a kulcsra a hasítófüggvényt, és annak a visszatérési értéke lesz az index. Ugyanígy működik a keresés is a kulcs alapján. A hasítófüggvény ezen a példán visszaadja a név első betűjének magyar ABC szerinti pozícióját.

indexhalmaz is (pl. 100 elemű), csak arra kell ügyelnünk, hogy a hasítófüggvény megfelelően képezzen le ebbe a kisebb halmazba (pl. $h(k) : k \ mod \ 100$, a hasítófüggvény indexként visszaadja a kulcs utolsó két számjegyét).

Egy tökéletes (tehát ütközéseket nem okozó) hasítófüggvény esetében, ha egy k kulcsú (ahol $k \in K$) elemet szeretnénk eltárolni, akkor a tömb $h(k)$ indexű helyére bemásoljuk ezt az elemet. Ugyanígy, egy k kulcsú elem keresésekor megvizsgáljuk a tömb $h(k)$ indexű elemét, ha az üres, akkor nincs ilyen elemünk, ha nem üres, akkor pedig az ott lévőt kell visszaadnunk. Törlés esetén pedig a tömb $h(k)$ indexű elemét megjelöljük töröltként.

Fontos megjegyezni, hogy mindez csak egy tökéletes hasítófüggvény esetében igaz, a gyakorlatban ilyennel nem nagyon fogunk találkozni. A hasító tábla említett előnye, hogy kisebb indexhalmazba képezzük le a sokkal nagyobb méretű kulcshalmazt, törvényszerűen azzal jár együtt, hogy felmerül az ütközések veszélye. A fenti példánál maradva elképzelhető, hogy két ember személyi számának megegyezik az utolsó két számjegye, így őket a hasítófüggvény ugyanarra a tömb indexre képezi le. Belátható általánosan is, hogy bármelyik esetben, ha a kulcshalmaz mérete nagyobb, mint az indexhalmaz mérete ($n > m$), akkor számolnunk kell az ütközésekkel. Ez nem lesz akkora probléma, mint amilyennek elsőre tűnik, és a későbbiekben számos módszert fogunk megismerni ennek a kezelésére, de érdemes már most megjegyezni, hogy a kiváló egy lépésből álló beszúrás/keresés/törlés műveleteket sajnos nem fogjuk tudni fenntartani ezekkel a technikákkal.

3.3. Hasítófüggvény megvalósítások

3.3.1. Az ideális hasítófüggvény

Ha megértettük a hasító táblázatok működésének alapjait, akkor már csak az lehet a kérdés, hogy miként válasszuk meg a hasítófüggvényt. Általánosan jó válasz sajnos nincs erre a kérdésre, a megfelelő függvény megtalálása mindig a megadott feladattól függ. Egyrészt technikai paraméterek korlátozzák a választható hasítófüggvények körét (pl. a kulcsok egész számok, szövegek, lebegőpontos számok, stb.), amelyekhez mindenképpen igazodni kell. Másrészt gyakran optimalizációval kapcsolatos döntéseket kell hoznunk a függvények, vagy azok paramétereit illetően (pl. egy kisméretű tömb használata jelentősen megnöveli az ütközések valószínűségét, túl nagy tömb azonban a tárhelyigényt fogja drasztikusan megnövelni). Tehát hasítófüggvényt csak a konkrét feladat ismeretében tudunk ajánlani.

Mindenesetre van néhány általános szabály, amit érdemes figyelembe venni. Ezek alapán az ideális hasítófüggvény jellemzői:

- Egyszerűen kiszámítható legyen: bár a hasítófüggvény számítása általában nagyságrendekkel kisebb erőforrásigénnyel bír, mint a tárhelyhez való hozzáférés, de célszerű itt is optimalizálni az időigényt.
- Kevés ütközést produkáljon: tudjuk majd kezelni az ütközéseket, de ezek mind további lépéseket, esetenként pedig további tárhelyhasználatot igényelnek. Emiatt célszerű már a hasítófüggvény megválasztással minél kevesebb ütközést előidézni.
- Egyenletesen ossza szét a kulcsokat: bár nem biztos, de várhatóan kevesebb ütközést fog produkálni egy olyan függvény, ami a kulcsokat egyenletesen osztja szét az indextartományon, mint az, amelyik bizonyos tartományokba több kulcsot képez le.
- Hasonló kulcsokat véletlenszerűen ossza szét: szintén nem biztos, de általában itt is kevesebb ütközésre számíthatunk, ha az egymástól csak kicsit különböző kulcsokat egymástól távolra képezi le a hasítófüggvény. A gyakorlatban ugyanis a kulcsok eloszlása sokszor nem egyenletes, így azonban a kulcsok sűrűsödése nem okoz majd ütközéseket az indexeknél.

Hasítófüggvények esetében szokás beszélni *lavina-hatásról* (avalanche effect) [2], ami azt jelenti, hogy egy jól megválasztott hasítófüggvény esetében, ha a bemenetet kicsit megváltoztatjuk (egy bit változás), akkor attól a kimenet jelentősen megváltozik (legalább a kimeneti bitek fele változik). Ennek a tulajdonságnak köszönhetően a hasítófüggvények gyakran megjelennek egyéb feladatok megoldásaiban is (pl. ellenőrző összegek).

3.3.2. Néhány hasítófüggvény megvalósítás

Az alábbiakban néhány hasítófüggvény megvalósítást vizsgálunk meg. Ismét megjegyezzük, hogy nincs egyetemes szabály a hasítófüggvények kialakítására, az alábbiak pusztán ötletadónak tekintendők. Magát a tényleges függvényt mindig a feladathoz alkalmazkodva kell elkészíteni, akár a lentiek közül választva egyet, azokat kombinálva, akár egy teljesen újszerű megoldás alkalmazásával.

A három minta esetében mindig feltételezzük, hogy a kulcs egy pozitív egész szám.

Osztó módszer

A hasítófüggvények fontos funkciója az átalakításon és szétszóráson túl, az intervallumba leképezés. Mivel általában a kulcsok halmaza jóval nagyobb, mint a lehetséges indexek halmaza, így a hasítófüggvénynek garantálnia kell, hogy a kulcs méretétől függetlenül a kimenete mindig az indexhalmaz tartományon belülre essen. Ennek a legegyszerűbb megvalósítása egy egyszerű maradékos osztás, amivel biztosítjuk, hogy a kimenet $0..(m-1)$ közötti tartományba essen.

Ennek megfelelően az osztó módszer általános alakja:

$$h(k) : k \ mod \ m \tag{3.1}$$

Ahol

- k: A kulcs, ami jelen esetben csak pozitív egész szám lehet.
- m: Az indexhalmaz mérete. A létrehozott tömb mérete megegyezik a osztóval, így biztosítható, hogy a visszaadott érték mindig $0..(m-1)$ tartományba essen.

Az m megválasztása alapvetően tetszőleges, nyilvánvaló, hogy nagyobb értékek általában kevesebb kulcsütközést jelentenek nagyobb tárhelyigény árán, míg kisebb m értékek több kulcsütközéssel járnak, viszont csökken a szükséges tömb mérete.

A lavina hatásnál láttuk, hogy a gyakorlatban olyan hasítófüggvényeket célszerű választani, amelyek a bemenet minden kisebb változtatására lehetőleg nagy kimeneti változást eredményeznek. Az osztó módszer esetében ezért gyakran nem célszerű m-nek 2 vagy 10 hatványait választani, mivel a 10 esetében a decimális számrendszerben felírt számoknak valójában csak az utolsó néhány számjegyét fogja figyelembe venni a függvény, a 2-re pedig ugyanez igaz, csak bináris számrendszerben. Ajánlásként célszerű a 2 hatványaihoz nem túl közeli prímeket választani [2].

Szorzó módszer

Véletlenszerűhöz hasonló eloszlást érhetünk el a szorzó módszer használatával. Ennek alapja:

$$h(k) : Közepe^M(Y * k) \tag{3.2}$$

Ahol

- k: A kulcs, ami jelen esetben csak pozitív egész szám lehet.
- Y: Egy tetszőleges konstans (szintén pozitív egész szám).
- $Közepe^M(x)$: Függvény, ami visszaadja az x szám tízes számrendszerbeli alakjának középső M darab számjegyét (Pl. $Közepe^3(24356) = 435$).

Az M paraméterrel tudjuk meghatározni, hogy mekkora tömböt szeretnénk a háttérben használni, mivel ennek értékből adódik, hogy a függvény mekkora számokat ad vissza (0 és $10^M - 1$ közöttieket). Az Y paraméter egy tetszőleges, általában nagy értékű konstans. Nyilván akkora számot célszerű választani, hogy a vele való szorzás legalább M jegyű számokat eredményezzen. A speciális esetek kivédése érdekében itt is célszerű prímet választani.

Amennyiben az index halmaz méretének nem 10 hatványát szeretnénk felhasználni, akkor az így visszakapott értéket egy maradékos osztással tudjuk egy intervallumba leképezni:

$$h(k) : Közepe^M(k * k) \ mod \ m \tag{3.3}$$

A módszer egy speciális alváltozata a négyzetközép módszer (a paraméterek ugyanazok, mint az előző leírásban). Ez olyan jó eloszlást eredményez, hogy kezdetleges véletlenszám-generátorként is lehet használni egymás után többszöri hívással:

$$h(k) : Közepe^M(k * k) \tag{3.4}$$

Számjegyes módszer

Szintén jó eloszlást kaphatunk, ha a kulcsokat 10-es (vagy akár más) számrendszerben leírva, azok számjegyei segítségével próbálunk meg egy indexet kiszámítani.

Erre egy egyszerű lehetőség:

$$h(k) : \sum_{i=1}^{|k|} Helyiérték^i(k) \ mod \ m \tag{3.5}$$

Ahol

- k: A kulcs, ami jelen esetben csak pozitív egész szám lehet.
- $| \ k \ |$: A k szám számjegyeinek a száma.
- m: A szokásos maradékos osztás paramétere, ezzel biztosítjuk, hogy a kimenet egy megadott tartományon belüli lesz.
- $Helyiérték^i(k)$: Visszaadja a k paraméter i. számjegyét.

Mivel a számok mérete és a számjegyeik összege között nincs szoros kapcsolat, ez a gyakorlatban szintén véletlenszerűhöz közeli eloszlást ad. Mivel érzéketlen arra, hogy melyik szám melyik helyiértéken szerepel (ami miatt pl. számjegycserék esetén ugyanoda képez), emiatt használható az alábbi módosított változat is:

$$h(k) : \sum_{i=1}^{|k|} Súly_i * Helyiérték^i(k) \ mod \ m \tag{3.6}$$

Ahol

- k: A kulcs, ami jelen esetben csak pozitív egész szám lehet.
- $| \ k \ |$: A k szám számjegyeinek a száma.
- m: A szokásos maradékos osztás paramétere, ezzel biztosítjuk, hogy a kimenet egy megadott tartományon belüli lesz.
- $Súly_i$: Megadja az i. számjegyhez tartozó súlyozó tényezőt.
- $Helyiérték^i(k)$: Visszaadja a k paraméter i. számjegyét.

Az alapötlet itt is ugyanaz, csak az egyes számjegyeket különböző súllyal vesszük figyelembe, attól függően, hogy melyik helyiértéken állnak.

3.3.3. További megfontolások

Szöveges típusú kulcsok

Az előző mintákban azt feltételeztük, hogy a kulcsok egész számok, a gyakorlatban azonban ez nem feltétlenül lesz mindig igaz. A személyi számra éppen teljesül, de pl. a személyi igazolvány számra, rendszámra már nem. És persze használhatunk kulcsként neveket is, ha tudjuk, hogy azok egyediek lesznek.

Azokban az esetekben, amikor egy karaktert kell használnunk kulcsként, ehhez egy egész számot kell hozzárendelnünk, amelyet majd az előző hasítófüggvény megvalósítások közül valamelyikkel (vagy akár egy teljesen más megvalósítással) leképezünk az indexhalmazra. Maga az átalakítás sokféleképpen megoldható, egy lehetséges megoldás (*jel-kód módszer* [8]), ha az egyes szimbólumokhoz (betűkhöz) számokat rendelünk. Ilyen táblázatok már eleve léteznek (ASCII, UniCode, stb.), de a kulcs és jellemzői (pl. a használt nyelv) alapján egy saját fordítótábla is készíthető.

Amennyiben nem csak egy karakter, hanem egy hosszabb szöveg a kulcs, akkor a karakterenként kapott kódokból kell valamilyen módszerrel egy számot előállítani. Ez jelentheti az így kapott számok összeadását, helyiértékenkénti „összeragasztását”, helyiértéktől függően súlyozott összeadását, stb.

Univerzális hasítási technika

Az előzőleg megismert módszerek gyakorlati használhatósága (kulcsütközések gyakorisága) nagyban függ a paraméterek megválasztásának módjától. Bár a hasító táblázatok általános esetben nagyon gyorsan tudják végrehajtani az alapműveleket, a helytelenül megválasztott paraméterekkel ez jelentősen leromlik.

Ez a legrosszabb eset fordulhat elő például akkor, ha az osztó módszernél olyan m paramétert határozunk meg, amivel minden kulcs osztható.

Ez ellen egy hatásos módszer az univerzális hasítási technika, ami azt jelenti, hogy a hasítófüggvényt a kulcsoktól függetlenül véletlenül választjuk ki, például az osztó módszernél az m paramétert egy intervallumból választjuk ki véletlenszerűen. Az átlagos esetet ez nem befolyásolja, viszont segíthet elkerülni a legrosszabb esetet, ami előfordulhat egyszerűen véletlen folytán is, de akár tudatos támadásként is (ismerve a hasítófüggvényünk paramétereit, valaki szándékosan olyan kulcsokat adhat, amelyek sok kulcsütközést generálnak). Néha persze előfordulhat, hogy a véletlen szám pont „eltalálja" a legrosszabb esetet, de a program következő futtatásakor már másik értékkel, jól fog működni.

Összetett kulcsok

Bizonyos esetekben az eltárolni kívánt típus összetett kulcsokkal rendelkezik (pl. egy háziállat esetében a kulcs lehet a gazda személyi igazolvány száma, az állat neve és születési dátuma). Az, hogy az ilyen összetett kulcsokat hogyan lehet felhasználni egy hasító táblázatnál, mindig egyedi megfontolást igényel. Azt mindenesetre célszerű betartani, hogy az összetett kulcs minden eleme jelenjen meg a kulcsgenerálás során.

Az egyik lehetőség a kulcs komponensek külön-külön hasítása (akár különböző függvényekkel), majd az így kapott értékek összegzése. Egy másik jól használható módszer, ha az összetett kulcs elemeit egyszerűen egymás mögé írjuk, és az így kapott adatsort egy nagyméretű bináris számként fogjuk fel, amire már az eddig megismert hasítófüggvény implementációk egyszerűen alkalmazhatók.

3.4. Kulcsütközések kezelése

A hasítófüggvények helyes megválasztásának módjához hasonlóan minden hasító táblázatnál ki kell alakítani egy módszert a kulcsütközések kezelésére. A *kulcsütközés* alatt azt értjük, ha két különböző kulcshoz a hasítófüggvény ugyanazt az indexet rendeli, tehát a tároláshoz használt tömb egy cellájában két elemet is el kellene tudnunk tárolni, ami nem lehetséges. Az előzőekben már beláttuk, hogy addig, amíg a kulcshalmaz nagyobb, mint az indexhalmaz, addig kulcsütközésekkel mindig számolnunk kell. Habár úgy próbáljuk megválasztani a hasítófüggvényt, hogy minél kevesebb ütközést okozzon, de ezek bármilyen ritkán fordulnak is elő, mi egyetlen ütközés miatti adatvesztést se engedhetünk meg, így mindenképpen foglalkoznunk kell ezzel a kérdéssel. A kulcsütközést tehát ne „hibaként" képzeljük el, hanem ez egyszerűen a hasító tábla adatszerkezet velejárója, amit valamilyen formában kezelnünk kell.

Megjegyzés

Ne zavarjon meg minket a némileg félreérthető „kulcsütközés" megnevezés. Ez nem azt jelenti, hogy vannak azonos kulcsú elemek (hiszen ezt már a kulcsok definíciójánál kizártuk), hanem azt jelenti, hogy két különböző kulcsú elemet a hasítófüggvény ugyanahhoz az indexhez rendel.

Természetesen a kulcsütközést is tetszőleges módszerekkel lehet kezelni, az alábbiakban csupán ötletadóként mutatunk be néhány nevezetes módszert.

3.4.1. Túlcsordulási terület

Az egyik legegyszerűbb megvalósítás, amely a hasító táblázatot tartalmazó tömbön túl egy további adatszerkezetet is használ. Működése megegyezik az eddig megismerttel, azonban azokban az esetekben, ahol egy hasítás kulcsütközéssel jár (tehát egy új elem beszúrása esetén a hasítófüggvény által kiszámolt indexű résben már van egy elem), az új elemet egyszerűen elhelyezi ebben a túlcsordulási területben. Értelemszerűen ennek megfelelően módosítani kell a keresést is, illetve a törlést is.

Beszúrás

A beszúrást mutatja be a 3.1. algoritmus. A paraméterként kapott beszúrandó elem kulcsára lefuttatjuk a hasítófüggvényt, ez alapján megkapjuk az indexet, hogy hova kellene elhelyezni azt a tömbben. A következő sorban következik az ellenőrzés, hogy ez a hely még szabad-e. Ha igen, akkor ide bemásoljuk az új elemet, ha pedig nem, akkor beszúrjuk a túlcsordulási területre. Ennek konkrét implementációjával nem foglalkozunk, lehet ez is egy tömb, egy láncolt lista (lásd 4. fejezet), stb.

Keresés

A keresést során is figyelembe kell vennünk a túlcsordulási területen lévő elemeket (3.2. algoritmus). A keresett elem itt már nem biztos, hogy benne lesz az A tömbben, ezért többféle lehetőséget kell kezelnünk. Látható, a hasítófüggvény meghívását követően (tehát miután megtudjuk, hogy hol kellene lennie az elemnek a tömbben) a feltételrendszer a következő lehetőségeket vizsgálja át:

- Ha a tömb i indexű eleme nem üres, akkor megvizsgáljuk ennek a kulcsát. Ha a keresett kulcsot találjuk itt, akkor megtaláltuk a keresett elemet.
- Minden egyéb esetben (ha az i indexű hely üres, vagy ott éppen egy másik elem van), akkor meg kell néznünk a túlcsordulási területet is. Ha az ottani kereső eljárás igazat ad vissza, akkor kiolvassuk az elemet és visszaadjuk (ezeknek a pontos implementációjával nem foglalkozunk).
- Ha a túlcsordulási területen se találjuk a keresett kulcsot, akkor jelezzük, hogy ilyen elem nincs.

3.1. Algoritmus Beszúrás túlcsordulási területtel

Bemenet: A - (**K**,**T**) **tömb**, h - ***függény***, *kulcs* - **K**, *érték* - **T**
Kimenet: A - (**K**,**T**) **tömb**

```
1: eljárás BESZÚRÁS(címszerint A : (K, T) tömb, h : függény, kulcs : K, érték : T)
2:     i ← h(kulcs)
3:     ha A[i].kulcs = ø akkor
4:         A[i].kulcs ← kulcs
5:         A[i].tart ← érték
6:     különben
7:         TÚLCSODULÁSITERÜLETBESZÚR(kulcs, érték)
8:     elágazás vége
9: eljárás vége
```

Felhasznált változók és függvények

- h: A választott hasítófüggvény.
- A: A hasító táblázat által használt tömb, amiben az elemeket tároljuk.
- *érték*: A beszúrandó érték.
- *kulcs*: A beszúrandó kulcs.
- TÚLCSODULÁSITERÜLETBESZÚR(*elem* : **T**): Beszúr egy új elemet a túlcsordulási területre.

Törlés

A törlés az elem helyétől függően történhet a tömbből, vagy a túlcsordulási területből is. Látható, hogy maga a törlés kódja nagyrészt megegyezik a keresés kódjával, hiszen itt is azt kell először tisztáznunk, hogy a keresett kulcs benne van-e az adatszerkezetben, és ha igen, akkor hol? Ezt követően már csak a megfelelő helyről törölnünk kell az elemet.

A törlést mutatja be a 3.3. algoritmus.

> Megjegyzés
>
> Valamivel optimálisabbnak tűnhet egy olyan megoldás készítése, ahol a törlés azon esetében, ha az A tömbből törlünk az i indexű helyről, átmásolja ide a túlcsordulási területen esetleg meglévő első olyan elemet, amelyiknek szintén az $A[i]$ helyen kellene lennie. Ezzel gyorsítható lenne a keresés, hiszen az $A[i] = ø$ esetben már nem is kell megnéznünk a túlcsordulási területet, hiszen biztos, hogy nincs adott kulcsú elemünk. A megoldás egyetlen hátránya, hogy a törlésnél kell hozzá egy olyan művelet, ami megtalálja a túlcsordulási területen azokat az elemeket, amelyek kulcsai „ugyanoda képeződnek le" (és ez nem összekeverendő azzal, hogy azonos a kulcs), és mi ilyen műveletet nem írtunk elő. De hatékonysági okokból célszerű lehet a bevezetése.

A módszer értékelése

A módszer első ránézésre nem tűnik hatékonynak, hiszen a túlcsordulási területen való munka meglehetősen erőforrás-igényes. Érdemes azonban figyelembe venni, hogy ideális esetben egy jól megválasztott hasítófüggvény már eleve kevés kulcsütközést fog produkálni, így remélhetőleg kevés elemmel kell majd a túlcsordulási területen dolgozni. Nagy adatmennyiség esetén itt is célszerű valamilyen optimalizált megoldást keresni.

3.4.2. Láncolás használata

Ebben az esetben egy láncolt lista nevű adatszerkezetet használhatunk az adatok tárolására (lásd 4. fejezet). A hasítófüggvény kiszámítása azonos az eddig tanultakkal, ennek eredményeképpen kapunk egy indexet. Az elemeket azonban nem közvetlenül a tömbbe helyezzük el az $A[i]$ résbe, ott ugyanis csak egy láncolt lista feje van, hanem ebbe a listába kell felvenni az elemet.

Ennek megfelelően a különféle műveletek.

- Beszúrás: Az $A[h(k)]$ helyen induló láncolt listába beszúrjuk az új elemet.

3.2. Algoritmus Keresés túlcsordulási területtel

Bemenet: A - (**K**,**T**) **tömb**, h - ***függény***, $kulcs$ - **K**
Kimenet: $elem$ - **T** (*a megadott kulcsú elem*)

```
1: függvény KERESÉS(A : (K, T) tömb, h : függény, kulcs : K)
2:     i ← h(kulcs)
3:     ha A[i].kulcs = kulcs akkor
4:         vissza A[i].tart
5:     különben
6:         ha TÚLCSODULÁSITERÜLETKERES(kulcs) akkor
7:             elem ← TÚLCSODULÁSITERÜLETOLVAS(kulcs)
8:             vissza elem
9:         különben
10:            hiba "Nincs ilyen kulcs"
11:        elágazás vége
12:    elágazás vége
13: függvény vége
```

Felhasznált változók és függvények

- h: A választott hasítófüggvény.
- A: A hasító táblázat által használt tömb, amiben az elemeket tároljuk.
- $kulcs$: A keresett kulcs.
- TÚLCSORDULÁSITERÜLETKERES($kulcs$: **K**) : **logikai**: Visszatérési értéke egy logikai érték, hogy a megadott kulcsú elem benne van-e a túlcsordulási területben.
- TÚLCSORDULÁSITERÜLETOLVAS($kulcs$: **K**) : **T**: Visszaadja a paraméterként átadott kulcsú elemet a túlcsordulási területről.

- Keresés: Az $A[h(k)]$ helyen induló láncolt listából megkeressük az adott kulcsú elemet.
- Törlés: Az $A[h(k)]$ helyen induló láncolt listából töröljük az adott kulcsú elemet.

A szükséges algoritmusokat a 4. fejezet tartalmazza.

A módszer értékelése

Ez a megoldás sok szempontól ideális lehet, főleg azért, mivel így ki tudjuk használni a láncolt lista dinamikus mivoltát (vagyis a mérete futásidőben változhat, attól függően, hogy éppen hány elemet kell ténylegesen eltárolnunk). Amennyiben a kulcsok egyenletesen helyezkednek el, akkor a keresés se lesz drasztikusan rosszabb, hiszen ideális esetben az egyes láncolt listákban csak néhány lépést kell megtennünk, ellentétben a túlcsordulási területtel, ahol egy helyen gyűjtjük az összes kulcsütközést okozó elemet.

Hátrányként azt érdemes megjegyezni, hogy a láncolt lista helyfoglalása nagyobb mint egy hagyományos tömbbé, mivel minden lista elemnél el kell tárolnunk a következő listaelem címét is.

3.4.3. Nyílt címzés

Beszúrás

A nyílt címzés ötlete az, hogy abban az esetben, ha látjuk, hogy a beszúrandó elem kulcsához tartozó index terület már foglalt az tömbben, akkor valamilyen szabályos módon elkezdünk egy új területet keresni. Például a hasítófüggvény által visszaadott k. rés már foglalt, akkor megpróbálhatjuk a $k+1$, $k+2$, ... cellákat is megvizsgálni (ügyelve persze arra, hogy a tömb túlcímzése esetén az első indextől folytassuk a keresést). Mindezt addig folytatjuk, amíg nem találunk egy üres (vagy törölt, erre még visszatérünk) pozíciót, ide elhelyezzük a beszúrandó elemet (3.4. algoritmus). Másik lehetőség, ha a tömb minden elemét átnéztük, és nincs szabad hely, ilyenkor hibát jelzünk.

Ehhez bevezethetünk egy második paramétert a hasító függvénynél, amely a próbálkozások számát mutatja. Erre mutat példát a 3.7. egyenlet, ahol az egyparaméteres h függvény egy hagyományos hasító

3.3. Algoritmus Törlés túlcsordulási területtel

Bemenet: A - (**K**,**T**) **tömb**, h - ***függény***, $kulcs$ - **K**
Kimenet: A - (**K**,**T**) **tömb**

```
1: eljárás TÖRLÉS(címszerint A : (K, T) tömb, h : függény, kulcs : K)
2:     i ← h(kulcs)
3:     ha A[i].kulcs = kulcs akkor
4:         A[i].kulcs ← ø
5:     különben
6:         ha TÚLCSODULÁSITERÜLETKERES(kulcs) akkor
7:             TÚLCSODULÁSITERÜLETTÖRÖL(kulcs)
8:         elágazás vége
9:     elágazás vége
10: eljárás vége
```

Felhasznált változók és függvények

- h: A választott hasítófüggvény.
- A: A hasító táblázat által használt tömb, amiben az elemeket tároljuk.
- $kulcs$: A törlendő kulcs.
- TÚLCSORDULÁSITERÜLETKERES($kulcs$: **K**) : **logikai**: Visszatérési értéke egy logikai érték, hogy a megadott kulcsú elem benne van-e a túlcsordulási területben.
- TÚLCSORDULÁSITERÜLETTÖRÖL($kulcs$: **K**): Törli a paraméterként átadott kulcsú elemet a túlcsordulási területről.

függvény, a kétparaméteres a kulcsütközést is kezelő kibővített változat, a k a kulcs, a j a próbálkozás száma, az m pedig a tömb mérete.

$$h(k, j) : (h(k) + j) \ mod \ m \tag{3.7}$$

A megoldás hátránya, hogy néha az egyébként kulcsütközést nem okozó elemeket se tudjuk berakni a helyükre, mert azt már előzőleg elfoglalta egy, a kulcsütközés miatt az eredeti helyéről elcsúsztatott elem.

Keresés

Természetesen fontos, hogy a keresést is ennek megfelelően módosítsuk, tehát ha a hasítófüggvény által visszaadott i. résben nem azt találjuk, amit valójában kerestünk, akkor az előzőhöz hasonló módon ilyenkor is meg kell vizsgálnunk a következő cellákat. Addig haladunk, a beszúráshoz hasonló módon, amíg más elemek által elfoglalt, vagy pedig töröltnek jelölt helyet találunk (3.5. algoritmus). Ezt még ki kell ki kell egészíteni egy ellenőrzéssel, hogy teljesen kitöltött, de a keresett kulcsot nem tartalmazó tömbben ne kerüljön végtelen ciklusba.

Törlés

A pszeudokódban is látható, hogy be kellett vezetnünk a már meglévő üres (ø) mellett egy új, törölt (⊗) jelölést. Ennek szerepe a törlés alapján érthető meg. A törlés egyébként egészen hasonló, mint a keresés, csak a végén nem visszaadja a megtalált elemet, hanem beállítja azt töröltként (3.6. algoritmus).

Azért lényeges, hogy ne üres értéket adjon, mivel elképzelhető, hogy előzőleg volt olyan beszúrás, aminek elvileg a most törölt elem helyére kellett volna elhelyeznie az elemet, de a foglalt rések miatt csak a törölt után tudta azt elmenteni. Viszont ha most a törölt cellát üresnek állítanánk be, akkor a keresés itt mindig megállna, így nem találná meg ezt az elemet.

> Megjegyzés
>
> Nem biztos, hogy a törölt jelet az elemekkel azonos típussal meg tudjuk valósítani. Ilyenkor egyszerűen felvehetünk egy logikai tömböt a hasító táblázatban, ami azt jelzi, hogy megadott rés törölt-e.

3.4. Algoritmus Beszúrás nyílt címzéssel

Bemenet: A - (**K**,**T**) **tömb**, h - ***függény***, *kulcs* - **K**, *érték* - **T**
Kimenet: A - (**K**,**T**) **tömb**

```
1: eljárás BESZÚRÁS(címszerint A : (K, T) tömb, h : függény, kulcs : K, érték : T)
2:     j ← 0
3:     ciklus amíg j < m ∧ A[h(kulcs, j)].kulcs ≠ ø ∧ A[h(kulcs, j)].kulcs ≠ ⊗
4:         j ← j + 1
5:     ciklus vége
6:     ha j < m akkor
7:         A[h(kulcs, j)].kulcs ← kulcs
8:         A[h(kulcs, j)].tart ← érték
9:     különben
10:        hiba "Nincs hely"
11:    elágazás vége
12: eljárás vége
```

Felhasznált változók és függvények

- h: A választott hasítófüggvény.
- A: A hasító táblázat által használt tömb, amiben az elemeket tároljuk.
- *érték*: A beszúrandó érték.
- *kulcs*: A beszúrandó kulcs.
- m: Az A tömb mérete.
- $\otimes$: Speciális jel, amivel a törölt állapotot jelöljük.

3.5. Algoritmus Keresés nyílt címzéssel

Bemenet: A - (**K**,**T**) **tömb**, h - ***függény***, *kulcs* - **K**
Kimenet: *elem* - **T** (*a megadott kulcsú elem*)

```
1: függvény KERESÉS(A : (K, T) tömb, h : függény, kulcs : K)
2:     j ← 0
3:     ciklus amíg j < m ∧ A[h(kulcs, j)].kulcs ≠ ø ∧ A[h(kulcs, j)].kulcs ≠ kulcs
4:         j ← j + 1
5:     ciklus vége
6:     ha j < m ∧ A[h(kulcs, j)].kulcs ≠ ø akkor
7:         vissza A[h(kulcs, j)].tart
8:     különben
9:         hiba "Nincs ilyen kulcs"
10:    elágazás vége
11: függvény vége
```

Felhasznált változók és függvények

- h: A választott hasítófüggvény.
- A: A hasító táblázat által használt tömb, amiben az elemeket tároljuk.
- m: Az A tömb mérete.
- *kulcs*: A keresett kulcs.
- $\otimes$: Speciális jel, amivel a törölt állapotot jelöljük.

3.6. Algoritmus Törlés nyílt címzéssel

Bemenet: A - (**K**,**T**) **tömb**, h - ***függény***, $kulcs$ - **K**
Kimenet: A - (**K**,**T**) **tömb**

```
1: eljárás TÖRLÉS(címszerint A : (K, T) tömb, h : függény, kulcs : K)
2:     j ← 0
3:     ciklus amíg j < m ∧ A[h(kulcs, j)].kulcs ≠ ø ∧ A[h(kulcs, j)].kulcs ≠ kulcs
4:         j ← j + 1
5:     ciklus vége
6:     ha j < m ∧ A[h(kulcs, j)].kulcs ≠ ø akkor
7:         A[h(kulcs, j)].kulcs ← ⊗
8:     különben
9:         hiba "Nincs ilyen kulcs"
10:    elágazás vége
11: eljárás vége
```

Felhasznált változók és függvények

- h: A választott hasítófüggvény.
- A: A hasító táblázat által használt tömb, amiben az elemeket tároljuk.
- m: Az A tömb mérete.
- $kulcs$: A törlendő kulcs.
- $\otimes$: Speciális jel, amivel a törölt állapotot jelöljük.

Kipróbálási stratégiák

Az előbb említett módszer, hogy az egymás utáni elemeket vizsgáljuk meglehetősen egyszerű volt. Emellett azonban többféle stratégiát alkalmazhatunk arra, hogy hogyan határozzuk meg az egymás után vizsgálandó cellákat:

- Lineáris próba:

$$h(k, j) : (h(k) + zj) \ mod \ m \tag{3.8}$$

  Ahol z egy tetszőleges konstans, a j pedig a probálkozások száma (0-tól kezdve). Ezt mutatja be a bevezetőben szereplő példa is $z = 1$ választással (bár magát a képletet ott nem láthatjuk így egyben, hiszen egyesével növelgetés esetén felesleges lenne minden esetben újra kiszámolni a $h(k)$ értéket), tehát a szabad helyek keresése közben mindig a következő réseket kezdjük el nézegetni. Érdemes észrevenni a maradékos osztás szerepét, ezzel biztosítjuk azt, hogy ha esetleg már túlcímeznénk a tömböt, akkor a függvény visszatérési értéke vissza fog ugrani 0-ra. Hátránya, hogy gyakran csoportosuláshoz vezet (clustering), amikor egyre hosszabb telített szakaszok jönnek létre a tömbben, és ide egyre nehezebb lesz beilleszteni elemeket. Tovább nehezíti a helyzetet, hogy a törlések sem segítenek a sebesség növekedésben, hiszen a keresés a törölt elemeken is mindig túl fog lépni. Természetesen nem csak 1-el növelhetjük az értéket, de az eredmény más esetekben is hasonló lesz.

- Négyzetes próba:

$$h(k, j) : (h(k) + z_1 j + z_2 j^2) \ mod \ m \tag{3.9}$$

  Ahol z_1 és z_2 tetszőleges konstansok, a j pedig a probálkozások száma (0-tól kezdve). Tehát minden próbálkozással egyre nagyobb távolságokban keresünk új szabad rést, ami jó hatással van a csoportosulásra (ebben az esetben kevésbé, illetve más jelleggel jelentkezik).

Ezeken túl természetesen tetszőleges függvény használható a következő elem indexének meghatározására.

A módszer értékelése

A módszer egyik nagy előnye az egyszerűsége, illetve az, hogy nem igényel különböző külső adatszerkezeteket a kulcsütközésben érintett elemek tárolásához. Ez azt is eredményezi, hogy a hasító táblázatunk tárhelyét jobban ki tudjuk használni még kulcsütközések esetén is. A hátránya az, hogy nagyszámú

kulcsütközés során az elemek elkezdenek eltávolodni a nekik szánt helyekről és ez lelassíthatja a keresési műveletet.

3.4.4. Többszörös hasítás

A többszörös hasítás egy általánosan jól használható technika. Lényege az, hogy nem egy darab, hanem kettő, vagy akár tetszőleges számú hasító függvénnyel próbáljuk meg a hasítást (legyenek ezek $h_1, h_2, ..., h_r$). Amennyiben a h_j-edik függvény egy olyan indexet adott vissza, ahol már van elem a tömbben, akkor egyszerűen megpróbáljuk a hasítást a h_{j+1}-edik függvénnyel.

Nem kell feltétlenül egymástól teljesen különböző hasítófüggvényekre gondolni, egyszerűen megoldható az is, hogy mindig ugyanazt a hasítófüggvényt használjuk, csak különböző paraméterekkel. Tehát pl. a szorzó módszer esetében az előzőleg használt Y szorzó konstans lehet a próbálkozások számának (j) a függvénye:

$$h_j(k) : Közepe^M(Y * j * j * k) \tag{3.10}$$

A keresésnek nyilvánvalóan ugyanezt az utat kell bejárnia, tehát az egymást követő hasítófüggvényeket addig kell egymás után alkalmazni, amíg valamelyikkel meg nem találjuk a keresett elemet (vagy el nem tudjuk dönteni, hogy az biztosan nincs benne a hasító táblázatban).

A módszer értékelése

A beszúrás/keresés/törlés pszeudokódokat külön nem tekintjük át, az alapelv teljesen megegyezik a nyílt címzésnél. Sőt, magát a többszörös hasítást tekinthetjük akár a nyílt címzés egy alváltozatának is. Csak annyi a különbség, hogy nem egy egyszerű összeadással változtatjuk folyamatosan a vizsgált indexet, hanem mindig az újabb hasítófüggvény meghívásával kapjuk meg a következő vizsgálandó címet.

4. fejezet

Láncolt lista

4.1. Láncolt listák felépítése

4.1.1. Lista elemeinek szerkezete

Az előzőleg megismert tömb és hasító táblázatok után megkezdjük a dinamikus adatszerkezetek vizsgálatát. A dinamikus jelleg arra vonatkozik, hogy az adatszerkezet mérete futás közben folyamatosan változhat, új elemek beszúrása során növekszik, törlés esetén pedig csökken.

Ezek közül a legegyszerűbb adatszerkezet a *láncolt lista* (linked list), amely egy olyan sorozat, amelyen az első elemtől kezdődően sorban lehet haladni [8]. Alapelve az, hogy az elemeket a memóriában nem közvetlenül egymás mellett tároljuk (mint a tömbök esetében), hanem azok tetszőleges helyen helyezkedhetnek el, a köztük lévő kapcsolatot pedig úgy valósítjuk meg, hogy mindegyik elem egy hivatkozást tárol az őt követő elemre. A legelső elem ilyen szempontból speciálisnak tekinthető, erre egy külső hivatkozás mutat.

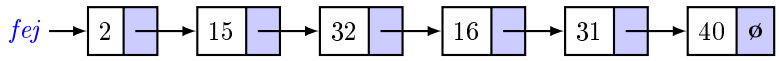
A konkrét technikai megvalósítás során a listaelemeket általában az alábbi mezők alkotják:

- Tartalmi rész: Ez a rész tartalmazza magát, a listaelemben eltárolni kívánt adatot. Nincs semmiféle megkötésünk ennek típusára, lehet szám, szöveg, vagy akár tetszőleges objektum típus is. Azt általában kikötjük, hogy az egyes listaelemek mind azonos típusú adatokat tartalmazzanak, de OOP környezetben persze erre sincs feltétlenül szükség. A példákban ezt a mezőt mindig *tart* néven fogjuk használni, típusát pedig **T**-vel fogjuk jelölni.

- Következő hivatkozás rész: A lista minden eleme tartalmaz egy hivatkozást a listában rákövetkező elemre. A legelső elemtől elindulva így (közvetve, a közbülső elemeken keresztül) elérhetjük a lista bármelyik elemét. A mező típusa a lista megvalósításától függően többféle lehet, erre még a későbbi, implementációs résznél visszatérünk (addig is a legegyszerűbb a lista elemeit objektumként, a következő mezőt pedig objektum referenciaként felfogni). A példákban ezt a mezőt mindig *köv* néven fogjuk használni, típusát pedig **M**-el fogjuk jelölni.

> **Megjegyzés**
>
> Mind az előző hasító táblázatnál, mind pedig a később tárgyalt fáknál megjelenik a már megismert kulcs mező, ami a tárolás/keresés során alapvető fontosságú. Természetesen a láncolt listát is kiegészíthetjük egy kulcs mezővel, a szakirodalomban azonban ez általában nem szokás, emiatt ezt mi sem tesszük (ennek az oka talán az, amit majd később látni fogunk, hogy a láncolt listák a keresés szempontjából elég rosszul teljesítenek).

A legelső elemre is kell rendelkeznünk egy hivatkozással, ez egy külső hivatkozás kell, hogy legyen, a példákban ezt mindig fej változóval fogjuk jelölni. A lista legutolsó eleme szintén speciálisnak tekinthető, hiszen a *köv* hivatkozás itt már nem tud egy valóban létező elemre mutatni. Emiatt ebbe a mezőbe mindig egy **ø**-al jelölt speciális lezáró értéket helyezünk (ennek tényleges értékét szintén később, az implementációval foglalkozó fejezetben vizsgáljuk).



4.1. ábra. Láncolt lista példa. Az eltárolt elemek: 2, 15, 32, 16, 31, 40.

Külön érdemes foglalkozni az üres listákkal. Ezeket mindig úgy fogjuk értelmezni, hogy a fej változó tartalma már eleve a ø lezáró elem. Ezt nem tekintjük hibának, ezzel csak azt jelöljük, hogy a lista ugyan létezik, de éppen nincsenek benne elemek.

A 4.1. ábra mutat egy minta láncolt listát. Látható, hogy a fej mutató a 2-es tartalmú elemre mutat, ezt tekintjük a lista első elemének. Ennek a tartalma "2", a *köv* mezője pedig a 15-ös tartalmú elemre hivatkozik. Amennyiben a *köv* mezőket követjük, akkor kialakul a láncolt listában tárolt elemek sorrendje: 2, 15, 32, 16, 31, 40.

A láncolt lista számos előnnyel rendelkezik a hagyományos tömbben való tárolással szemben:

- Mérete változtatható: A tömb egyik legnagyobb hátránya az volt, hogy már létrehozásakor meg kellett határozni a méretét, és a későbbiekben ez nem változtatható. A láncolt lista esetében ilyen problémánk nincs, mivel a dinamikus megvalósításokban az egyes elemek egymástól független objektumokként léteznek, így bármikor felvehetünk egy újat, vagy akár törölhetünk is egyet, csak arra kell ügyelni, hogy a *köv* mutatókon keresztül végigjárt út mindig pontosan a lista elemeit tartalmazza.

- Nem igényel összefüggő memóriaterületet: Bár napjainkban ez kevésbé okoz problémákat, de érdemes megjegyezni, hogy a tömb (felépítéséből adódódan) mindig egy összefüggő memóriaterületen helyezkedik el. A lista elemei a memóriában bárhol lehetnek szétszórva, azok egymáshoz való közelsége, illetve sorrendje lényegtelen, hiszen úgyis csak a *köv* mutatók határozzák meg az elemek logikai sorrendjét. Ezzel elkerüljük azt a futásidőben jelentkező jelenséget, amikor van még ugyan elég szabad hely a memóriában, de ez nem összefüggő, így nem tudunk létrehozni megfelelő méretű tömböt.

- Gyorsabb módosítás: A tömbök esetén az új elem beszúrása és az elemek törlése meglehetősen erőforrás-igényes művelet. Az előbbi esetben ugyanis (ha a tömb közepébe szúrunk be), akkor hátrébb kell tolni a mögötte lévő elemeket, és ehhez hasonlóan a törléskor is előre kell húzni a törölt mögötti elemeket. A későbbiekben részletesen megvizsgáljuk a lista módosítási műveleteket, de talán a felépítéséből is látható, hogy ezek a műveletek jóval egyszerűbbek lesznek, hiszen az elemek tényleges mozgatása nélkül, csak a *köv* mezők lokális módosításával könnyen tudjuk az új elemeket beláncolni, illetve a törlendő elemeket kiláncolni a listából. Mivel ezek a műveletek nem érintik a teljes listát, csak a módosítandó elem közvetlen környezetét, így ezek lépésszáma független az elemszámtól, konstans időben megvalósíthatók.

A láncolt lista persze néhány hátránnyal is jár:

- Nagyobb helyfoglalás: Minden listaelemben a tényleges eltárolt adattartalom mellett tárolnunk kell egy újabb mutatót is, ami a következő elemre mutat. Amennyiben az adattartalom csak egy egyszerű szám, akkor ez akár a tárigény duplázódását is jelentheti.

- Közvetlen elérés (véletlen elérés) hiánya: A tömb elemei mindig közvetlenül egymás mellett helyezkednek el a memóriában, így bármikor kiszámítható egy elem kezdőcíme. A láncolt lista elemei azonban tetszőleges helyeken szétszórva lehetnek a memóriában, így itt ilyen lehetőség nincs. Az $n.$ elemet csak úgy tudjuk elérni, ha az első elemtől elindulva $(n-1)$ darab lépésen keresztül követjük a *köv* mezők által megadott útvonalat. Ez jelentős hátrányt jelent azoknál az algoritmusoknál, amelyeknél közvetlenül szeretnénk elérni az adatszerkezet belső elemeit (pl. logaritmikus keresés), ezek a láncolt listával csak nagyon nehézkesen valósíthatók meg (és ezzel gyakran az egész algoritmus értelmét veszti).

Az előnyök és hátrányok figyelembevételével elmondható, hogy a láncolt listák a gyakorlatban jól használhatók olyan környezetekben, ahol nagy mennyiségű adatot kell eltárolnunk, és az adatszerkezetet gyakran kell módosítanunk (elemek törlése, új elemek felvétele), viszont nincs szükségünk a belső elemek közvetlen elérésére. Az eddig tanult programozási tételek alapján ez utóbbi meglehetősen hátrányos korlátnak tűnik, hiszen ott folyamatosan tömb indexeléssel dolgoztunk, azonban érdemes átgondolni,

hogy a tanult algoritmusok jelentős része (tipikusan azok, amelyeknél a ciklus az elemeket egyesével vizsgálja meg 1-től az elemszámig) egyszerűen módosítható úgy, hogy láncolt listákkal is hatékonyan működjön.

A láncolt listáknak számos változata ismert, az alábbiak szerint:

- Irányítottság alapján
  - Egyirányú láncolt lista: Minden lista elem csak a következő elemre tartalmaz hivatkozást. Erre mutat példát a 4.1. ábra.
  - Kétirányú láncolt lista: Minden lista elem a következő mellett az előző elemre is tartalmaz hivatkozást (4.5.1. alfejezet).
- Láncolások száma alapján
  - Egyszeresen láncolt (másnéven egyszerű) láncolt lista: A lista a benne lévő elemeknek egy sorrendjét tartalmazza (4.2. alfejezet).
  - Többszörösen láncolt lista: A lista nem csak egy, hanem több sorrendet is tartalmaz (4.5.2. alfejezet).

Emellett még léteznek különféle specializált listák, mint például a rendezett (4.3. alfejezet) vagy ciklikus (4.5.3. alfejezet) listák, illetve kiegészítő technikák, mint például a strázsa elemek használata (4.4.2. alfejezet).

4.2. Egyirányú, egyszerű láncolt lista alapműveletei

4.2.1. Inicializálás

Az előzőekben láthattuk a láncolt listák alapvető felépítését. Az üres listának azt tekintettük, amikor a lista *fej* elemének értéke a speciális lezáró jellel egyenlő (**ø**). Egy frissen létrehozott listánál biztosítanunk kell ezt a tulajdonságot, ezt valósítja meg a 4.1. algoritmus. Egy még inicializálatlan változót paraméterként átadva, beállítja a kezdőértéket a *fej* változónak (egy már meglévő lista esetén nem célszerű meghívni, mivel az elemeket ezzel nem szabadítottuk fel, azonban ezt követően már nem tudjuk őket elérni az első elemre való hivatkozás nélkül).

Érdemes kiemelni az algoritmusban használt címszerinti paraméterátadás fontosságát. Értékszerinti paraméterátadás esetében hiába változtatnánk meg a *fej* paraméter értékét, az nem lenne hatással a híváskor átadott változó értékére. A címszerinti paraméterátadás azonban biztosítja, hogy az átadott (még inicializálatlan) listafej valóban felvegye a szükséges **ø** értéket.

A későbbiekben bemutatott algoritmusok során mindig feltételezzük, hogy a láncolt lista inicializálva lett, tehát a *fej* vagy az első elemre mutat, vagy üres lista esetén a **ø** értéket tárolja.

4.1. Algoritmus Láncolt lista inicializálása

Bemenet: *fej* - **M**
Kimenet: *fej* - **M**

```
1: eljárás ListaInicializálás(címszerint fej : M)
2:     fej ← ø
3: eljárás vége
```

Felhasznált változók és függvények

- *fej*: Láncolt lista feje.

4.2.2. Új elem felvétele

Egy egyszerű láncolt lista esetében (akárcsak a rendezetlen tömböknél) nincs szabály arra, hogy az eltárolt értékek milyen sorrendben jelenjenek meg a listában. Az újonnan elhelyezendő elemek esetében tehát azt is tisztázni kell, hogy a listán belül hová kerüljön ez az elem. Ennek megfelelően többféle listába való beszúrást is áttekintünk.

Első elem elé beszúrás

Tömbök esetében az egyik leglassabban kivitelezhető művelet a tömb legelső eleme elé beszúrás, hiszen ilyenkor a tömb minden már meglévő elemét eggyel hátrébb kell tolni (feltéve, ha meg szeretnénk tartani az elemek sorrendjét). Az egyirányú, egyszerű láncolt listák esetében azonban éppen ez a beszúrás valósítható meg a legegyszerűbben (4.2. algoritmus).

Az algoritmus hívásakor két paramétert kell átadnunk, a lista aktuális *fej* elemét, illetve a beszúrni kívánt tartalmat (tehát nem egy teljes listaelemet, csak annak leendő tartalmi részét). Első lépésként létrehozunk egy új, üres listaelemet, ahol a tartalom mezőnek azonnal értékül is adhatjuk a beszúrni kívánt értéket.

Ezt követi az új elem beláncolása a listába. Ezt végzik a 4. és 5 sorok. A jelenlegi első elem a beszúrás után a második elem lesz közvetlenül a most beszúrandó elem mögött, ezért az erre hivatkozó *fej* értéket eltároljuk az *új* elem *köv* mezőjében. Ezzel a beszúrandó új elem és az eddigi első elem már megfelelő sorrendbe kerülnek.

A lista korábbi állapotától függően két eset lehetséges:

- Első elem elé beszúrás: Ebben az esetben (4.2. ábra) már az elem beszúrása előtt is voltak elemek a láncolt listában. Ez tekinthető az általános esetnek, ilyenkor az új elem a fent leírt módon tárolódik el.
- Első elem felvétele: Ebben az esetben a beszúrás meghívása előtt még nem volt elem a listában. Az újonnan felvett elemet tehát nem egy már meglévő elé kell felvenni, hanem ez az új elem lesz

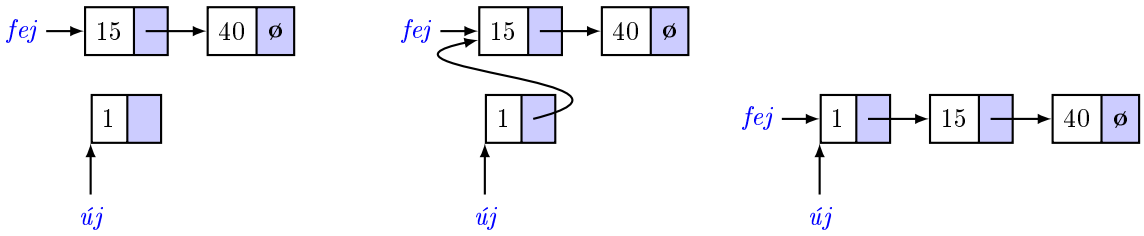
4.2. Algoritmus Láncolt lista első eleme elé beszúrás

Bemenet: fej - **M**, *érték* - **T**
Kimenet: fej - **M**

```
1: eljárás ListábaBeszúrásElejére(címszerint fej : M, érték : T)
2:     új ← Létrehoz(M)
3:     új.tart ← érték
4:     új.köv ← fej
5:     fej ← új
6: eljárás vége
```

Felhasznált változók és függvények

- fej: Láncolt lista feje (ami esetleg meg is változhat).
- *érték*: Beszúrandó tartalom.
- Létrehoz(**M**) : **M**: Létrehoz egy **M** típusú elemet, ez a visszatérési értéke.



(a) Kételemű lista, és a létrehozott *új* elem (b) Az *új.köv* a fej-re mutat (c) A fej az új elemre mutat

4.2. ábra. Láncolt lista elejére beszúrás. A példában az "1" értéket szúrjuk be a listába.

a lista első (és egyben összes) eleme. Az algoritmus ebben az esetben is jól működik, jól látható, hogy az így felvett *új* elem *köv* mezőjébe ilyenkor a fej aktuális tartalma kerül, aminek (üres lista lévén) az értéke **ø**. Ez pedig helyes végeredmény, hiszen az első elem egyben az utolsó is, tehát a lezáró elemnek kell ide kerülnie.

Utolsó elem mögé beszúrás

Az első elem elé beszúrás hatékonynak tekinthető, hiszen a láncolt lista méretétől függetlenül mindössze két lépést igényel az új elem beláncolása. A módszer egyik mellékhatása azonban az, hogy az elemek a listában a beszúrás sorrendjével ellentétes sorrendben tárolódnak. Bizonyos feladatoknál ez nem jelent hátrányt (pl. ahol a sorrend lényegtelen), más esetekben viszont ez gondot jelenthet. Amennyiben szükséges, hogy a lista pont ugyanabban a sorrendben tárolja az elemeket, mint ahogy azokat elhelyeztük benne, akkor mindig a lista végére kell felvenni az új elemet.

A lista végére beszúrás meglehetősen erőforrás-igényes művelet, ugyanis külső hivatkozással csak a lista első elemét érjük el, az ezt követőkhöz további lépések szükségesek. Az utolsó elem eléréséhez tehát egyesével végig kell lépkednünk a lista elemein, majd az utolsót megtalálva, ott beláncolhatjuk az újat. Ennek megfelelően a 4.3. algoritmusban is jól elkülöníthető ez a két fő lépés. A 4.3. ábra pedig egy példát mutat a beszúrásra.

Az algoritmus indulásakor azonnal létrehozhatjuk az elemet, illetve kitölthetjük a *tart* és a *köv* mezők tartalmát is, hiszen az elem a lista végére fog kerülni, így ezek az értékek mindig a megadott tartalom, illetve a **ø** értéket kapják meg (még abban az esetben is, ha egy üres listába szúrjuk be az elemet, tehát ez lesz az első és egyben utolsó láncszem is).

Az 5. sorban található ellenőrzés azt vizsgálja meg, hogy van-e már elem a listában. A beláncolás általános esete azt jelentené, hogy megkeressük a lista végét, majd az utolsó elem *köv* mezőjébe egy hivatkozást helyezünk el az újonnan létrehozott elemre. Speciális esetnek tekinthetjük azt, ha üres listába akarunk beszúrni, hiszen itt még egy elem sincs a listában, amihez hozzá lehetne fűzni. Emiatt ilyenkor a fej hivatkozást kell módosítanunk, az így lefutó beláncolás tulajdonképpen megegyezik a lista elejére beszúrásnál láthatóval.

Amennyiben vannak elemek a listában, akkor meg kell keresni ezek közül az utolsót. Ehhez egy p segédváltozót használunk, amelynek kezdeti értéke (8. sor) egy hivatkozás lesz a lista első elemére (mivel az előbb tisztáztuk, hogy a lista nem üres, így ez biztosan egy létező elemre fog mutatni). Az ezt követő ciklus (9. sor) szerepe, hogy elkezd egyesével lépegetni a listában, egészen addig, amíg el nem jut az utolsó elemig. A keresett elem jellegzetessége, hogy a *köv* mezőjének értéke **ø** lesz, ez a listában csak a legutolsó elemre lehet igaz. Emiatt a ciklus egészen addig fut, amíg a p hivatkozással nem találtuk meg ezt az elemet. Amennyiben a kilépési feltételünk nem teljesült, akkor éppen a lista valamelyik belső elemén állunk. Ilyenkor a p változóval egyszerűen átléphetünk a következő elemre (10. sor). Ezt a technikát a későbbiekben még sokszor fogjuk látni, mivel ez az egyetlen mód, hogy a láncolt lista belső elemeit feldolgozzuk.

Az egyirányú egyszerű láncolt listáról tanultaknak megfelelően az előző ciklus előbb-utóbb véget ér, és a kilépéskor a p éppen a lista utolsó elemére hivatkozik (4.3c. ábra). Ezen a ponton az *új* hivatkozik az általunk létrehozott új elemre, a p pedig a lista eddigi utolsó elemére. Az új elem beláncolása itt már egyszerűen megoldható, csak az utolsó elem *köv* hivatkozását kell átirányítanunk az újonnan létrehozott elemre (12. sor)

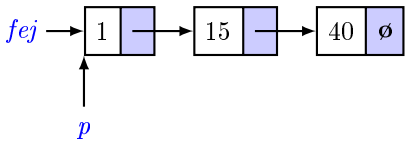
4.3. Algoritmus Láncolt lista utolsó elem mögé beszúrás

Bemenet: *fej* - **M**, *érték* - **T**
Kimenet: *fej* - **M**

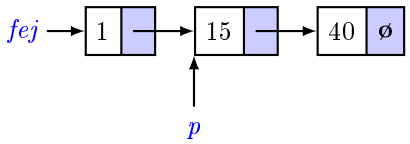
```
 1: eljárás ListábaBeszúrásVégére(címszerint fej : M, érték : T)
 2:     új ← Létrehoz(M)
 3:     új.tart ← érték
 4:     új.köv ← ø
 5:     ha fej = ø akkor
 6:         fej ← új
 7:     különben
 8:         p ← fej
 9:         ciklus amíg p.köv ≠ ø
10:             p ← p.köv
11:         ciklus vége
12:         p.köv ← új
13:     elágazás vége
14: eljárás vége
```

Felhasznált változók és függvények

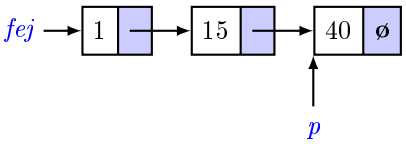
- *fej*: Láncolt lista feje (ami esetleg meg is változhat).
- *érték*: Beszúrandó tartalom.
- Létrehoz(**M**) : **M**: Létrehoz egy **M** típusú elemet, ez a visszatérési értéke.



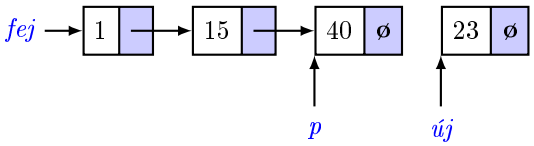
(a) Keresés első lépése.



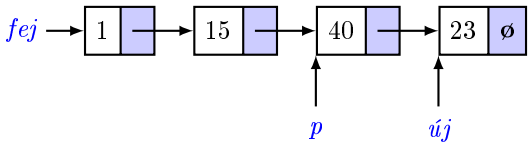
(b) Keresés második lépése.



(c) Keresés utolsó lépése. Jelenleg a p változó az utolsó elemre hivatkozik.



(d) Új elem létrehozása (a pszeudokódban ezt már az első lépésben megtettük).



(e) Az új elem beláncolása az eddigi utolsó elem mögé (amire a p hivatkozik).

4.3. ábra. Láncolt lista végére beszúrás. A példában a 23 értéket szúrjuk be a lista utolsó eleme mögé.

Lista n. helyére beszúrás

Ezt tekinthetjük a beláncolás általános esetének, amikor a lista két, már meglévő eleme közé kell elhelyeznünk egy új elemet. A feladatot itt is két részre bonthatjuk, először meg kell keresnünk a megfelelő elemeket, amelyek közé az újat be kell majd szúrni, majd pedig ennek megfelelően el kell végezni magát a beláncolást.

A 4.4. algoritmus ennek egy lehetséges megvalósítását mutatja (a 4.4. ábra pedig egy példát mutat ennek működésére). Az első két sor itt is ugyanaz, mint az előzőekben, hiszen ez biztos: létrehozzuk az új elemet, és ebben elhelyezzük a beszúrandó tartalmi értéket. Ezt követi egy ellenőrzés, mivel két különböző esetet kell megkülönböztetni:

- A lista első helyére kell beszúrnunk, ami két esetben fordulhat elő: az n értékének 1-et adunk meg, tehát az aktuális első elem elé kell beszúrni, vagy pedig üres a lista, így a beszúrt új elem (az n értékétől függetlenül) az első lesz. A két eset abban hasonlít egymáshoz, hogy mindkét eset a fej változó módosításával jár (mivel a fej mindig az első elemre mutat, így az első elem cseréje értelemszerűen ezt vonja magával).

- A lista közbülső vagy utolsó helyére kell beszúrnunk. Egy közbülső helyre beszúrás során az előző elem *köv* mezőjének az új elemre kell majd mutatnia, míg az újonnan felvett elem *köv* mutatójának az őt követő elemre. Speciális esetnek tűnik a legutolsó elem mögé beszúrás, de az algoritmus áttekintése során látni fogjuk, hogy ez nem igényel külön programágat.

A 4. sor felelős a fent említett ellenőrzésért. A fej változó aktuális értékéből meg tudjuk határozni, hogy a lista éppen üres-e, illetve az n értékéből látható, hogy az első elem elé szeretnénk-e beszúrni. Amennyiben a kettő közül bármelyik is igaz, akkor az 5. és a 6. sorok segítségével felvesszük az új elemet a lista elejére (ezek tulajdonképpen megegyeznek a már megismert, lista elejére való beszúrás lépéseivel).

Abban az esetben, ha egy belső elemet kell felvennünk, egy ciklussal meg kell keresni annak a helyét. A lista végére beszúrásnál már láthattuk, hogy hogyan tudunk egyesével lépkedni a listában. A különbség pusztán annyi, hogy ebben az esetben nem kell egészen a lista végéig eljutni, elég megtalálni az n. helyet (ehhez a jelenlegi $(n-1)$. elemet kell megkeresnünk, hiszen az újonnan felvett listaelemet majd ez után kell beláncolni). A 10. sorban látható ciklusfeltételek figyelik, hogy a ciklus csak addig fusson, amíg nem léptünk az utolsó elem mögé (ez csak abban az esetben fordulhat elő, ha a paraméterként átadott n érték

nagyobb, mint a listában található elemek száma), vagy elértük a szükséges elemet. Az i segédváltozó minden továbblépésnél eggyel növekszik, így mindig azt mutatja, hogy éppen hányadik elem *előtt* állunk.

A ciklusból való kilépést követően megtörténik a beláncolás (14–15. sor). Ennek két lépését mutatják a 4.4d. és 4.4e. ábrák.

A legutolsó elem mögé beszúrás speciális esetnek tűnik, de valójában a megadott algoritmus ezt is kezeli. A ciklus megállításakor a p ilyenkor a lista utolsó elemén áll. A 14. sor ennek megfelelően átmásolja a *p.köv* mezőben található **ø** értéket az *új* lista elembe, ami helyes, mivel valóban ő lesz a lista utolsó eleme. A p után láncolás (15. sor) pedig már hasonlóan működik, mint az általános esetben.

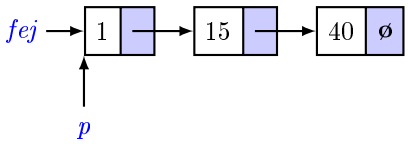
4.4. Algoritmus Beszúrás a láncolt lista n. helyére

Bemenet: fej - **M**, *érték* - **T**, n - **egész**
Kimenet: fej - **M**

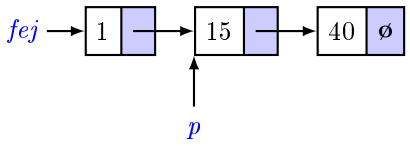
```
 1: eljárás ListábaBeszúrásN(címszerint fej : M, érték : T, n : egész)
 2:     új ← Létrehoz(M)
 3:     új.tart ← érték
 4:     ha (fej = ø) ∨ (n = 1) akkor
 5:         új.köv ← fej
 6:         fej ← új
 7:     különben
 8:         p ← fej
 9:         i ← 2
10:         ciklus amíg (p.köv ≠ ø) ∧ (i < n)
11:             p ← p.köv
12:             i ← i + 1
13:         ciklus vége
14:         új.köv ← p.köv
15:         p.köv ← új
16:     elágazás vége
17: eljárás vége
```

Felhasznált változók és függvények

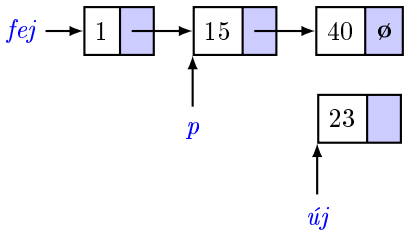
- fej: Láncolt lista feje (ami esetleg meg is változhat).
- *érték*: Beszúrandó tartalom.
- n: Meghatározza a beszúrandó elem helyét. A beszúrás után az új elem az n. lesz a listában (amennyiben a beszúrás előtt nincs legalább $(n-1)$ darab elem, akkor az utolsó).
- Létrehoz(**M**) : **M**: Létrehoz egy **M** típusú elemet, ez a visszatérési értéke.



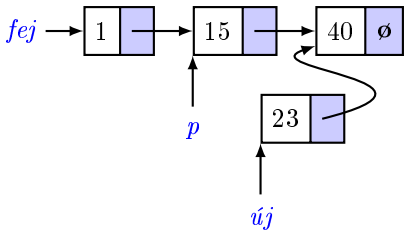
(a) Keresés első lépése ($i = 2$).



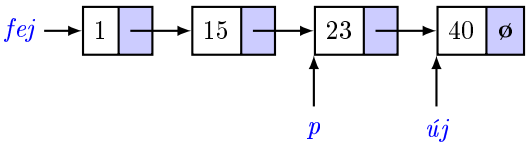
(b) Keresés második lépése ($i = 3$, ezért a ciklus véget ért).



(c) Új elem létrehozása, a pszeudokódban ezt már az első lépésben megtettük.



(d) Az új elem hivatkozzon az eddigi n. elemre.



(e) A p által jelölt elem hivatkozzon a létrehozott *új* elemre.

4.4. ábra. Láncolt lista n. helyére beszúrás. A példában a 23 értéket szúrjuk be a 3. elem elé.

4.2.3. Bejárás és keresés

Minden általunk tárgyalt adatszerkezet esetében megvizsgáljuk annak bejárási lehetőségeit. Bejárás alatt értjük azt a műveletet, amelynek segítségével az adatszerkezet valamennyi elemét pontosan egyszer elérjük. Az elérés során természetesen tetszőleges feldolgozási műveletekre is szükség lehet, ez lehet akár az elemek listázása, akár az elemeken valamilyen egyéb művelet elvégzése (pl. elemek összegzése, legkisebb elem megkeresése, stb.). A pontos művelet számunkra lényegtelen, mivel maga a bejárás módszere minden esetben azonos lesz.

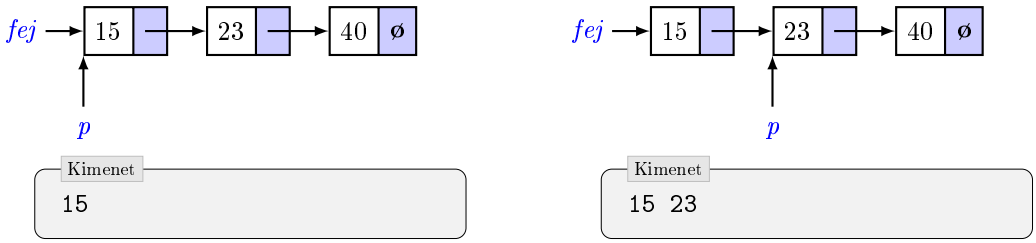
Láncolt lista bejárása

Egyszerű, egyirányú láncolt lista bejárását mutatja be a 4.5. algoritmus (ennek megértését pedig a 4.5. ábra segíti). A láncolt lista egyes elemeit nem tudjuk közvetlenül elérni, mindegyikhez csak az előző elemen keresztül rendelkezünk egy hivatkozással, kivéve persze az első elemet, amelyre egy külső hivatkozás mutat. A bejárás alapelve ennek megfelelően az, hogy elindulunk az első elemtől, majd egy ciklusba szervezve feldolgozzuk az aktuális elemet, és továbblépünk a következőre. A ciklust addig ismételjük, amíg fel nem dolgozzuk a teljes listát.

Az algoritmus egy p nevű segédváltozót használ az aktuális elem eltárolásához. Az algoritmus 2. sorában ennek a segédváltozónak értékül adjuk a fej változó értékét. Hivatkozásokról van szó, tehát itt nem a láncolt listában tárolt tartalmi adatok másolása történik meg, pusztán a p hivatkozás ugyanarra az elemre hivatkozik, ahová a fej. Amennyiben a feldolgozandó lista üres volt, akkor a p értéke a fej-hez hasonlóan **ø** lesz, ez azonban nem okoz problémát, mivel ilyenkor a következő ciklus magja egyszer se fog lefutni, ennek megfelelően az algoritmus véget ér érdemi feldolgozás nélkül.

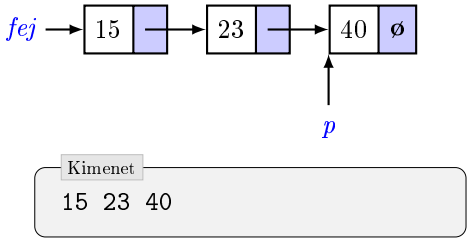
A ciklus feladata a fent említett lépések ismétlése (feldolgozás, következő elemre lépés) egészen addig, amíg nem jutunk a lánc végére (üres lista esetén tekinthetjük úgy, hogy már eleve ott állunk).

A ciklusmag első lépése maga a feldolgozás, ami jelen esetben egy FELDOLGOZ($elem$: **T**) függvény meghívását jelenti (4. sor). A függvény egy darab, a láncolt lista tartalmának megfelelő paramétert vár, és

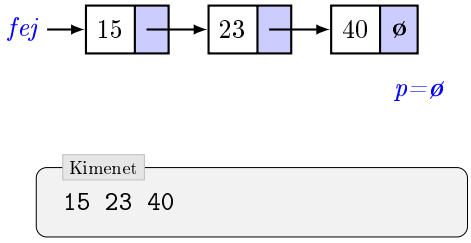


(a) Lista első elemének feldolgozása.

(b) Lista második elemének feldolgozása.



(c) Lista harmadik elemének feldolgozása.



(d) A lista végére ért a mutató.

4.5. ábra. Példa láncolt lista bejárására. A feldolgozás a listaelemek tartalmának kimenetre írását jelenti.

ezt a feladatnak megfelelően feldolgozza (a gyakorlatban persze elképzelhető, hogy az implementációban itt valójában nem egy függvényhívás, hanem közvetlenül a feldolgozás programkódja szerepel). Érdemes észrevenni, hogy a bejárás általános formájában nincs lehetőség az egyes elemek közötti kapcsolatok felhasználására (pl. előző, következő elemmel együtt feldolgozás).

A feldolgozást követően még a ciklusmagon belül átléptetjük a p hivatkozást a következő elemre (5. sor). Amíg ez egy valóban létező elemet jelent, addig a ciklus újra lefut, egészen addig, amíg el nem érünk a lánc végére (4.5c. ábra). Az utolsó elem feldolgozását követő léptetés hatására a p változó értéke **ø** lesz, emiatt a ciklusfeltétel többé nem lesz igaz (3. sor), így a program kilép a ciklusból. Ez egyben a bejárás végét is jelenti.

A 4.5. ábra egy egyszerű példát mutat egy háromelemű láncolt lista bejárására. A feldolgozás jelen esetben a tartalmak egyszerű listázását jelenti.

4.5. Algoritmus Láncolt lista bejárása

Bemenet: fej - **M**

```
1: eljárás ListaBejárás(fej : M)
2:     p ← fej
3:     ciklus amíg p ≠ ø
4:         Feldolgoz(p.tart)
5:         p ← p.köv
6:     ciklus vége
7: eljárás vége
```

Felhasznált változók és függvények

- fej: Láncolt lista feje.
- Feldolgoz($elem$: **T**): Tetszőleges feldolgozási művelet, amely minden láncolt listában eltárolt tartalomra pontosan egyszer fut le.

Keresés egyszerű láncolt listában

A láncolt listában való lineáris keresés tulajdonképpen a bejárás egy kisebb módosításának is tekinthető. A 4.6. algoritmus mutatja a megadott feltétel szerinti keresés algoritmusát. Az első sorban a p segédváltozóval az első elemre próbálunk állni. Ha ilyen elem nincs (mivel üres a lista), akkor a fej értékéhez hasonlóan a p értéke is **ø** lesz, ennek következtében nem lépünk be a következő ciklusba, és a függvény visszatérési értéke *hamis* lesz.

Amennyiben a segédváltozó már az első elemre hivatkozik, akkor a bejáráshoz hasonló ciklussal tudjuk elkezdeni a keresést (3. sor). A lista végének az ellenőrzése azonos, azonban itt megjelenik egy további feltétel, amely azt ellenőrzi, hogy megtaláltuk-e a keresett elemet. Ha ez utóbbi igaz, akkor szintén kiléphetünk a ciklusból, hiszen már tudjuk a választ a kérdésre. Amíg ezek a feltételek teljesülnek, addig a p mutatót folyamatosan léptetjük a következő elemre (4. sor).

A ciklusból való kilépés után a keresés programozási tételnek megfelelően ellenőrizzük, hogy miért ért véget a ciklus. A 6. sorban található feltétel akkor ad vissza *hamis* értéket, ha már a lista utolsó eleme után állunk (tehát nincs meg a keresett elem), és csak akkor ad vissza *igaz* értéket, ha egy valódi elemen állunk (tehát megtaláltuk a keresett elemet, és a p változó éppen erre hivatkozik).

A függvény visszatérési értéke tehát a fent megállapított eredmény, illetve a p változó által mutatott elem tartalma, amennyiben a keresés sikeres volt (amennyiben nem, akkor ez a második visszatérési érték nem értelmezhető).

4.6. Algoritmus Megadott feltételnek megfelelő tartalmú elem keresése láncolt listában

Bemenet: fej - **M**, F - **logikai feltétel**
Kimenet: van - **logikai**, $vissza$ - **T**

```
1:  függvény ListábanKeresés(fej : M, F : logikai feltétel)
2:      p ← fej
3:      ciklus amíg (p ≠ ø) ∧ ¬F(p.tart)
4:          p ← p.köv
5:      ciklus vége
6:      van ← (p ≠ ø)
7:      ha van akkor
8:          vissza (van, p.tart)
9:      különben
10:         vissza van
11:     elágazás vége
12: függvény vége
```

Felhasznált változók és függvények

- fej: Láncolt lista feje.
- F : A keresési feltétel, ami megadja, hogy minek kell megfelelnie a keresett elem tartalmának.
- van : Visszatérési értéke mutatja, hogy a lista tartalmaz-e keresett feltételnek megfelelő tartalommal bíró elemet.
- $vissza$: Amennyiben a van változó értéke igaz, akkor a keresett tartalmat adja vissza. Amennyiben nincs ilyen, akkor a visszatérési értéke nem értelmezhető. Amennyiben több, a feltételnek megfelelő tartalmú elem is megtalálható a listában, akkor a visszatérési érték ezek közül az első.

4.2.4. Törlés láncolt listából

Tetszőleges elem törlése a láncolt listából

Hogy a lista a későbbiekben is megtartsa előnyös tulajdonságait, így általában nem csak a lista elemen belül megtalálható tartalmat töröljük (vagy jelöljük valahogy, hogy az törölt), hanem az egész lista elemet eltávolítjuk. Ezt nevezzük kiláncolásnak, hiszen önmagában a törölni kívánt elem megszüntetése még nem lenne elegendő, módosítani kell az előtte lévő elemet is (illetve a legelső elem esetében a fej hivatkozást), hogy megmaradjon a lista épsége.

A feladat megoldása két részből áll: meg kell keresni a törlendő elemet (jelen esetben megadott tartalommal rendelkező elemet keresünk), majd pedig végre kell hajtani a kiláncolást (4.7. algoritmus). A kiláncolás során két alapvető esetet különböztethetünk meg:

- Középső vagy utolsó elem törlése: Egyszerű egyirányú láncolt listák esetében minden elem csak a következőre hivatkozik, az előzőre nem. Tehát a kiláncolás során csak a törlendő elem előtti elem *köv* hivatkozását kell majd módosítani a törlendő elem utáni elemre. Ezt tekinthetjük a törlés általános esetének. Ezeket az eseteket mutatják be a 4.6. és a 4.7. ábrák.

- Első elem törlése: Az első elem törlésekor ez természetesen nem valósítható meg, hiszen nincs „előző elem”, aminek a *köv* hivatkozását módosíthatnánk a kiláncolás során. Emiatt ilyenkor a fej értékét kell hasonló módon megváltoztatni, hogy az eddigi első (törlendő) elem helyett a másodikra hivatkozzon (4.8. ábra).

Az algoritmus keresés része hasonló a már megismert kereséshez. A p segédváltozó felveszi a fej értéket, majd egy ciklus segítségével addig lépeget a következő elemekre (6. sor), amíg meg nem találja a törlendő elemet (a ciklus feltételek is azonosak).

Lényeges kiegészítés azonban, hogy egy e változó folyamatosan, egy elem késéssel követi a p értékét (5. sor), ezzel ez mindig a p által mutatott előtti listaelemre hivatkozik. Ez amiatt szükséges, mert amikor a p megtalálja a törlendő elemet, akkor az előzőt kell majd módosítanunk. A láncolt listában azonban nem lehet visszafele lépni, tehát ezzel a technikával az e mindig pont a p előzőjét fogja mutatni, így ez a kiláncolásnál hasznos lesz számunkra.

A ciklusból való kilépés során az p értékéből tudjuk meghatározni azt, hogy megtaláltuk-e a törlendő elemet (8. sor). Amennyiben ezen a ponton az értéke **ø**, az azt jelenti, hogy nem találtunk törlendő elemet (a lista végén áll), ha pedig egy valódi elemre hivatkozik, akkor az éppen a törlendő elem.

Amennyiben ez utóbbi feltétel teljesül, akkor az e értékéből tudunk következtetni arra, hogy az első, vagy egy későbbi elemet kell törölnünk. Amennyiben az e értéke még mindig **ø**, az azt jelenti, hogy az előző ciklus egy lépést sem tett, tehát a p-vel az első elemen állunk. Ebben az esetben a fej értékét kell módosítanunk, hogy az a jelenlegi második elemre hivatkozzon (10. sor). Ezt az esetet mutatja be a 4.8. ábra.

Amennyiben az e értéke sem **ø**, akkor a változó éppen a törlendő előtti elemre hivatkozik. Ebben az esetben a kiláncolást úgy tudjuk megvalósítani, hogy ennek az előző elemnek a *köv* hivatkozását átállítjuk a törlendő utáni elemre (12. sor). Ezzel a törlendő elemet kiláncoltuk a listából. Érdemes megfigyelni, hogy egy közbülső, illetve az utolsó elem esetében is ugyanezeket a lépéseket hajtjuk végre, és mindkét esetben helyes eredményhez jutunk (4.6d. és 4.7d. ábrák).

Bármelyik eset is következik be, törlés esetében a p éppen a törlendő elemre mutat, így a kiláncolást követően ezt az elemet felszabadíthatjuk (14. sor).

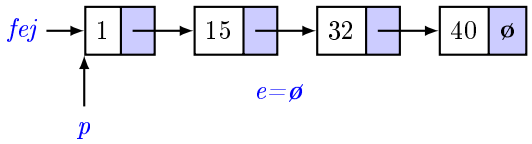
4.7. Algoritmus Láncolt lista megadott tartalmú elemének törlése

Bemenet: fej - **M**, *törlendő* - **T**
Kimenet: fej - **M**

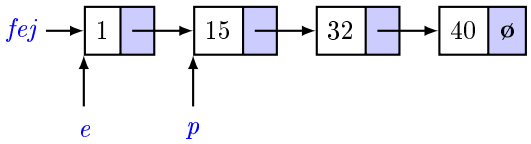
```
1:  eljárás ListábólTörlés(címszerint fej : M, törlendő : T)
2:      p ← fej
3:      e ← ø
4:      ciklus amíg (p ≠ ø) ∧ (p.tart ≠ törlendő)
5:          e ← p
6:          p ← p.köv
7:      ciklus vége
8:      ha p ≠ ø akkor
9:          ha e = ø akkor
10:             fej ← p.köv
11:         különben
12:             e.köv ← p.köv
13:         elágazás vége
14:         Felszabadít(p)
15:     különben
16:         hiba "nincs ilyen elem"
17:     elágazás vége
18: eljárás vége
```

Felhasznált változók és függvények

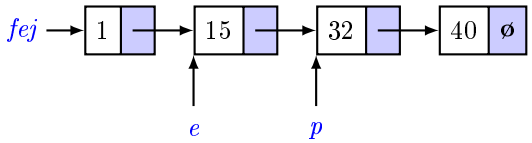
- fej: Láncolt lista feje (ami esetleg meg is változhat).
- *törlendő*: Törlendő érték. Amennyiben az érték nem szerepel a listában, akkor hibát jelez.
- Felszabadít(p : **M**): A p által mutatott elem felszabadítása.



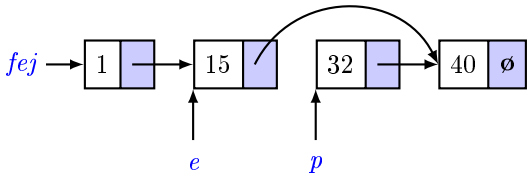
(a) Keresés első lépése. A p még nem a törlendő elemre mutat.



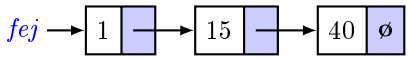
(b) Keresés következő lépése. A p még nem a törlendő elemre mutat.



(c) A következő lépés után a p a törlendő elemre mutat, az e pedig az azt megelőzőre.

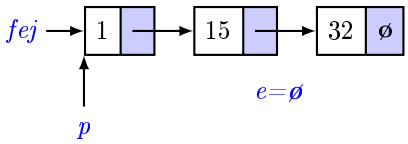


(d) Az $e.köv$ hivatkozás módosítása a törlendő utáni elemre (ezzel a törlendő kiláncolása).

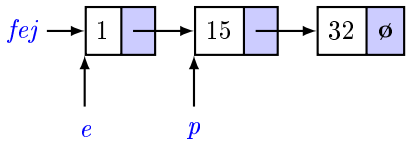


(e) A p által mutatott elem megszüntetése.

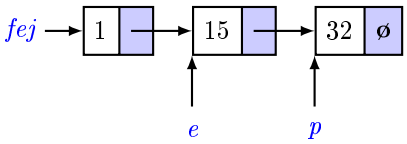
4.6. ábra. Láncolt lista középső elemének törlése.



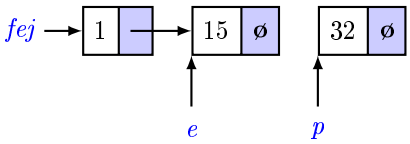
(a) Keresés első lépése. A p még nem a törlendő elemre mutat.



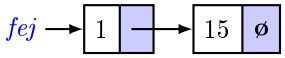
(b) Keresés következő lépése. A p még nem a törlendő elemre mutat.



(c) A következő lépés után a p a törlendő elemre mutat, az e pedig az azt megelőzőre.

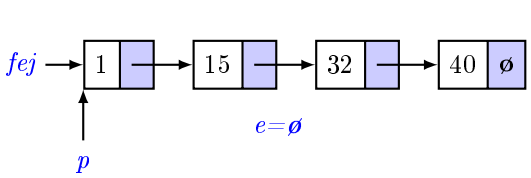


(d) Az $e.köv$ hivatkozás értékül kapja a $p.köv$ hivatkozás értékét, ami jelen esetben a lezáró ø.

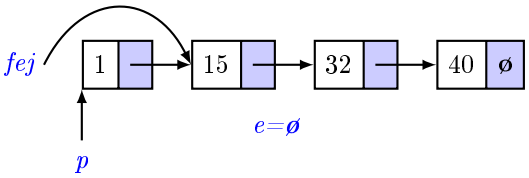


(e) A p által mutatott elem megszüntetése.

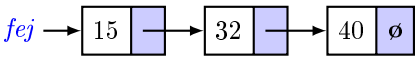
4.7. ábra. Láncolt lista utolsó elemének törlése.



(a) Keresés első lépése. A p azonnal a törlendő elemre hivatkozik.



(b) A fej mutató átléptetése a p utáni elemre.



(c) A p által mutatott elem megszüntetése.

4.8. ábra. Láncolt lista első elemének törlése.

Teljes láncolt lista törlése

Szükség lehet a teljes lista törlésére is, amely természetesen megvalósítható az előzőleg megismert, tartalom szerinti törlés többszöri meghívásával. Hatékonyabb módszer azonban az első elemek folyamatos törlése egészen addig, amíg kitöröljük az utolsó elemet. Ehhez csak egy ciklusra van szükség, amely addig törli a listát, amíg az nem üres (2. sor). A ciklusmag pedig az előzőleg megismert módon mindig törli a legelső elemet, tehát a fej mutatót átlépteti a másik elemre (4. sor), majd felszabadítja az ezzel kiláncolt elemet (5. sor).

Egy rövid példa látható a teljes lista törlésére a 4.9b. ábrán. Ezen is látható, hogy az utolsó elem törlését követően a fej mutató értéke ø lesz, tehát ezt nem szükséges külön beállítani (4.9a. ábra).

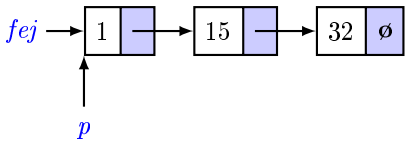
4.8. Algoritmus Láncolt lista összes elemének törlése

Bemenet: fej - **M**
Kimenet: fej - **M**

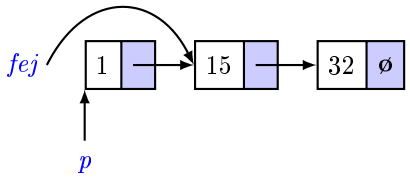
```
1: eljárás ListaTeljesTörlése(címszerint fej : M)
2:     ciklus amíg (fej ≠ ø)
3:         p ← fej
4:         fej ← fej.köv
5:         Felszabadít(p)
6:     ciklus vége
7: eljárás vége
```

Felhasznált változók és függvények

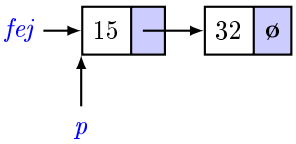
- fej: Láncolt lista feje. Az eljárás végén értéke ø lesz.
- Felszabadít(p : **M**): A p által mutatott elem felszabadítása.



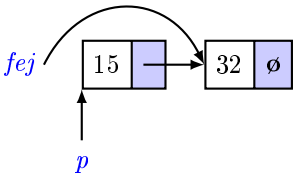
(a) A p mutató szerepe, hogy eltárol egy hivatkozást a törlendő elemre.



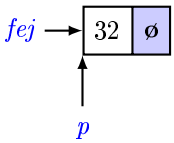
(b) A fej mutató átléptetése a p utáni elemre.



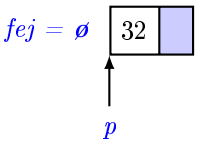
(c) A kiláncolt elem megszüntetése, majd a p beállítása az első elemre.



(d) Aktuális elem kiláncolása.



(e) Újabb elem megszüntetés, majd a p beállítása az első elemre.



(f) A fej felveszi az első elem *köv* mezőjének értékét, aminek az értéke ø.

$fej = ø$

(g) Az utolsó elem felszabadítása után az eredmény egy üres láncolt lista.

4.9. ábra. Teljes láncolt lista törlése.

4.3. Rendezett láncolt lista

4.3.1. Rendezett láncolt lista felépítése

Tömbökhöz hasonlóan a láncolt listákat is rendezhetjük. Ahhoz, hogy a rendezettséget egyszerűen tudjuk kezelni, kibővítjük a lista elemeket egy új, kulcs fogalommal. Ennek megfelelően a lista elemét az alábbiak alkotják:

- Tartalmi rész: Ez a rész tartalmazza magát, a listaelemben eltárolni kívánt adatot. Nincs semmiféle megkötésünk ennek típusára, lehet szám, szöveg, vagy akár tetszőleges objektum típus is. Azt általában kikötjük, hogy az egyes listaelemek mind azonos típusú adatokat tartalmazzanak, de OOP környezetben persze erre sincs feltétlenül szükség. A példákban ezt a mezőt mindig *tart* néven fogjuk használni, típusát pedig **T**-vel fogjuk jelölni.

- Kulcs rész: A rendezett láncolt listában ez alapján tudjuk az adatokat eltárolni, illetve visszakeresni. Általában feltételezzük, hogy egyedi, bár a legtöbb algoritmusunk működni fog azonos kulcsokkal is. Ennek típusa a tartalomhoz hasonlóan tetszőleges lehet, azonban lényeges megkötés, hogy összehasonlíthatónak kell lennie. A tényleges megvalósítást tekintve ez a kulcs lehet egy tartalomtól teljesen független mező, de lehet maga a tartalom (ha pl. csak számokat tárolunk a listában, akkor a kulcs lehet egyben a tartalom is), vagy akár a tartalomnak egy része is (pl. személyek adatait tartalmazzuk a listában, és a kulcs ezek személyi száma). A példákban mindig egy külön mezőnek tekintjük a kulcsot, és ezt *kulcs* néven fogjuk elérni, típusát pedig **K**-val fogjuk jelölni. Az ábrákon a könnyebb olvashatóság érdekében a kulcsot és a tartalmat nem különböztetjük meg.

- Következő hivatkozás rész: A lista minden eleme tartalmaz egy hivatkozást a listában rákövetkező elemre. A legelső elemtől elindulva így (közvetve, a közbülső elemeken keresztül) elérhetjük a lista bármelyik elemét. A mező típusa a lista megvalósításától függően többféle lehet, erre még a későbbi, implementációs résznél visszatérünk (addig is a legegyszerűbb a lista elemeit objektumként, a következő mezőt pedig objektum referenciaként felfogni). A példákban ezt a mezőt mindig *köv* néven fogjuk használni, típusát pedig **M**-el fogjuk jelölni.

4.3.2. Beszúrás rendezett láncolt listába

Egyszerű, de hosszabb változat

Tömbök esetében többnyire azzal foglalkoztunk, hogy hogyan lehet egy már feltöltött, de rendezetlen tömböt helyben rendezni (esetleg bizonyos megvalósításoknál, pl. a beillesztéses rendezésnél megemlítettük, hogy az akkor is nagyon hatékonyan használható, ha már a tömb feltöltésekor futtatjuk az algoritmust). Láncolt listák esetében is járható ez az út, de itt célszerű lehet kihasználni azt a tényt, hogy nagyon könnyen tudunk két meglévő elem közé egy újat felvenni, emiatt célszerű lehet már a lista felépítésekor az új elemeket a rendezésnek megfelelő helyre beszúrni.

Könnyen belátható, hogy a többi művelet változatlanul hagyható, hiszen ha a beszúrás során ügyelünk az elemek rendezésére, akkor az a későbbiekben is rendezett marad a lista. A keresés/bejárás nem változtatja meg az elemek tartalmát, a törlés során pedig szintén nem változhat meg az elemek sorrendje. Módosítással nem foglalkozunk, de értelemszerűen, ha a módosítás során megváltozik a tartalom, az nem befolyásolja a kulcs szerinti rendezettséget (a kulcsot pedig tekintsük úgy, hogy nem változtatható).

A rendezett láncolt lista felépítésére két algoritmust is megvizsgálunk. Mindkét módszer kimenete teljesen azonos, vizsgálatuk oka pusztán didaktikai, az első egyszerűbben áttekinthető, a második viszont rövidebb és elegánsabb megoldás.

Az első lehetőséget mutatja a 4.9. algoritmus. Ebben négy, egymástól jól elkülöníthető programág felel a felmerülő lehetőségek kezeléséért:

- Üres listába való beszúrás: Itt a beszúrandó elem lesz a lista első és egyben egyetlen eleme.

- Első elem elé beszúrás: Ebben az esetben a beszúrandó elem kulcsa kisebb, mint a listában lévő bármelyik másik kulcs, emiatt az új elem lesz a lista első eleme.

- Két elem közé beszúrás: Ezt tekinthetjük a rendezett listába beszúrás általános esetének. Meg kell keresnünk azt az elempárt, ahol az egyik kulcsa kisebb (vagy egyenlő), a másiké viszont már

nagyobb (vagy egyenlő), mint a beszúrandó kulcs. Ilyenkor ez a két elem közé kell elhelyezni az új elemet.

- Utolsó elem mögé beszúrás: Amennyiben az újonnan beszúrandó elem kulcsa nagyobb a lista minden már meglévő kulcsánál, akkor az új tartalmat tároló elemet az eddigi utolsó elem után kell láncolni.

Az algoritmus felépítése is a fenti eseteket követi. A 2–4. sorok létrehoznak egy új elemet, és ebbe elhelyezik a beszúrandó kulcsot és tartalmat.

Az ezt követő ellenőrzés (5. sor) megvizsgálja, hogy üres-e a lista. Amennyiben igen, akkor a legegyszerűbb módon felveszi ezt az elemet elsőként. A fej mutató az új elemre hivatkozik, annak *köv* mezőjének értéke pedig **ø** lesz, hiszen ő egyben az utolsó elem is.

A következő vizsgálatra (9. sor) már csak akkor kerül sor, ha nem üres a lista. Ebben az esetben meg lehet vizsgálni a lista első elemének kulcsát, hogy az nagyobb-e, mint a beszúrandó. Ha igen, akkor az új elemnek még a lista első eleme elé kell bekerülnie, erre jól használható a már megismert *elejére beszúrás* művelet (4.2.2. alfejezet).

Minden egyéb esetben egy ciklust kell indítanunk, amelyik megkeresi azt a két elemet, amelyek közé be kell láncolni az újat (15. sor). Ehhez elég megtalálnunk az első, nála nagyobb kulccsal rendelkezőt. Az ez előtti elem lesz az őt közvetlenül megelőző. Kivételes esetnek tekinthető, ha a beszúrandó elemnél nincs nagyobb a listában, emiatt a ciklus egy kiegészítő feltételt is tartalmaz, hogy csak addig fusson, amíg a p nem jutott a lista végére. A beszúráshoz szükségünk lesz az előző elemre, és mivel a láncolt listában nem tudunk visszafelé lépni, ezért a törlésnél már megismert módon, egy e nevű segédváltozóval a cikluson belül mindig eltároljuk a p előző értékét.

A két kilépési feltétel közül valamelyik biztosan teljesülni fog, akkor kilépünk a ciklusból. A 19. sor szerepe, hogy megvizsgálja, melyik feltétel miatt léptünk ki a ciklusból. Amennyiben a p értéke ezen a ponton **ø**, az azt jelenti, hogy végignéztük a listát, és nincs az újnál nagyobb kulcsú elem. Amennyiben egy valódi elemre mutat, akkor az nem más, mint a listában a beszúrandónál nagyobb kulcsot tartalmazó elemek közül az első.

Az előbbi esetben a lista végére beszúrás algoritmusnál már megismert módszerrel felvesszük az új elemet a lista végére (20–21. sorok).

A másik esetben pedig a lista n. helyére beszúrás algoritmusnál már megismert módszerrel felvesszük az új elemet az e és a p által hivatkozott listaelemek közé (23–24. sorok).

4.9. Algoritmus Rendezett láncolt listába új elem beszúrása

Bemenet: fej - **M**, *kulcs* - **K**; ahol **K** összehasonlítható, *érték* - **T**
Kimenet: fej - **M**

```
1:  eljárás RendezettListábaBeszúrás(címszerint fej : M, kulcs : K, érték : T)
2:      új ← Létrehoz(M)
3:      új.kulcs ← kulcs
4:      új.tart ← érték
5:      ha fej = ø akkor                                          ▷ Üres lista
6:          új.köv ← ø
7:          fej ← új
8:      különben
9:          ha fej.kulcs > kulcs akkor                            ▷ Első elem elé
10:             új.köv ← fej
11:             fej ← új
12:         különben
13:             p ← fej
14:             e ← ø
15:             ciklus amíg (p ≠ ø) ∧ (p.kulcs < kulcs)
16:                 e ← p
17:                 p ← p.köv
18:             ciklus vége
19:             ha p = ø akkor
20:                 új.köv ← ø                                    ▷ Utolsó elem mögé
21:                 e.köv ← új
22:             különben
23:                 új.köv ← p                                    ▷ Két elem közé
24:                 e.köv ← új
25:             elágazás vége
26:         elágazás vége
27:     elágazás vége
28: eljárás vége
```

Felhasznált változók és függvények

- fej: Kulcs szerint rendezett láncolt lista feje.
- *kulcs*: Beszúrandó elem kulcsa.
- *érték*: Beszúrandó tartalom.
- Létrehoz(**M**) : **M**: Létrehoz egy **M** típusú elemet, ez a visszatérési értéke.

Összevont, rövidebb változat

Az előző fejezetben megadott algoritmus tökéletesen működik, helyesen kezeli az összes felmerülő lehetőséget. Azonban meglehetősen hosszú, és érdemes észrevenni, hogy bizonyos programágak gyakorlatilag ugyanazt a kódot tartalmazzák. Ezek alapján az alábbi egyszerűsítési lehetőségekkel élhetünk (a sorszámok a 4.9. algoritmusra vonatkoznak):

- Az nyilvánvaló, hogy a 7. és a 11. sorok tulajdonképpen ugyanazt hajtják végre (mivel a kettő között nem változik sem a fej, sem pedig a *új* értéke). Azonban megfigyelhető, hogy a felettük lévő a 6. és a 10. soroknak is mindig ugyanaz lesz az eredményük (a $fej = \text{ø}$ feltétel teljesülése miatt a 6. sorban található ø értéket helyettesíthetjük a fej-jel, így már formailag is ugyanaz a két sor.

- Hasonló módon látható, hogy a 21. és a 24. sorok is teljesen azonosak. Itt szintén megfigyelhető, hogy a felettük lévő a 20. és a 23. soroknak is mindig ugyanaz lesz az eredményük (a $p = \text{ø}$ feltétel teljesülése miatt a 20. sorban található ø értéket helyettesíthetjük a p változóval, így már formailag is ugyanaz a két sor.

Ezek a sorok tehát összevonhatók, így az előzetesen felvázolt négy esetet le tudtuk egyszerűsíteni két felmerülő esetre. A beszúrás algoritmusát átgondolva egyébként is belátható, hogy tulajdonképpen tényleg csak kétféle esetet kell megkülönböztetnünk:

- A beszúrandó elem közvetlenül a fej mögé kerül (tehát üres listába, vagy az első elem elé akarunk beszúrni).

- A beszúrandó elem egy már meglévő elem mögé kerül (tehát két elem közé, vagy az utolsó elem mögé akarunk beszúrni).

Hasonló két ággal már találkozhattunk a törlés algoritmusánál is (4.2.4. alfejezet). Mivel ott is szükség volt egy előző elemre, a ciklus egy p és egy e változó értékeit módosította, majd a kilépést követően az e értéke alapján tudtuk eldönteni, hogy történt-e lépés, vagy sem (tehát az első elemet kell törölni, vagy egy későbbit). Ugyanezt a technikát itt is felhasználhatjuk, a ciklusból való kilépés után az e változó értéke megadja, hogy a fej változót kell módosítani, vagy pedig egy előző elem *köv* hivatkozását.

A fenti összevonásokkal nyert a 4.10. algoritmus első lépése ugyanúgy az új elem létrehozása, és a tartalom kitöltése. Az ezt követő p és e változók inicializálást követően lefut egy, a törléshez hasonló ciklus (7. sor), amely segítségével megkeressük az első, a beszúrandónál nagyobb kulcsot tartalmazó elemet (közben persze ügyelve arra is, hogy a láncolt lista végére jutva is kilépjünk a ciklusból).

A 11. sor szerepe, hogy meghatározza a szükséges beszúrás módját. Ha az e változó értéke ø, akkor a ciklusmag egyszer sem futott le, tehát vagy üres a lista (4.10a. ábra), vagy már az első elem is nagyobb volt a beszúrandónál (4.11a. ábra). Mindkét esetben közvetlenül a fej után kell felvenni az új elemet, ezt végzik el a 12–13. sorok).

Ha az e egy valódi elemre mutat, akkor az nem más, mint a beszúrandónál közvetlenül kisebb kulcsot tároló elem. A beszúrást ebben az esetben a 15–16. sorok végzik el. A közvetlenül előtte lévő elem *köv* mezője az új elemre mutat, annak *köv* mezője pedig felveszi a p értékét. Ez utóbbi abban az esetben is jó eredményt ad vissza, ha a p egy létező elemre mutat (tehát két elem közé szúrunk be, ezt mutatja a 4.13a. ábra), illetve akkor is, ha annak értéke ø (tehát az utolsó elem mögé szúrunk be, ezt mutatja a 4.12a. ábra). Ez utóbbi esetben az új elem *köv* mezőjének tartalma ø lesz, ami megfelel annak, hogy ő lett a lista utolsó eleme.

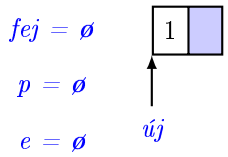
4.10. Algoritmus Rendezett láncolt listába új elem beszúrása (rövidebb változat)

Bemenet: *fej* - **M**, *kulcs* - **K**; ahol **K** összehasonlítható, *érték* - **T**
Kimenet: *fej* - **M**

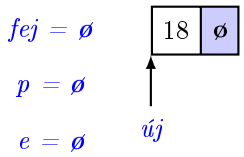
```
1:  eljárás RendezettListábaBeszúrás(címszerint fej : M, kulcs : K, érték : T)
2:      új ← Létrehoz(M)
3:      új.kulcs ← kulcs
4:      új.tart ← érték
5:      p ← fej
6:      e ← ø
7:      ciklus amíg (p ≠ ø) ∧ (p.kulcs < kulcs)
8:          e ← p
9:          p ← p.köv
10:     ciklus vége
11:     ha e = ø akkor
12:         új.köv ← fej
13:         fej ← új
14:     különben
15:         új.köv ← p
16:         e.köv ← új
17:     elágazás vége
18: eljárás vége
```

Felhasznált változók és függvények

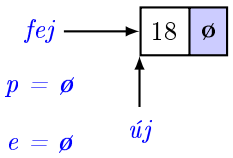
- *fej*: Kulcs szerint rendezett láncolt lista feje (ami esetleg meg is változhat).
- *kulcs*: Beszúrandó elem kulcsa.
- *érték*: Beszúrandó tartalom.
- Létrehoz(**M**) : **M**: Létrehoz egy **M** típusú elemet, ez a visszatérési értéke.



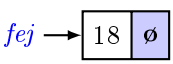
(a) Üres listába beszúrás esetén a változók állapota a ciklus után.



(b) Fej értékének átmásolása.

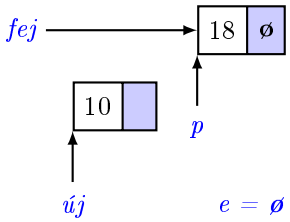


(c) Fej értékének beállítása.

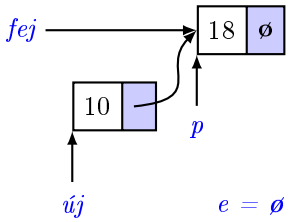


(d) Beszúrás eredménye.

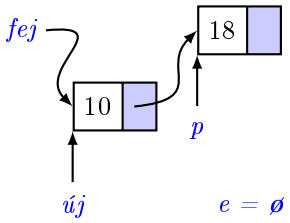
4.10. ábra. Első elem beszúrása üres rendezett láncolt listába. Az új elem tartalma: 18.



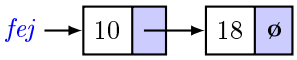
(a) Első elem elé beszúrás esetén a változók állapota a ciklus után.



(b) Fej értékének átmásolása.

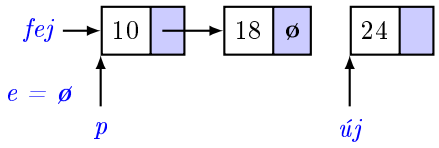


(c) Fej értékének beállítása.

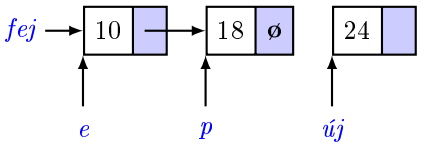


(d) Beszúrás eredménye.

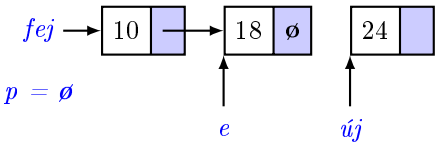
4.11. ábra. Első elem elé beszúrás rendezett láncolt listába. Az új elem tartalma: 10.



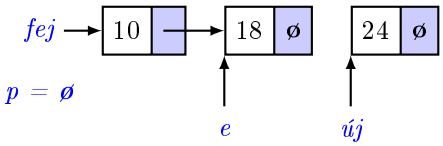
(a) A keresés első lépése, az első elem vizsgálata.



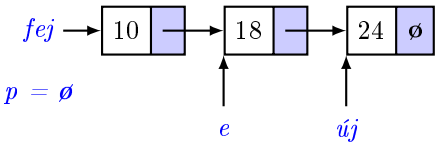
(b) Továbblépés a második elemre. Az e követi a p változót.



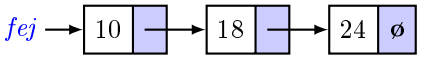
(c) A beszúrandó érték nagyobb, mint az utolsó elem, emiatt újabb lépés.



(d) A p érték átmásolása az *új* elem *köv* mezőjébe.

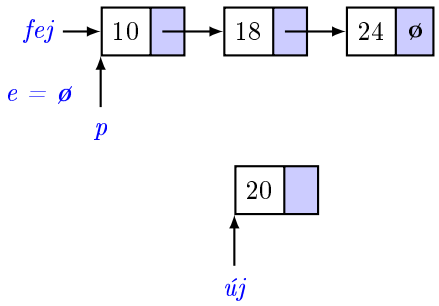


(e) Az előző elem átirányítása az újonnan beszúrt elemre.

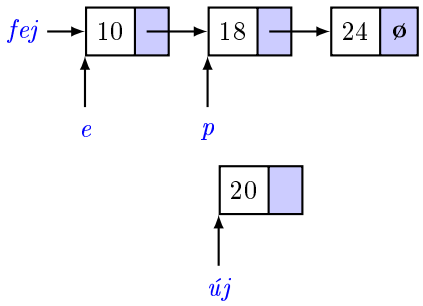


(f) A beszúrás végeredménye.

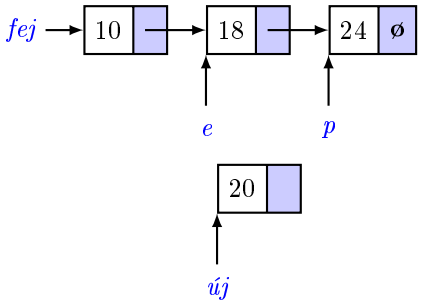
4.12. ábra. Utolsó elem mögé beszúrás rendezett láncolt listába. Az új elem tartalma: 24.



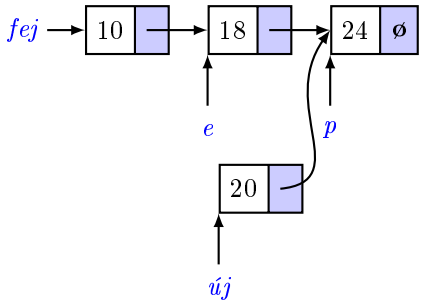
(a) A keresés első lépése, az első elem vizsgálata.



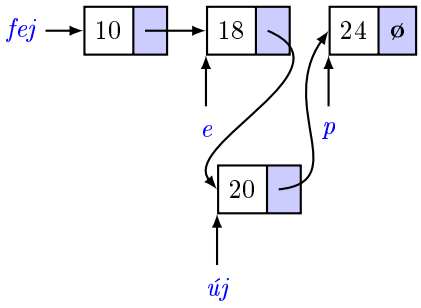
(b) Továbblépés a második elemre. Az e követi a p változót.



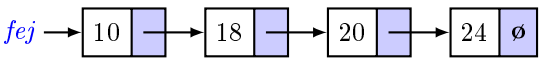
(c) A p már egy nagyobb elemen áll, emiatt kilépés a ciklusból.



(d) A p érték átmásolása az *új* elem *köv* mezőjébe.



(e) Az előző elem átirányítása az újonnan beszúrt elemre.



(f) A beszúrás végeredménye.

4.13. ábra. Két elem közé beszúrás rendezett láncolt listában. Az új elem tartalma: 20.

4.3.3. Keresés rendezett láncolt listába

Adatszerkezeteket többféle ok miatt is rendezhetünk, ezek közül az egyik leggyakoribb a gyorsabb keresés biztosítása. A hagyományos programozási tételek között is jól látható az egyszerű, nem rendezett tömbön is működőképes „lineáris keresés" és a rendezett tömböket igénylő „logaritmikus keresés" közötti különbség az utóbbi javára. A láncolt listák esetében a rendezettség szerepe kevésbé markáns, hiszen az előbb említett logaritmikus keresés algoritmusa itt nem valósítható meg hatékonyan. Ez annak köszönhető, hogy ez a keresés erősen támaszkodik az elemek gyors véletlen elérésére, ami pl. a tömbök esetében egyszerűen megvalósítható, hiszen a keresett elem indexe alapján egy egyszerű szorzással és összeadással megállapítható az elem memóriabeli címe. A láncolt listák esetében azonban az adatszerkezet felépítéséből adódóan, erre nincs mód, hiszen a lista n. elemét csak a legelső elemtől induló $(n-1)$ darab lépés segítségével tudjuk elérni. Emiatt, hiába tudjuk, hogy a lista elemei rendezettek, a logaritmikus keresés lépései (ami első körben a középső elemet olvasná be, ami jelen esetben $n/2$ darab lépést igényelne) itt nem kivitelezhetőek hatékonyan.

Valamelyest azonban a lineáris keresés is gyorsítható a rendezettség kihasználásával, hiszen ez egy új kilépési feltételt jelenthet a kereső ciklusból. Ennek megfelelően a keresés addig fut, amíg az alábbiak közül valamelyik bekövetkezik:

- Megtaláltuk a keresett elemet (tehát benne van a listában, és vissza tudunk adni rá egy hivatkozást).
- Eljutottunk a lista végére, és nem találtuk meg a keresett elemet (tehát nincs benne a listában).
- A keresés során egy nála nagyobb elemre léptünk, ami a rendezettség miatt szintén azt jelenti, hogy a keresett elem nincs a listában (mivel a lista további elemei már csak az aktuálisnál is nagyobb elemeket tartalmazhatnak). Ezt mutatja a 4.14a. ábra.

Tömbök esetében a lineáris keresés módosítása a ciklusfeltételt, illetve a visszatérési érték kiszámításának módját érintette. Láncolt lista esetében ehhez hasonlóan szintén kiindulhatunk az alapvető keresési algoritmusból (4.6. algoritmus), azt csak két ponton kell megváltoztatnunk.

A 4.11. algoritmus tartalmazza ezeket a módosításokat. A 3. sorban látható a ciklusfeltétel változása. A lista végének ellenőrzése nem változott, a második részfeltételben azonban az egyenlőség vizsgálata helyett itt már egy kisebbségi vizsgálat szerepel. A ciklus tehát csak addig fog futni, amíg a p által hivatkozott elem kulcsa kisebb a keresettnél. Mind egyenlőség (megtaláltuk), mind pedig nagyobb elemek esetében (biztos nem lesz meg) kilépünk a ciklusból.

Nem rendezett esetben elég volt a p hivatkozás értéke annak meghatározásához, hogy megtaláltuk-e az elemet vagy sem. Amennyiben a hivatkozás értéke $\emptyset$, akkor az a módosított algoritmusban is azt jelenti, hogy a keresett elem nincs benne a listában. Amennyiben azonban a változó egy valódi elemre mutat, az már további vizsgálatokat igényel. Nem rendezett listában ugyan már ekkor biztos volt, hogy a keresett elemet megtaláltuk, és a p éppen ide mutat, most azonban foglalkoznunk kell azzal a lehetőséggel is, hogy a változó már az első, a keresettnél nagyobb elemen áll. Ezt a kérdést válaszolja meg a 6. sorban található feltétel második fele. Ennek köszönhetően a *van* változó értéke (tehát a függvény visszatérési értéke) csak akkor lesz *igaz*, ha a p egy létező elemre mutat, és annak a kulcsa éppen megegyezik a keresettel.

Megjegyzés

A láncolt listák esetében kevésbé tudjuk kihasználni a rendezettséget, mint a tömbök esetében, ettől függetlenül néha jó hasznát vehetjük ennek a tárolási módnak:

- A keresésnél némi gyorsulást így is sikerült elérnünk. Ez akkor jelenik meg, ha olyan tartalmat keresünk, ami nincs a listában, és emiatt már az első nála nagyobb elemnél kilépünk a ciklusból. Amennyiben gyakoriak az olyan keresések, amelyek nem létező elemekre vonatkoznak, akkor ez a különbség lényeges lehet.
- A keresésen túl olyankor is hasznos lehet a rendezettség, amikor a listában lévő adatokat valamilyen sorrendben szeretnénk kinyerni. Például, ha a listában személyi adatokat tárolunk, és különféle szempontok szerint szeretnénk ebből adatokat kiválogatni úgy, hogy azok ABC sorrendben legyenek. Itt célszerű magát a listát rendezni, így a kiválogatások eleve ebben a sorrendben adják majd meg az eredményeket (és nem kell minden válogatás után egy rendezést is végrehajtani).

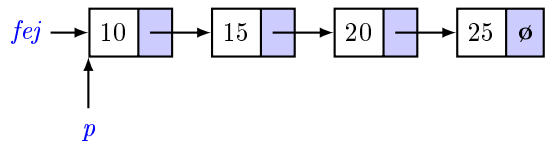
4.11. Algoritmus Megadott tartalmú elem keresése rendezett láncolt listában

Bemenet: *fej* - **M**, *keresett* - **K**; ahol **K** összehasonlítható
Kimenet: *van* - **logikai**, *vissza* - **T**

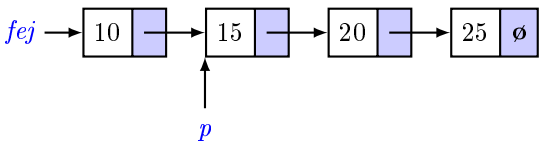
```
1:  függvény RendezettListábanKeresés(fej : M, kulcs : K)
2:      p ← fej
3:      ciklus amíg (p ≠ ø) ∧ (p.kulcs < kulcs)
4:          p ← p.köv
5:      ciklus vége
6:      van ← (p ≠ ø) ∧ (p.kulcs = kulcs)
7:      ha van akkor
8:          vissza (van, p.tart)
9:      különben
10:         vissza van
11:     elágazás vége
12: függvény vége
```

Felhasznált változók és függvények

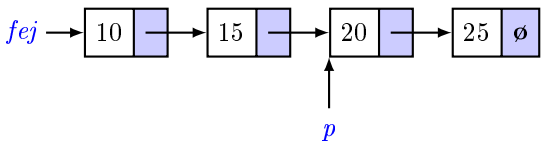
- *fej*: Egy tartalom szerint rendezett láncolt lista feje.
- *kulcs* : A keresett kulcs.
- *van* : Visszatérési értéke mutatja, hogy a lista tartalmaz-e keresett tartalmú elemet?
- *vissza* : Amennyiben a *van* változó értéke igaz, akkor a keresett elem tartalmát tartalmazza. Egyébként nem értelmezett.



(a) Első elem vizsgálata. Értéke kisebb, mint a keresett.



(b) Második elem vizsgálata. Értéke kisebb, mint a keresett.



(c) A *p* már a keresettnél nagyobb elemre mutat, így nincs értelme tovább futtatni a keresést.

4.14. ábra. Keresés rendezett láncolt listában, a keresett elem: 18.

4.4. Láncolt lista strázsa elemekkel

4.4.1. Beszúrás és törlés helyi műveletekkel

A különböző láncolt lista műveletek áttekintése során feltűnő lehetett, hogy sok esetben maga a művelet általános esete egyszerűen kezelhető lett volna (pl. elem beszúrása két másik elem közé), azonban a különféle kivételes esetek miatt a végeredmény egy jóval összetettebb, hosszabb algoritmus lett (pl. üres lista, első elem elé beszúrás, utolsó elem mögé beszúrás, stb.). Ez nem pusztán hosszabb programkódot eredményez, de a program futásideje szempontjából se szerencsés (pl. még több ezer elem beszúrása után is mindig meg kell vizsgálnunk, hogy a lista nem üres-e véletlenül).

A kivételes esetek elhagyásával néhány nagyon egyszerű és rövid megoldást is tudunk használni, ezek közül át is tekintünk kettőt.

Mutatott elem elé beszúrás

A mutatott elem elé beszúrás algoritmusnak átadott paraméterek jelentősen különböznek az eddig megismert beszúrásoktól. Itt ugyanis nem a lista *fej* elemét adjuk át, hanem csak egy hivatkozást arra az elemre, amely elé be szeretnénk szúrni. Ez egyrészt azt jelenti, hogy nem férünk hozzá a fejhez (tehát az eddig megismert módszerek, amelyeknél a fejtől kiindulva egyesével lépegetve kerestük meg a szükséges helyet, itt már nem működnek), másrészt nem is tudjuk módosítani a *fej* értékét.

A 4.15. ábrán látható módon azonban a beszúrás így is működik a legtöbb esetben. Egy elem elé beszúrást eddig mindig úgy képzeltünk el, hogy mindenképpen szükségünk van az azt megelőző elemre is, hiszen a beláncolásnál annak a *köv* hivatkozását is át kell majd irányítani az új elemre. Itt azonban ez nem szükséges, hiszen látható, hogy az új elemet valójában a meglévő után helyezzük el a láncba (4.15c–4.15e. ábrák), a sorrendet pusztán azzal az ötlettel állítjuk helyre, hogy az eddigi p által mutatott elem tartalmát átmásoljuk az új elembe, és a régi tartalmat írjuk felül az új értékkel (4.15f. ábra).

Látható, hogy a beszúrás így jóval egyszerűbben megvalósítható, habár néhány mellékhatással is számolnunk kell.

- A legnagyobb problémánk, hogy üres listába a legelső elem beszúrásánál ez nem fog működni. Itt ugyanis nincs semmilyen elem, amit át tudnánk másolni, így ezt a speciális esetet valamilyen formában mindenképpen kezelnünk kell.
- Ez a módszer nem használható akkor, ha a lista elemeire külső hivatkozások is megengedettek. Ez ugyanis kimondottan hátrányos mellékhatásokhoz vezet, ha éppen a külső változó által hivatkozott elem elé szúrunk be, hiszen ilyenkor a hivatkozás továbbra is ugyanarra a lista elemre mutat, azonban ott már más tartalom fog elhelyezkedni.

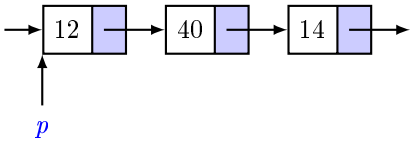
4.12. Algoritmus Mutatott elem elé beszúrás

Bemenet: *p* - **M**, *érték* - **T**

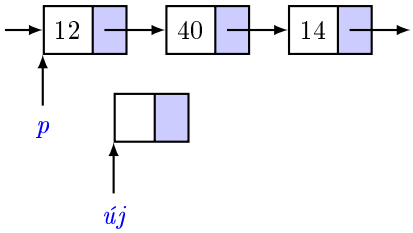
```
1: eljárás MutatottElemEléBeszúrás(p : M, érték : T)
2:     új ← Létrehoz(M)
3:     új.tart ← p.tart
4:     új.köv ← p.köv
5:     p.köv ← új
6:     p.tart ← érték
7: eljárás vége
```

Felhasznált változók és függvények

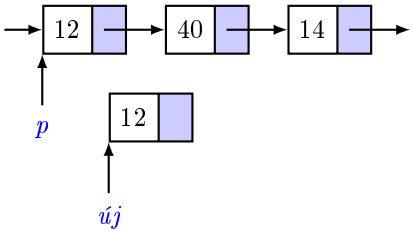
- *p*: Hivatkozás egy már létező listaelemre, amely elé be kell szúrni az új elemet.
- *érték*: Beszúrandó tartalom.
- Létrehoz(**M**) : **M**: Létrehoz egy **M** típusú elemet, ez a visszatérési értéke.



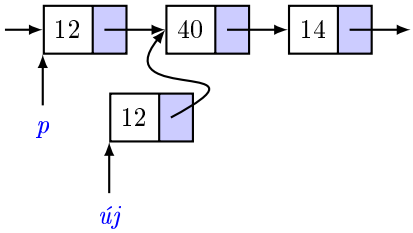
(a) A lista p által mutatott eleme elé kell beszúrni az új elemet.



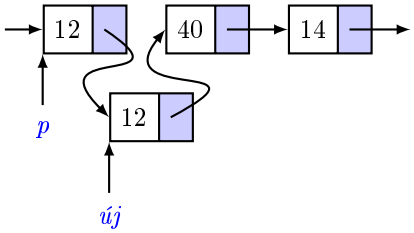
(b) Az *új* elem létrehozása.



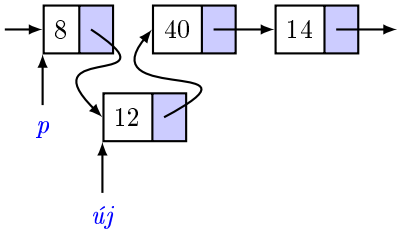
(c) Tartalom átmásolása az *új* elembe.



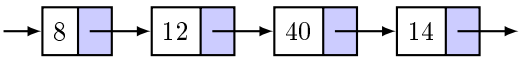
(d) Következő hivatkozás átmásolása az *új* elembe.



(e) Az *új* elem beláncolása.



(f) A beszúrandó tartalom bemásolása a p által hivatkozott elembe.



(g) A beszúrás végeredménye.

4.15. ábra. Mutatott elem elé beszúrás. A példában a 8 értéket szúrjuk be a lista p által mutatott eleme elé.

Mutatott elem törlése

Az előzőleg megismert ötlet alapján már könnyen kivitelezhető a mutatott elem törlése is. Ennek az algoritmusnak ugyanaz az előnye, nincs hozzá szükségünk a *fej* mutató értékére, az algoritmus csak azt a mutatót kapja meg paraméterként, amit törölnünk kell. Az eddigi megoldásaink során ez azért okozott volna problémát, mert a kiláncoláshoz szükségünk volt a megelőző elemre (annak is módosítani kell a *köv* hivatkozását), ezt azonban csak a p ismeretében nem tudjuk megtalálni.

Látható, hogy a 4.13. algoritmus a legtöbb esetben enélkül is működik, felhasználva azt az ötletet, hogy a törlendő elem adatait felülírjuk a mögötte lévő elem adataival (3–4. sorok), majd ezt követően a mögötte lévő elemet láncoljuk ki (ez már könnyen megvalósítható, hiszen ismerjük az őt megelőző elemet, így nem kell azt külön megkeresni). A módszer nagyon hatékony és elegáns, azonban néhány kivételes esetben sajnos ez sem működik megfelelően:

- A leglátványosabb hiányossága az utolsó elem törlésekor tapasztalható. Mivel nincs már mögötte elem, ahonnan adatokat lehetne átmásolni, így a legutolsó elem törlésére ez a módszer nem használható.
- Ha a listánkban már csak egy elem van, és ezt szeretnénk törölni, akkor szintén csődöt mond ez a megoldás (bár ez felfogható az előző probléma egy speciális esetének is, hiszen az egyetlen elem egyben nyilván az utolsó is).
- Itt szintén igaz, hogy ez a módszer nem használható akkor, ha a lista elemekre külső hivatkozások is megengedettek.

4.13. Algoritmus Mutatott elem törlése

Bemenet: p - **M**

```
1: eljárás MutatottElemTörlése(p : M)
2:     q ← p.köv
3:     p.tart ← q.tart
4:     p.köv ← q.köv
5:     Felszabadít(q)
6: eljárás vége
```

Felhasznált változók és függvények

- p: Hivatkozás egy már létező listaelemre, amit ki kell törölni (nem lehet a lista utolsó eleme).
- Felszabadít(q : **M**): A q által mutatott elem felszabadítása.

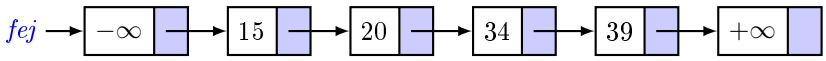
4.4.2. Strázsa elemek használata

A felmerülő problémák ellenére azonban célszerű alaposabban megvizsgálnunk ezeket a módszereket. Mindkét algoritmusnál csak a kivételes esetek jelentik a problémát (első elem, egyetlen elem, utolsó elem, stb.), általános esetben (amikor egy lista egy belső eleméről beszélünk) azonban jól használhatóak. Ezen alapul a strázsa technika alapötlete, hogy próbáljuk meg egy olyan láncolt lista megvalósítást létrehozni, amelyben nincsenek ilyen speciális esetek.

A hagyományos listát úgy alkottuk meg, hogy a *fej* mutató közvetlenül az első elemre mutat, üres lista esetében ennek az értéke pedig a lezáró **ø** érték. Ugyanezt használtuk a lista végének jelzésére, ezért az utolsó elem következő mezőjébe is ezt helyeztük.

Ehelyett készítsünk egy olyan láncolt listát, ahol mindig van egy-egy listaelem az első valós adatot tároló elem előtt, illetve után is. Ezeket nevezzük a későbbiekben *strázsa* (sentinel) elemeknek. Ennek megfelelően az üres, tehát értékes adatot nem tartalmazó listát is úgy képzeljük el, hogy már van benne két lista elem (4.16. ábra).

A strázsa elemek felépítésüket tekintve pont ugyanolyanok, mint bármelyik másik listaelem, valahogy azonban jó lenne megkülönböztetni őket a többi elemtől. Erre használhatnánk egy új mezőt is, de az megnövelné a teljes lista helyfoglalását, ezért ehelyett, (amennyiben annak értékkészlete ezt megengedi) használhatjuk a *tart* mezőt, amibe olyan értékeket töltünk, amelyek egyértelműen jelzik a két elem speciális voltát. Ha ez nem megoldható, akkor is rá lehet jönni, hogy melyik elemek a strázsák, hiszen az



4.16. ábra. Láncolt lista strázsa elemekkel.

egyikre mindig a fej mutat, a másiknak pedig a *köv* mezőjének értéke **ø**, ezek egyike se lehet igaz más listaelem esetében.

A *tart* mezőbe tehát olyan értékeket töltünk, amelyek más elemekre biztosan nem igazak. Amennyiben rendezett a listánk, akkor az első strázsába célszerűen az értékkészlet legkisebb elemét töltjük (jelölése: $-\infty$), az utolsó strázsába pedig az értékkészlet legnagyobb elemét (jelölése: $+\infty$). Ezek konkrét értéke attól függ, hogy milyen típusú a tartalom, számok esetén az adott típus legkisebb/legnagyobb értéke lehet, szövegek esetén ASCII(0), stb. Ha sikerül ilyen értékeket választanunk, az azért is hatékony, mivel rendezett láncolt lista esetén, a rendezettség szempontjából nem kell külön kezelnünk a strázsa elemeket, azok teljesen illeszkednek a valós tartalmat tároló elemek közé. Ennek köszönhetően pl. beszúrás esetén mindig élhetünk azzal a feltételezéssel, hogy találni fogunk egy olyan elempárt, amelyek egyike kisebb a beszúrandónál, a másik pedig nagyobb:

- Ha az eddigieknél kisebb elemet akarunk beszúrni, akkor ez az elempár az első strázsa és az eddigi legkisebb elem lesz.
- Ha az eddigieknél nagyobb elemet akarunk beszúrni, akkor ez az elempár az eddigi legnagyobb és a mögötte lévő strázsa lesz.
- Ha pedig az eddigi legkisebb és legnagyobb közötti tartalmat szúrunk be, akkor értelemszerűen találni fogunk két, a fenti feltételnek megfelelő elemet.
- Üres lista esetén is létezik a két strázsa elem, tehát ilyenkor ezek közé fog kerülni az új érték.

A konkrét pszeudokód bemutatása nélkül is nyilvánvaló, hogy ezzel pl. a rendezett beszúrást jelentősen leegyszerűsíthetjük, hiszen a négyféle lehetőség közül (üres lista, első elem elé, két elem közé, utolsó elem mögé) már csak eggyel kell foglalkoznunk. Ugyanez az előny látható a törlésnél is.

A strázsa technika egyetlen hátránya a nagyobb helyfoglalás, viszont ez sem tekinthető jelentősnek, hiszen csak két új elem felvételét jelenti, tehát a lista méretétől függetlenül konstans lesz a helyfoglalási többlet (ami értelemszerűen sokkal jobb, mintha minden lista elemnél növelnünk kellett volna a helyfoglalást).

4.5. Speciális láncolt listák

4.5.1. Kétirányú láncolt lista

Az eddigiek során mindig egyirányú listákkal foglalkoztunk, tehát minden elem a tartalmi rész mellett egy másik elemre hivatkozott, amelyik a listában a következő. Néhány algoritmusnál láttuk, hogy ez számos problémát felvet, néha hasznos lett volna egy visszalépési lehetőség, amivel egy elemből el tudunk jutni a láncban előtte lévőre.

A *kétirányű láncolt listák* (doubly linked list) (esetenként kétszeresen láncolt listának is nevezik, bár ez félreérthető lehet [4]) egy egyszerű megoldást nyújtanak erre a problémára, az egyes elemeket ugyanis kiegészítik egy visszafelé mutató hivatkozással. Ennek megfelelően minden elem felépítése az alábbi:

- Tartalmi rész: Ez a rész tartalmazza magát a listaelemben eltárolni kívánt adatot. Megfelel az egyirányú láncolt listában is tárgyalt *tart* mezőnek.
- Következő hivatkozás rész: A lista minden eleme tartalmaz egy hivatkozást a listában rákövetkező elemre. A legelső elemtől elindulva így (közvetve, a közbülső elemeken keresztül) elérhetjük a lista bármelyik elemét. A mező típusa a lista megvalósításától függően többféle lehet.
- Előző hivatkozás rész: A lista minden eleme tartalmaz egy hivatkozást a listában őt megelőző elemre. Ennek típusa ugyanaz, mint amit a következő mezőnél láthattunk.

A kétirányú láncolt lista előnye nyilvánvaló, lehetővé teszi a visszafelé lépkedést. A 4.17. ábra egy ilyen láncolt listát mutat be. Ugyan nem nélkülözhetetlen, de gyakran hasznos lehet egy új fejet is felvenni, ami a legutolsó elemre mutat, ebből indulva visszafelé is be tudjuk járni a listát. Amennyiben az általunk használt algoritmusok ezt a tulajdonságot kihasználják, akkor ezzel a módszerrel hatékony megoldásokat készíthetünk.

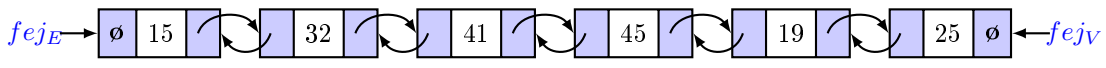
A hátránya a hagyományos listához képest az, hogy a két mutató miatt a lista elemeinek a helyfoglalása nagyobb (ami már az előzőleg megismert láncolt listánál is problémát okozott). Továbbá a különféle módosító algoritmusok valamelyest bonyolultabbak, hiszen minden beszúrásnál/törlésnél be kell álltani az új elem előző mutatóját is, illetve a listában utána következő elemnél is ezt el kell végezni. Hasonló módon, két fej esetén mindkét fejet folyamatosan karban kell tartani az elvégzett módosításnak megfelelően.

Bár ezek a módosító műveletek bonyolultabbak, mint az egyszerű egyirányú listánál, teljesítmény szempontjából akár hatékonyabbak is lehetnek. Elég csak arra gondolunk, hogy a kétirányú láncolt listával egyszerűen megoldhatjuk a mutatott elem elé beszúrást, illetve a mutatott elem törlését, hiszen nincs szükség egy hosszadalmas keresésre a megelőző elem megtalálásához.

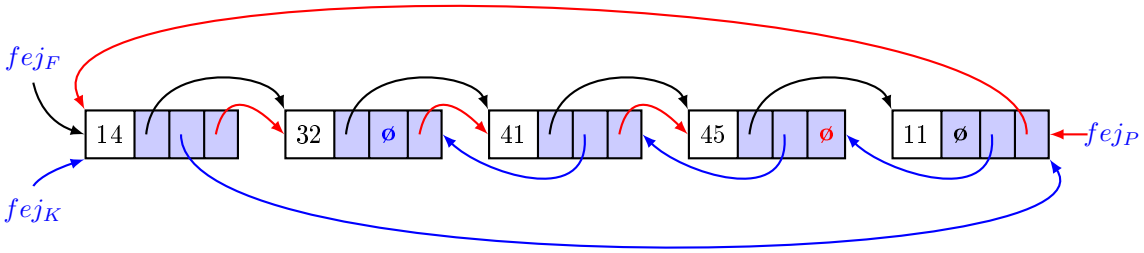
4.5.2. Többszörösen láncolt lista

A *többszörösen láncolt lista* (multiply linked list) tekinthető a kétirányú láncolt lista általánosításának abból a szempontból, hogy itt már nem csak két, hanem akár három, vagy annál több elemre is hivatkozhatnak az egyes elemek. A két struktúra közötti lényeges különbség, hogy míg a kétirányú láncolt listánál az előre és a hátra mutató hivatkozások mindig egymásnak ellenkező irányokat adtak meg, addig egy többszörösen láncolt lista esetében ezek teljesen függetlennek tekintendők. Tehát elképzelhető egy kétszeresen láncolt lista, ahol minden elem két következő mezőt tartalmaz, és ezek egymástól független sorrendeket adnak meg.

Egy háromszorosan láncolt listára mutat példát a 4.18. ábra. A lényeges különbség a hagyományos listához képest, hogy minden listaelem három darab következő hivatkozást tartalmaz, amelyek egymástól függetlenül mutatnak egy-egy további elemre (vagy éppen a ø értékkel jelzik, hogy nincs ilyen). Célszerű úgy elképzelni ezt a szerkezetet, mintha három, egymástól teljesen független láncolt listánk lenne, amelyek a tartalom részt megosztják egymás között. Ennek megfelelően rendelkezünk három *fej* változóval is,



4.17. ábra. Kétirányú láncolt lista.



4.18. ábra. Többszörösen láncolt lista. Az példában az egyes fejekből kiinduló listák adatai: fekete: 14, 32, 41, 45, 11; kék: 14, 11, 45, 41, 32; piros: 11, 14, 32, 41, 45.

amelyek az egyes listáknak az első elemeire mutatnak (és látható, hogy a megadott *köv* mezőt követve, mindhárom lista esetében találunk egy lezáró **ø** jelet is).

Egy N-szeresen láncolt lista felépítése tehát az alábbi:

- Tartalmi rész: Ez a rész tartalmazza magát, a listaelemben eltárolni kívánt adatot. Ez megfelel az egyirányú láncolt listában is tárgyalt *tart* mezőnek.
- Következő$_i$ hivatkozás rész (ahol $1 \leq i \leq N$): Az i. lánc alapján a következő elemre való hivatkozás.

A szerkezet hátránya természetesen itt is az, hogy minél több hivatkozást tárolunk, annál nagyobb tárterületet fog követelni az adatszerkezet. A módosító algoritmusok is több odafigyelést igényelnek, hiszen pl. minden törlésnél ügyelni kell arra, hogy minden sorrend szerint megfelelően láncoljuk ki az elemet (ami a lépésszámok tekintetében is meglehetősen hátrányos lehet, ha ez mindig újabb kereséseket jelent).

Bizonyos esetekben azonban mégis érdemes lehet használni ezt a megoldást, főleg olyankor, amikor ugyanazokat az adatokat többféle sorrendben is el szeretnénk tárolni. Pl. személyek adatait eltárolhatjuk életkor szerint, név szerint és igazolvány szám szerint rendezve. A feldolgozáskor pedig mindig azon a láncon haladunk végig, amelyik az aktuális szempontnak a leginkább megfelel.

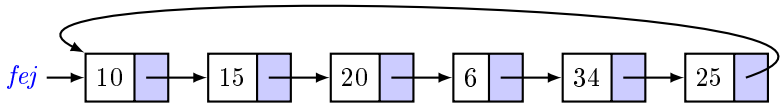
Szintén előnyös tulajdonsága ennek a módszernek, hogy függetlenül attól, hogy több láncot kezelünk, a tartalmi részt csak egyszer kell eltárolnunk, hiszen minden lánc ezt ugyanabban az elemben tárolja. Ez egyrészt hatékony tárhely-kihasználást tesz lehetővé, másrészt a módosításoknál ilyenkor csak egy helyen kell változtatni az adatokat, nem szükséges azt minden láncban külön-külön megtenni.

4.5.3. Ciklikus láncolt lista

A *ciklikus láncolt lista* (circular list) [4] elemeinek a felépítése nem különbözik jelentősen az egyszerű egyirányú láncolt listánál tanultaktól. Minden elem ugyanúgy egy tartalmat és egy következő elemre vonatkozó hivatkozást tárol.

A különbség pusztán az (miként a 4.19. ábrán is látható), hogy hiányzik a lista végéről a lezáró elem. A lista utolsó eleme (már ha egyáltalán utolsónak nevezhető ebben a szerkezetben) az első elemre hivatkozik, így az egész lánc egy kört alkot.

A megoldás egyik előnye az, hogy ezzel megszabadulunk néhány kivételes esettől. Néhány algoritmusnál külön kellett foglalkoznunk az első és utolsó elemekkel, mivel ezek előtt/után már nincsenek további elemek. A ciklikus láncolt lista esetében ez láthatóan nem áll fenn, hiszen (ha már vannak elemeink a listában), akkor minden elemnél fogunk találni rákövetkezőt, illetve előzőt.



4.19. ábra. Ciklikus láncolt lista.

A gyakorlatban ez a szerkezet főleg olyan feladatoknál használható ki jól, amelyek eleve hasonló, gyűrűszerű tárolást igényelnek. Ilyen lehet például a már megismert véges méretű *sor* adatszerkezet, ahol a folyamatosan behelyezett elemek egy idő után felül kell, hogy írják a legrégebben felvett elemeket. A ciklikus lista ezt természetszerűen támogatja, hiszen mindig a következő elemre lépegetéssel az „utolsó" elem után azonnal az első elemre lépünk, így megkezdődhet a legrégebbi elemek felülírása.

Mindez persze jelentős változtatásokat igényel a megismert algoritmusok esetében is, hiszen pl. a már megismert bejárás ebben az esetben egy végtelen ciklusba kerülne.

Esetenként érdemes lehet a különféle megismert láncolt lista megvalósításokat kombinálni is. Például, ha az algoritmusaink ki tudják használni annak speciális tulajdonságait, érdemes lehet használni kétirányú ciklikus listákat is akár.

4.6. Láncolt listák implementációja

Az eddigiek során megvizsgáltunk számos láncolt lista algoritmust, illetve a listák különféle variációit is, azonban mindvégig elkerültük az implementációval kapcsolatos kérdéseket. A pszeudokódok jelentős része egyértelmű ebből a szempontból is, pusztán néhány (a pszeudokódban szándékosan nem definiált) típust, és műveletet kell pontosítanunk.

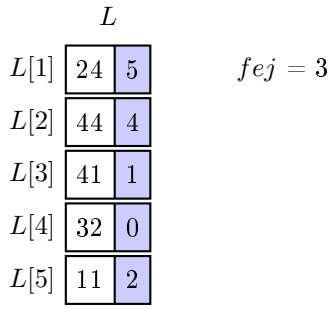
Ezek az alábbiak:

- **T** típus: Ezt már a bevezetőben tisztáztuk, hogy alapvetően tetszőleges típusú adatot tárolhatunk az adatszerkezetben. A rendezett láncolt listák esetében ezt csak annyival egészítettük ki, hogy rendezhetőnek kell lennie a típusnak. Az algoritmusok során tipikusan a listaelemek *tart* mezője volt ilyen típusú, illetve segédváltozók, paraméterek esetében is használtuk.
- **M** típus: Ezek a változók egy másik láncolt lista elemre tartalmaznak egy hivatkozást. Az implementációtól függően különböző típusúak lehetnek. A tárgyalt algoritmusokban a *köv* mező és fej volt ilyen típusú, illetve néhány segédváltozó (pl. p, e, *új*).
- **ø** érték: A pszeudokódokban különösebb probléma nélkül tudtuk használni ezt a speciális jelet, ezzel jelezve, hogy a hivatkozás nem egy tényleges elemre mutat. A tényleges implementáció során természetesen itt is pontosan definiálni kell, hogy a **M** típus melyik lehetséges értéke reprezentálja majd a **ø**-t.
- LÉTREHOZ(**M**) : **M** függvény: Ez a függvény létrehoz egy új láncolt lista elemet, és a visszatérési értéke egy hivatkozás erre az új elemre (a hivatkozás típusa az előzőleg említettnek felel meg, tehát **M**).
- FELSZABADÍT(p : **M**) eljárás: Felszabadítja a paraméterként átadott p által hivatkozott listaelemet. Megvalósítása függ mind a választott implementációs módtól, mind pedig a használt programozási nyelvtől/futtatási környezettől (pl. automatikus szemétgyűjtéssel rendelkező nyelvek esetében nincs szükség semmilyen műveletre a nem használt elemek felszabadításához).

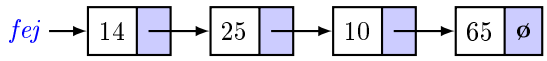
4.6.1. Implementáció tömbökkel

A láncolt lista elemek tárolásakor az egyik lehetőség, ha ezeket egy tömbben helyezzük el. Miként a 4.20. ábrán is látható, ebben az esetben a tömbök egyes elemei maguk a láncolt lista elemek lesznek. A tömb típusa célszerűen egy olyan összetett típus, amely egyben tartalmazza a tartalmat és a következő elemre vonatkozó hivatkozást is.

- A **T** típus természetesen bármi lehet, ami a feladat szempontjából megfelelő. Elképzelhető, hogy magában a tömbben tároljuk a teljes értéket, de az is, hogy a tömb csak egy referenciát tartalmaz a valódi tartalomhoz (pl. objektumok esetében).



4.20. ábra. Tömb segítségével implementált láncolt lista. Az 5 elemű L tömb a láncolt lista elemeit tartalmazza. A fej értéke mutatja, hogy a tömb 3. eleme lesz az első a láncban, majd a következő mező értékek jelzik a következő címeket, egészen a lezáró 0 értékig. A listában lévő tartalmak sorrendje tehát: 41, 24, 11, 44, 32.



4.21. ábra. Mutatókkal megvalósított láncolt lista. A fej mutató az első elem kezdőcímére mutat, majd ezt követően minden elem *köv* mezője a következő elem címét tartalmazza.

- A **M** típus ebben az implementációban egy egész szám. A számot úgy használjuk hivatkozásként, hogy értéke azt mutatja, hogy a tömb hányadik elemére hivatkozik. A tömb egyes elemeinek a *köv* mezőjében lévő értéke tehát azt jelzi, hogy melyik tömb elem lesz a lista következő eleme. A fej változóban lévő egész szám pedig azt mutatja, hogy a tömb hányadik eleme lesz a lista első eleme.
- A **ø** érték a típus ismeretében már könnyen definiálható. Ha a tömböket 1-től címezzük, akkor pl. a 0 szám jelezheti ezt a speciális értéket, amely azt reprezentálja, hogy nem egy tényleges elemről van szó. Mind a fej változó esetében, mind pedig az utolsó elem *köv* mezőjében használhatjuk ezt az értéket.
- A LÉTREHOZ(**M**) : **M** függvény megvalósítása során a tömbben keresni kell egy szabad helyet, majd pedig ennek a sorszámát kell visszaadni. Ennek megvalósítása során elvégezhetünk egy keresést, ahol megtaláljuk az első, még el nem foglalt helyet, de szükség esetén különféle segédtáblákkal is gyorsíthatjuk ezt a módszert. Tömbök esetében tényleges memóriafoglalásra nincs szükség, hiszen az adatszerkezet működéséből adódóan már a tömb létrehozásakor le kellett foglalnunk a teljes szükséges memóriaterületet.
- A FELSZABADÍT(p : **M**) eljárás a tömb esetén tényleges memóriafelszabadítást nem végez, pusztán valamilyen formában jelzi, hogy a megadott elem már felhasználható egy következő beszúrás esetén. Ennek megvalósításának természetesen igazodnia kell a LÉTREHOZ(**M**) függvény által várt állapotokhoz.

A tömbben eltárolt láncolt lista ötvözi a tömbök és a láncolt listák előnyeit, és persze a hátrányait is. Egyszerűen kezelhető (dinamikus memóriakezelés nélkül), és a láncon keresztüli hozzáférés esetében a láncolt lista műveleteknél megismert egyszerű módon tudunk új elemeket beszúrni, illetve törölni a lista közepében. Ugyanakkor továbbra is fennáll az a probléma, hogy már a tömb létrehozásakor meghatározzuk az adatszerkezet maximális méretét, illetve ennél kevesebb elem esetében jelentős mennyiségű, feleslegesen lefoglalt memóriát igényelhet.

4.6.2. Implementáció mutatókkal/referenciákkal

A láncolt listákat az előző megoldással szemben többnyire olyan dinamikus adatszerkezetként képzeljük el, ahol maguk az elemek a program futása során folyamatosan jönnek létre (az egyes beszúrás műveleteknél), illetve szabadulnak fel (a törlés műveletek során). A megvalósítás alapelve, hogy az egyes elemek a memória tetszőleges helyén elhelyezkedhetnek, az elemekre vonatkozó hivatkozások pedig az egyes elemek memóriabeli címét tartalmazzák. A tényleges megvalósítás történhet mutatók segítségével, illetve az OOP nyelvek esetében referenciák segítségével.

A megismert algoritmusok ebben az esetben is azonosak, pusztán a fejezet elején már említett típusokat, illetve függvényeket kell pontosabban definiálnunk:

- A **T** típus természetesen itt is bármi lehet, ami a feladat szempontjából megfelelő. A lista elemben eltárolhatjuk magát a tényleges tartalmat, de elképzelhető, hogy csak egy hivatkozást tárolunk rá.
- A **M** típus ebben az implementációban egy mutató, vagy egy referencia. A fej elem ezen keresztül közvetlenül tartalmaz egy mutatót az első elem címére, míg az egyes lánc elemek *köv* mezői mind a listában őket követő elemekre hivatkoznak.
- A **ø** érték esetében a programozási nyelvek által nyújtott lehetőségek közül választhatunk. Általában a dinamikus memóriakezelést támogató nyelvekben létezik egy *null*, *NULL*, *nil*, stb. kulcsszó, amely egy olyan értéket reprezentál, amit értékül adhatunk egy mutatónak/referenciának, és ezzel jelezhetjük, hogy az valójában nem mutat valódi elemre.

- A LÉTREHOZ($\mathbf{M}$) : $\mathbf{M}$ függvény megvalósítása szintén a választott programozási nyelv lehetőségeitől függ. Mutatók esetében általában az adott nyelv közvetlen memóriafoglalási műveleteit tudjuk használni (pl. egy listaelemnyi méretű memóriaterület lefoglalása), míg a referenciák esetében egy objektumot hozhatunk létre, és az így visszakapott objektum referenciával dolgozhatunk tovább.
- A FELSZABADÍT(p : $\mathbf{M}$) szintén a konkrét programozási nyelvtől függő művelet. Jelentheti egy közvetlenül címzett, előzőleg lefoglalt memóriaterület felszabadítását, vagy akár egy referencia által hivatkozott objektum destruktorának hívását. Külön érdemes kiemelni az automatikus szemétgyűjtő mechanizmussal rendelkező nyelveket, itt a felszabadítás műveleteket egyszerűen elhagyhatjuk (mivel a felszabadítandó elemekre nincs hivatkozás, így azokat előbb-utóbb a szemétgyűjtő megszünteti).

Alapvetően ez utóbbi megoldást (4.21. ábra) tekintjük a láncolt lista megvalósítások során a követendő útnak, hiszen csak ez tud biztosítani a fejezet elején említett előnyök közül néhányat: nem igényel összefüggő memóriaterületet, továbbá a lista mindig csak annyi helyet foglal a memóriából, amennyit az általa tartalmazott lista elemek ténylegesen igényelnek.

5. fejezet

Bináris keresőfa

5.1. Bináris keresőfa felépítése

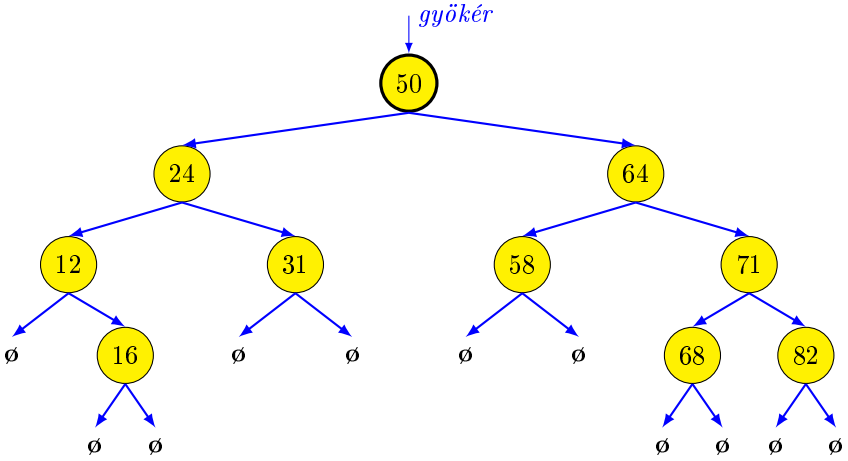
A gráfelméletben fának nevezzük a körmentes, összefüggő gráfokat. Mi azonban egy kicsit más definíció alapján közelítjük meg ezeket [1]: egy *fa* nem más, mint *csomópontok* (nodes) halmaza, amelyeket *élek* (edges) kötnek össze, és teljesülnek az alábbi feltételek:

- létezik egy kitüntetett csomópont: a *gyökér* (root),
- a gyökértől különböző minden más csomópont egy éllel van összekötve a szülőjével,
- összefüggő, tehát bármelyik nem-gyökér csomópontból kiindulva, a szülőkön keresztül a gyökérhez eljutunk.

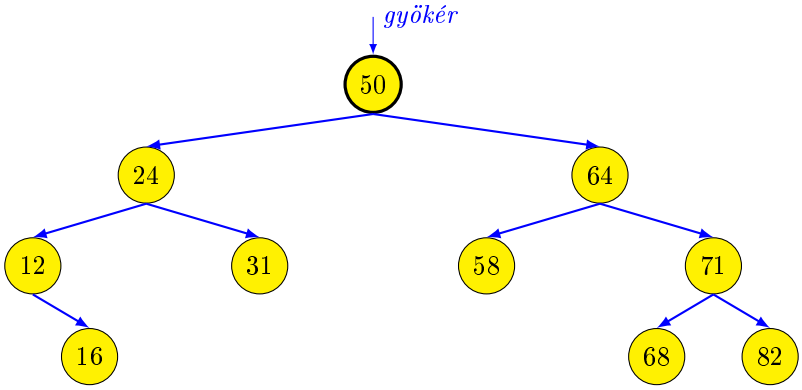
A két definíció közti (számunkra a megvalósítás szempontjából nagyon fontos) különbség az, hogy ez utóbbi esetben a fa egy csomópontját kitüntetettként kezeljük. A láncolt lista fejéhez hasonlóan, ez lesz a fa egyetlen kívülről is elérhető eleme, amelyen keresztül (közvetlenül vagy közvetve) hozzáférhetünk majd a többihez.

Ebben a fejezetben csak úgynevezett *bináris fákról* lesz szó, ahol minden szülőelemnek legfeljebb kettő gyereke lehet (tehát a gyerekek száma 0, 1 vagy 2). A bináris keresőfa tárolása során ezt a két gyerekekre való hivatkozást gyakran nevezzük *bal-*, illetve *jobboldali* hivatkozásnak. Ennek megfelelően beszélhetünk bal- illetve jobboldali gyerekről, és bal- illetve jobboldali részfáról is. Néhány további, a későbbiekben használni kívánt fogalom (ahol x és y fa csomópontok):

- Ha létezik egy x-ből y-ba vezető él, akkor x-et az y *szülőjének*, y-t pedig az x *gyerekének* nevezzük. Például az 5.1. ábrán a 31-es elem szülője a 24-es, illetve a 24-es gyereke a 31-es. A gyökérelemnek nincs szülője, minden más csomópontnak pontosan egy van.
- Az x és y elemek testvérek, ha azonos a szülőjük. Az 5.1. ábrán két testvér például az 58 és a 71 (szükség esetén az emberi családfákhoz hasonló módon definiálható a nagyszülő, nagybácsi, unokatestvér, stb. fogalmak is).
- Az út csomópontok és élek váltakozó sorozata, amely mindig csúccsal kezdődik és végződik, és minden csúcsot valamelyik gyereke követi a sorozatban, illetve minden él két végpontja az őt megelőző és követő csomópont (mivel egy fáról beszélünk, egy séta mindig út is egyben). Az út hossza az őt alkotó élek száma.
- Ha vezet út x-ből y-ba, akkor x az y *őse*, az y pedig az x *leszármazottja*. Amennyiben $x \neq y$, akkor valódi ősről, illetve valódi leszármazottról beszélünk. Az 5.1. ábrán az 50-es elem (valódi) őse a 16-osnak, míg a 16-os (valódi) leszármazottja az 50-esnek.
- Valódi leszármazottal nem rendelkező csomópontokat *leveleknek* nevezzük. Az 5.1. ábrán a levelek az alábbiak: 12, 16, 31, 58, 68, 82.
- Egy x csomópont az összes leszármazottjával egy részfát alkot, aminek a gyökere az x. Tehát az 5.1. ábrán a 64 gyökerű részfa elemei: 64, 58, 71, 68, 82.



(a) A teljes fa, ahol ábrázoljuk a levelek lezáró bal és jobb hivatkozásait.



(b) Ugyanaz a bináris fa, csak a későbbiekben is látható módon, itt már nem ábrázoljuk a levelek alatt található lezáró ø jeleket.

5.1. ábra. Egy minta bináris keresőfa, amelyben egész számokat tárolunk.

- Egy *csomópont mélysége* egyenlő a gyökértől hozzá elvezető út hosszával.
- Egy *csomópont magassága* a leghosszabb, innen kiinduló és a levelekig vezető út hossza.
- A *fa magassága* a gyökér magassága.

A bináris fák egy speciális esetének tekinthetők a bináris keresőfák. A *keresőfák* célja a hatékonyabb keresés érdekében az adatok olyan reprezentációja, amely gyors visszakeresést és dinamikus használatot tesz lehetővé [7]. Ez az igény a bináris fát annyiból specializálja, hogy a fában található kulcsok rendezve vannak az alábbiak szerint: minden egyes csomópontra igaz, hogy a baloldali részfájában lévő csomópontok nála kisebb, a jobboldali részfájában lévő csomópontok pedig nála nagyobb kulcsokat tartalmaznak (vagy épp fordítva). Az 5.1. ábra egy példát mutat bináris keresőfára.

A konkrét technikai megvalósítás során a bináris keresőfa csomópontokat általában az alábbi négy részre bonthatjuk:

- Tartalmi rész: Ez a rész tartalmazza magát, a fában eltárolni kívánt adatot. Nincs semmiféle megkötésünk ennek a típusára, lehet szám, szöveg, vagy akár tetszőleges osztály típus is. Azt azonban általában kikötjük, hogy az egyes csúcsok mind azonos típusú adatokat tartalmazzanak. A példákban ezt a mezőt mindig *tart* néven fogjuk használni, ami **T** típusú.
- Kulcs rész: A bináris keresőfa esetében a rendezettséget mindig ehhez a kulcsmezőhöz kötjük. Típusa a tartalomhoz hasonlóan bármilyen típus lehet, persze csak olyan, ami összehasonlítható. A kulcsot mindig egyedinek tekintjük, két azonos kulcsú elem nem lehet egy bináris keresőfában. A kulcs gyakran szorosan összekapcsolódik a tartalommal, pl. ha a tartalom egy összetett érték

(hallgatói adatok tárolása esetén név, életkor, évfolyam stb.), akkor a kulcs ennek lehet az egyik komponense (pl. a név mező). Egyszerűbb esetekben nincs is szükség a tartalom és a kulcs megkülönböztetésére, mi is ezt követjük majd a példáink során, a fa egyes elemeiben egész számokat fogunk tárolni, így maga a szám lesz az eltárolt tartalom, és egyben a rendezés alapját jelentő kulcs is. A példákban *kulcs* néven hivatkozunk majd erre a mezőre, ami **K** típusú.

- Bal hivatkozás rész: A fa minden eleme tartalmaz egy hivatkozást a baloldali részfájának első elemére. A mező típusa a lista megvalósításától függően többféle lehet, erre még a későbbi, implementációs résznél visszatérünk (addig is a legegyszerűbb a fa elemeit objektumként, a bal és jobb mezőt pedig objektum referenciaként felfogni). A példákban ezt a mezőt mindig *bal* néven fogjuk használni. Amennyiben egy csomópontnak nincs baloldali részfája, úgy ennek a mezőnek az értéke **ø** lesz. A mező **M** típusú.

- Jobb hivatkozás rész: Értelemszerűen a jobboldali részfa első elemére hivatkozik. A példákban *jobb* néven hivatkozunk erre a mezőre. Amennyiben nincs jobboldali részfa, akkor a mező értéke **ø** lesz. A mező **M** típusú.

A bináris fa egy eleme technikailag nagyon hasonlít az előző fejezetben megismert láncolt listáéhoz, azzal a kiegészítéssel, hogy a tartalmi rész mellett nem csak egy *köv*, hanem egy-egy *bal* illetve *jobb* hivatkozást tartalmaz. Látni fogjuk azonban, hogy ez a kis kiegészítés jelentősen megváltoztatja a feldolgozó és módosító műveleteinket (pl. már a legegyszerűbb bejárásnál is felmerül a kérdés, hogy a gyökérelemből kiindulva mit értünk „következő" elem alatt).

A láncolt listánál szükségünk volt egy fej nevű külső hivatkozásra, ami a lista első elemére mutatott. A fánál erre szintén szükségünk lesz, itt azonban hagyományosan *gyökér*-nek szoktuk elnevezni, habár a szerepe teljesen azonos. Ezen keresztül tudjuk elérni a fa gyökérelemét, majd azon keresztül a többit. Üres fa esetében a *gyökér* változó értéke **ø**.

Érdemes észrevenni a bináris keresőfa esetében annak rekurzív felépítését. A fát úgy is elképzelhetjük, hogy az áll egy gyökérelemből, illetve egy bal és jobboldali részfából. Ahol mindkét részfa tulajdonképpen egy gyökérelem és az ahhoz tartozó bal és jobboldali részfa, stb. A rekurzív adatszerkezeteket gyakran rekurzív algoritmusokkal tudjuk a leghatékonyabban feldolgozni, itt is erre látunk majd példákat. Érdemes megjegyezni, hogy az összes felsorolt művelet megvalósítható nem rekurzív formában (sőt, ez még hatékonyabb megoldást is adhat).

Megjegyzés

A láncolt lista esetében egyébként ugyanígy megfigyelhető egy rekurzív jellemző. Hiszen maga a lista felfogható úgy, hogy áll egy fej elemből, illetve egy ahhoz kapcsolt további láncolt listából. Ennek megfelelően a láncolt lista feldolgozások egyszerűen megadhatók rekurzív műveletekkel is. Míg azonban a fák esetében a rekurzív megoldást fogjuk gyakran egyszerűbbnek és természetesebbnek találni, addig a láncolt listák esetében ezek gyakran öncélúnak és feleslegesen bonyolultnak tűnnek.

5.2. Bejárások

A láncolt listáknál már láttuk, hogy a bejárás egy olyan művelet, amely az adatszerkezet minden egyes elemét pontosan egyszer dolgozza fel. A listánál szinte nem is találhatunk más módot a bejárásra, csak amit meg is vizsgáltunk. A *fej* hivatkozáson keresztül ugyanis csak az első elemet tudjuk elérni, innen pedig csak mindig a következő elemekre tudunk tovább lépegetni. Ebből adódóan az elemek bejárási sorrendje mindig megfelelt azok láncbeli sorrendjének.

A fák esetében ez már nem ilyen egyértelmű. Ha megnézzük az 5.1. ábrát, akkor látható, hogy maga a kezdőpont itt is egyértelmű (hiszen a fából kívülről csak a *gyökér* hivatkozást látjuk, ami a gyökér elemre hivatkozik), azonban az innen való továbblépés már több irányba is történhet. Továbbléphetünk elsőként a 24, illetve a 64 kulcsú elemre is. Majd az ezekről való továbblépés ugyanígy több irányban is megtörténhet.

A fentieknek megfelelően a bináris keresőfánál többféle bejárás műveletet is megkülönböztetünk. Érdemes megjegyezni, hogy maga a bejárás definíciója ugyanaz maradt így is, mindegyik bejárás minden elemet pontosan egyszer fog feldolgozni. Az egyetlen különbség pusztán az, hogy milyen sorrendben érjük el a fában található csomópontokat, illetve mikor dolgozzuk fel az azokban tárolt tartalmakat.

5.2.1. Preorder bejárás

Az egyik lehetséges bejárási mód a fa *preorder* bejárása. Az 5.1. algoritmus mutatja a hozzá tartozó pszeudokódot. A bejárás lépései az alábbi sorrendben dolgozzák fel a csomópont elemeit:

- Csomópont tartalmának feldolgozása.
- Csomópont baloldali részfájának feldolgozása.
- Csomópont jobboldali részfájának feldolgozása.

Az eljárás hívását követően mindig ellenőrizzük, hogy a paraméterként átadott p hivatkozás egy valódi csomópontra mutat-e. Üres fa esetében ugyanis elképzelhető, hogy már a meghíváskor az ilyenkor szokásos **ø** értéket kapjuk első paraméterként. Amennyiben az ellenőrzés azt mutatja, hogy a változó értéke megfelelő, akkor feldolgozhatjuk az általa hivatkozott csomópont tartalmát.

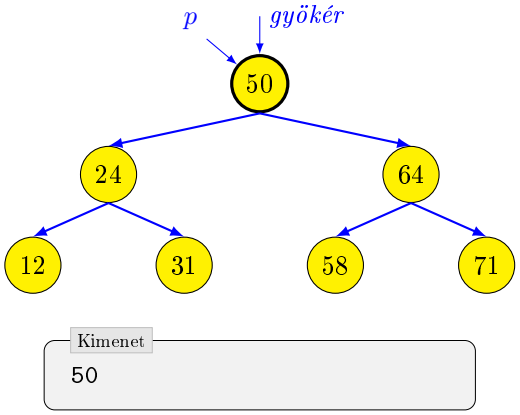
Ezt követi a csomópont részfáinak a feldolgozása. Először a csomópont baloldali részfájában lévő elemeket dolgozzuk fel, ezt egy rekurzív hívással egyszerűen meg tudunk tenni. Miután a rekurzió visszatért (tehát a baloldali részfa minden eleme fel lett dolgozva), következhet a jobboldali részfa feldolgozása egy újabb rekurzív hívással. Miután mindhárom művelet befejeződött, az eljárás véget ér, ezzel a rekurzió visszalép egy szinttel (illetve a legelső szinten visszalép az őt meghívóhoz).

A 2. sorban található p ellenőrzésnek a rekurzió szempontjából is fontos szerepe van. Az egyes részfák feldolgozása során egyre mélyebbre kerülünk a fában, és mivel feltételezzük, hogy az véges számú elemet tartalmaz, így előbb-utóbb elérünk a levelekig. Ezeknek már nincsenek bal-, illetve jobboldali gyerekeik, de a rekurzió ugyanúgy meghívódik ezekre az ágakra is. Ilyenkor az eljárás következő szintje paraméterként mindig **ø** értéket kap majd (hiszen ezzel jelöltük a leveleknél, hogy hivatkozásaik nem valódi elemekre mutatnak), így ezzel állítjuk meg a rekurziót. Mindez persze nem csak a levél elemek esetében, hanem az egy gyerekkel rendelkező csomópontoknál is hasonlóan működik.

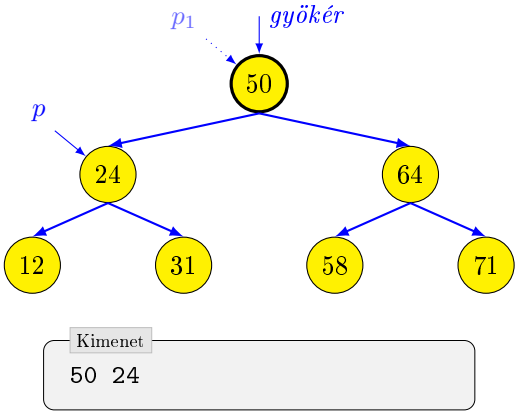
Az 5.2. ábra lépésenként mutatja be egy bináris keresőfa preorder bejárásának lépéseit (az alsó indexszel rendelkező p változók az előző rekurziós szintek értékeit mutatják). A végeredményként kapott számsor fontos szerepet kap a teljes fa elmentésekor, mivel ez alapján fel tudunk építeni egy, az eredetivel megegyező fát.

Megjegyzés

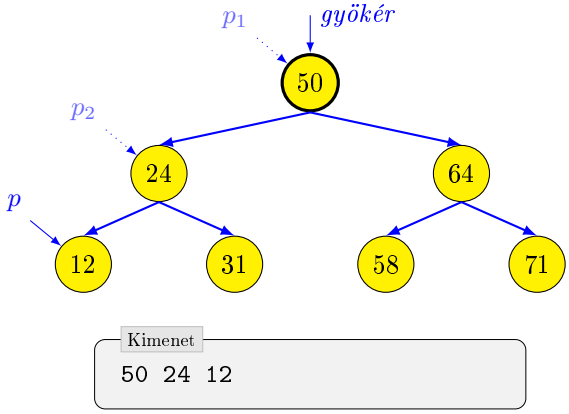
A számsor értelmezéséből talán már sejthető, hogy a beszúrások sorrendje befolyásolja a létrehozott fa szerkezetét. Tehát még ha ugyanazokat az elemeket is helyezzük el a fában, de különböző sorrendben, akkor is elképzelhető, hogy különböző szerkezetű fákat kapunk. Mindez persze nem törvényszerű, könnyen találhatunk olyan példákat, ahol különböző sorrendben megadott (de természetesen azonos) számokat felvéve ugyanahhoz a fához jutunk. Érdemes egy-egy példát keresni ezekre.



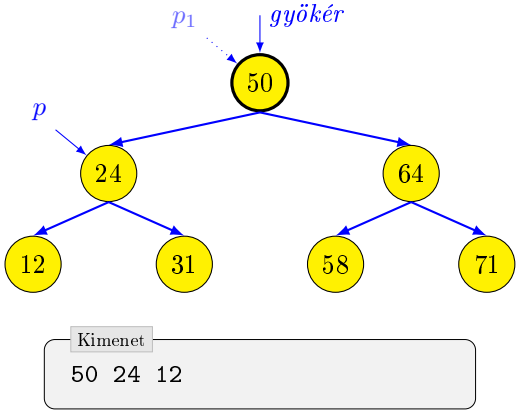
(a) A p az eljárás indításakor a gyökérelemre hivatkozik. A rekurzív algoritmus első lépésének megfelelően feldolgozza ezt az elemet, majd balra lép.



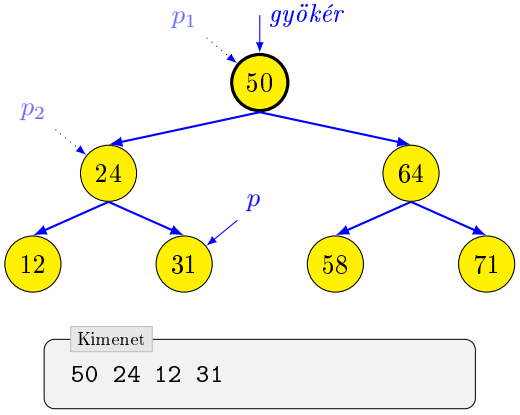
(b) A rekurzív algoritmus ellenőrzi, hogy az elem nem ø. Mivel nem az, így ismét feldolgozza az aktuális elemet, majd balra lép.



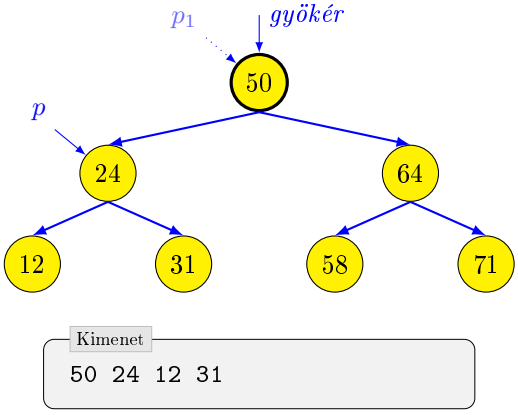
(c) A 12-es elemre érve, azt azonnal feldolgozza. Mivel már sem balra, sem jobbra nem tud továbblépni, Így visszatér a rekurzió előző szintjére.



(d) Ezen a szinten a feldolgozás és a bal részfa bejárása már megtörtént. Emiatt továbblép a jobboldali részfára.

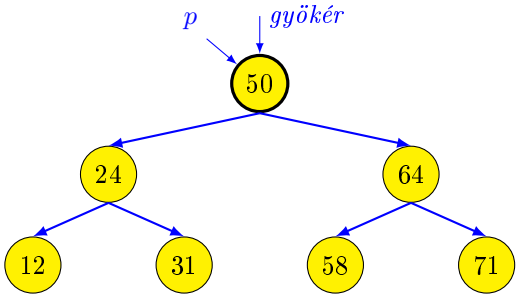


(e) A 31-es elem esetében is megtörténik a feldolgozás. Itt sincs lehetőség sem balra, sem pedig jobbra lépni. Ezért visszalépés az előző szintre.



(f) Megtörtént a feldolgozás, baloldal és jobboldal bejárás, így innen is visszaébblép a rekurzió az előző szintre.

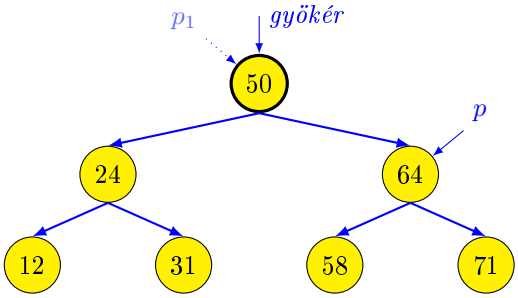
5.2. ábra. Bináris keresőfa preorder bejárása. A feldolgozás a csomópontok tartalmát kiírja a kimenetre.



Kimenet

```
50 24 12 31
```

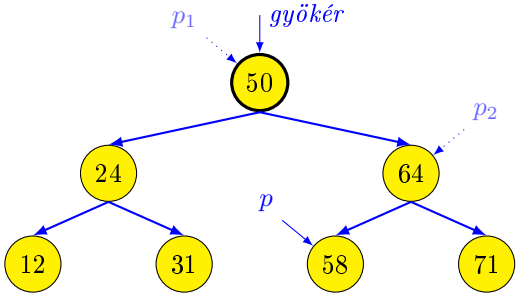
(g) A legfelső szinten a feldolgozás és a bal oldal bejárása már megtörtént. Ezt követi az a lépés, ahol az eljárás meghívja önmagát, paraméterként átadva a jobboldali részfát.



Kimenet

```
50 24 12 31 64
```

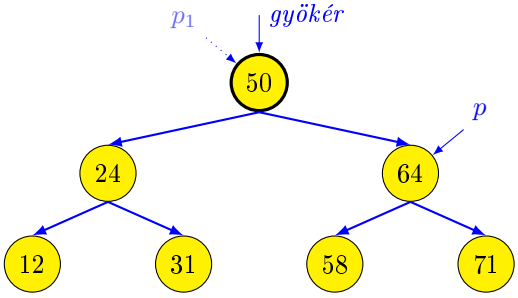
(h) A 64-es elem szintjén először megtörténik a feldolgozás, majd ellenőrzi a bal oldalt. Mivel itt vannak elemek, így azonnal tovább is lép erre.



Kimenet

```
50 24 12 31 64 58
```

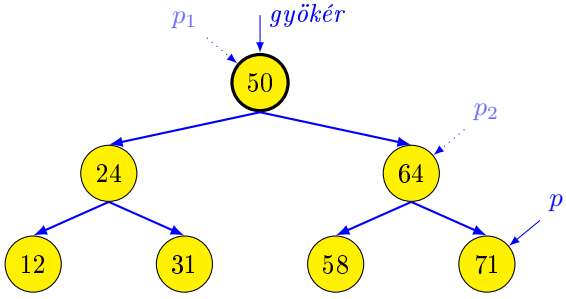
(i) Az eljárás feldolgozza az 58-as elemet. Majd a bal részfáját, ami üres. Ezt követi a jobb oldal vizsgálata, ami szintén üres. Ezt követően a rekurzió visszábblép egy szinttel.



Kimenet

```
50 24 12 31 64 58
```

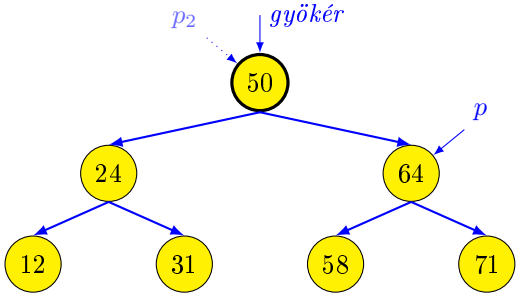
(j) A 64-es elem, és a baloldala már fel lett dolgozva, ezért az algoritmus meghívja önmagát a jobboldali részfára.



Kimenet

```
50 24 12 31 64 58 71
```

(k) Az eljárás feldolgozza az 71-es elemeet. Ezt követi az elem bal részfája, majd a jobb oldal vizsgálata, ami szintén üres. Ezt követően a rekurzió visszábblép egy szinttel.

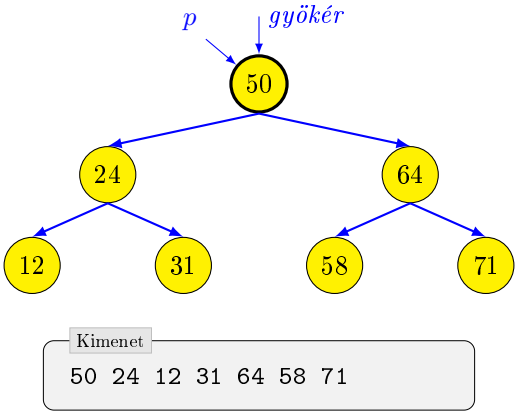


Kimenet

```
50 24 12 31 64 58 71
```

(l) Ezen a szinten már lefutott mindhárom művelet, így a rekurzió egy szinttel visszalép.

5.2. ábra. Bináris keresőfa preorder bejárása. A feldolgozás a csomópontok tartalmát kiírja a kimenetre. (folytatás)



(m) A gyökérelemnél is megtörtént a bal és jobboldali részfa bejárása, illetve az elem feldolgozása. Ezzel az eljárás véget ért.

5.2. ábra. Bináris keresőfa preorder bejárása. A feldolgozás a csomópontok tartalmát kiírja a kimenetre. (folytatás)

5.1. Algoritmus Bináris keresőfa preorder bejárása

Bemenet: p - **M**

```
1: eljárás PreorderBejárás(p : M)
2:     ha p ≠ ø akkor
3:         Feldolgoz(p.tart)
4:         PreorderBejárás(p.bal)
5:         PreorderBejárás(p.jobb)
6:     elágazás vége
7: eljárás vége
```

Eljárás hívása: PreorderBejárás(*gyökér*)

Felhasznált változók és függvények

- p: A fa aktuálisan vizsgált eleme.
- Feldolgoz(*elem* : **T**): Eljárás, ami feldolgozza a paraméterként átadott tartalmat.

5.2.2. Inorder bejárás

A bináris keresőfa *inorder* bejárása ez előzőek alapján már valószínűleg könnyen érthető lesz (5.2. algoritmus). A bejárás egészen hasonló módon járja be az elemeket, pusztán abban különbözik a megoldás a preordernél láthatótól, hogy más az egyes csomópontokban elvégzett műveletek sorrendje:

- Csomópont baloldali részfájának feldolgozása.
- Csomópont tartalmának feldolgozása.
- Csomópont jobboldali részfájának feldolgozása.

Az eljárás itt is a p ellenőrzésével kezdődik, hiszen üres fa esetében elképzelhető, hogy már a legelső hívásnál is **ø** értéket kaptunk paraméterként. Az eljárás hasonló módon hívja önmagát, először feldolgozza a baloldali részfában található elemeket. Miután végzett a részfával, utána dolgozza csak fel az aktuális csomópontban található tartalmat, majd ezt követően tér át a jobb oldali részfára.

Az 5.3. ábrán is látható, hogy az algoritmus hasonlóan működik, ugyanaz a levélelemeknél is a **ø** mutató szerepe, ez állítja meg a rekurzív hívások láncolatát. Érdemes megvizsgálni a feldolgozás sorrendjeként kapott számsort, bár itt jóval nyilvánvalóbb ennek gyakorlati haszna, tulajdonképpen kulcs szerint növekvő sorrendben kaptunk meg az elemeket. Jelen példánkban a fát úgy rendeztük, hogy minden csomópont baloldalán nála kisebb, jobb oldalán pedig nála nagyobb elemek vannak. Mivel az inorder bejárás alapelve az volt, hogy minden csomópont feldolgozása előtt végigjárja az ő teljes baloldali részfáját (tehát az összes nála kisebb elemet), majd pedig a csomópont feldolgozása után annak jobboldali részfáját (tehát az összes nála nagyobb elemet), így könnyen belátható, hogy miért lesz rendezett az így kapott lista.

> **Megjegyzés**
>
> Érdekes lehet a fában levő elemek kulcs szerint fordított sorrendjében való feldolgozása. Ehhez csak meg kell cserélnünk a bal- és jobboldali részfa meghívást, hiszen így minden csomópont feldolgozás előtt a nála nagyobb, utána pedig a nála kisebb elemeket fogja feldolgozni a bejárás.

5.2. Algoritmus Bináris keresőfa inorder bejárása

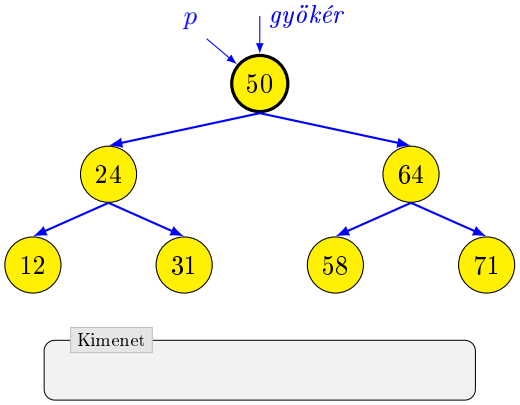
Bemenet: p **- M**

```
1: eljárás InorderBejárás(p : M)
2:     ha p ≠ ø akkor
3:         InorderBejárás(p.bal)
4:         Feldolgoz(p.tart)
5:         InorderBejárás(p.jobb)
6:     elágazás vége
7: eljárás vége
```

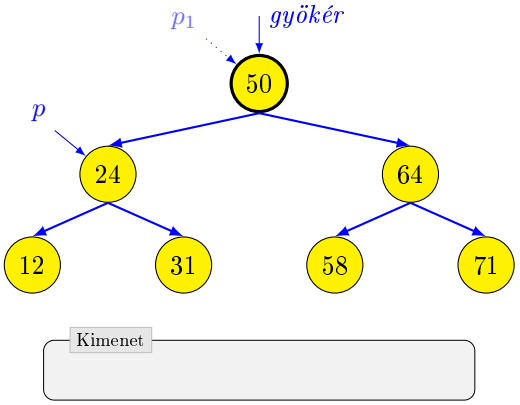
Eljárás hívása: InorderBejárás(*gyökér*)

Felhasznált változók és függvények

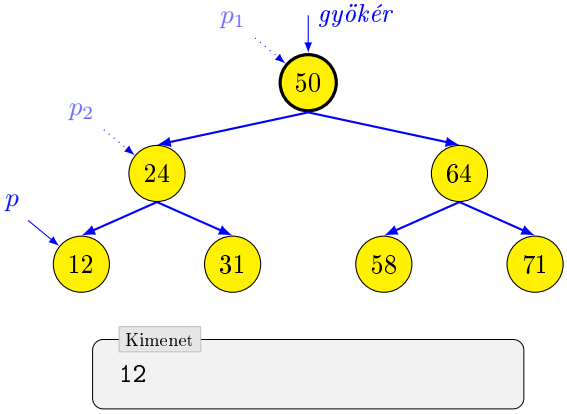
- p: A fa aktuálisan vizsgált eleme.
- Feldolgoz(*elem* : **T**): Eljárás, ami feldolgozza a paraméterként átadott tartalmat.



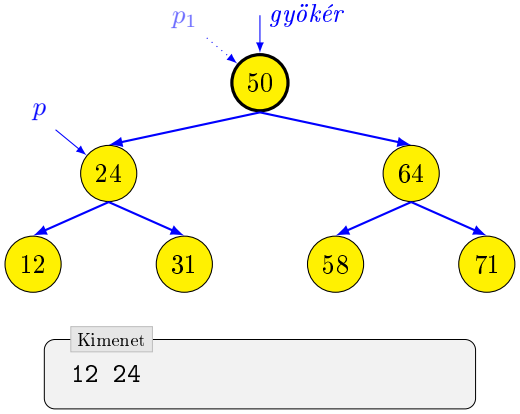
(a) A p az eljárás indításakor a gyökérelemre hivatkozik. A rekurzív algortmus első lépésének megfelelően balra lép.



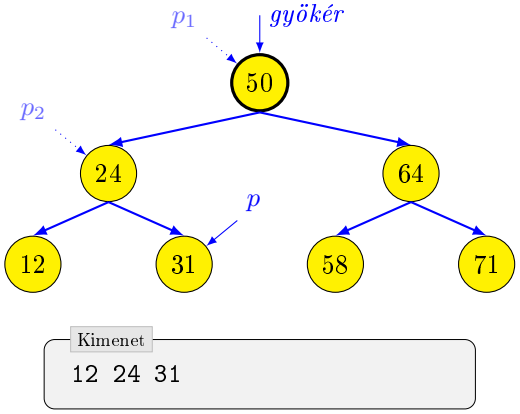
(b) A rekurzív algoritmus ellenőrzi, hogy az elem nem ø. Mivel nem az, így ismét azonnal balra lép.



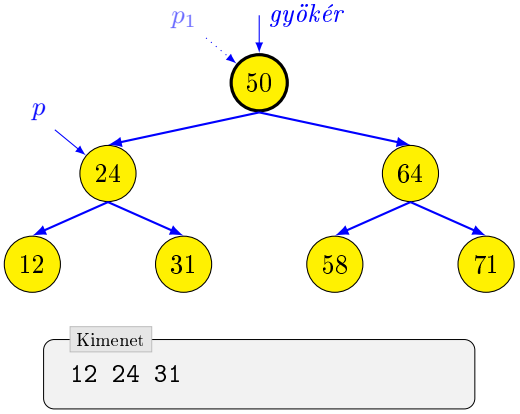
(c) Mivel balra már nem tud lépni, így feldolgozza a p által mutatott elemet. Ezt követően jobbra ellenőrzi a fát, de arra se tud továbbhaladni. Így visszalép a rekurzió előző szintjére.



(d) Ezen a szinten a bal részfa feldolgozása már megtörtént. Emiatt feldolgozza az aktuális elemet, majd továbblép a jobboldali részfára.

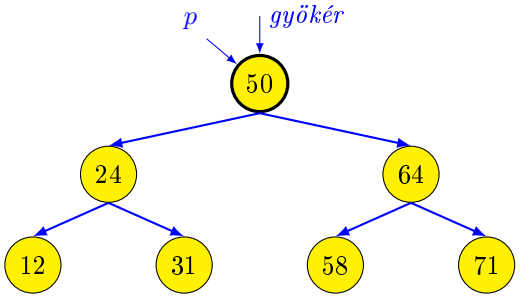


(e) A 31-es elem esetében sincs lehetőség balra lépni. Ezt követi a feldolgoás, a jobb oldal ellenőrzése, majd visszalépés az előző szintre.



(f) Megtörtént a baloldal bejárása, a feldolgozás, a jobboldali bejárás, így innen is visszalép a rekurzió az előző szintre.

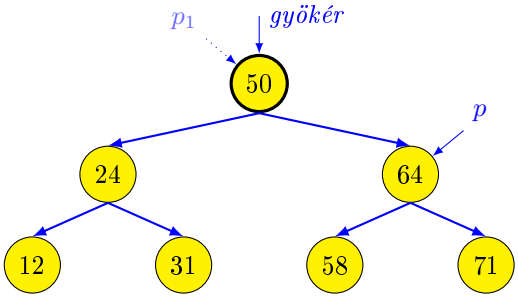
5.3. ábra. Bináris keresőfa inorder bejárása. A feldolgozás a csomópontok tartalmát kiírja a kimenetre.



Kimenet

```
12 24 31 50
```

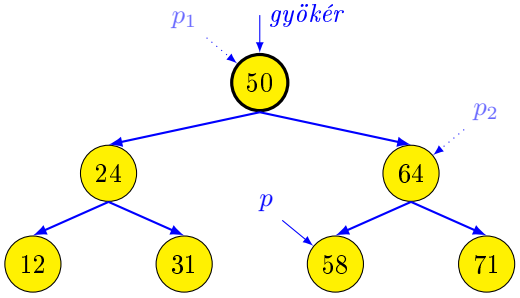
(g) A legfelső szinten a bal oldal bejárása már megtörtént. Ezt követi a feldolgozás, majd az eljárás meghívja önmagát, paraméterként átadva a jobboldali részfát.



Kimenet

```
12 24 31 50
```

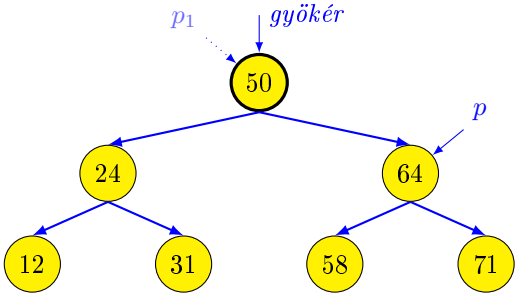
(h) A 64-es elem szintén először ellenőrzi a bal oldalt. Mivel itt vannak elemek, így azonnal tovább is lép erre.



Kimenet

```
12 24 31 50 58
```

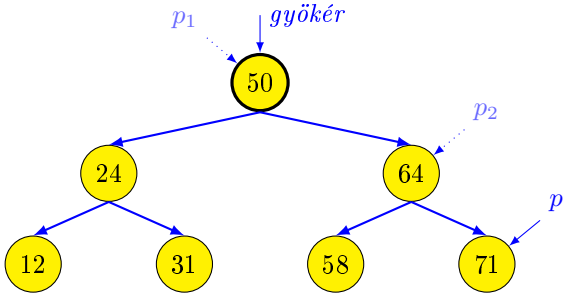
(i) Az eljárás feldolgozza az 58-as elem bal részfáját, ami üres. Ezt követi az 58 feldolgozása, majd a jobb oldal vizsgálata, ami szintén üres. Ezt követően a rekurzió visszábblép egy szinttel.



Kimenet

```
12 24 31 50 58 64
```

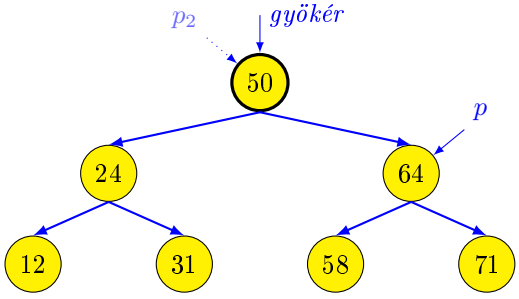
(j) A 64-es elem baloldala már fel lett dolgozva, ezt követi az elem feldolgozása, majd az algoritmus meghívja önmagát a jobboldali részfára.



Kimenet

```
12 24 31 50 58 64 71
```

(k) Az eljárás feldolgozza az 71-as elem bal részfáját, ami üres. Ezt követi a 71 feldolgozása, majd a jobb oldal vizsgálata, ami szintén üres. Ezt követően a rekurzió visszábblép egy szinttel.

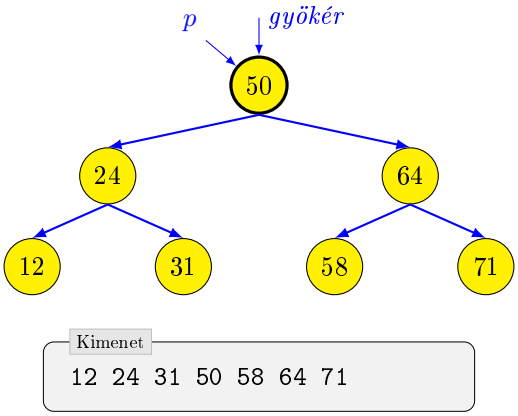


Kimenet

```
12 24 31 50 58 64 71
```

(l) Ezen a szinten már lefutott mindhárom művelet, így a rekurzió egy szinttel visszalép.

5.3. ábra. Bináris keresőfa inorder bejárása. A feldolgozás a csomópontok tartalmát kiírja a kimenetre. (folytatás)



(m) A gyökérelemnél is megtörtént a bal és jobboldali részfa bejárása, illetve az elem feldolgozása. Ezzel az eljárás véget ért.

5.3. ábra. Bináris keresőfa inorder bejárása. A feldolgozás a csomópontok tartalmát kiírja a kimenetre. (folytatás)

5.2.3. Postorder bejárás

A *postorder* bejárás során is is ugyanazt a három műveletet végezzük el minden egyes csomópontban, azonban a tartalom feldolgozása a sorrendben az utolsó helyre kerül (5.3. algoritmus). Ennek megfelelően minden elem esetében az alábbi lépések történnek meg:

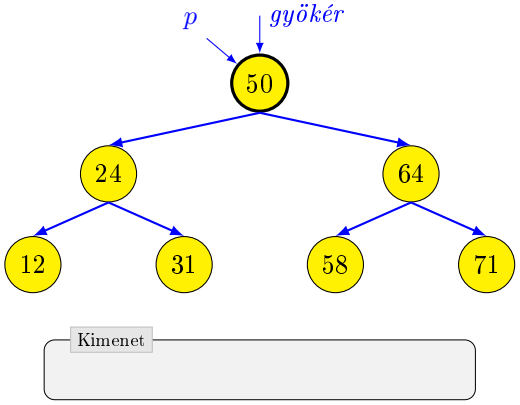
- Csomópont baloldali részfájának feldolgozása.
- Csomópont jobboldali részfájának feldolgozása.
- Csomópont tartalmának feldolgozása.

Egy lehetséges megvalósítást mutat az 5.3. algoritmus. Kezdetben ez is ellenőrzi, hogy a p változó értéke nem ø, aminek köszönhetően a rekurzió megáll a levélelemek alatt, illetve ugyanennek köszönhetően működik az algoritmus üres fa esetében is.

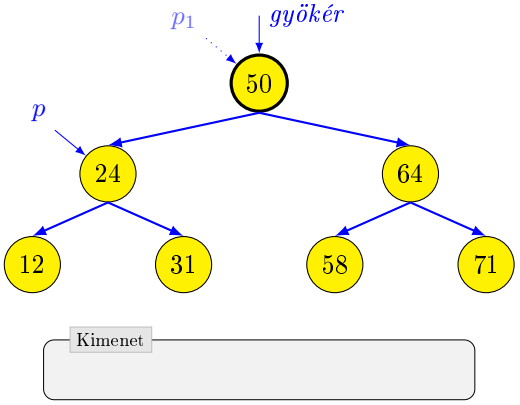
Az 5.4. ábra mutat egy példát a postorder bejárásra. Az algoritmus alapja, hogy először mindig bejárja a két részfát, majd csak ezt követően dolgozza fel az aktuális elem tartalmát. Ennek köszönhetően a teljes fa felszabadításakor jól használható, hiszen a feldolgozás mindig csak levélelemeken fog lefutni. Bár a bináris keresőfából való törlést csak később tárgyaljuk, de sejthető, hogy a leveleket lesz a legegyszerűbb kitörölni (az ábrán is látható, hogy a 12 és a 31 feldolgozása után következik a 24 feldolgozása (ez ugyan alapesetben nem levél, de ha a 12 és 31 elemeket már töröltük, akkor már ez is levél lesz).

Megjegyzés

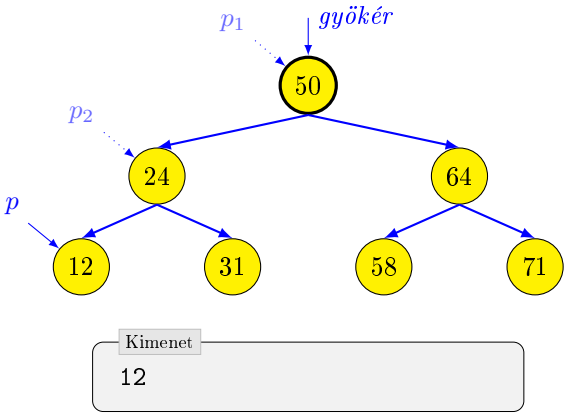
Érdemes lehet megfigyelni, hogy mindhárom bejárás esetében a rekurzió ugyanolyan sorrendben érintette az elemeket. Ha a feldolgozástól függetlenül csak a p mutató útját figyeljük, akkor látható, hogy a pre-, in- és postorder bejárások során is ugyanazt az utat járta be, a különbség pusztán annyi, hogy maga a feldolgozás mikor történt: amikor rálépett az elemre (preorder), amikor elhagyta az elemet (postorder) vagy a két részfa feldolgozása közben (inorder).



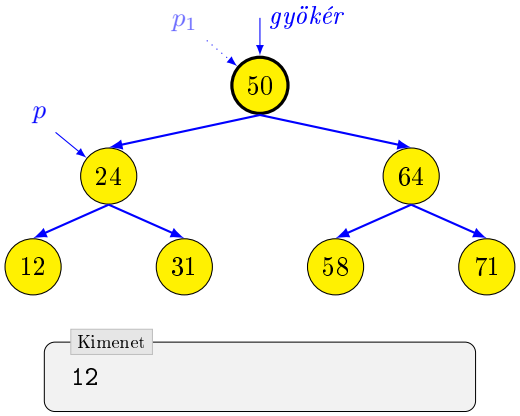
(a) A p az eljárás indításakor a gyökérelemre hivatkozik. A rekurzív algortmus első lépésének megfelelően először balra lép.



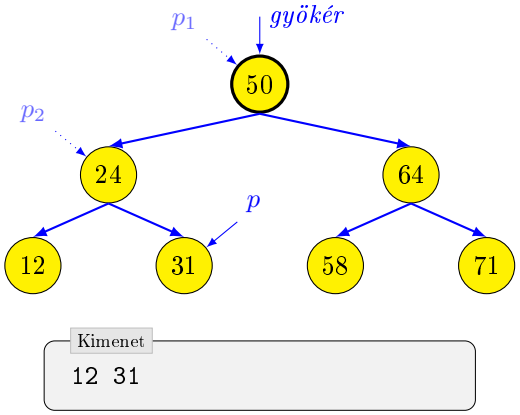
(b) A rekurzív algoritmus ellenőrzi, hogy az elem nem ø. Mivel nem az, így ismét balra lép.



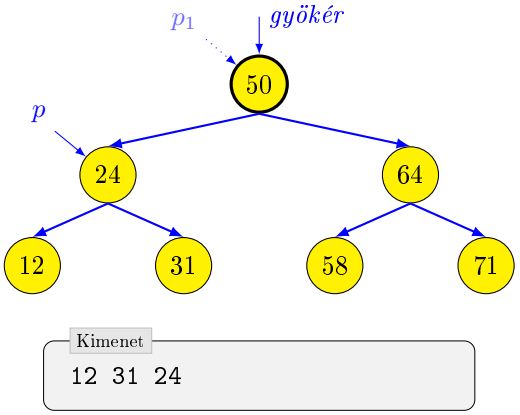
(c) A 12-es elemre érve, mivel már sem balra, sem jobbra nem tud továbblépni, feldolgozza az elemet. Ezt követően visszatér a rekurzió előző szintjére.



(d) A 24-re visszalépre azonnal megkezdi a jobboldali részfa feldolgozását.

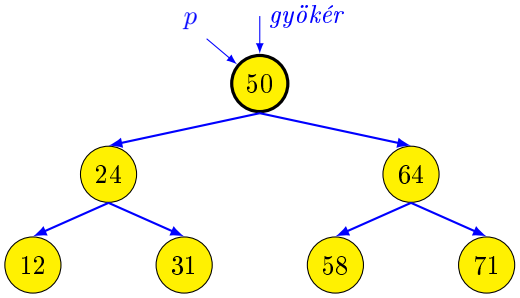


(e) A 31-es elem esetében már nincs lehetőség sem balra, sem pedig jobbra lépni. Ezért ezt az elemet is feldolgozza, majd visszalépés az előző szintre.



(f) Megtörtént a baloldal és jobboldal bejárása, így itt is megtörténik a feldolgozás, majd innen is visszalép a rekurzió az előző szintre.

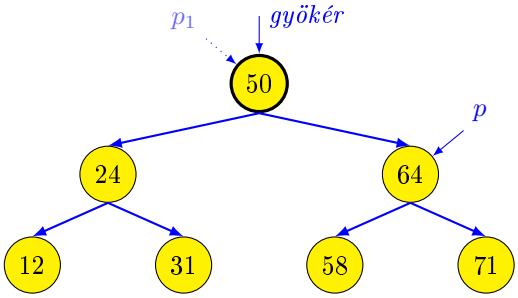
5.4. ábra. Bináris keresőfa postorder bejárása. A feldolgozás a csomópontok tartalmát kiírja a kimenetre.



Kimenet

```
12 31 24
```

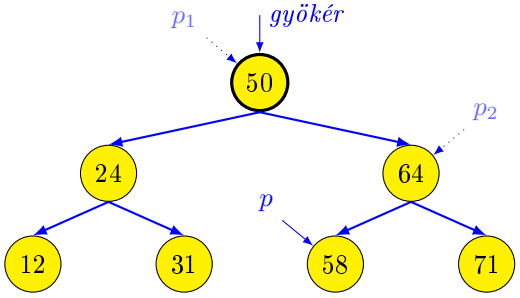
(g) A legfelső szinten a bal oldal bejárása már megtörtént. Ezt követi az a lépés, ahol az eljárás meghívja önmagát, paraméterként átadva a jobboldali részfát.



Kimenet

```
12 31 24
```

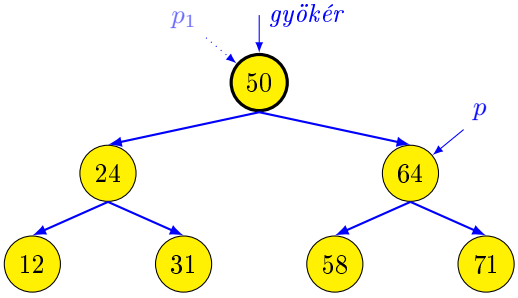
(h) A 64-es elem szintjén először ellenőrzi a bal oldalt. Mivel itt vannak elemek, így azonnal tovább is lép erre.



Kimenet

```
12 31 24 58
```

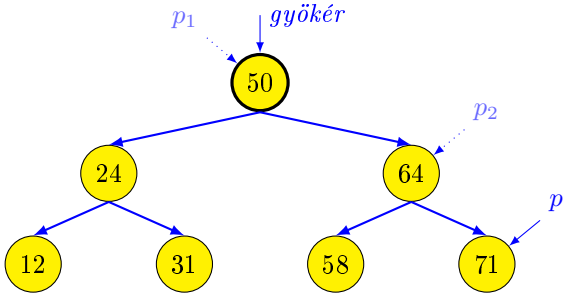
(i) Az eljárás ellenőrzi a bal részfáját, ami üres. Ezt követi a jobb oldal vizsgálata, ami szintén üres. Ezt követően az 58-as elemet feldolgozza, majd a rekurzió visszábblép egy szinttel.



Kimenet

```
12 31 24 58
```

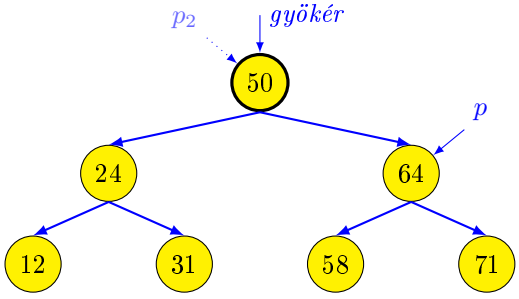
(j) A 64-es elem baloldala már fel lett dolgozva, ezért az algoritmus meghívja önmagát a jobboldali részfára.



Kimenet

```
12 31 24 58 71
```

(k) Az eljárás ellenőrzi az 71-es elemet. Első lépés a bal részfája, majd a jobb oldal vizsgálata, ami szintén üres. Végül feldolgozza az elemet, majd a rekurzió visszábblép egy szinttel.

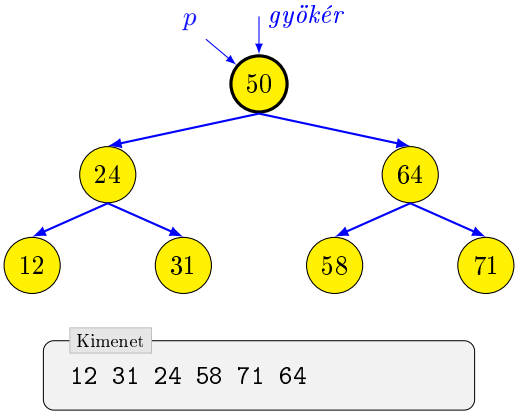


Kimenet

```
12 31 24 58 71 64
```

(l) Ezen a szinten már lefutott a két részfa bejárás, ezért most jön a 64 feldolgozása. Ezután a rekurzió egy szinttel visszalép.

5.4. ábra. Bináris keresőfa postorder bejárása. A feldolgozás a csomópontok tartalmát kiírja a kimenetre. (folytatás)



(m) A gyökérelemnél is megtörtént a bal és jobboldali részfa bejárása, ezért most következik az elem feldolgozása. Ezzel az eljárás véget ért.

5.4. ábra. Bináris keresőfa postorder bejárása. A feldolgozás a csomópontok tartalmát kiírja a kimenetre. (folytatás)

5.3. Algoritmus Bináris keresőfa postorder bejárása

Bemenet: p - **M**

```
1: eljárás PostorderBejárás(p : M)
2:     ha p ≠ ø akkor
3:         PostorderBejárás(p.bal)
4:         PostorderBejárás(p.jobb)
5:         Feldolgoz(p.tart)
6:     elágazás vége
7: eljárás vége
```

Eljárás hívása: PostorderBejárás(*gyökér*)

Felhasznált változók és függvények

- p: A fa aktuálisan vizsgált eleme.
- Feldolgoz(*elem* : **T**): Eljárás, ami feldolgozza a paraméterként átadott tartalmat.

5.2.4. Keresés

Keresés szempontjából meg kell különböztetni egymástól két esetet: ha a kulcs alapján keresünk, illetve ha a tartalomra vonatkozó szabályok alapján keresünk. A fa felépítése az előbbit támogatja, erről szól részletesebben ez a fejezet, itt a keresés paramétere egy kulcs típusának megfelelő érték, és vissza kell adnunk azt a tartalmat, ami ehhez a kulcshoz tartozik (tehát amit ezzel a kulccsal együtt szúrtunk be a fába). A tartalom alapján való keresés jóval rugalmasabban kezelhető, itt kereshetünk a tartalom típusának megfelelő konkrét értéket, de a programozási tételeknél megszokott valamilyen (F) feltételnek megfelelő elemet is. Mindkét esetben az előzőekben megismert bejárások egyikét kell úgy módosítani, hogy a feldolgozás tulajdonképpen a keresési feltétel kiértékelése legyen, és ha találtunk megfelelő elemeket, akkor az legyen a keresés visszatérési értéke.

A kulcs szerinti keresés ennél sokkal hatékonyabban megvalósítható. A rekurzív adatszerkezet felépítéséből adódóan itt is megadhatjuk a keresést rekurzív formában: megvizsgáljuk a fa gyökérelemét, amennyiben az a keresett kulcsot tartalmazza, akkor végeztünk (megtaláltuk), ha nem, akkor a kulcs értékétől függően keresünk a jobb-, illetve a baloldali részfában. A keresés akkor ér véget, ha megtaláltuk a keresett elemet, vagy elértünk a fa aljára (tehát a levél alatti **ø** értékekhez), és nincs hová tovább lépni.

A keresés algoritmusa (5.4. algoritmus) némileg hasonlít a bejárásokhoz, a lényeges különbség az, hogy a p nem mindig egy előre rögzített útvonalat jár be, hanem mindig a keresett értéknek megfelelően döntünk, hogy az egyes csomópontokból milyen irányba lépjen tovább.

Az algoritmus 2. sorában itt is megvizsgáljuk a p értékét, hogy az egy valódi elemre hivatkozik-e. Ez lesz a rekurzió egyik megállási feltétele, ugyanis egy **ø** érték azt jelenti, hogy az előző rekurziós szinten olyan irányba léptünk tovább, ahol már nincsenek elemek, így biztos nincs benne a keresett elem a bináris keresőfában. Ugyanez az ellenőrzés felelős azért is, hogy üres fa esetében az eljárás hiba nélkül visszatérjen (értelemszerűen egy nemleges válasszal, hiszen abban biztos nem szerepel a keresett elem).

Ezt követi a kulcs ellenőrzése, először megvizsgáljuk, hogy a p által aktuálisan hivatkozott csomópont kulcsa kisebb-e mint a keresett. Amennyiben igen, akkor a rendezettségnek megfelelően a keresett elem a jobboldali részfában kell, hogy legyen, így a függvény meghívja önmagát ebbe az irányba. Elképzelhető persze, hogy nincsenek gyerekei ezen az ágon, ilyenkor maga a meghívás még megtörténik, azonban a p értéke **ø** lesz, így az előző bekezdésben leírtaknak megfelelően a rekurzió a következő lépéssel végetér.

Amennyiben az előző feltétel nem teljesült, akkor megvizsgáljuk, hogy az aktuálisan vizsgált elem kulcsa nagyobb-e, mint a keresett. Ha igen, akkor ez utóbbi a baloldali részfában fog elhelyezkedni (ha egyáltalán van ilyen), így a függvény ismét meghívja önmagát, paraméterként átadva a baloldali gyerek hivatkozását.

Amennyiben a program futása a 9. sorhoz jut, akkor tudjuk, hogy a p egy létező bináris fa elemre mutat, amelyiknek a kulcsa nem is kisebb, illetve nem is nagyobb, mint a keresett elem, tehát megtaláltuk azt. A függvény visszatérési értéke tehát ennek a tartalmi része lesz. Érdemes megfigyelni, hogy ez a visszatérési érték csak az előző rekurziós szintre kerül visszaadásra, innentől minden szintnek biztosítania kell, hogy a megtalált elem referenciája továbbítódjon az őt hívó szint felé. Emiatt lényeges a 4. és 7. sorban látható módszer, ahol a rekurzívan meghívott függvénytől kapott visszatérési értéket adja vissza az aktuális szint is az őt hívónak.

A keresés lépésszáma ideális esetben $\mathcal{O}(log_2 N)$, de ez sajnos nem lesz minden esetben igaz. A bejárt út hosszát ugyanis jelentősen befolyásolja a fa szerkezete. Amennyiben ez egy, az 5.5. ábrán is látható „kiegyensúlyozott fa" (a kiegyensúlyozottság definíciójával foglalkozik a 6.1. alfejezet), akkor az $\mathcal{O}(log_2 N)$-es lépésszám valóban igaz. Nem kiegyensúlyozott esetekben ez azonban jóval több lépést igényel, pl. legrosszabb esetben, ha a fa minden csúcsának csak egy gyereke van, akkor ez a lineáris kereséshez hasonlóan $\mathcal{O}(N)$ lesz (a fa ilyenkor sokkal inkább hasonlít egy láncolt listára, így a keresés lépésszáma is annak megfelelő lesz).

A kulcs szerinti keresés mellett érdemes megvizsgálni további műveleteket, például a fában a legnagyobb, illetve legkisebb kulcsú elem kiválasztását. Amennyiben van legalább egy elem a fában, akkor a rendezettségből adódik, hogy ezek a fa legjobboldalibb illetve legbaloldalibb elemei lesznek (tehát a gyökérelemből elindulva folyamatosan balra, illetve jobbra lépkedve találjuk meg őket). A fának ezt a jellemzőjét még később, a törlésnél fel fogjuk használni.

5.4. Algoritmus Bináris keresőfában keresés

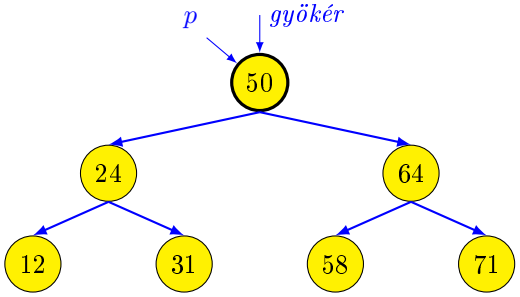
Bemenet: p - **M**, *kulcs* - **K**; ahol **K** összehasonlítható
Kimenet: *tartalom* - **T** (*a keresett kulcshoz tartozó tartalom*)

```
1:  függvény KERESÉS(p : M,  kulcs : K)
2:      ha p ≠ ø akkor
3:          ha p.kulcs > kulcs akkor
4:              vissza KERESÉS(p.bal, kulcs)
5:          különben
6:              ha p.kulcs < kulcs akkor
7:                  vissza KERESÉS(p.jobb, kulcs)
8:              különben
9:                  vissza p.tart
10:             elágazás vége
11:         elágazás vége
12:     különben
13:         hiba "nincs ilyen kulcs"
14:     elágazás vége
15: függvény vége
```

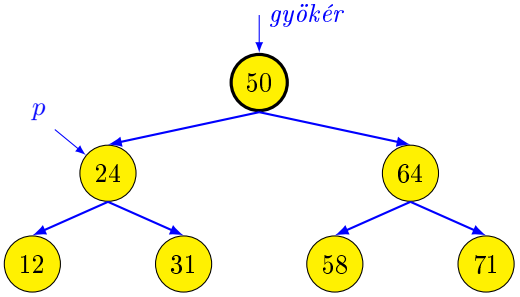
Függvény hívása: KERESÉS(*gyökér*, *kulcs*)

Felhasznált változók és függvények

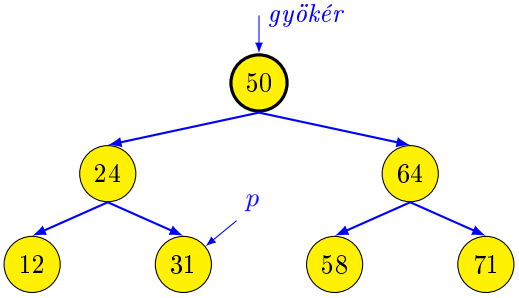
- p: A fa aktuálisan vizsgált eleme.
- *kulcs*: A keresendő kulcs.



(a) A p az eljárás indításakor a gyökérelemre hivatkozik. A keresett érték kisebb, mint 50, emiatt a függvény meghívja önmagát a baloldali részfára.



(b) A következő híváskor a p a 24-es elemre hivatkozik. Mivel ez kisebb a keresettnél, emiatt újrahívja magát a jobboldali részfára.



(c) A p által mutatott elem tartalma a keresett elem. Emiatt nem lép mélyebb szintre, hanem visszaadja ezt az értéket (a rekurzió visszatérését már nem tartalmazza az ábra).

5.5. ábra. Adott tartalmú elem keresése bináris keresőfában (a keresett érték a 31).

5.3. Új elem felvétele

A bináris keresőfa esetében is megvizsgáljuk az új elem elhelyezésének lehetőségét. A láncolt lista esetében erre számos megoldást áttekintettünk (elejére, végére, közepére, rendezetten beszúrás), ezekre a bináris keresőfa esetében nincs szükség, hiszen a fában lévő elemeket mindig kulcs szerint rendezetten szeretnénk eltárolni. Ez persze nem jelenti azt, hogy ne lehetne többféle beszúró algoritmust kidolgozni, de a mi esetünkben olyan megoldást keresünk, amelyik az új elemek beszúrásakor nem módosítja a már meglévő elemek egymáshoz viszonyított helyzetét, hanem csak az új elemet hozzáláncolja valamelyik már meglévő elem *bal* vagy *jobb* hivatkozásához. Figyelembe véve, hogy az elemeket rendezetten szeretnénk elhelyezni, ez egyben egyértelműen meg is határozza az elem leendő helyét.

A fába való beszúrás emiatt két fő lépésből áll: meg kell keresni az új elem leendő helyét, majd ezt az elemet hozzá kell kapcsolni a fa eddigi csomópontjaihoz.

Az 5.5. algoritmus 9–16. sorai már ismerősek lehetnek a keresésből. Itt keressük meg a beszúrandó elem helyét. Az új elem helyének megállapítása, és a keresés egymással nyilván szorosan összefügg, hiszen az új elemet olyan helyre kell elhelyeznünk, ahol az előzőleg megismert keresés majd meg fogja találni. Emiatt itt is ugyanazokat a lépéseket hajtjuk végre, attól függően, hogy az aktuálisan vizsgált elem kulcsa milyen viszonyban áll a beszúrandó kulccsal.

- Ha a p által hivatkozott csomópontban található kulcs nagyobb, mint a beszúrandó kulcs, akkor az új elemet az aktuálisan vizsgált elem baloldali részfájába kell majd elhelyezni (hiszen a keresés is ebbe az irányba fog majd haladni, amikor ezt vissza akarjuk olvasni). Ezért az eljárás rekurzívan meghívja önmagát, paraméterként pedig a p baloldali gyerekét adja át.

- Ha a p által hivatkozott csomópontban található kulcs kisebb, mint a beszúrandó kulcs, akkor az új elemet az aktuálisan vizsgált elem jobboldali részfájába kell majd elhelyezni. Ezért az eljárás rekurzívan meghívja önmagát, paraméterként pedig a p jobboldali gyerekét adja át.

- Ha az aktuális elem kulcsa egyenlő azzal, mint amit be szeretnénk szúrni, akkor jelezzük, hogy a beszúrást nem tudjuk végrehajtani, hiszen a fa felépítésekor már tisztáztuk, hogy a kulcsokat egyedinek tekintjük.

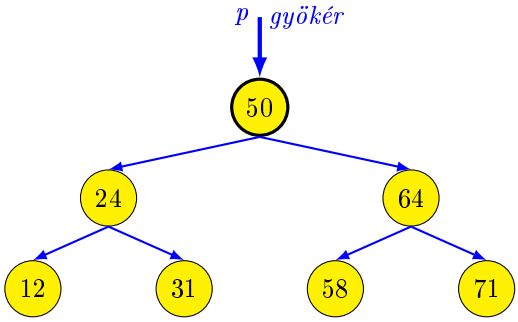
A fenti három lehetőség persze csak akkor értelmezhető, ha a p egy valódi elemre mutat a fában, amelyiknek tudjuk vizsgálni a kulcsát. Amennyiben a p változó értéke $\emptyset$, az két dolgot jelenthet: a rekurzió előző szintjén olyan irányba próbáltunk meg továbblépni, amerre már nincsenek elemek; vagy eleve üres volt a fa, és már a legelső híváskor is $\emptyset$ lett a paraméter értéke. Egyik se jelent hibát, hiszen éppen egy ilyen helyet kerestünk, ami üres, és a rendezettség szempontjából megfelel az új elemnek.

Hogy megértsük magát a beláncolási műveletet, fontos észrevennünk, hogy a p paramétert mindvégig címszerinti paraméterátadással adtuk át. Ennek hatása, a fent említett két esetben:

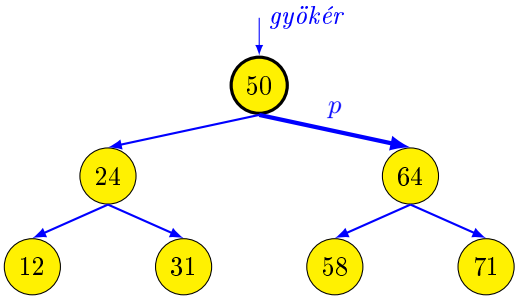
- Amennyiben a fa üres, akkor már a legelső híváskor igaz lesz a 2. sorban található feltétel, így lefut az alatta lévő létrehozás művelet. Mivel az első híváskor a *gyökér* változót adtuk át címszerinti paraméterátadással, így a p paraméter módosítása egyben ennek a megváltoztatását is jelenti. Tehát amikor az újonnan létrehozott elemre ráirányítjuk a p paramétert, akkor ezzel egyben a fa *gyökér* mutatóját is erre irányítjuk. Így az üres fából lesz egy egyelemű, a kért adatokat tartalmazó bináris keresőfánk.

- Amennyiben a fa nem üres, akkor biztos, hogy a rekurzió egy későbbi szintjén lesz csak igaz a 2. sorban található feltétel. Ebben az esetben a hívó függvény is ugyanez a függvény volt, és vagy a 10. sorban, vagy pedig a 13. sorban történt meg a hívás, mindkét esetben az előző rekurziós szinten található p által hivatkozott csomópont *bal* vagy *jobb* mezőjét adtuk át paraméterként. Ez számunkra most amiatt fontos, mivel a címszerinti paraméterátadás miatt ilyenkor a p változó módosításával megváltoztatjuk az előző csomópont paraméterként átadott gyerek hivatkozását is. Mivel a p-t a létrehozott elemre irányítjuk, így ennek megfelelően az előző szinten lévő csomópont megfelelő gyerek mutatója is erre fog hivatkozni.

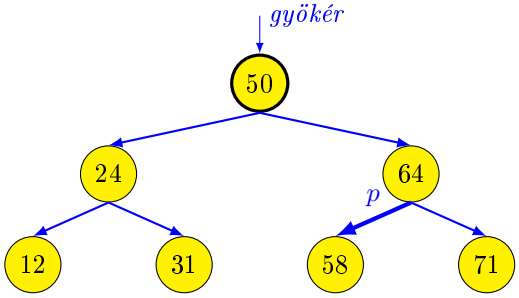
A címszerinti paraméterátadás fontosságát kiemeltük az 5.6. ábrán is. A p címke helyzete azt próbálja jelezni, hogy a p minden egyes híváskor az előző szinten lévő csomópont bal vagy jobb gyereke címszerinti paraméterátadással átadva.



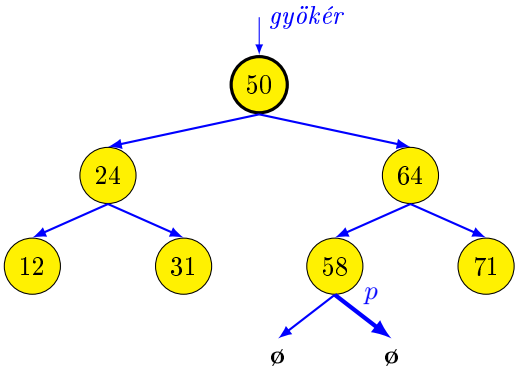
(a) A p az eljárás indításakor a gyökérelemre hivatkozik. Mivel ez egy létező elem, emiatt a rendezettségnek megfelelően jobbra lép.



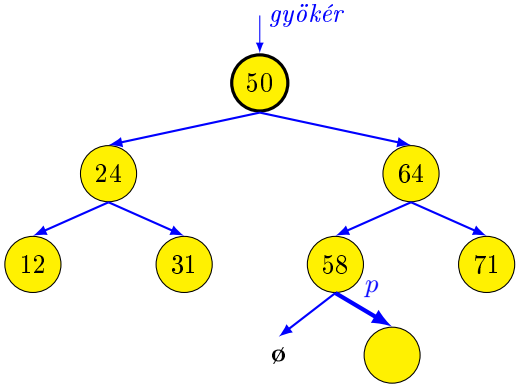
(b) A p által mutatott elem értéke miatt az algoritmus most balra lép.



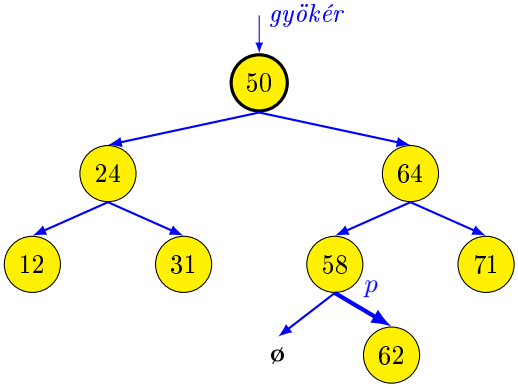
(c) A rekurzív algoritmus ellenőrzi, hogy az elem nem ø. Mivel nem az, így ismét jobbra lép.



(d) Nem szoktuk ugyan jelölni, de most kivételesen ábrázoljuk a fa leveleiből kiinduló lezáró hivatkozásokat. Látható, hogy a p az egyik ilyen értékre mutat.

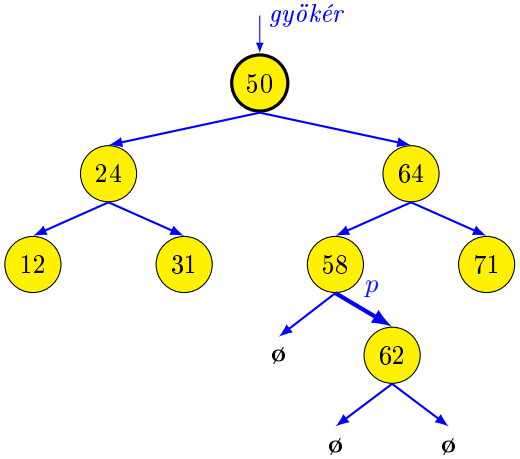


(e) Az új elem létrehozása. A p hivatkozás az új elemre állítódik, és mivel ez egy címszerint átadott paraméter, megváltozik a szülő jobb mutatója is.

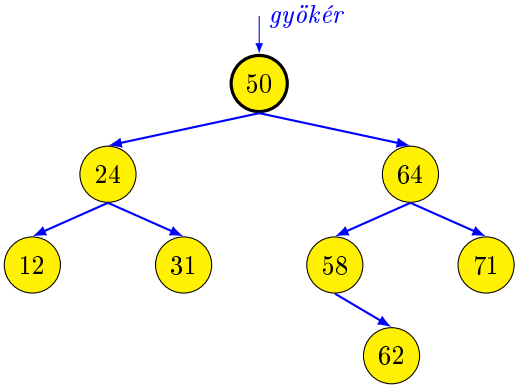


(f) Az algoritmus beállítja az új elem kulcs és tartalom mezőit.

5.6. ábra. Új elem felvétele bináris keresőfába (62-es kulcs).



(g) Az algoritmus beállítja az új elem bal és jobb mutatóját is a lezáró elemekre.



(h) Az új elem beszúrás végeredménye.

5.6. ábra. Új elem felvétele bináris keresőfába (62-es kulcs). (folytatás)

5.5. Algoritmus Bináris keresőfába új elem beszúrása

Bemenet: p - **M**, *kulcs* - **K**; ahol **K** összehasonlítható, *érték* - **T**
Kimenet: p - **M**

```
1:  eljárás BESZÚRÁS(címszerint p : M,  kulcs : K,  érték : T)
2:      ha p = ø akkor
3:          p ← LÉTREHOZ(M)
4:          p.kulcs ← kulcs
5:          p.tart ← érték
6:          p.bal ← ø
7:          p.jobb ← ø
8:      különben
9:          ha p.kulcs > kulcs akkor
10:             BESZÚRÁS(p.bal, kulcs, érték)
11:         különben
12:             ha p.kulcs < kulcs akkor
13:                 BESZÚRÁS(p.jobb, kulcs, érték)
14:             különben
15:                 hiba "már van ilyen kulcs"
16:             elágazás vége
17:         elágazás vége
18:     elágazás vége
19: eljárás vége
```

Eljárás hívása: BESZÚRÁS(*gyökér*, *érték*)

Felhasznált változók és függvények

- p: A fa aktuálisan vizsgált eleme.
- *kulcs*: A beszúrni kívánt kulcs.
- *érték*: A beszúrni kívánt tartalom.
- LÉTREHOZ(**M**) : **M**: Létrehoz egy **M** típusú elemet, ez a visszatérési értéke.

Látható, hogy a fenti beszúrás műveletet használva a kialakuló fa szerkezete jelentősen függ az elemek beszúrásának sorrendjétől. Ugyanabban a sorrendben megadott elemek természetesen mindig ugyanazt a fát adják, különböző sorrendek azonban más-más eredményhez vezethetnek.

A fában való keresés hatékonysága jelentősen függ a beszúrás során kialakított szerkezettől. Érdekes megfigyelni, hogy abban az esetben, ha az elemeket kulcs szerint rendezett sorrendben szúrjuk be a bináris keresőfába, akkor minden elem az előtte beszúrt (tehát addig legnagyobb) elem jobb gyerekeként kerül felfűzésre, így a lehető legrosszabb fa szerkezetet kapjuk meg.

Megjegyzés

A beszúrás sorrendjétől való függés meglehetősen nagy hátrányt jelent a hagyományos bináris keresőfa esetében, hiszen nem tudjuk garantálni az egyébként igen csábító $\mathcal{O}(log_2 N)$ lépésszámú keresést. Emiatt léteznek egyéb, az általunk tárgyalt bináris keresőfához hasonló adatszerkezetek, amelyek a beszúrás során igyekeznek úgy elhelyezni az új elemet (amit a már meglévő fa átstrukturálásával tudnak csak megoldani), hogy minél inkább megmaradjon a fa kiegyensúlyozott jellege (pl. kiegyensúlyozott fák [5], piros-fekete fák [2]).

5.4. Elem törlése

Az elem törlése valamivel bonyolultabb művelet, mint a beszúrás, ugyanis míg ott mi határozhattuk meg, hogy hová helyezzük el az új elemet (és ezt úgy tettük meg, hogy minél egyszerűbb legyen a beszúrás), addig a törlésnél már nincs hasonló szabadságunk, értelemszerűen onnan kell törölni az elemet, ahol az megtalálható. Bizonyos esetekben, pl. a levelek törlésénél ez kimondottan egyszerű lesz, de a két gyerekkel is rendelkező belső csomópontoknál már némileg összetettebb algoritmusra lesz szükségünk.

A törlés eljárásnak paraméterként a törlendő elem kulcsát adjuk át, emiatt első lépésként meg kell találnunk magát a csomópontot, ahol ezt eltároltuk (az 5.6. algoritmus). Ehhez igazodva a 3–8. sorok a már megismert keresést mutatják, a p által aktuálisan mutatott elem kulcsa alapján a rekurzió meghívja önmagát a bal, illetve a jobb oldali részfára, attól függően, hogy a törlendő elem kulcsa kisebb, vagy éppen nagyobb, mint az aktuálisan vizsgált. Miként az algoritmusban is látható (illetve az ábrákon is igyekeztünk ezt jelölni), a p paramétert mindig címszerinti paraméterátadással adjuk át, a későbbiekben ez még fontos lesz a működés szempontjából.

Amennyiben a program eljut a 9. sorig, akkor már tudjuk, hogy a p a törlendő elemre hivatkozik. A következő lépés ennek az elemnek a kiláncolása, majd pedig felszabadítása. Attól függően, hogy a törlendő elemnek hány darab gyereke van, három lehetőséget különböztetünk meg, amelyeket külön-külön vizsgálunk meg részletesebben: nincs gyereke, egy gyereke van, két gyereke van.

Elem törlése egy gyerek esetén

Elsőként célszerű ezt az esetet megvizsgálni, mivel látni fogjuk, hogy a levélelemek törlése ennek egy speciális változataként kezelhető. Vizsgáljuk meg az 5.7c. ábrát. Látható, hogy ez azt az állapotot mutatja, amikor már lefutott a keresés, és a p változó a törlendő elemre mutat. Az algoritmus ebben a pillanatban a 9. soron áll.

Ezt követi egy vizsgálat, ahol megnézzük, hogy hány gyereke van a törlendő elemnek. Elsőként megnézzük, hogy van-e neki baloldali gyereke (és ezzel együtt persze baloldali részfája). Amennyiben a $p.bal$ értéke $\emptyset$, akkor nincs. Az ábrán jól látható, hogy ha az egész fának csak a p közvetlen környezetét vizsgáljuk (annak szülőjével, és egyetlen jobb oldali gyerekével együtt), akkor egy láncolt listához hasonló szerkezetet találunk. Ennek megfelelően a törlés is hasonlóan történik, a szülő jobboldali hivatkozását át kell állítanunk a törlendő elem jobboldali elemére.

Ehhez szükség lenne egy referenciára a szülő elemre, amely azonban nem áll rendelkezésre, a bináris keresőfa egyes csomópontjai nem tárolnak el külön hivatkozásokat a szülőkre (miként a láncolt listánál sem volt hivatkozás az előző elemre). A listánál alkalmazott technika (egy e változóval a keresés közben mindig eltároltuk az előző elem címét) lehet az egyik lehetséges megoldás itt is, azonban itt már kicsit nehézkesebb lenne a törlés végrehajtása, hiszen tudnunk kell, hogy a szülő *bal* vagy *jobb* mezőjét kell-e átírni, továbbá azt is, hogy a törlendő elem bal-, vagy jobboldali gyerekére kell átirányítani. Az így megjelenő négy eset mellett még külön kellene foglalkoznunk a gyökérelem törlésével is, amely esetben magát a *gyökér* változót kellene módosítanunk.

Ehelyett a p paraméter címszerinti átadásával egy elegáns és egyszerű megoldást használhatunk, ami jól látható az algoritmus 11. sorában is. Mivel itt tudjuk, hogy a p által mutatott elemnek nincs baloldali gyereke, így egyszerűen átléptetjük a p paramétert annak jobboldali gyerekére. A címszerinti paraméterátadás miatt a p változtatása hatása lesz az előző rekurziós szinten paraméterként átadott változó értékére is, tehát ennek megfelelően megváltozik a szülő elem *jobb* hivatkozása is. Ez az eddigi p elem helyett annak jobboldali gyerekére fog hivatkozni. Ezzel az elem kiláncolása megtörtént.

A felszabadításhoz szükségünk van egy hivatkozásra a törlendő elemre, emiatt szükséges a q változó. Ez még a kiláncolás előtt eltárolja a felszabadítandó elem címét, majd a 12. sorban ennek segítségével ténylegesen fel is szabadítjuk ezt (amennyiben olyan programozási nyelven implementáljuk az algoritmust, ahol van automatikus szemétgyűjtés, akkor a q változóra egyáltalán nem is lesz szükség). Ezzel végeztünk is a törléssel.

A fent vázolt módszer csak abban az esetben működik, ha a törlendő elemnek nincs baloldali gyereke. Egészen hasonló módon tudjuk kezelni azt az esetet is, amikor a törlendő elemnek a jobboldali gyereke (és ezzel együtt a jobboldali részfa) hiányzik. A következő sorokban alapvetően a fent már részletesen áttekintett kódot láthatjuk, pusztán az irányok megcserélésével.

Érdemes belegondolni, hogy mi történik akkor, ha a gyökér elemet szeretnénk törölni. Az eljárás ilyenkor még nem hívta meg önmagát rekurzívan, hanem már az első híváskor megtaláltuk a törlendő

5.6. Algoritmus Bináris keresőfából elem törlése

Bemenet: p - **M**, *kulcs* - **K**; ahol **K** összehasonlítható
Kimenet: p - **M**

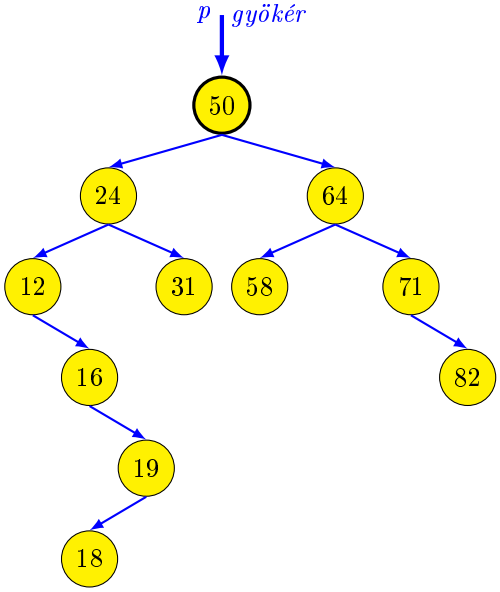
```
1: eljárás Törlés(címszerint p : M,  kulcs : K)
2:     ha p ≠ ø akkor
3:         ha p.kulcs > kulcs akkor
4:             Törlés(p.bal, kulcs)
5:         különben
6:             ha p.kulcs < kulcs akkor
7:                 Törlés(p.jobb, kulcs)
8:             különben
9:                 ha p.bal = ø akkor
10:                    q ← p
11:                    p ← p.jobb
12:                    Felszabadít(q)
13:                else
14:                    ha p.jobb = ø akkor
15:                        q ← p
16:                        p ← p.bal
17:                        Felszabadít(q)
18:                    különben
19:                        TörlésKétGyerek(p, p.bal)
20:                    elágazás vége
21:                elágazás vége
22:            elágazás vége
23:        elágazás vége
24:    különben
25:        hiba "nincs ilyen kulcs"
26:    elágazás vége
27: eljárás vége

28: eljárás TörlésKétGyerek(e : M,  címszerint r : M)
29:    ha r.jobb ≠ ø akkor
30:        TörlésKétGyerek(e,  r.jobb)
31:    különben
32:        e.tart ← r.tart
33:        e.kulcs ← r.kulcs
34:        q ← r
35:        r ← r.bal
36:        Felszabadít(q)
37:    elágazás vége
38: eljárás vége
```

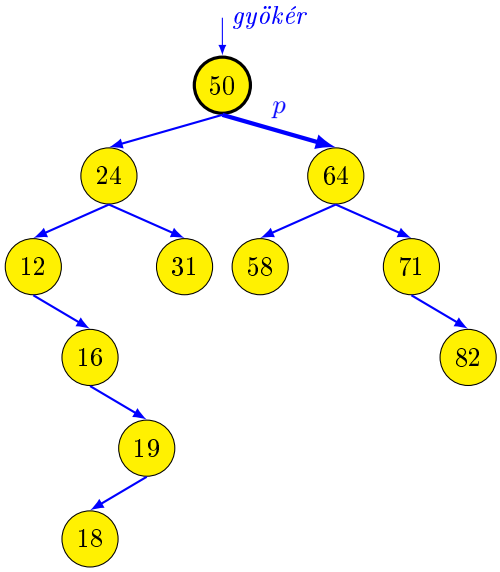
Eljárás hívása: Törlés(*gyökér*, *érték*)

Felhasznált változók és függvények

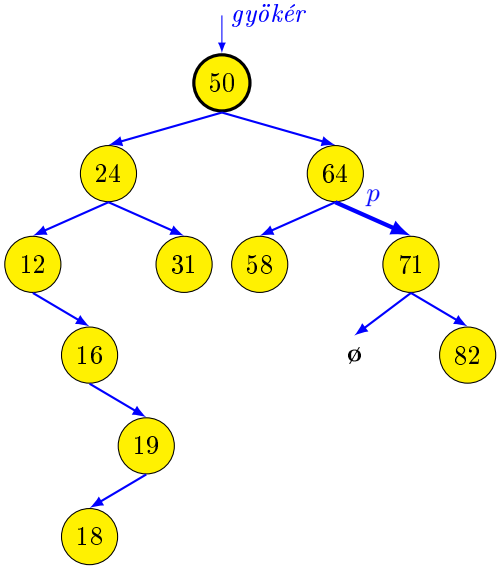
- p: A fa aktuálisan vizsgált eleme.
- *kulcs*: A törölni kívánt kulcs.
- e, r, q: Fa csúcs típusú segédváltozók.
- Felszabadít(p : **M**): Felszabadítja a paraméterként átadott elemet.



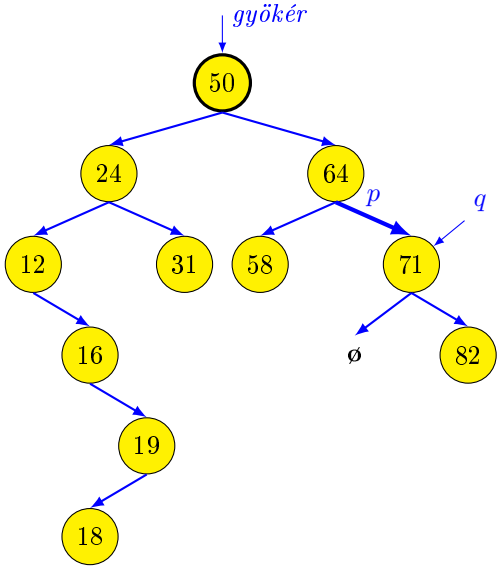
(a) A művelet első lépése a törlendő elem megkeresése. Jelen esetben a gyökértől jobbra kell továbblépni.



(b) Újabb lépés jobbra. Mint az ábrán is látható, a p mindig címszerinti paraméter.

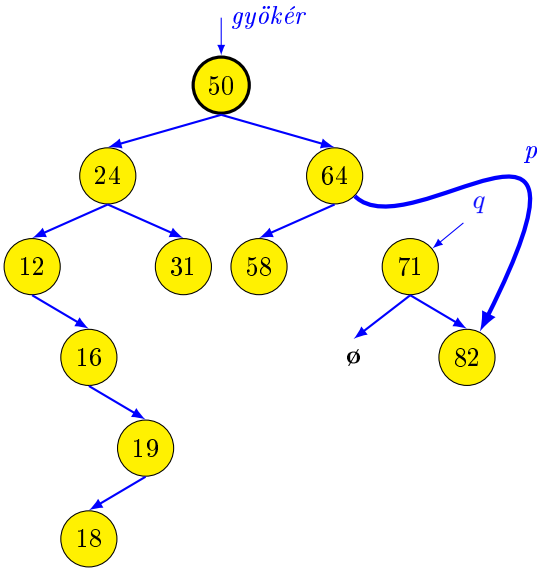


(c) A lefelé haladó rekurzió megtalálta a keresett elemet. Kivételesen berajzoljuk a lezáró jelet is, így látható, hogy nincs baloldali gyereke.

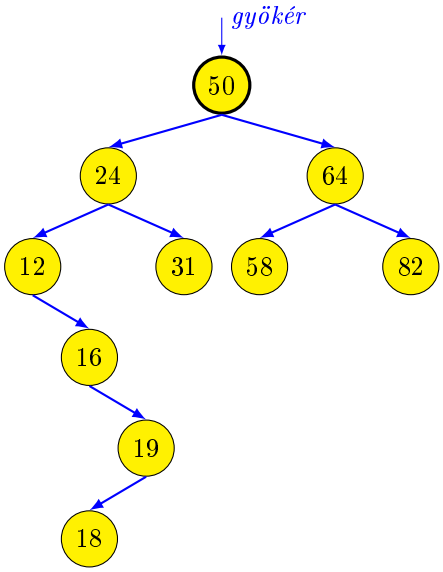


(d) A q hivatkozás eltárolja a törlendő elem címét, mivel a p-t innen tovább fogjuk léptetni.

5.7. ábra. Bináris keresőfából, egy gyerekkel rendelkező elem törlése (71-es elem).



(e) A baloldali részfa üres, a jobb oldaliban azonban vannak adatok. Emiatt a p átléptetése a jobb oldali részfára. A címszerinti paraméterátadás miatt változik a 64-es elem jobb hivatkozása is.



(f) A q elem törlése. Ezzel előáll a végeredmény.

5.7. ábra. Bináris keresőfából, egy gyerekkel rendelkező elem törlése (71-es elem). (folytatás)

elemet, amire a p hivatkozik. Az első híváskor paraméterként a *gyökér* változót adtuk át, tehát a p az nem más, mint a *gyökér* címszerinti paraméterátadással átadva. A p megváltoztatása (átléptetése a gyerek elemre) tehát együtt jár a paraméterként átadott *gyökér* változó változásával.

Megjegyzés

Habár a címszerinti paraméterátadás itt egy nagyon egyszerű és látványos eredményt adott, de néha problémákat okozhat ennek használata, pl. olyan implementációknál, ahol a választott programozási nyelv nem támogatja ezt (Java). Ebben az esetben egy másik megoldást kell keresnünk, de tekintve, hogy csak egy paramétert adtunk át címszerint, és az eljárásnak jelen esetben nincs visszatérési értéke, a módosítást egyszerűen megtehetjük egy függvénnyé való átalakítással, és a címszerinti paraméterátadás helyett a függvény visszatérési értékének kihasználásával.[6]

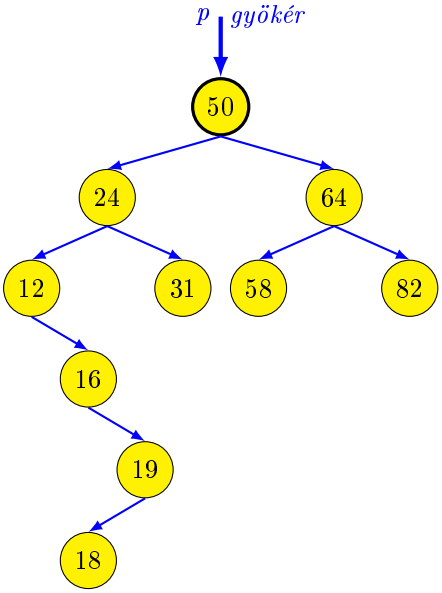
Elem törlése levél esetén

Bár külön esetként vizsgáljuk, de valójában ez az előző egy speciális részeseteként kezelhető. Erre mutat egy példát az 5.8. ábra, ezen belül is vizsgáljuk meg az 5.8c. ábrát. Ez ismét azt az állapotot mutatja, amikor már lefutott a keresés, és a p változó a törlendő elemre mutat. Az algoritmus ebben a pillanatban a 9. soron áll.

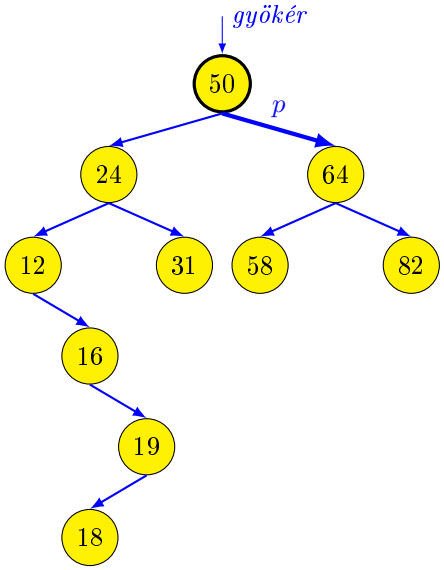
Ezt követi egy vizsgálat, ahol megnézzük, hogy hány gyereke van a törlendő elemnek. Elsőként megnézzük, hogy van-e baloldali gyereke (és ezzel együtt persze baloldali részfája). Amennyiben a $p.bal$ értéke ø, akkor nincs. Jelen esetben ez a feltétel igaz, tehát lefut az elágazás igaz ága.

Észrevehető, hogy az előző példánál, amikor csak egy gyereke volt a törlendő elemnek, akkor is pont ez az ág futott le. Valójában ugyanis a teendők itt is azonosak. Már az előző példánál tisztáztuk a címszerinti paraméterátadás jelentőségét a p paraméter esetében. Jelen esetben itt is ugyanaz történik, a p változóba betöltjük a jobboldali gyerekére mutató hivatkozásának értékét. Míg az előző példában ez egy tényleges elemre való hivatkozás volt, addig itt ennek az értéke most ø.

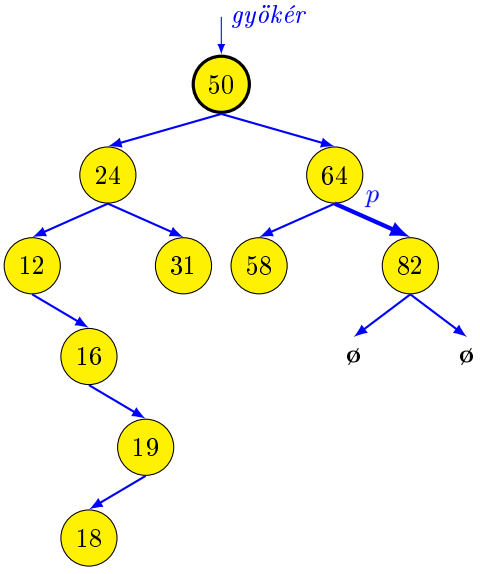
A p új értéke tehát ø lesz, és a címszerinti paraméterátadás szabályainak megfelelően ezzel együtt megváltozik az a változó is, amit paraméterként átadtunk a rekurzió előző szintjén. Ez pedig nem más volt, mint a szülő elem jobb oldali mutatója. Így ebbe a mezőbe is ø érték kerül.



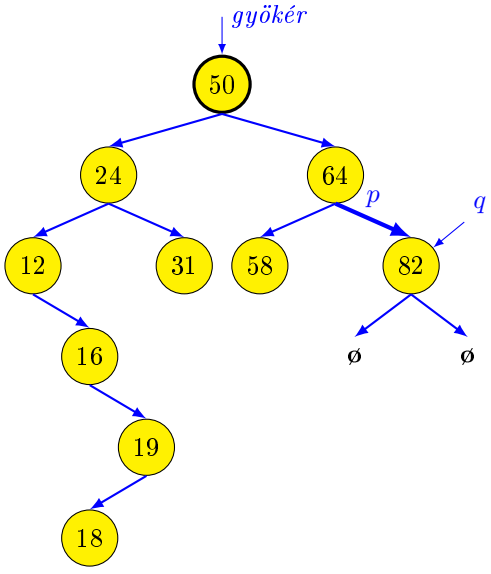
(a) Az első lépés itt a törlendő elem keresése. A gyökéből a rekurzió jobbra indul.



(b) Újabb lépés jobbra. Mint az ábrán is látható, a p mindig címszerinti paraméter.

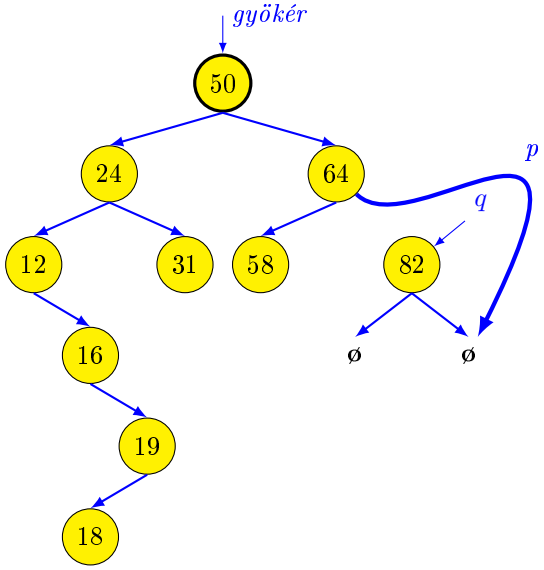


(c) Egy újabb jobbra lépés után a p a törlendő elemre hivatkozik. Kivételesen ábrázoljuk a lezáró jeleket is, így látható, hogy nincs gyereke.

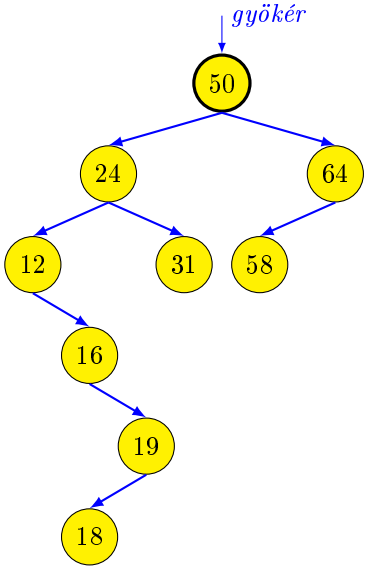


(d) A q hivatkozás eltárolja a törlendő elem címét, mivel a p-t innen tovább fogjuk léptetni.

5.8. ábra. Bináris keresőfából levélelem törlése (82-es elem).



(e) A baloldali részfa üres, a jobb oldaliban azonban vannak adatok. Emiatt a p átléptetése a jobb oldali részfára. A címszerinti paraméterátadás miatt változik a 64-es elem jobb hivatkozása is.



(f) A q elem törlése. Ezzel előáll a végeredmény.

5.8. ábra. Bináris keresőfából levélelem törlése (82-es elem). (folytatás)

Az elem felszabadítás is hasonlóan történik, a q segédváltozóval eltároljuk a felszabadítandó elemet, majd a p átléptetése után ezt felszabadítjuk.

Itt is célszerű lehet megvizsgálni azt az állapotot, amikor a gyökérelemet kell törölnünk. A p ilyenkor nem más, mint maga a *gyökér* változó címszerinti paraméterátadással átadva. Amikor a p-nek értékül adjuk a ø értéket, azzal tulajdonképpen a *gyökér* változónak is ezt adjuk, tehát az eredmény egy üres fa lesz. Ez helyes megoldás, hiszen ez az állapot azt jelentette, hogy már csak egy darab levélelemünk van a fában, és ezt kitöröltük.

Elem törlése két gyerek esetében

Amennyiben a törlendő csúcsnak két gyereke is van (tehát mindkét oldalra egy-egy részfa lett kapcsolva), akkor a törlés már nem oldható meg ilyen egyszerűen. Itt már nem tudjuk a kiláncolást a láncolt listánál látható módon megvalósítani, hiszen a szülőből (vagy gyökérből) csak egy hivatkozás mutat a törlendő elemre, onnan viszont két további hivatkozás halad tovább a bal-, illetve jobboldali részfára.

Emiatt az 5.9. ábrán látható módon próbáljuk megvalósítani a törlést. Ismét átugorhatjuk a keresés lépéseit, és érdemes a az 5.9c. ábránál folytatni a vizsgálatokat. Az algoritmus ilyenkor már a 19. sornál áll, tehát megtaláltuk a törlendő elemet, a (címszerinti paraméterként átadott) p változó erre mutat, és az előző vizsgálatok eredményeképpen már látjuk, hogy mindkét oldalon vagy egy-egy gyereke.

A módszer alapja az, hogy egy kisebb átrendezést végzünk el a fában, mielőtt ténylegesen törölnénk belőle a szükséges elemet. Az átrendezés az alábbi lépéseket igényli:

- A törlendő elemnél kisebb kulcsú elemek közül a maximális kulcsú elem megkeresése. A fa rendezettségéből adódik, hogy ez az elem a törlendő csomópont baloldali részfájának legjobboldalibb eleme lesz (tehát a nála kisebbek közül a legnagyobb).
- Ennek az elemnek a kulcsát és tartalmát átmásoljuk a törlendő elembe. Könnyen belátható, hogy ezt megtehetjük, mivel nem szegjük vele meg a fa rendezettségét. Érdemes az 5.9e. ábrán egy konkrét példán is megnézni ezt az állapotot. A másolás három helyen befolyásolhatja a rendezettséget, de egyik sem okoz problémát:
  - Az 50-es elem szempontjából nincs változás, hiszen eddig is mindkét elem a baloldali részfájában volt, most is így maradtak.

  - A 24-es eddigi bal részfája szempontjából a rendezettség továbbra is jó lesz, hiszen a felmásolt elem a részfa legnagyobb kulcsát tartalmazta, tehát ezek az elemek az ő baloldalán jó helyen vannak.
  - A 24-es eddig jobb részfáját se kell módosítani, hiszen itt eddig is a 24-nél nagyobb elemek lehettek csak, ha a 24-et kicseréljuk egy kisebbre, attól a rendezettség még biztosan megmarad.

- Töröljük az előbb megtalált csomópontot a fából, hiszen nem szükséges, hogy az elem kétszer is benne legyen. Maga a törlés itt már biztosan megoldható egyszerűen, hiszen ennek a csomópontnak nem lehet két gyereke (akkor nem lehetne ő a legjobboldalibb elem a részfában).

A TörlésKétGyerek( ) eljárás tartalmazza ennek a módszernek a lépéseit. Mivel egy részfa legnagyobb kulcsú elemét kell megtalálnunk, egy rekurzióval addig lépegetünk az aktuális csomópont jobboldali gyerekére, amíg az létezik. Az eljárás tehát először addig hívogatja magát a jobboldali részfájára, amíg a jobboldali hivatkozás értéke ø nem lesz. Az eljárásnak két paramétere van, az első (e) a törlendő elemre mutat, ez a hívások között nem is változik. A második (r) pedig magát az itt leírt keresést végzi.

Ezt követi a tartalom és kulcs átmásolása. A 32–33. sorokban elvégzi ezt a másolást, felülírva a törlendő elem tartalmát és kulcsát. Az előzőekben már beláttuk, hogy ez a másolás bármikor megtehető, nem fogja befolyásolni a fa rendezettségét.

Az utolsó lépés az átmásolt tartalmú elem kitörlése. Ez az r mutató balra léptetésével történik. Miként az ábrákon is jeleztük, ezt a mutatók mindig címszerinti paraméterátadással adtuk át. Az r paraméter változtatása tehát hatással van a mutatott elem szülőjének a jobboldali mutatójára is (jelen esetben tudjuk, hogy ő a szülő jobb gyereke, hiszen a keresésből ez adódik). Két különböző esetet vizsgálhatunk itt meg:

- Ha az r által mutatott csomópontnak van baloldali gyereke, akkor az r változót erre léptetjük át. Ez pedig egyben azt is jelenti, hogy az r által mutatott elem szülőjének jobb mutatója is erre hivatkozik majd (ez látható az ábrán).

- Ha az r által mutatott elemnek nincs baloldali gyereke, akkor a *bal* mező értéke ø. Ilyenkor ez lesz az r változó értéke is, ami a címszerinti paraméterátadásnak köszönhetően egyben a szülő jobboldali mutatójába is kerül. Tehát ilyenkor a szülő elem jobb mezőjét lezárjuk, hiszen abba az irányba már nem folytatódik a fa.

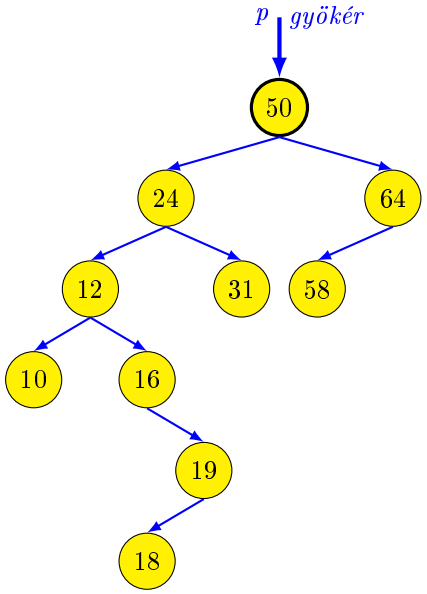
A q mutató szerepe itt is hasonló, mint a többi törlési módszernél, ezzel eltároltuk a ténylegesen felszabadítandó elem referenciáját (ami jelen esetben nem egyenlő a törlendő kulcsot tartalmazó elemmel!). A kiláncolást követően ez az elem már nyugodtan felszabadítható.

Megjegyzés

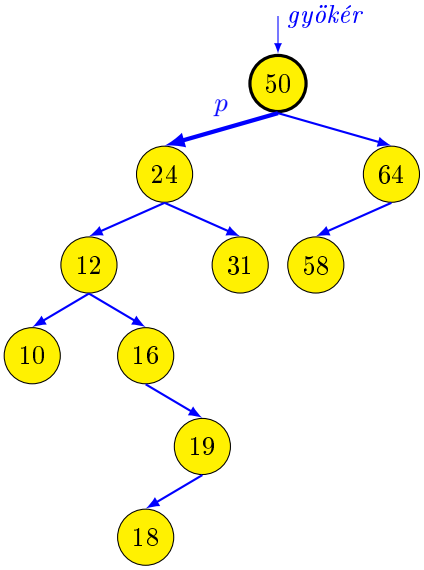
Érdemes még néhány speciális esetet figyelembe venni a két gyerekkel rendelkező elemek törlésénél (5.9. ábra):

- Hogyan működik az algoritmus akkor, ha a 12-es elemnek nincs egy jobboldali gyereke sem, tehát a részfa gyökéreleme egyben a legnagyobb elem is.
- Hogyan működik az algoritmus akkor, ha a 12-es elemnek se jobb-, sem pedig baloldali gyereke nincs. Tehát valójában ő a részfa egyetlen (így értelemszerűen legnagyobb) eleme.

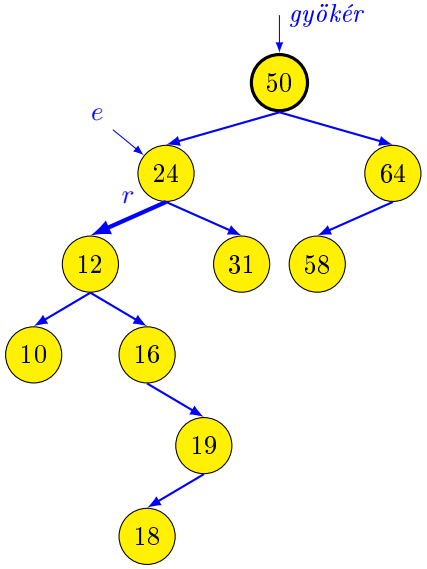
Mindkét esetben működni fog a tanult módszer, de érdemes egy-egy példán végigpróbálni, hogy pontosan milyen lépéseket hajt végre.



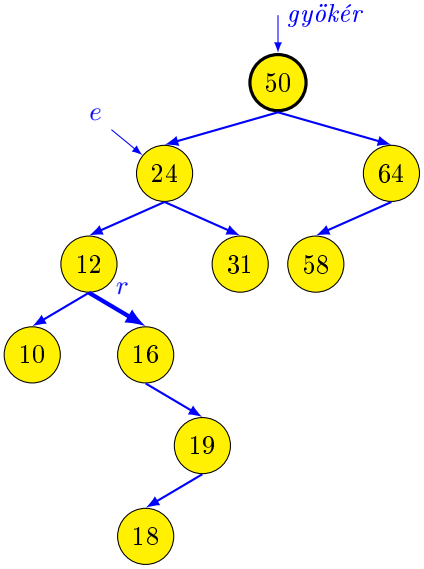
(a) Az első lépés itt is a törlendő elem keresése. A gyökérből a rekurzió balra indul.



(b) A következő szinten már meg is találtuk a törlendő elemet. Mint látható, mindkét hivatkozása létező elemekre hivatkozik, emiatt megkeressük a baloldali részfa legjobboldalibb elemét.

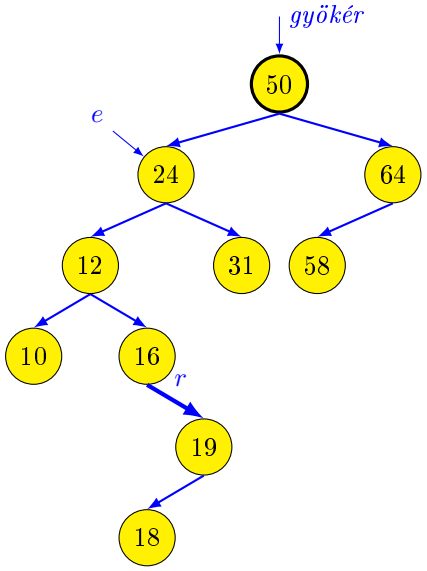


(c) A következő rekurzió során a e mindig a törlendő elemre mutat, az r pedig kezdetben annak a baloldali gyerekére (címszerinti paraméterátadással). Mivel ez egy másik eljárás, így az előző p itt nem érthető el, ezért nem is ábrázoljuk. Mivel van az r-nek jobboldali gyereke, így átlépünk oda.

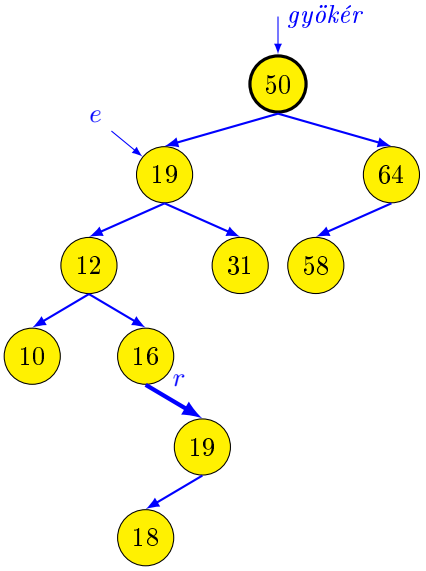


(d) Az r által mutatott elemnek még mindig van jobboldali gyereke, így a függvény meghívja önmagát, paraméterként átadva ezt az elemet. A r mindig címszerint kerül átadásra.

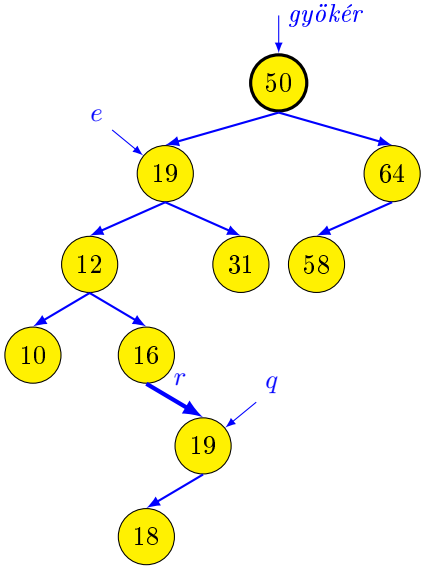
5.9. ábra. Bináris keresőfából belső csúcs törlése (24-es elem).



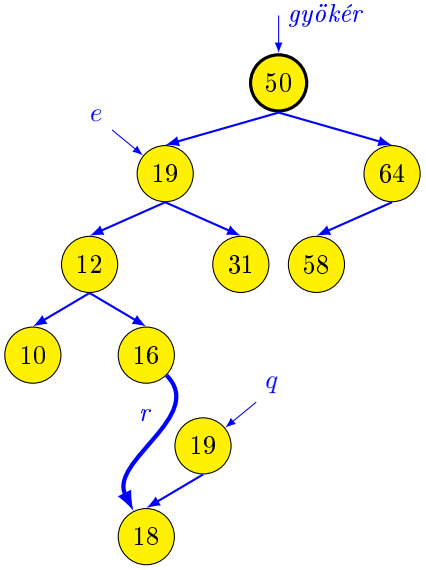
(e) A r által mutatott elemnek már nincs jobb oldali gyereke, így a rekurzió nem halad tovább, hanem folytatja a törlést.



(f) Tudjuk, hogy az r által mutatott elem kulcsa a törlendő elemnél kisebbek közül a legnagyobb. Emiatt ennek értékét felmásolhatjuk a törlendő tartalom helyére.

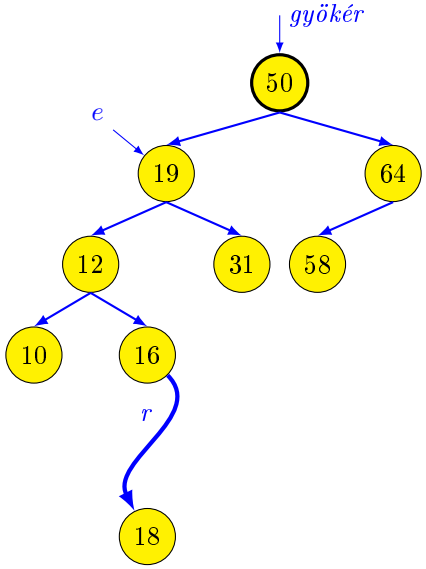


(g) A q változóval eltároljuk az r jelenlegi értékét, mivel hamarosan ezt kell majd kiláncolnunk.

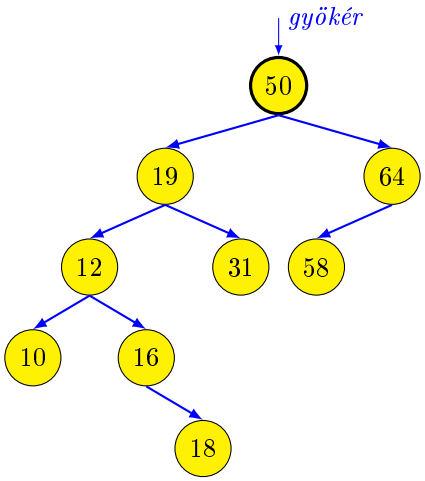


(h) Majd átléptetjük az r-t az általa mutatott elem baloldali gyerekére. A címszerinti paraméterátadás miatt itt megváltozik a 16-os elem jobb hivatkozása is. Ezzel az eredeti 19-et tartalmazó elemet kiláncoltuk a fából.

5.9. ábra. Bináris keresőfából belső csúcs törlése (24-es elem). (folytatás)



(i) Ezt követi a q által hivatkozott elem felszabadítása.



(j) A törlés végeredménye.

5.9. ábra. Bináris keresőfából belső csúcs törlése (24-es elem). (folytatás)

6. fejezet

B-fa

6.1. Fák kiegyensúlyozottsága

Az előző fejezetben megismert bináris keresőfa esetében előnyként hoztuk fel a gyors keresési lehetőséget, aminek a várható lépésszáma N darab kulcs és megfelelően felépített fa esetében $\mathcal{O}(log_2 N)$. Ugyanakkor az is könnyen belátható, hogy amennyiben a fa szerkezete kevésbé ideális, tehát az egyes levelek mélysége egymástól jelentősen különbözik, akkor ez a várható átlagos lépésszám is jóval nagyobb lesz, legrosszabb esetben akár $\mathcal{O}(N)$ lépésre is szükség lehet. Annak érdekében, hogy valamilyen formában pontosítsuk a fák „jóságát", vezessük be az alábbi fogalmakat:

- Kiegyensúlyozott fa: Az azonos szinten található részfák mélységei között legfeljebb egy szintnyi különbség lehet.
- Teljesen kiegyensúlyozott fa: Minden csúcsra igaz, hogy az abból kiinduló részfák csúcsainak száma legfeljebb eggyel különbözik egymáshoz képest.
- Teljes fa: Minden csúcsra igaz, hogy az abból kiinduló részfák csúcsainak száma azonos.

Célunk természetesen a minél inkább kiegyensúlyozott fák felépítése, bár a fenti feltételeket nem is tudjuk minden esetben teljesíteni (pl. két kulcsot tartalmazó bináris keresőfa nyilvánvalóan nem lehet teljes). A megismert bináris keresőfa beszúró és törlő algoritmusok nem foglalkoztak a fa kiegyensúlyozottságával, így ezekben az esetekben a fa felépítésében csak a beérkező kulcsok sorrendje játszott szerepet. Ettől függően (szerencsés esetben) a fa lehetett akár teljes is, de (szerencsétlen esetben) elfajult is, ami jelentősen lerontotta a keresés hatékonyságát. A bináris keresőfának is léteznek továbbfejlesztései (pl. piros-fekete fa [2]), amelyek a beszúrás és a törlés során igyekeznek olyan formába átalakítani a fát, hogy az minél inkább közelítsen a kiegyensúlyozott állapothoz, ebben a fejezetben azonban egy másik lehetséges megoldással, a B-fák használatával foglalkozunk.

6.2. B-fa felépítése

A B-fa tulajdonképpen a bináris keresőfa általánosításaként is tekinthető, azt annyiból egészíti ki, hogy egy csomópont egynél több kulcsot is tartalmazhat, és ezzel együtt a gyerekek száma is lehet kettőnél több (6.1. ábra). Mindez persze jelentősen megváltoztatja a fa felépítését, illetve a rajta különböző műveleteket végző algoritmusok működését.

A B-fa minden egyes csomópontja az alábbi mezőket tartalmazza (minden csomópont **M** típusú):

- *n*: Egy egész szám, ami a csomópontban aktuálisan eltárolt kulcsok darabszámát mutatja (később látni fogjuk, hogy erre különféle korlátok fognak majd vonatkozni).
- *levél*: Logikai érték, ami azt mutatja, hogy az adott csúcs levél (nincs gyereke), vagy pedig nem levél (vannak gyerekei). Értelemszerűen el is tárolható ez az érték egy külön mezőben, de akár futás közben is számolható.
- $kulcs[1], kulcs[2], ..., kulcs[n]$: A csomópontban eltárolt kulcsok. Ezek típusa a bináris keresőfához hasonlóan bármi lehet, egyetlen követelményünk, hogy összehasonlítható legyen. A kulcsok típusa a pszeudokódokban: **K**.
- $gyerek[1], gyerek[2], ..., gyerek[n+1]$: A belső csúcsok hivatkozásai a gyerekekre. A gyerekmutatók száma mindig eggyel több, mint a kulcsok száma. A gyerek hivatkozások típusa a kódokban: **M**.
- $tartalom[1], tartalom[2], ..., tartalom[n]$: A kulcsok mellett természetesen valamilyen tartalmat is tárolhatunk az egyes csomópontokban. Mivel ezek a tartalmak mindig a kulcsokkal együtt jönnek létre, mozognak, illetve törlődnek, így külön nem fogjuk a pszeudokódokban ezt jelölni. Minden kulcs művelethez odaértjük a hozzátartozó tartalom módosítást is. A tartalom mező típusa: **T**.

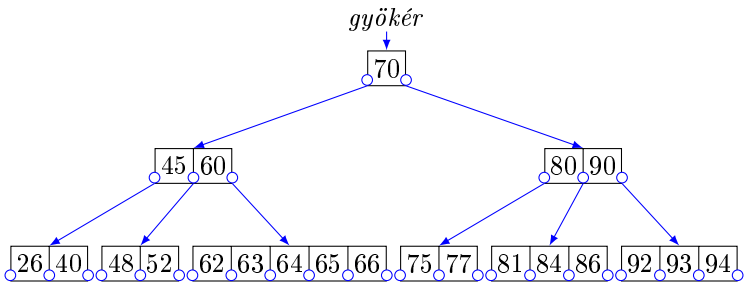
A B-fa jellemzői:

- Rendezett: Minden csúcs esetében igaz, ha k_i egy kulcs a $gyerek[i]$ gyökerű részfában, akkor:

$$k_1 \leq kulcs[1] \leq k_2 \leq kulcs[2] \leq ... \leq k_n \leq kulcs[n] \leq k_{n+1} \tag{6.1}$$

- Kiegyensúlyozott: Minden levél mélysége azonos.
- Korlát a kulcsok darabszámára: Az egyes csúcsokban található kulcsok darabszáma alulról és felülről is korlátos, ezt egy előre rögzített t számmal jelöljük (B-fa minimális fokszáma). Ez alapján:
  - Minden csúcsnak legfeljebb $(2t-1)$ darab kulcsa lehet (tehát legfeljebb $2t$ darab gyereke). *Telítettnek* nevezzük azt a csúcsot, amelyiknek már $(2t-1)$ kulcsa van.
  - Minden nemgyökér csúcsnak legalább $(t-1)$ darab kulcsa kell, hogy legyen (tehát legalább t darab gyereke).

Az általunk használt beszúrás és törlés művelet garantálni fogja, hogy a B-fa minden módosítás után teljesíteni fogja mindhárom feltételt. Érdemes megjegyezni, hogy a minimumfeltétel a gyökérelemre nem vonatkozik (ami logikus, hiszen egy olyan fa esetében, ami csak egy elemet tárol, nem tudnánk azt teljesíteni).



6.1. ábra. B-fa példa. A számok a fában tárolt kulcsok, a tartalmakat nem jelöljük. Az egyes kulcsok előtt/között/mögött látható körök a gyerekekre hivatkozó mutatók (a mutatók száma minden csomópontban eggyel több, mint a kulcsok száma). A fa legalsó szintjén a gyerekmutatók értéke **ø**, ezt nem ábrázoljuk.

Másodlagos tároló használata

Az ábrán is feltűnő, hogy a bináris keresőfához képest a legnagyobb különbséget az jelenti, hogy a B-fa adatszerkezet esetében egy csomópontban nem csak egy, hanem egyszerre több kulcsot is tárolunk, ami értelemszerűen nem csak kettő, hanem annál több gyerek hivatkozással is jár. Ennek köszönhetően azonos kulcsszám mellett csökken a fa magassága (a gyökérből a levelekig vezető leghosszabb út hossza). A kiegyensúlyozott bináris fa esetében n csomópont tárolásához $log_2 n$ magas fára volt szükség, a B-fánál a képlet hasonló, azonban a csomópontok nagyobb „elágazási tényezőjének" hatására a logaritmus alapja 2-nél nagyobb lehet. Bizonyítható, hogy a fa magasságára igaz az alábbi összefüggés [2]:

$$h \leq log_t \frac{n+1}{2} \tag{6.2}$$

Ahol

- n: A B-fában lévő kulcsok száma ($n \geq 1$).
- t: A fent említett minimális fokszám ($t \geq 2$).
- h: A fa magassága.

Tévedés lenne azonban azt gondolni, hogy ez a tény önmagában indokolná a B-fák létezését. Míg a bináris keresőfa esetében a keresés lépésszáma a gyökérből a levelekig vezető út hosszával volt jellemezhető, addig a B-fa esetében ez már nem igaz. Itt ugyanis az egyes csomópontokon belül már nem tudjuk egy egyszerű összehasonlítással eldönteni, hogy melyik irányba folytassuk a keresést, hiszen nem csak egy, hanem akár $(2t-1)$ darab kulcsunk is lehet. A kulcsokat rendezve tároljuk, tehát használhatjuk a logaritmikus keresést, de belátható, hogy a t értékének növelésével (ami a fa magasságának csökkenését jelenti) arányosan növekszik a kulcsokon belüli műveletek lépésszáma.

A B-fa hatékonyságának megértéséhez tudnunk kell azt is, hogy ezt az adatszerkezetet tipikusan a másodlagos tárolókon való működésre tervezték. A bináris keresőfa esetében eddig mindig úgy képzeltük, hogy a teljes fa a memóriában helyezkedik el, így meglehetősen gyorsan tudunk az egyes csomópontok között mozogni. Nagy mennyiségű adat esetében már kénytelenek vagyunk a fát a másodlagos háttértáron elhelyezni, ez esetben viszont minden egyes csomópont hozzáférés egy-egy háttértár írási/olvasási műveletet is jelent, ami drasztikusan lecsökkenti a teljesítményt.

Ez az a pont, ahol megjelenik a B-fa igazi erőssége. Napjainkban másodlagos tárolóként még mindig elsősorban merevlemezeket használunk, amelyek az adatokat sávokra, illetve azon belül blokkokra bontva tárolják. Egy írási/olvasási műveletnél a legtöbb időt a fej mozgatása, illetve a lemezek megfelelő irányba állítása igényli, maga az írás/olvasás ehhez képest már rövid ideig tart. Emiatt a technika sajátossága, hogy egy teljes blokk kiírása/beolvasása közel ugyanannyi ideig tart mind egy byte írása/olvasása. A B-fa pedig nagyon jól kihasználja ezt a lehetőséget, ugyanis ha célszerűen úgy választjuk meg a t paramétert, hogy egy B-fa csomópont egy merevlemez blokkot foglaljon el, akkor egy hozzáféréssel mindig egy teljes csomópontot tudunk írni/olvasni. Tehát a B-fa egy $(2t-1)$ darab kulcsot tartalmazó csomópontjának a kiírása/beolvasása ugyanannyi ideig tart, mint a bináris keresőfa egyetlen kulcsot tartalmazó csomópontjáé.

Tehát az valóban igaz, hogy a B-fa keresés elvi lépésszáma hasonló, mint a bináris keresőfáé, azonban ha megkülönböztetjük az új csomópontokra lépés műveletét (ami merevlemez műveletet igényel) és a csomóponton belüli keresést (ami már jóval gyorsabb memóriaművelet), akkor látható, hogy ez előbbi jóval hatékonyabban működik a jelenleg használt másodlagos tárolók esetében.

> **Megjegyzés**
>
> Éppen ez a megkülönböztetés miatt a szakirodalomban gyakran külön jelzik a B-fa algoritmusok esetében a háttértár olvasási és írási műveleteket. Az könnyebb olvashatóság kedvéért mi ezeket elhagyjuk, de értelemszerűen ezeket odaérthetjük minden csomópont váltás elé és módosítás mögé.

6.3. Beszúrás B-fába

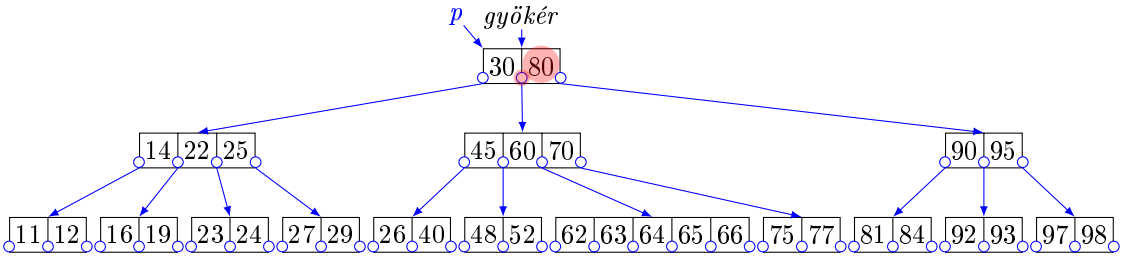
A B-fába beszúrás alapgondolata hasonló a bináris keresőfánál tanulthoz. B-fa esetében is élünk azzal a lehetőséggel, hogy mindig csak levélelembe szúrunk be új elemeket, mivel ebben az esetben nem kell a fa már meglévő részeit átalakítani, elég lokálisan, a megfelelő levélnél végrehajtani a módosítást (új kulcs felvétele). Ennek megfelelően a beszúrás első lépése itt is egy keresés, amivel a B-fában megkeressük, hogy melyik levélnek kell majd tartalmaznia az új kulcsot. Ez a keresés nyilván azonos az általános B-fában való kulcs szerinti kereséssel, hiszen az új elemet oda kell elhelyeznünk, ahol az utóbbi majd keresni fogja.

A kiegyensúlyozottság fenntartása felvet azonban néhány további követelményt, amelyekhez igazodnunk kell. A bináris keresőfa esetében a beszúrást megelőző keresés egy olyan üres helyet keresett, ahova új levélként fel lehetett venni az új kulcsot. A B-fa esetében ez azonban nem járható út, mivel egy már kiegyensúlyozott B-fába nem tudunk felvenni egy új levelet egy már meglévő alá anélkül, hogy ne szegjük meg azt a feltételt, hogy a gyökérből minden levélhez vezető út hosszának azonosnak kell lennie. Szerencsére erre nincs is szükség, mivel a B-fa egy csúcsa több kulcsot is tartalmazhat, így azt a megoldást választjuk, hogy a keresés során talált legalsó, még létező csúcsba felvesszük az új kulcsot. Ezt az esetet mutatja a 6.2. ábra.

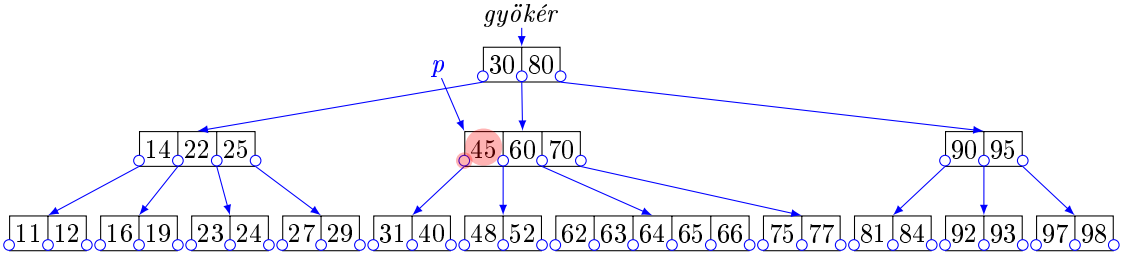
Ez persze csak akkor valósítható meg, ha legalább egy szabad hely még van ebben a levélben. A fa aktuális tartalmától függően ez vagy fennáll, vagy sem, biztosak nem lehetünk benne, tehát úgy kell elkészítenünk az algoritmust, hogy ha szükséges, tudja garantálni ezt az üres helyet. A gyakorlatban ezt a vágás nevű művelettel tudjuk megvalósítani, aminek a célja az, hogy egy telített csúcsot szétvágunk két kisebb (a minimumfeltételt éppen teljesítő) csúcsra, míg a középső kulcsot felvisszük a szülőbe.

A vágás csak akkor végezhető el, ha a szülőben van szabad hely még egy kulcs számára. Emiatt az előbbi követelményt, hogy a keresés során csak olyan belső csúcsra lépjünk, amelyiknek van szabad kulcs helye általánosíthatjuk, hogy általában csak olyan elemre lépjünk tovább, ahol van szabad hely. Az új elem helyének keresése során tehát mielőtt a rekurzió meghívná önmagát valamelyik részfa gyökerére, előtte ellenőrzi, hogy az telített-e. Ha nem, akkor azonnal továbblép ebbe az irányba, ha pedig igen, akkor szétvágja azt. Erre a vágásra mutat példát a 6.3. ábra (illetve maga a vágási művelet a 6.3b–6.3c. ábrákon látható).

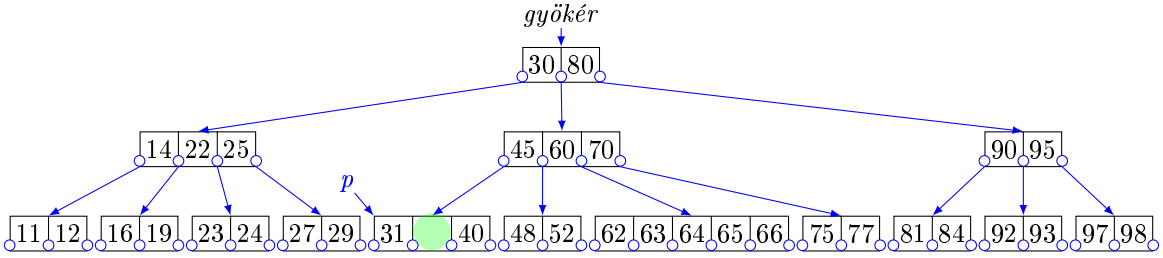
Bár a fentieknek megfelel ez is, de mégis speciális esetként kezelhető az, amikor a gyökérelem telített. A fenti szabály itt is alkalmazható, emiatt szétvágjuk ezt a csúcsot, annyiból viszont különbözik az általános esettől, hogy itt nincs szülő elem, ahová fel lehetne vinni a középső elemet. Emiatt ilyenkor létrehozunk egy új csomópontot, ez lesz az új gyökér. A gyökérelem szétvágására mutat példát a 6.4. ábra.



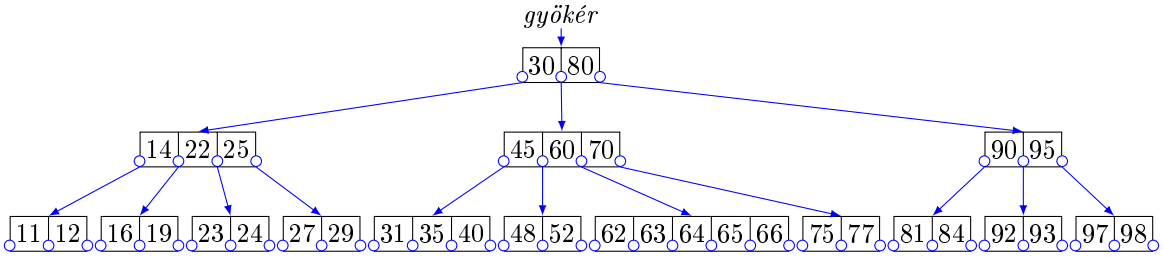
(a) Kezdetben a p a gyökérelemre hivatkozik. A kulcskeresés megadja, hogy az új elemet melyik részfába kell elhelyezni.



(b) A következő szint még egy belső csúcs, így itt is továbblépünk a keresés eredményének megfelelően.

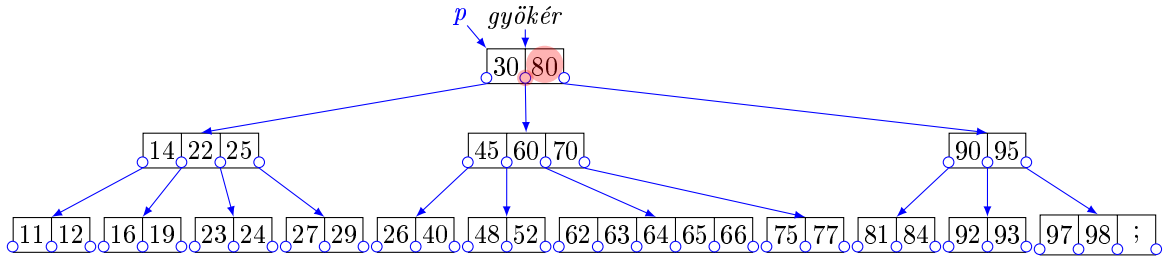


(c) A keresés eljutott egy levélelemhez. Ide el lehet helyezni az új kulcsot.

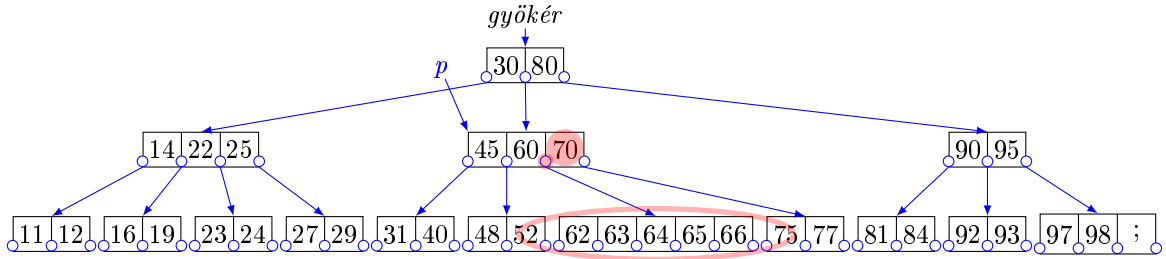


(d) A B-fa a beszúrás után.

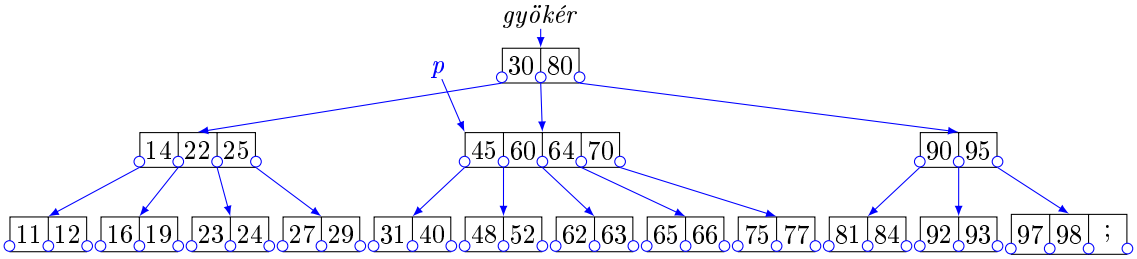
6.2. ábra. B-fába való beszúrás általános esete (35 beszúrása).



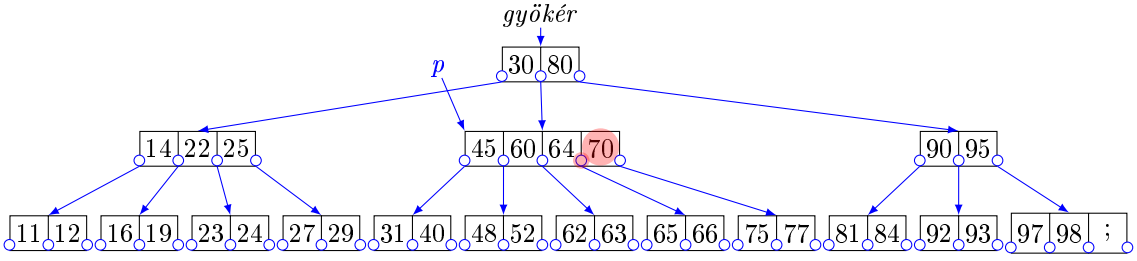
(a) Kezdetben a p a gyökérelemre hivatkozik. A kulcskeresés megadja, hogy az új elemet melyik részfába kell elhelyezni.



(b) A következő szintről nem tudunk továbblépni, mivel a következő csúcs már telített.

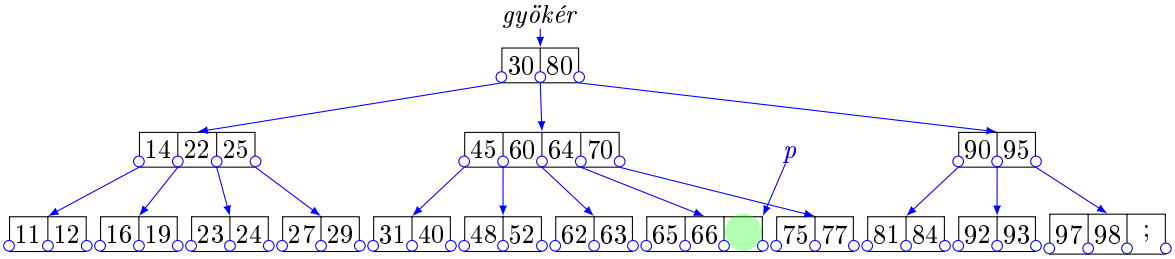


(c) Emiatt a következő elemet szétvágjuk, ez azzal jár, hogy egy kulcsot fel kell vinni a szülőbe.

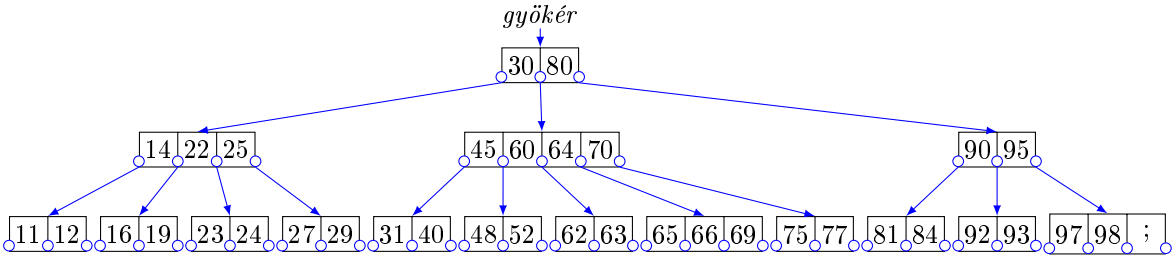


(d) A módosítás miatt újra meg kell állapítani a továbblépési irányt.

6.3. ábra. B-fába való beszúrás során elem szétvágása (69 beszúrása).

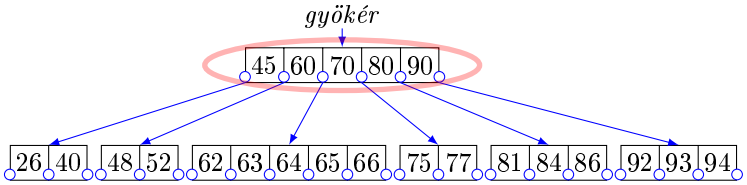


(e) Ismét eljutottunk egy levélig, ide már elvégezhető a beszúrás.

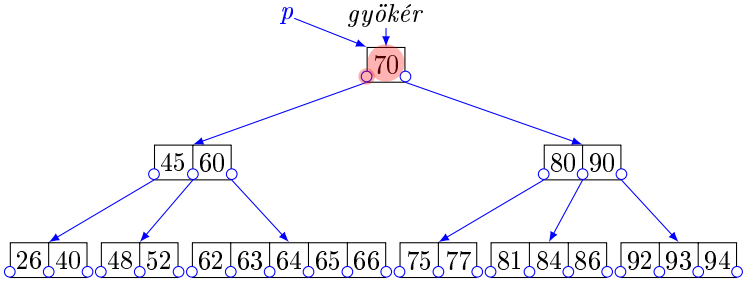


(f) A beszúrás eredménye.

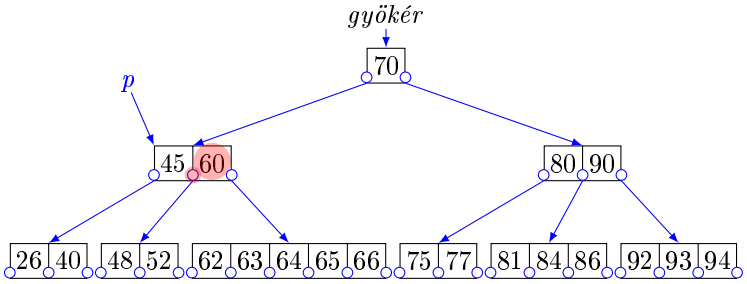
6.3. ábra. B-fába való beszúrás során elem szétvágása (69 beszúrása). (folytatás)



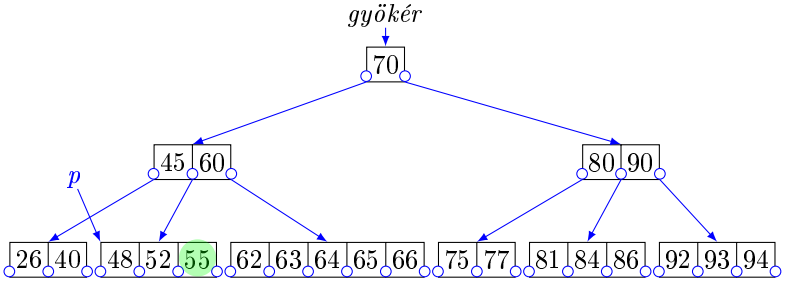
(a) A beszúrásnál már az első továbblépést se tudjuk megtenni, mivel a gyökérelem telített. Emiatt még a rekurzió indítása előtt szétvágjuk. A vágás során létrehozott új elem az új gyökér.



(b) A szétvágást követően a keresés a szokásos módon folytatódik.



(c) A keresés folytatható a következő szintre.



(d) A következő iterációban a levélbe már elhelyezhető a beszúrandó elem.

6.4. ábra. B-fába való beszúrás során a gyökér szétvágása (55 beszúrása).

6.3.1. Beszúrás pszeudokódja

A beszúrás a bináris keresőfához hasonlóan rekurzív, itt azonban több speciális esettel is kell foglalkoznunk, emiatt nem közvetlenül a rekurzív eljárást hívjuk meg, hanem először egy segédfüggvényt. Ennek első paramétere a B-fa gyökere, a második pedig a beszúrandó kulcs értéke (miként említettük, a kulcs mellett külön tartalommal a pszeudokódokban nem foglalkozunk, de természetesen ez is megjelenhetne harmadik paraméterként). Ezt mutatja be a 6.1. algoritmus. A két említett speciális eset:

6.1. Algoritmus Beszúrás a B-fába

Bemenet: *gyökér* - **M**, *érték* - **K**
Kimenet: *gyökér* - **M**

```
1:  eljárás BESZÚRÁS(címszerint gyökér : M, érték : K)
2:      ha gyökér = ø akkor
3:          gyökér ← LÉTREHOZ(M)
4:          gyökér.levél ← igaz
5:      különben
6:          ha gyökér.n = 2 * t − 1 akkor
7:              p ← gyökér
8:              gyökér ← LÉTREHOZ(M)
9:              gyökér.levél ← hamis
10:             SZÉTVÁGÁS(p, gyökér)
11:         elágazás vége
12:     elágazás vége
13:     KERESÉSBESZÚR(gyökér, érték)
14: eljárás vége
```

Felhasznált változók és függvények

- *érték*: A kulcs, amit be kell szúrni a B-fa megfelelő helyére.
- *gyökér*: A B-fa gyökere, ami esetleg meg is változhat az eljárás futása közben.
- *p*: A szétvágáshoz használt segédváltozó.
- LÉTREHOZ(**M**) : **M**: Létrehoz és visszaad egy új B-fa csúcsot.
- SZÉTVÁGÁS(p : **M**, *szülő* : **M**): Szétvágja a p mutató által hivatkozott csomópontot, amelyiknek a szülőjére a *szülő* paraméter hivatkozik (6.5. algoritmus).
- KERESÉSBESZÚR(p : **M**, *érték* : **K**): Megkeresi az *érték* kulcs helyét a p gyökerű B-fában, majd elvégzi annak beszúrását (6.2. algoritmus).

A 2. sorban lévő ellenőrzés azt vizsgálja meg, hogy a fa gyökérelemének értéke **ø**-e. Amennyiben igen, ez azt jelenti, hogy a fa jelenleg üres, így a beszúrás egy új gyökérelem felvételét jelenti. Az újonnan létrehozott elemnek egyetlen kulcsa lesz csak (ennek megfelelően beállítjuk a n változó értékét), ez a kulcs pedig a paraméterként kapott érték, de ezt majd a rekurzió fogja felvenni a többi esethez hasonló módon.

Amennyiben a fa nem üres, úgy meg kell vizsgálnunk egy másik speciális esetet, a telített gyökér esetét. Ezt teszi a 6. sor, amely ellenőrzi, hogy a gyökérelemben hány kulcs található. Amennyiben a kulcsok száma egyenlő a maximális értékkel, akkor a már fent említett módon létrehoz egy új elemet (ez lesz majd az új gyökér), majd szétvágja úgy az aktuális gyökérelemet, hogy a középső elem ebbe az új csúcsba kerüljön. Ezt követi a *gyökér* mutató aktualizálása.

A kezdeti karbantartó funkciókat követően, a fentiektől függetlenül elindítjuk magát a beszúrást végző rekurziót a 13. sorban. Az eljárásnak átadjuk paraméterként a fa aktuális gyökerét, illetve a beszúrandó kulcsot.

A 6.2 eljárás mutatja a rekurziót, ami szintről szintre lépkedve megkeresi azt a levelet, ahová majd az újonnan beszúrandó elemet el kell helyezni. A KERESÉSBESZÚR eljárás a rekurzió első szintjén a fa gyökerét kapja paraméterként, amennyiben ez nem levél, akkor megkeresi a kulcsok alapján, hogy melyik irányba kell tovább folytatni a keresést, és rekurzívan meghívja önmagát arra a gyerekre. Amennyiben egy levélhez érkezett, akkor pedig meghívja a BESZÚRÁSLEVÉLBE eljárást, ami elhelyezi az új kulcsot a levélben. Miután ez megtörtént, végeztünk is a beszúrással, így ezen a szinten további műveletek nem futnak le. A rekurzió szintenként visszatér a gyökérelemig.

6.2. Algoritmus Beszúrandó elem helyének megkeresése és az új kulcs beszúrása

Bemenet: p - **M**, *érték* : **K**

```
1: eljárás KeresÉsBeszúr(p : M, érték : K)
2:     ha p.levél akkor
3:         BeszúrásLevélbe(p, érték)
4:     különben
5:         gy ← KulcsKeresés(p, érték)
6:         ha p.gyerek[gy].n = 2 * t − 1 akkor
7:             Szétvágás(p.gyerek[gy], p)
8:             ha p.kulcs[gy] < érték akkor
9:                 gy ← gy + 1
10:            elágazás vége
11:        elágazás vége
12:        KeresÉsBeszúr(p.gyerek[gy], érték)
13:    elágazás vége
14: eljárás vége
```

Felhasznált változók és függvények

- p: A csomópont, amelyen belül keresünk.
- *gy*: A továbblépéshez szükséges gyerekre hivatkozó segédváltozó.
- *érték*: A kulcs, amit keresünk.
- BeszúrásLevélbe(p : **M**, *érték* : **K**): Beszúrja az *érték* kulcsot a p paraméter által hivatkozott levélbe (6.3. algoritmus).
- KulcsKeresés(p : **M**, *érték* : **K**) : **egész**: Megkeresi a p mutató által hivatkozott csomópontban az *érték* kulcs helyét. Ha már van ilyen kulcs a csomópontban, akkor annak az indexét, ellenkező esetben az első nála nagyobb kulcs indexét adja vissza (vagy $p.n$-t ha ilyen sincs) (6.4. algoritmus).
- Szétvágás(p : **M**, *szülő* : **M**): Szétvágja a p mutató által hivatkozott csomópontot, amelyiknek a szülőjére a *szülő* paraméter hivatkozik (6.5. algoritmus).

Ha azonban nem levélelemen állunk, akkor biztosan tovább kell majd lépnünk egy szinttel lejjebb. Az 5. sorban található KulcsKeresés eljárás megadja, hogy a csúcsban lévő kulcsok értékei alapján milyen irányban kell majd továbblépni a kereséskor.

A tényleges továbblépés előtt azonban ellenőriznünk kell, hogy a folytatáshoz kiválasztott elem nem telített-e. Ezt a 6. sorban látható módon egyszerűen elvégezhetjük, hiszen elég megvizsgálni, hogy a kiszemelt gyereknek hány kulcsa van. Amennyiben ez a feltétel igaz, akkor a már említett módon szét kell vágni ezt az elemet. A szétvágás paramétereként meg kell adni a szétvágandó gyereket, illetve annak szülőjét (ami most az aktuális, p által hivatkozott csúcs), mivel a szétvágás során egy kulcsot fel fog majd hozni ez utóbbiba. Amennyiben történt szétvágás, akkor az eddig gyerekből kettő lett, így újra ellenőriznünk kell, hogy a kettő közül melyikre kell folytatni a rekurzív keresést. Ha a felhozott kulcsnál nagyobb a beszúrandó kulcs, akkor aktualizálnunk kell a megfelelő gyerek indexet, hiszen nem az eddigi jelölt felé, hanem annak jobb testvére felé (az újonnan létrehozott elem felé) kell folytatni a keresést.

Függetlenül attól, hogy történt-e szétvágás, vagy pedig sem, az eljárás a 12. sorban meghívja önmagát, az előzőleg kikeresett gyerek csúcs felé folytatva a keresést.

6.3.2. Beszúrás segédalgoritmusok

A teljes beszúrás algoritmust darabokra tördelve mutatja be ez a jegyzet, hogy ne vesszen el az alapvető mondanivaló a részletekben. Egy konkrét programnyelvi implementációhoz azonban szükség lehet a legapróbb részleteket megvalósító eljárásokra is, így ezeknek a pszeudokódjai itt, elkülönítve szerepelnek.

Az algoritmusban megjelent a levélbe való beszúrás lépése. Ezt ott külön nem részleteztük, mivel meglehetősen egyszerű műveletről van szó, ennek algoritmusát tartalmazza a 6.3 algoritmus. Alapvető lépései: az új kulcs helyének megkeresése, a mögötte lévő kulcsok hátrébb csúsztatása (mivel levélről beszélünk, így itt a gyerekmutatókkal nem kell foglalkozni), majd az új kulcs elhelyezése, végül a darabszám változó beállítása.

6.3. Algoritmus Beszúrás egy szabad helyet tartalmazó levélelembe

Bemenet: p - **M**, *érték* - **K**

```
1: eljárás BeszúrásLevélbe(p : M, érték : K)
2:     i ← KulcsKeresés(p, érték)
3:     ciklus j ← p.n-től i-ig
4:         p.kulcs[j + 1] ← p.kulcs[j]
5:     ciklus vége
6:     p.kulcs[i] ← érték
7:     p.n ← p.n + 1
8: eljárás vége
```

Felhasznált változók és függvények

- p: A levélelem, amibe el kell helyezni az új értéket.
- *érték*: A kulcs, amit be kell szúrni a levélelem megfelelő helyére.
- KulcsKeresés(p : **M**, *érték* : **K**) : **egész**: Megkeresi a p mutató által hivatkozott csomópontban az *érték* kulcs helyét. Ha már van ilyen kulcs a csomópontban, akkor annak az indexét, ellenkező esetben az első nála nagyobb kulcs indexét adja vissza (vagy $p.n$-t ha ilyen sincs) (6.4. algoritmus).

A kulcs helyének megkereséséhez a logaritmikus keresést használja [9]. A csúcson belüli keresés megvalósítása is egyszerűen lett megvalósítva (mivel ez több helyen is meghívásra kerül, így külön függvénybe került: 6.4. algoritmus).

Végül, az egyik legfontosabb segédeljárás a szétvágást végző algoritmus, ennek működése látható a 6.5. ábrán. A beszúrás során mind a gyökér, mind pedig a belső csúcsok szétvágásakor is ez hívódik meg.

6.4. Algoritmus A B-fa egy csomópontjában egy kulcs keresése

Bemenet: p - **M**, *érték* - **K**
Kimenet: k−**egész**

```
1: függvény KulcsKeresés(p : M, érték : K)
2:     e ← 1
3:     v ← p.n
4:     ciklus
5:         k ← ⌊(e + v)/2⌋
6:         ha érték < p.kulcs[k] akkor
7:             v ← k − 1
8:         különben
9:             ha érték > p.kulcs[k] akkor
10:                e ← k + 1
11:            elágazás vége
12:        elágazás vége
13:    amíg e ≤ v ∧ p.kulcs[k] ≠ érték
14:    ha p.kulcs[k] ≥ érték akkor
15:        vissza k
16:    különben
17:        vissza (k + 1)
18:    elágazás vége
19: függvény vége
```

Felhasznált változók és függvények

- p: A csomópont, amelyen belül keresni.
- e, v: A logaritmikus keresésnél használt segédváltozók.
- *érték*: A kulcs, amit keresünk.
- k: Az első nem kisebb kulcs indexe. Ha ilyen nincs, akkor értéke a kulcsok száma+1.

Bár az algoritmus meglehetősen hosszúnak, valójában csak a sokféle adatmozgatás miatt tűnik összetettnek. A szétvágáshoz létre kell hoznunk egy új elemet, ez lesz majd a szétvágandó elem új jobboldali testvére. Egy ciklus segítségével átmásoljuk ebbe a szétvágandó elem utolsó $(t-1)$ darab kulcsát (emiatt is lényeges, hogy a szétvágást csak telített elemeken tudjuk végrehajtani), illetve belső csúcs esetében az ezekhez a kulcsokhoz tartozó gyerekmutatókat. Mivel azonban az új elemet nem tudjuk hova láncolni a szülő kulcsai közé, emiatt a maradék kulcsok közül a legnagyobbat (tehát az eredeti szétvágandó csúcs középső elemét) felvisszük a szülőbe (ami bár meglehetősen egyszerű művelet, de mégis a szülő jelentős átalakítását igényli, hogy helyet csináljunk az új kulcsnak a rendezettség szerinti megfelelő helyen). Ezt követi még néhány technikai lépés, hogy a szükséges változók (n, gyerekmutatók) a megfelelő értékeket tartalmazzák. Ezzel a szétvágás megvalósult. Amennyiben a kulcsok mellett külön tartalom mezőket is szeretnénk, akkor a ciklusokat ki kellene egészíteni ezek másolásával is.

6.5. Algoritmus B-fa egy csomópontjának szétvágása

Bemenet: p - **M**, *szülő* : **M**

```
 1: eljárás SZÉTVÁGÁS(p : M, szülő : M)
 2:     új ← LÉTREHOZ(M)
 3:     új.levél ← p.levél
 4:     új.n ← t − 1
 5:     ciklus i ← 1-től t − 1-ig
 6:         új.kulcs[i] ← p.kulcs[t + i]
 7:     ciklus vége
 8:     ha ¬új.levél akkor
 9:         ciklus i ← 1-től t-ig
10:             új.gyerek[i] ← p.gyerek[t + i]
11:         ciklus vége
12:     elágazás vége
13:     p.n ← t − 1
14:     i ← 1
15:     ha szülő.n ≠ 0 akkor
16:         ciklus amíg szülő.gyerek[i] ≠ p
17:             i ← i + 1
18:         ciklus vége
19:         ciklus j ← szülő.n-től i-ig
20:             szülő.kulcs[j + 1] ← szülő.kulcs[j]
21:         ciklus vége
22:         ciklus j ← szülő.n + 1-től (i + 1)-ig
23:             szülő.gyerek[j + 1] ← szülő.gyerek[j]
24:         ciklus vége
25:     elágazás vége
26:     szülő.kulcs[i] ← p.kulcs[t]
27:     szülő.gyerek[i] ← p
28:     szülő.gyerek[i + 1] ← új
29:     szülő.n ← szülő.n + 1
30: eljárás vége
```

Felhasznált változók és függvények

- p: A szétvágandó elem.
- *új*: A szétvágáskor létrehozott új elemre hivatkozó mutató.
- i, j: A másolásnál használt segédváltozók.
- *szülő*: A szétvágandó elem szülője.
- LÉTREHOZ(**M**) : **M**: Létrehoz és visszaad egy új B-fa csúcsot.

6.4. Törlés B-fából

6.4.1. Kulcs eltávolítása

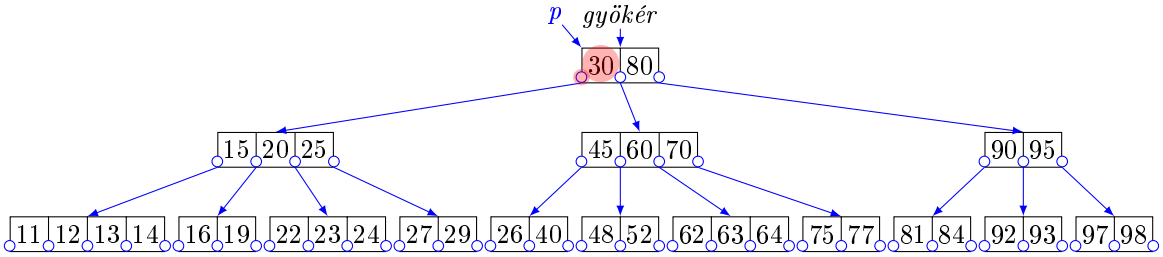
B-fából való törlés szintén egy kereséssel kezdődik, hiszen első körben meg kell találnunk a törlendő kulcsot. A keresés egészen hasonló a bináris keresőfánál tanultakhoz, csak itt igazodnunk kell ahhoz a tényhez, hogy egy csúcson belül több kulcs, és ennek megfelelően kettőnél több gyerek is elhelyezkedhet. A keresés tehát ugyanúgy rekurzív, minden szinten megvizsgáljuk az aktuális csúcsban lévő kulcsokat, amennyiben köztük van a törlendő elem, akkor meg is kezdhetjük a törlést, ha nincs, akkor a kulcsok alapján kikereshető, hogy melyik részfa felé kell folytatni a keresést, és a törlést végző eljárást rekurzívan meghívjuk erre.

Miután megtaláltuk a törlendő kulcsot, az alábbi eseteket kell megkülönböztetnünk, attól függően, hogy a kulcs milyen csúcsban található (levél, vagy sem), illetve annak a környezete milyen kulcsokat tartalmaz.

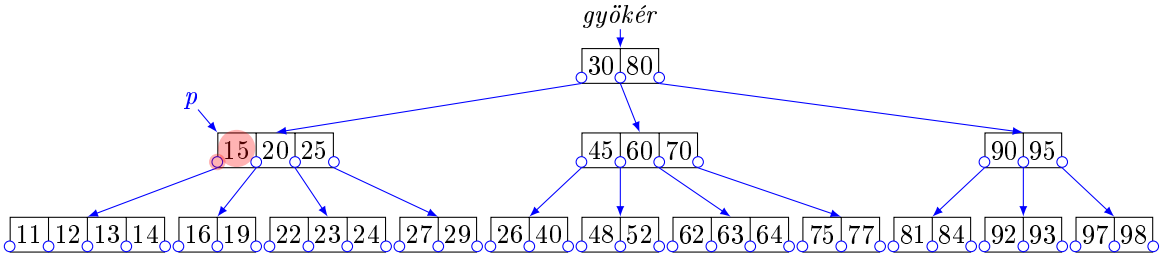
- Levélből való törlés: Amennyiben a levélnek van legalább t darab kulcsa, akkor a törlendő kulcs egyszerűen elhagyható. Mivel levélről van szó, nincs szükség a gyerekmutatók karbantartására, ugyanígy nem kell azzal foglalkozni, hogy a kulcs törlése milyen módon érinti az ezekből kiinduló részfákat. A törlés tehát lokálisan megoldható (6.5. ábra).
- Belső csúcsból való törlés: Amennyiben belső csúcsból kell törölnünk egy X kulcsot, és ennek a csúcsnak van legalább t kulcsa, akkor belátható, hogy az alábbiak közül legalább az egyik módszer használható:
  - Amennyiben az X kulcshoz tartozó baloldali gyerek tartalmaz legalább t darab kulcsot: Keressük meg ennek a részfának a legnagyobb elemét (X'), ezt másoljuk az X helyére, majd rekurzívan folytassuk a törlést ebben a részfában a X'-re (6.6. ábra, illetve konkrétan az áthozatal látható a 6.6c. ábrán).
  - Amennyiben az X kulcshoz tartozó jobboldali gyerek tartalmaz legalább t darab kulcsot: Keressük meg ennek a részfának a legkisebb elemét (X'), ezt másoljuk az X helyére, majd rekurzívan folytassuk a törlést ebben a részfában a X'-re (6.7. ábra, illetve konkrétan az áthozatal látható a 6.7c. ábrán).
  - Amennyiben az X kulcs bal- és jobboldalán is a fentieknél kisebb elemek vannak, akkor belátható, hogy a két gyerek összevonható az X kulccsal. Végezzük el ezt az összevonást, majd az újonnan kapott elemben rekurzívan töröljük az X-t (6.8. ábra, illetve konkrétan az összevonás látható a 6.8d. ábrán).

A legutolsó esethez megállapíthatunk egy speciális esetet is, amikor a törlendő elem a gyökérben található, és ez egyben az utolsó kulcs is ebben a csúcsban (a gyökérben nem kell teljesülnie a kulcsokra vonatkozó minimumfeltételnek, így ez elképzelhető. A belső csúcsokban - ésszerű t értéket választva - ezzel a lehetőséggel nem kell számolnunk). Ebben az esetben is az utolsó pontban megadott folyamatot kell követnünk, azonban egy kisebb módosítással: amikor a K kulcsot levisszük az új, összevont elembe, akkor tulajdonképpen kivesszük az utolsó kulcsot is a gyökérből. Ilyenkor ezt a már üres elemet megszüntetjük, és az új összevont csúcs lesz a B-fa új gyökere.

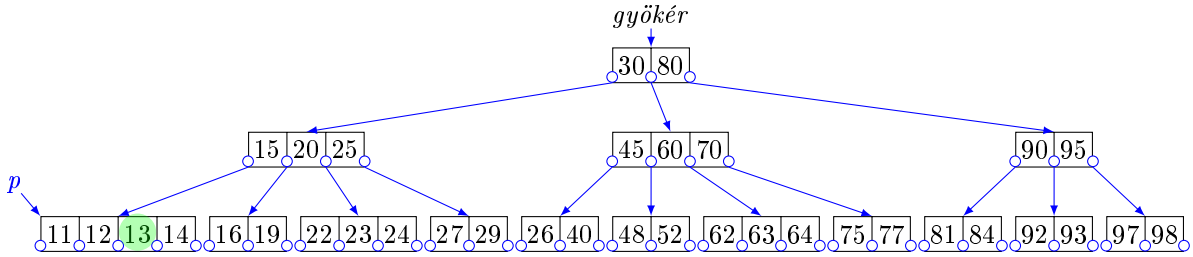
Fontos észrevennünk, hogy a törlés során eddig használt módszereink mindig éltek azzal a feltételezéssel, hogy a törlendő K kulcsot tartalmazó belső csúcsnak/levélnek legalább t darab kulcsa van. Ez természetesen nem minden esetben áll fenn, ilyen esetekben ezt nekünk kell biztosítanunk, erre visszatérünk a 6.4.2. fejezetben.



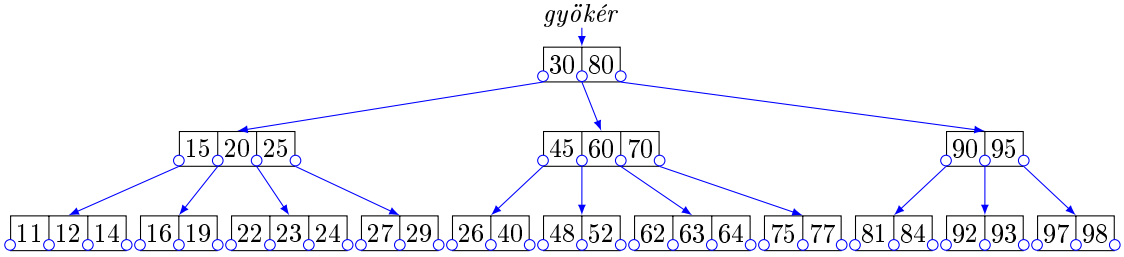
(a) Kezdetben a p a gyökérelemre hivatkozik. A kulcskeresés megadja, hogy a keresett elem nincs itt, illetve azt is, hogy milyen irányba kell tovább folytatni a törlendő elem keresését.



(b) Újra tovább kell lépni a rendezettségnek megfelelő gyerekelemre.

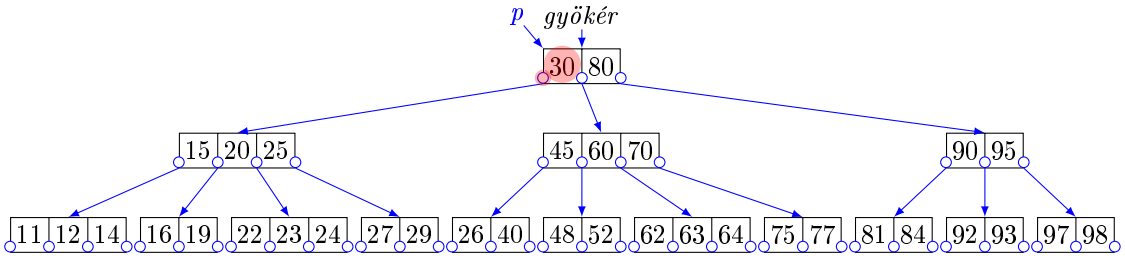


(c) A kulcskeresés itt már eredményes, hiszen megtalálta a törlendő értéket.

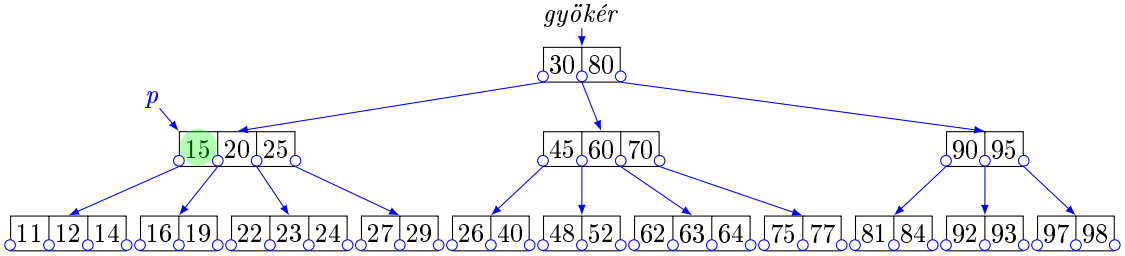


(d) Mivel a törlendő kulcs egy levélelemben található, aminek több, mint $(t-1)$ darab kulcsa van, így az egyszerűen elhagyható.

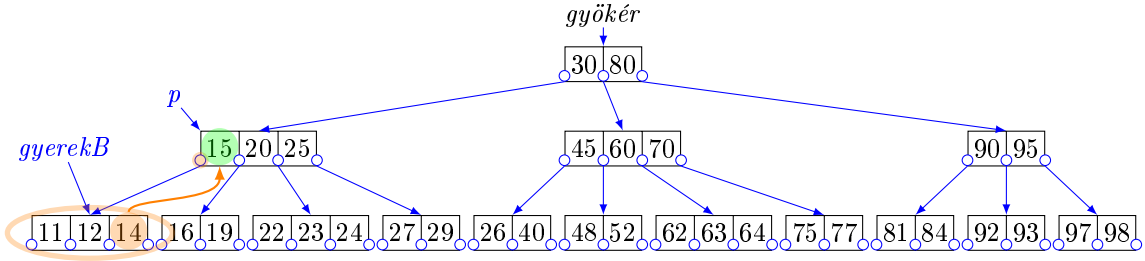
6.5. ábra. B-fa levelében található kulcs törlése (13-as elem).



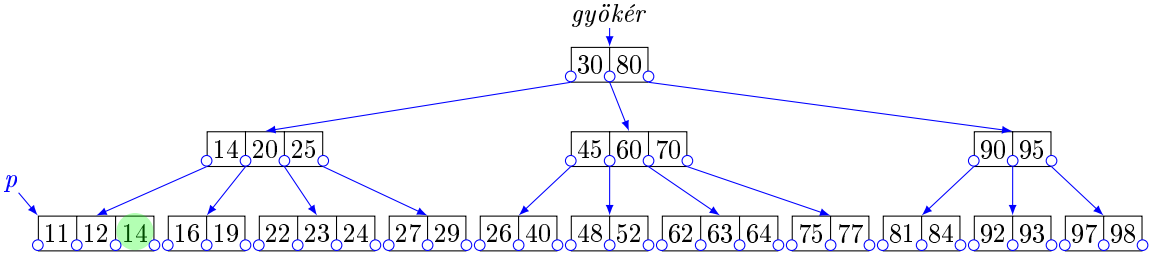
(a) Kezdetben a p a gyökérelemre hivatkozik. A kulcskeresés megadja, hogy a keresett elem nincs itt, illetve azt is, hogy milyen irányba kell tovább folytatni a törlendő elem keresését.



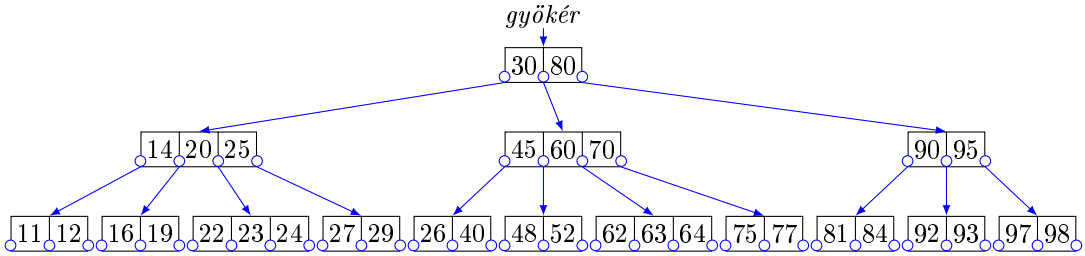
(b) A p továbblépése után a kulcskeresés megtalálja a törlendő kulcsot, látható, hogy ez nem levélelem, tehát nem lehet egyszerűen csak elhagyni a kulcsot.



(c) A törlendő kulcshoz tartozó baloldali gyereknek több mint $(t-1)$ darab kulcsa van, emiatt innen egy kulcs felhozható.

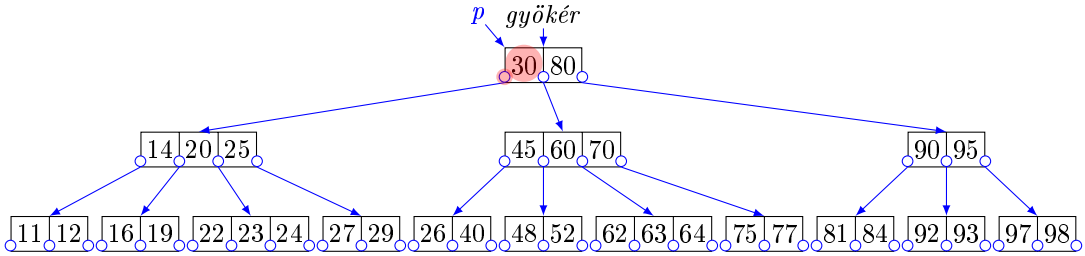


(d) Ezt követően folytatjuk a törlést a gyerekelemen a felhozott kulcsra.

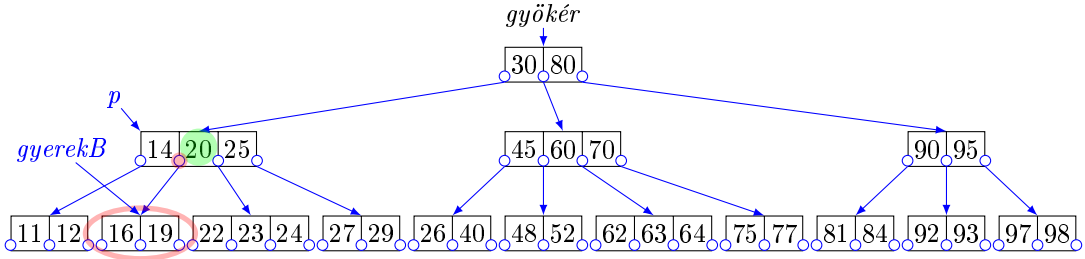


(e) Mivel ez a gyerek egy levélelem, így a törlés egyszerűen megoldható.

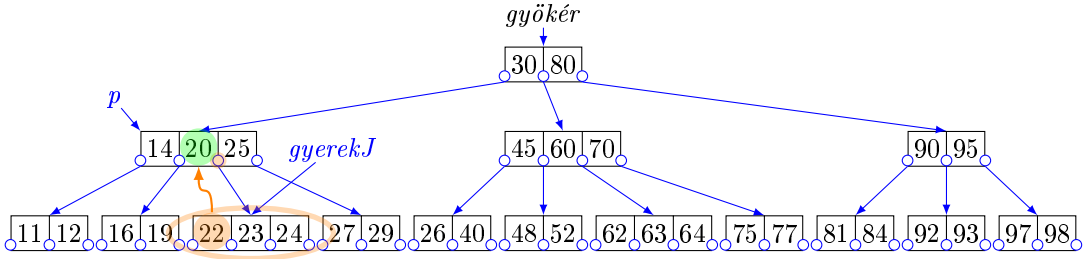
6.6. ábra. B-fa belső csúcsában található kulcs törlése, bal gyerekből felhozás (15-ös elem).



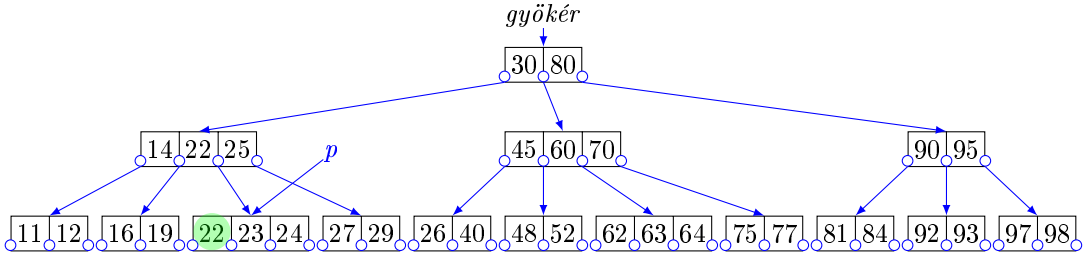
(a) Kezdetben a p a gyökérelemre hivatkozik. A kulcskeresés megadja, hogy a keresett elem nincs itt, illetve azt is, hogy milyen irányba kell tovább folytatni a törlendő elem keresését.



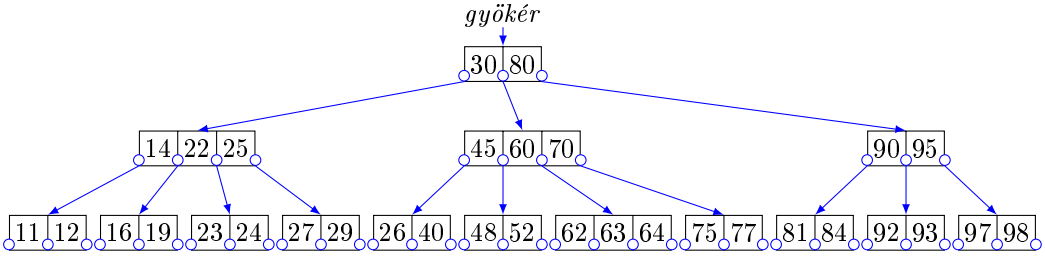
(b) Megtaláltuk a törlendő kulcsot, látható, hogy ez nem levélelem, tehát nem lehet egyszerűen csak elhagyni a kulcsot. Mint látható, a baloldali gyereknél nincs felhozható elem, hiszen a kulcsok száma itt csak $(t-1)$.



(c) A jobboldali gyerekből viszont fel lehet hozni ismét egy kulcsot a törlendő kulcs helyére.

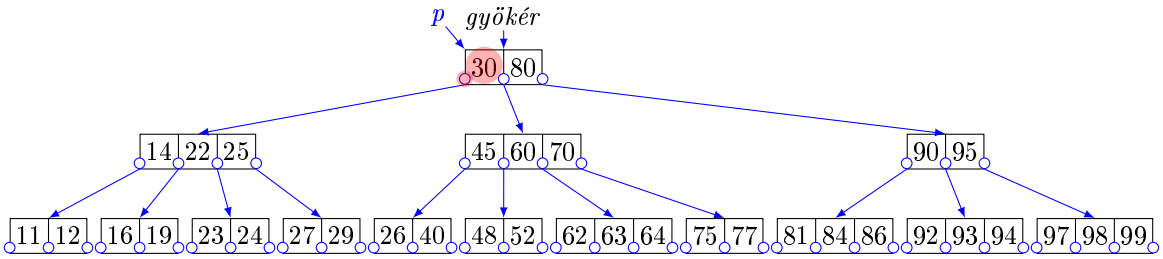


(d) Ezt követően folytatjuk a törlést a gyerekelemen a felhozott kulcsra.

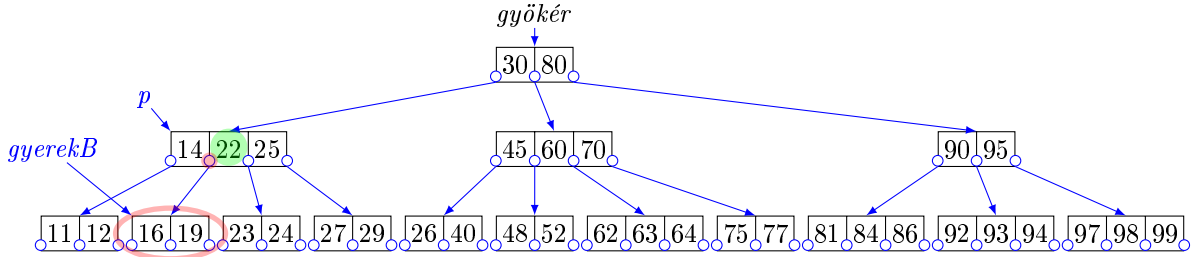


(e) Mivel ez a gyerek egy levélelem, így a törlés egyszerűen megoldható.

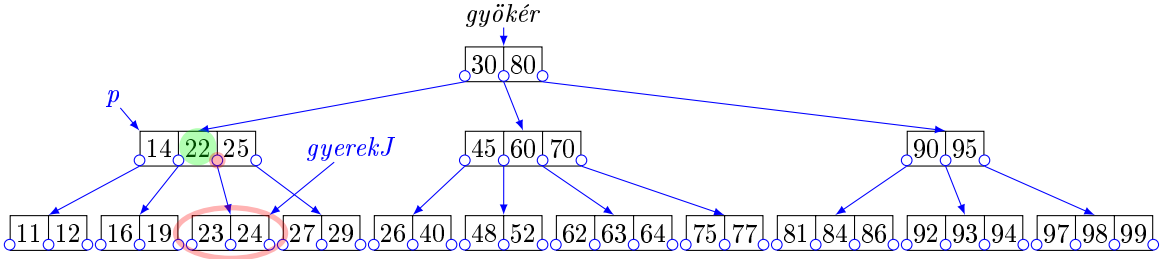
6.7. ábra. B-fa belső csúcsában található kulcs törlése, jobb gyerekből felhozás (20-as elem).



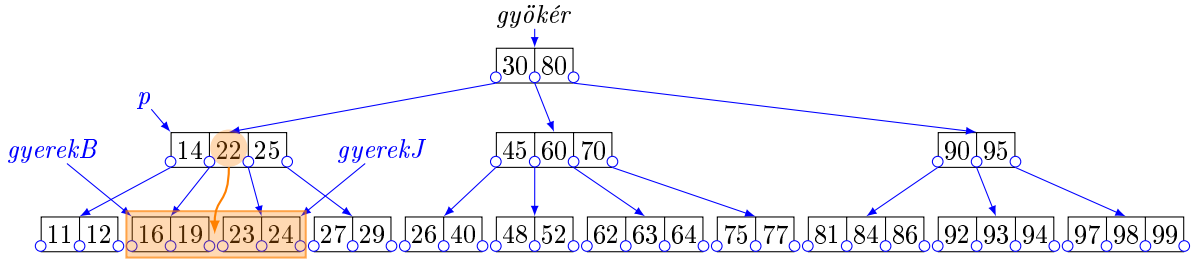
(a) Kezdetben a p a gyökérelemre hivatkozik. A kulcskeresés megadja, hogy a keresett elem nincs itt, illetve azt is, hogy milyen irányba kell tovább folytatni a törlendő elem keresését.



(b) Megtaláltuk a törlendő kulcsot, látható, hogy ez nem levélelem, tehát nem lehet egyszerűen csak elhagyni a kulcsot. A baloldali gyerek csak $(t-1)$ darab kulcsot tartalmaz, így innen nem lehet kulcsot felhozni.

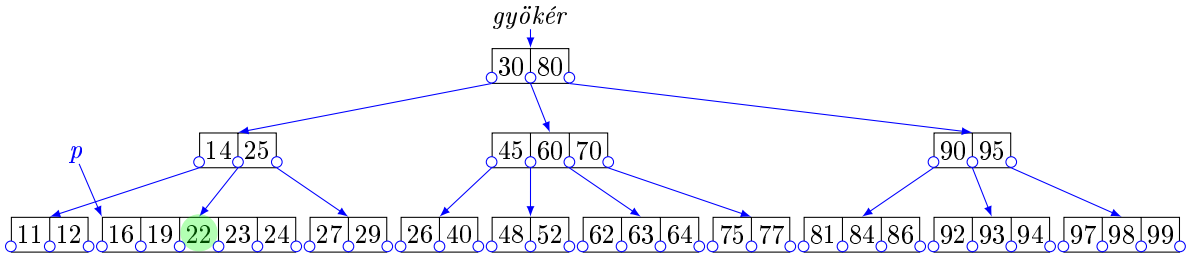


(c) A jobboldali gyereknél szintén nincs meg a szükséges kulcsszám a felhozatalhoz.

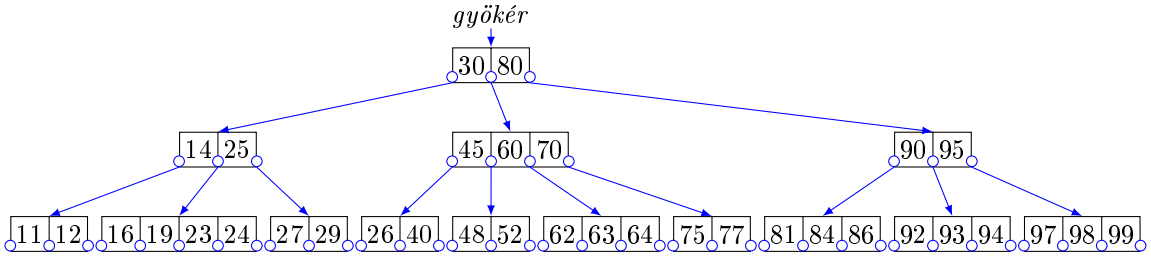


(d) Következő lépésként emiatt összevonjuk a két gyereket, és levisszük az új elembe a törlendő kulcsot is.

6.8. ábra. B-fa belső csúcsában található kulcs törlése, két gyerek összevonása (22-es elem).



(e) Ezt követően rekurzívan folytatjuk a törlést az új elemen a lehozott kulcsra.



(f) A p által mutatott elem egy levél, emiatt a törlés csak az elem elhagyását jelenti.

6.8. ábra. B-fa belső csúcsában található kulcs törlése, két gyerek összevonása (22-es elem). (folytatás)

6.4.2. Legalább t darab kulcs biztosítása

Az előző fejezetben láttuk a kulcs törlésének módját, azonban végig éltünk azzal a feltételezéssel, hogy a kulcsot tartalmazó levélnek van legalább t darab kulcsa, tehát a minimumfeltételhez képest legalább eggyel több (a gyökér esetében a minimumfeltétel nem ad korlátot, így itt elég ha csak 1 darab kulcs található).

A beszúrásnál már megismertünk egy technikát, amivel már a beszúrás helyének keresése közben tudtuk biztosítani, hogy maga a beszúrás mindig olyan levélre jusson, ahol van legalább egy szabad kulcs hely. Ezt úgy oldottuk meg, hogy a keresést végző rekurzió minden részfára (gyerekre) való újrahívás előtt ellenőrizte, hogy a továbblépésre megjelölt csúcs teljesíti-e ezt a feltételt, mert ha nem, akkor azt szétvágva tudtuk garantálni a szükséges kulcsszámot.

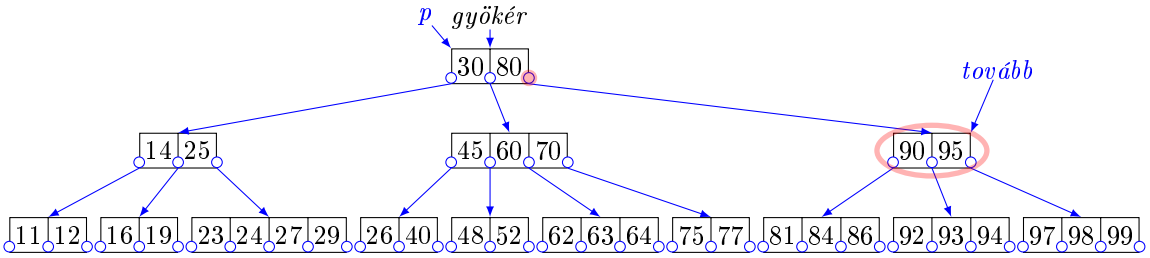
A törlésnél hasonlóan fogunk eljárni, csak itt éppen ellenkező követelményt kell garantálnunk, a gyökér kivételével minden egyes elemnek legalább t darab kulcsot kell tartalmaznia. A beszúráshoz hasonlóan itt is már a keresési fázisban átalakítjuk a fát, amennyiben az szükséges a feltétel biztosításához. Tehát mielőtt a keresést végző rekurzió meghívná önmagát, ellenőrzi, hogy a folytatáshoz kiszemelt elem (mivel a pszeudokódban erre *tovább* mutatóval hivatkozunk, hívjuk így a későbbiekben itt is) rendelkezik-e megfelelő számú kulccsal. Amennyiben nem, akkor könnyen belátható, hogy az alábbi három módszer közül valamelyik minden esetben használható:

- A *tovább* elem baloldali testvérének van legalább t darab kulcsa: Ebben az esetben a két testvérhez tartozó szülőbeli kulcsot levisszük a *tovább* elem első kulcsaként, a baloldali testvér legnagyobb kulcsát pedig felvisszük a szülőbe ennek a helyére. Ezt követően a baloldali testvér legjobboldalibb gyerekét átláncoljuk a *tovább* csúcs legbaloldalibb gyerekeként. Ezt a folyamatot mutatja a 6.9. ábra, illetve magát a kulcsmozgatást a 6.9b. ábra.

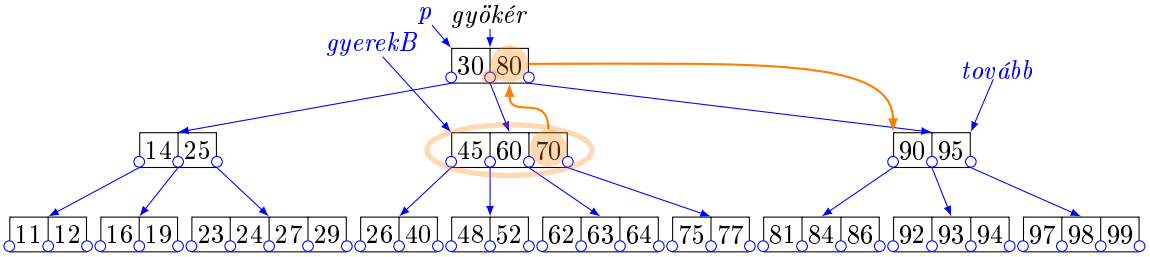
- A *tovább* elem jobboldali testvérének van legalább t darab kulcsa: Ebben az esetben a két testvérhez tartozó szülőbeli kulcsot levisszük a *tovább* elem utolsó kulcsaként, a jobboldali testvér legkisebb kulcsát pedig felvisszük a szülőbe ennek a helyére. Ezt követően a jobboldali testvér legbaloldalibb gyerekét átláncoljuk a *tovább* csúcs legjobboldalibb gyerekeként. Ezt a folyamatot mutatja a 6.10. ábra, illetve magát a kulcsmozgatást a 6.10c. ábra.

- A *tovább* elem mindkét testvérének csak $(t-1)$ darab kulcsa van: Ebben az esetben az egyik gyerek (a mi példánkban a baloldali) összevonható a *tovább* csúccsal, úgy, hogy az elembe levisszük a szülő rájuk hivatkozó kulcsát is. Ezt a folyamatot mutatja a 6.11. ábra. Amennyiben nincs baloldali gyerek, akkor ugyanezt megtehetjük a jobboldalival is.

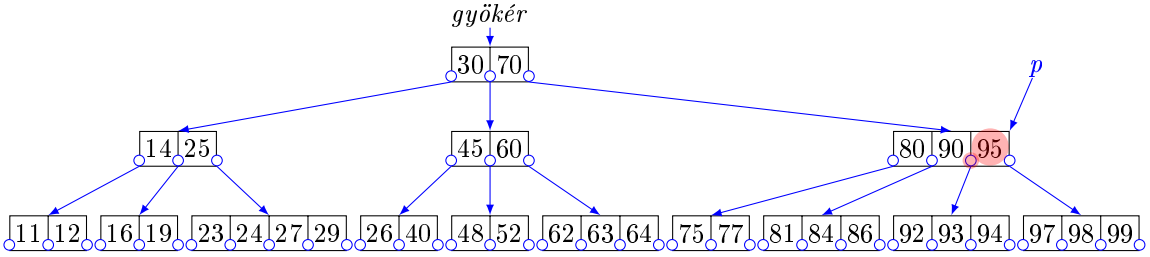
Az utolsó esetben a szülőből le kell vinnünk egy kulcsot az összevont gyerekekből álló csúcsba. Ez akkor eredményezhet problémát, ha ezt a gyökérből kellene levinnünk, ahol csak egy darab kulcsunk van. Emiatt az utolsó esetben itt is előfordulhat, hogy amennyiben a gyökérelemnek csak egy kulcsa van, és a túl kevés kulcs miatt az ehhez tartozó két gyerekelemet összevonjuk, akkor meg kell szüntetnünk a meglévő gyökért, és az új, összevont csúcsot kell a későbbiekben gyökérnek tekintenünk. Ezt a folyamatot mutatja a 6.12 ábra.



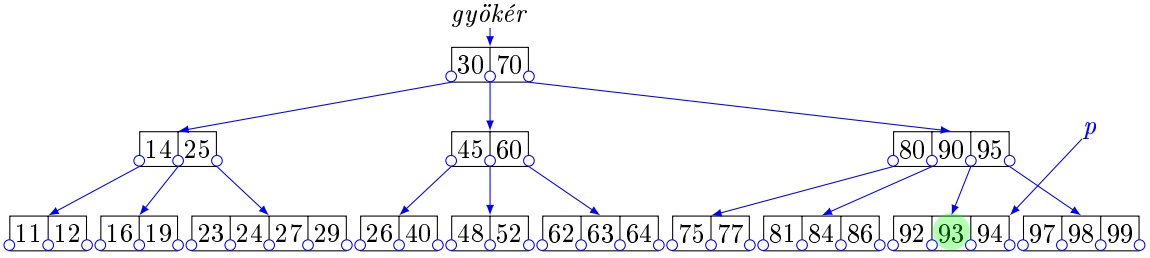
(a) A kulcskeresés alapján látható, hogy a keresett elem nincs a gyökérelemben. A továbblépés iránya is adott, erre azonban nem tudunk továbbmenni, mivel nincs meg a szükséges t darab kulcs.



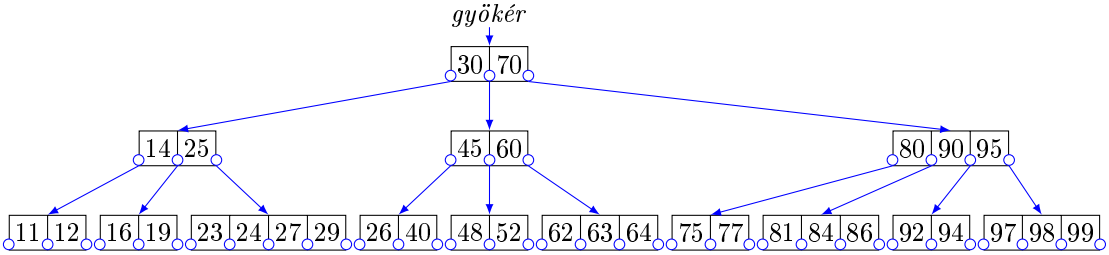
(b) A baloldali testvérben azonban van „felesleg”, ezért innen a legnagyobb kulcsot felvisszük a szülőbe, majd onnan a megfelelő kulcsot levisszük a *tovább* által mutatott elembe. A leírtaknak megfelelően átláncoljuk a legjobboldalibb gyereket is.



(c) Az átalakítás után már tovább tud lépni a kereső rekurzió erre az elemre.

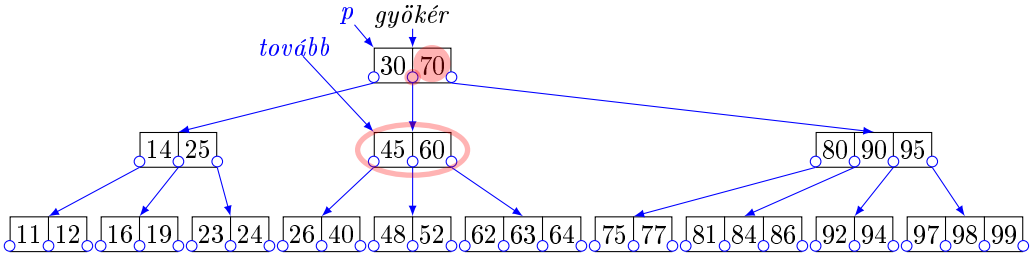


(d) A keresés folytatása ebben a részfában.

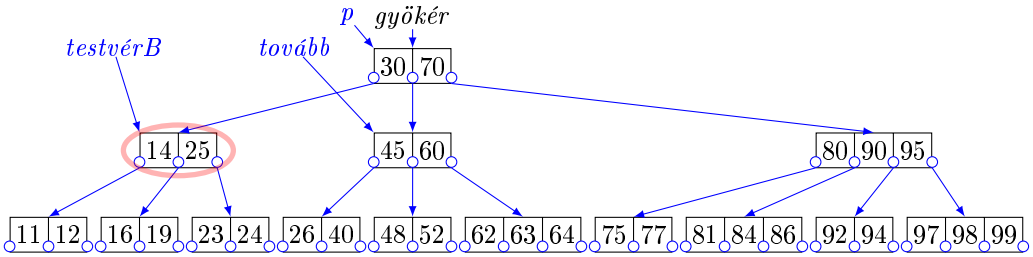


(e) A törlendő elem egy levélben volt, így onnan egyszerűen eltávolítható.

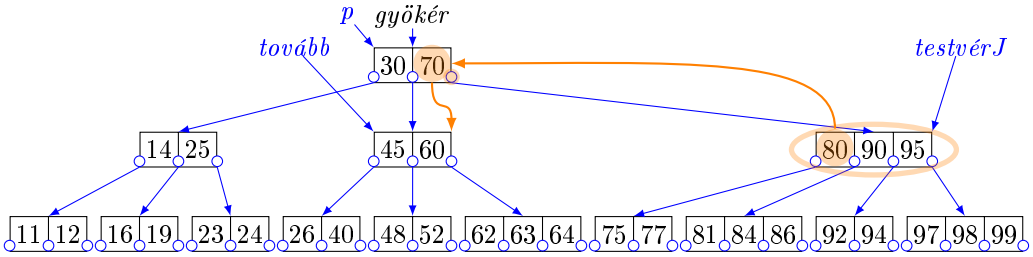
6.9. ábra. Legalább t darab kulcs biztosítása. Elem áthozása baloldali testvérből (93-as elem törlése).



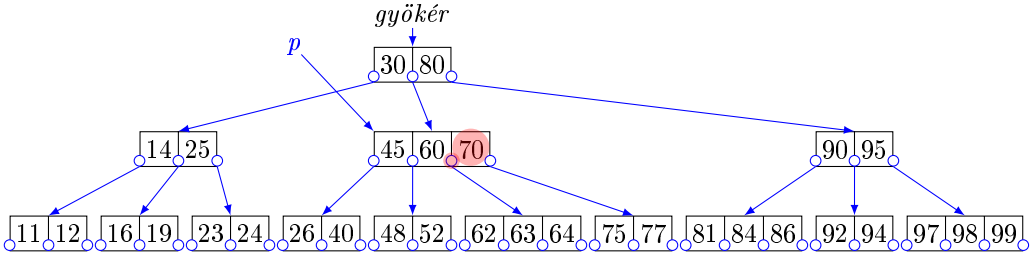
(a) A kulcskeresés alapján látható, hogy a keresett elem nincs a gyökérelemben. A továbblépés iránya is adott, azonban nem tudunk továbbmenni, mivel nincs meg a szükséges t darab kulcs.



(b) A baloldali testvérnek csak $(t-1)$ darab kulcsa van, így innen nem tudunk elemet áthozni.

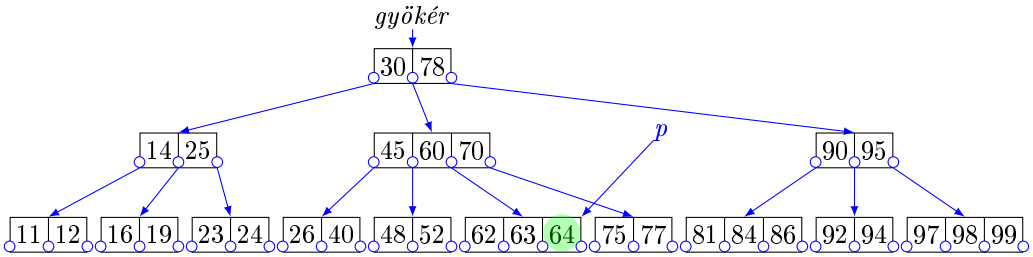


(c) A jobboldali testvérnek több eleme van, így innen a szülőn keresztül át tudunk hozni egy kulcsot. Ezzel együtt átláncoljuk annak legbaloldalibb gyerekét is.

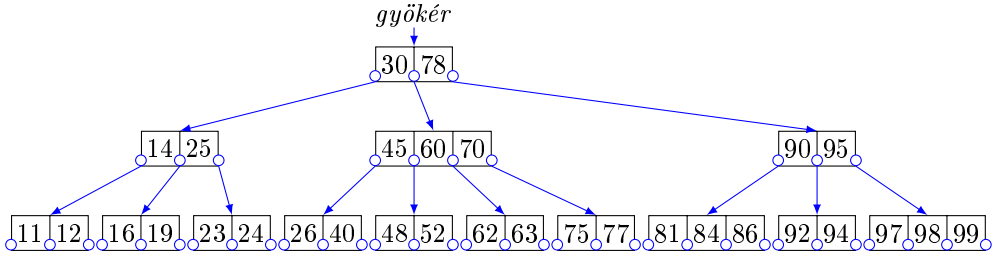


(d) Az átalakítás után már lehet folytatni a keresést a *tovább* által mutatott elemre.

6.10. ábra. Legalább t darab kulcs biztosítása. Elem áthozása jobboldali testvérből (64-es elem törlése).

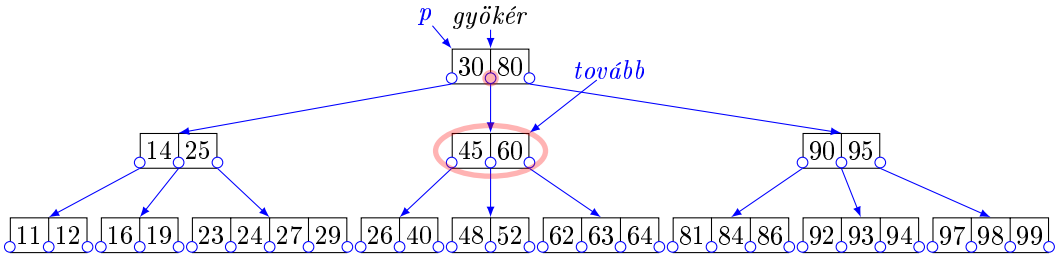


(e) A levélelemben megtaláltuk a törlendő kulcsot, itt már a törlés egyszerűen megoldható.

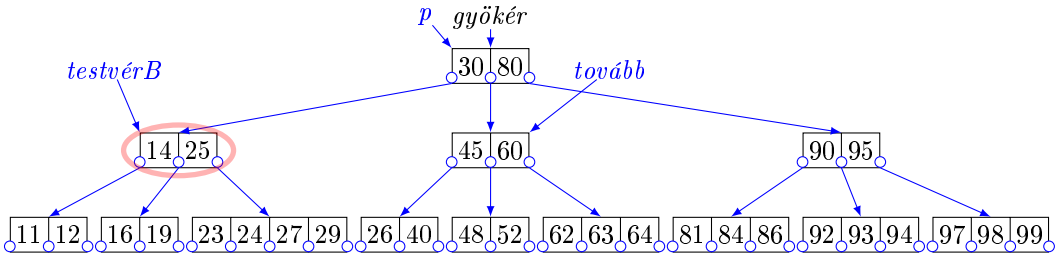


(f) A törlés végeredménye.

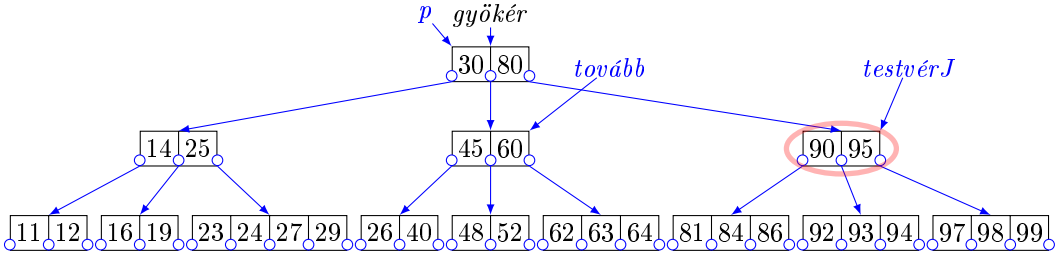
6.10. ábra. Legalább t darab kulcs biztosítása. Elem áthozása jobboldali testvérből (64-es elem törlése). (folytatás)



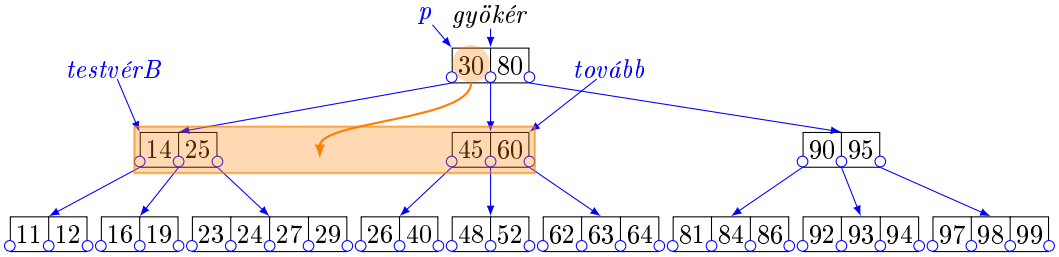
(a) A kulcskeresés alapján látható, hogy a keresett elem nincs a gyökérelemben. A továbblépés iránya is adott, erre azonban nem tudunk továbbmenni, mivel nincs meg a szükséges t darab kulcs.



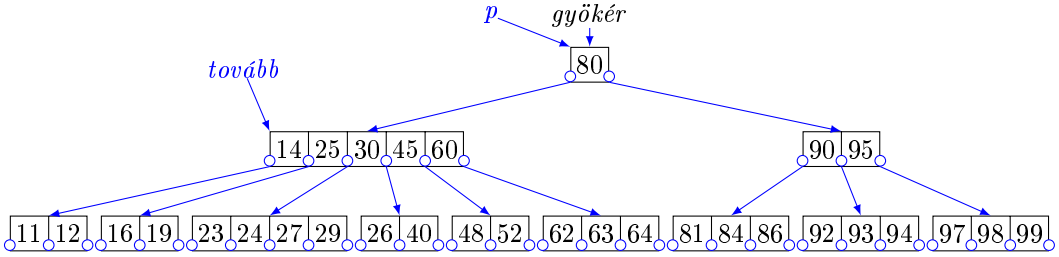
(b) A baloldali testvérnek csak $(t-1)$ darab kulcsa van, így innen tudunk elemet áthozni.



(c) A jobboldali testvérnek is csak $(t-1)$ darab kulcsa van, így innen sem tudunk elemet áthozni.

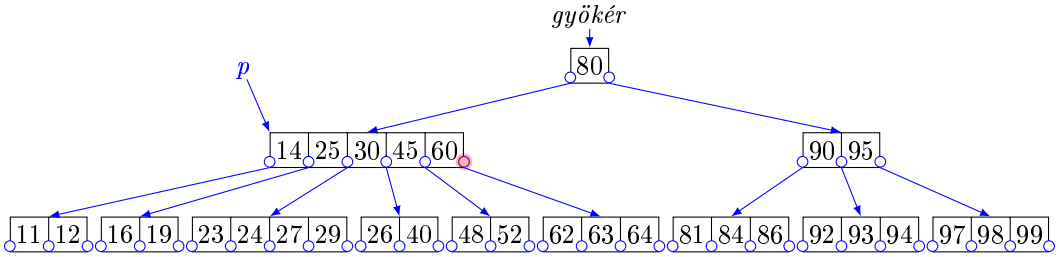


(d) Emiatt összevonjuk az elemet a baloldali testvérével, ehhez a szülőből le kell hozni egy kulcsot.

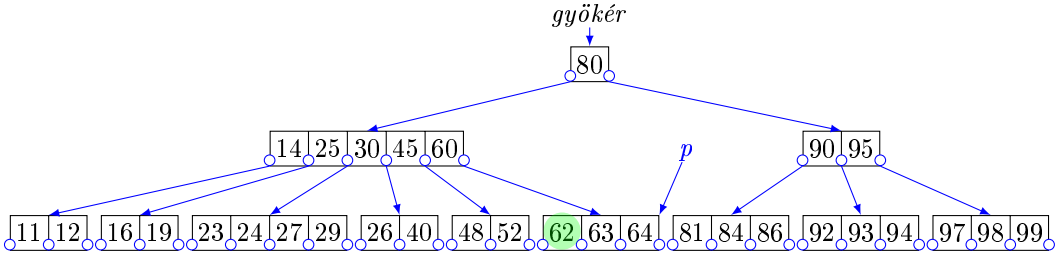


(e) Az összevonást követően az új, összevont elem lesz a továbbléps iránya.

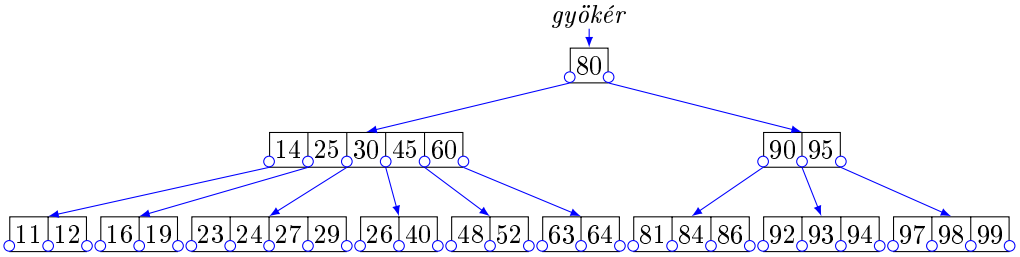
6.11. ábra. Legalább t darab kulcs biztosítása. Testvérek összevonása (62-es elem törlése).



(f) Ezt követően folytatjuk a keresést az összevont elemen keresztül. Továbblépés a 62-es kulcs irányába.

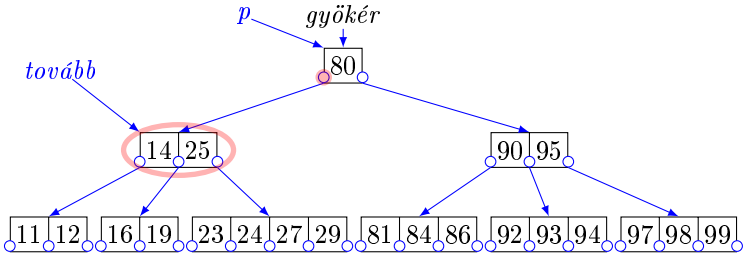


(g) Megtaláltuk a törlendő kulcsot. Ez egy levélelemben helyezkedik el, így egyszerűen elhagyható.

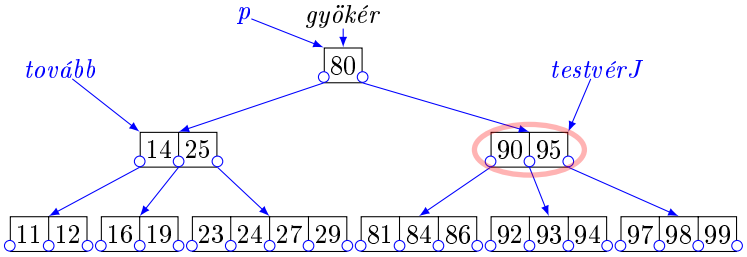


(h) A törlés utáni állapot.

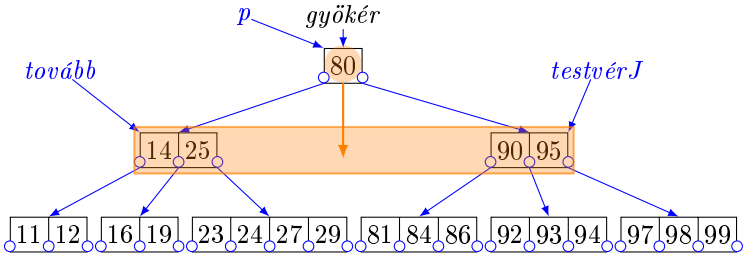
6.11. ábra. Legalább t darab kulcs biztosítása. Testvérek összevonása (62-es elem törlése). (folytatás)



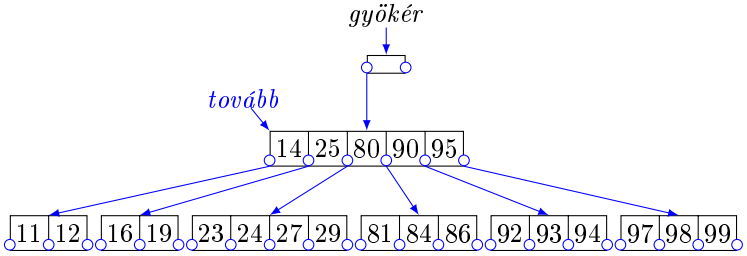
(a) Már a keresés első lépésénél sem tudunk továbblépni, mivel a következő elemnek csak $t-1$ darab kulcsa van. Bal szomszédja nincs, így innen nem tudunk kulcsot áhozni.



(b) A jobboldali testvérnek csak $t-1$ darab kulcsa van, így innen sem tudunk elemet áthozni.

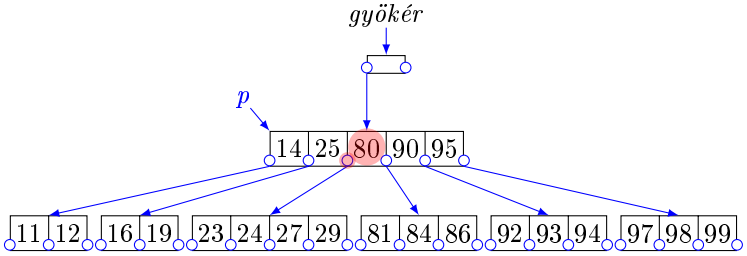


(c) Emiatt összevonjuk az elemet a jobboldali testvérével, ehhez a szülőből le kell hozni egy kulcsot.

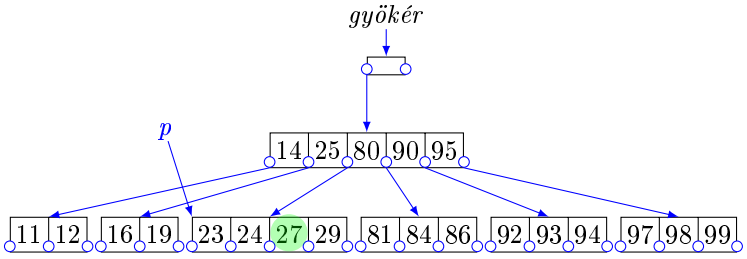


(d) A gyökérben már egy kulcs sincs, a gyerekek száma azonban még így is 1, ez mutat az összevont elemre.

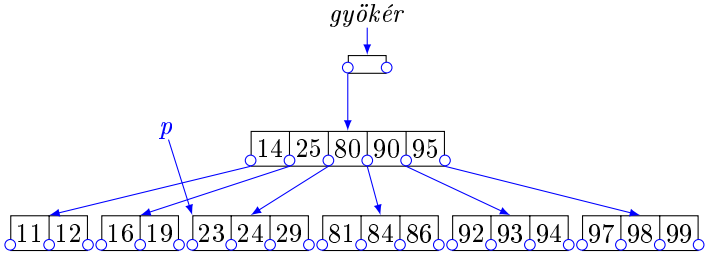
6.12. ábra. Legalább t darab kulcs biztosítása. Testvérek összevonása (27-es elem törlése).



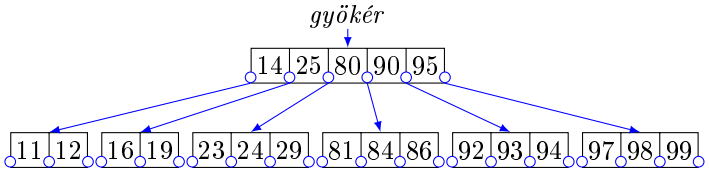
(e) A hiányos gyökért figyelmen kívül hagyva, folytatjuk a törlést. Az elemen belüli kereséssel továbblépünk.



(f) Megtaláltuk a törlendő kulcsot. Ez egy levélelemben helyezkedik el, így egyszerűen elhagyható.



(g) A megtalált elem törlése.



(h) A törlés utáni ellenőrizzük a gyökér állapotát. Ha kulcsainak száma 0, akkor az első gyerek mutató által hivatkozott elem lesz az új gyökér.

6.12. ábra. Legalább t darab kulcs biztosítása. Testvérek összevonása (27-es elem törlése). (folytatás)

6.4.3. Törlés pszeudokódja

Törlést indító eljárás

A fentiek alapján már megadható a törlés pszeudokódja. A kódban itt is szükség lehet a törlés után némi karbantartásra (pl. amikor a gyökér alatti elemek összevonása miatt meg kell szüntetnünk a már létező gyökérelemet), emiatt itt sem közvetlenül a törlést végző rekurziót hívjuk meg, hanem egy segédeljárást (6.6. algoritmus).

Látható, hogy ez az eljárás azonnal meghívja a rekurziót, paraméterként átadva a B-fa gyökerét. Miután a vezérlés visszaérkezik, meg kell vizsgálni (3. sor), hogy történt-e összevonás a gyökér utolsó kulcsával. Ha igen, akkor a gyökérben lévő kulcsok száma 0 lesz, míg a gyökérelem első gyereke az összevonás során kialakított (egyébként a régi baloldali) elemre hivatkozik. Ez jól látható a 6.12. ábrán is. Az, hogy az összevonás melyik lépésben történt, valójában lényegtelen, itt már csak az lényeges, hogy a visszamaradt egyetlen kulcsot se tartalmazó gyökérelemet kicseréljük annak első gyerekére.

Speciális esetként jelenik meg az az állapot, amikor már az egész B-fa csak egyetlen kulcsot tartalmazott, és ezt töröltük. Az algoritmus lépéseit végigkövetve látható, hogy ilyenkor a gyökérben lévő utolsó kulcs is törlődik, az első gyerek mutató értéke pedig **ø** lesz. Ezt betöltve a *gyökér* változóba, helyesen egy üres B-fát kapunk.

6.6. Algoritmus Kulcs törlése a B-fából

Bemenet: *gyökér* - **M**, *érték* - **K**
Kimenet: *gyökér* - **M**

```
1: eljárás TÖRLÉS(címszerint gyökér : M, érték : K)
2:     KeresésTöröl(gyökér, érték)
3:     ha gyökér.n = 0 akkor
4:         gyökér ← gyökér.gyerek[1]
5:     elágazás vége
6: eljárás vége
```

Felhasznált változók és függvények

- *gyökér*: A B-fa gyökere. A törlendő elem helyzetétől függően esetleg meg is változhat az eljárás végén.
- *érték*: A törlendő kulcs.
- KeresésTöröl(p : **M**, *érték* : **K**): Megkeresi az *érték* kulcs helyét a p gyökerű B-fában, majd elvégzi annak törlését (6.7. algoritmus).

Törlést végző rekurzió

Az előbbi indító eljárásnál természetesen jóval összetettebb magát a keresést, törlést, majd az ezt követő karbantartást végző rekurzív eljárás. Ezt mutatja be a 6.7. algoritmus.

Magát az eljárást három fő részre oszthatjuk:

- A törlendő kulcs megkeresése.
- A törlendő kulcs eltávolítása.
- Keresés közben a legalább t darab kulcs biztosítása.

Az eljárás első sora pusztán azt ellenőrzi, hogy a törlendő elem megtalálható-e a fában. Amennyiben a p paraméter értéke **ø**, az azt jelenti, hogy vagy eljutottunk a fa aljára, és nem találtuk meg az elemet, vagy pedig el se indult a rekurzió, mivel már eleve egy üres fán indítottuk el a törlést. Mindkét esetben hibát jelzünk.

Ezt követi a szokásos keresés. A rekurzió minden szintjén megvizsgáljuk, hogy az ott lévő kulcsok között szerepel-e a törlendő. A csomóponton belüli kereséshez a már megismert KulcsKeresés függvényt használja. Ennek eredménye alapján a 4. sorban ellenőrzi, hogy a visszakapott index a törlendő kulcsot mutatja-e.

Amennyiben igen, akkor meg kell kezdenünk a kulcs törlését. Ez egy vizsgálattal kezdődik (5. sor), hogy egy levélben van-e a megtalált elem. Ha igen, akkor egyszerűen töröljük ezt a kulcsot (a későbbiekben részletezett módszernek köszönhetően biztosak lehetünk benne, hogy legalább t kulcs van ebben a levélben, gyerekei pedig értelemszerűen nem lehetnek). A kulcsokat előrébb kell tolni, majd csökkenteni az n értékét.

Amennyiben megtaláltuk az elemet, azonban az egy belső csúcsban van, akkor a fent leírtak szerint kell eljárnunk, a háromféle módszert egyesével megvizsgáljuk, hogy használhatóak-e ebben az esetben. A 12. sor vizsgálja meg, hogy a baloldali gyereknek van-e legalább t kulcsa. Ha igen, akkor megkeressük az ebből a gyerekből kiinduló részfa legnagyobb elemét. Ezzel az elemmel felülírjuk a törlendő kulcsot, majd rekurzívan folytatjuk a törlést a részfában, az előbb megtalált kulcsra.

A másik lehetőség, hogy a jobboldali gyereknek van legalább t darab kulcsa, ezt ellenőrzi a 17. sor. Amennyiben a feltétel igaz, akkor az előzőhöz hasonló műveletek futnak le, csak éppen ellenkező irányba. Megkeressük a jobboldali gyerekből kiinduló részfa legkisebb elemét. Ezzel felülírjuk a törlendő kulcsot, majd rekurzívan folytatjuk a törlést a jobboldali részfában, az előbb megtalált kulcsra.

Amennyiben a fenti kettő közül egyik feltétel sem teljesült, akkor a harmadik módszert kell használnunk (hiszen ilyenkor mindkét gyerek pontosan $(t-1)$ elemmel rendelkezik), ami a 21. sorban indul. Ebben az esetben összevonjuk a két gyereket a p-beli i. kulccsal (ami nem más, mint a törlendő kulcs indexe). Ezt követően folytatjuk a törlést az új csomópont által meghatározott részfában, továbbra is a törlendő kulcsot keresve.

Érdemes észrevenni, hogy a három közül bármelyik módszert is alkalmaztuk, mindhárom esetben folytatjuk a rekurziót. A belső csúcsokból ugyanis nem tudjuk a törlést megvalósítani egy lokális módosítással, csak egy szinttel lejjebb tudtuk tolni a problémát. Persze mindezt egymás után többször alkalmazva előbb-utóbb eljutunk a levél elemekig, ahol pedig már egyszerűen befejezhető a törlési művelet.

A törlést végző eljárás harmadik fontos szerepe a legalább t darab kulcs folyamatos biztosítása a rekurzió újabb szintjének átadott csúcson. Ennek módszereit tartalmazza a 6.4.2. fejezet, ugyanazokat a technikákat láthatjuk a kódban is. A 27–29. sorok beállítják néhány segédváltozó értékét a későbbi egyszerűbb használat érdekében. A *tovább* hivatkozás mutat arra az elemre, ahová a rekurzió következő szintjén lépnünk kellene, a *testvérB* és *testvérJ* változók pedig ennek a bal- illetve jobboldali testvéreire hivatkoznak (feltéve persze, ha ilyenek léteznek, ha nem, akkor értékük **ø** érték lesz).

Ezt követi az első vizsgálat, hogy a baloldali testvérnek van-e legalább t darab kulcsa (31. sor). Amennyiben van, akkor a 6.9b. ábrán látható módon áthozunk egy kulcsot a baloldali testvértől.

Ha ez a feltétel nem teljesül, akkor megvizsgáljuk a jobboldali testvért, hogy annak van-e legalább t darab kulcsa (34. sor). Amennyiben van, akkor az előzőhöz hasonló műveletet végezzük el, csak éppen a másik irányba: áthozunk egy kulcsot a jobboldali testvértől (mikét ez a 6.10c. ábrán is látható).

Amennyiben egyik feltétel sem teljesült, akkor biztosak lehetünk benne, hogy mindkét testvérben (amennyiben léteznek) pontosan t darab kulcs található. Ilyenkor az összevonást kell választanunk (6.11. ábra). Mindegy, hogy melyik testvérrel vonunk össze, ezért eljárásban található elágazás csak azt ellenőrzi, hogy van-e egyáltalán baloldali testvér, mert ha van, akkor azzal von össze, ellenkező esetben pedig a jobboldalival. Az összevonás során mindig a baloldali elembe másoljuk a kulcsokat a jobboldaliból, emiatt a *testvérB* és *tovább* elemek összevonását követően minden kulcs a *testvérB* csúcsba kerül. Ilyenkor a *tovább* változó értékét módosítanunk kell, hogy a keresés itt folytatódjon (39. sor).

A különböző módosítások után (ha egyáltalán szükség volt ezek közül bármelyikre is) bár biztosak lehetünk benne, hogy a *tovább* által mutatott csúcsnak legalább t darab kulcsa van. Így folytathatjuk a rekurzív hívást erre az elemre (46. sor).

6.7. Algoritmus B-fából való törlés rekurzív része (keresés és törlés)

Bemenet: p - **M**, *érték* - **K**

```
1: eljárás KeresÉsTöröl(p : M, érték : K)
2:     ha p = ø akkor hiba "nincs ilyen kulcs"
3:     i ← KulcsKeresés(p, érték)
4:     ha i ≤ p.n ∧ p.kulcs[i] = érték akkor
5:         ha p.levél akkor
6:             ciklus j ← i-től p.n − 1-ig
7:                 p.kulcs[j] ← p.kulcs[j + 1]
8:             ciklus vége
9:             p.n ← p.n − 1
10:        különben
11:            gyerekB ← p.gyerek[i − 1]
12:            ha gyerekB.n ≥ t akkor
13:                p.kulcs[i] ← RészfaMax(gyerekB)
14:                KeresÉsTöröl(gyerekB, p.kulcs[i])
15:            különben
16:                gyerekJ ← p.gyerek[i + 1]
17:                ha gyerekJ.n ≥ t akkor
18:                    p.kulcs[i] ← RészfaMin(gyerekJ)
19:                    KeresÉsTöröl(gyerekJ, p.kulcs[i])
20:                különben
21:                    ÖsszevonÉsSzülőbőlLehoz(gyerekB, gyerekJ, p , i)
22:                    KeresÉsTöröl(gyerekB, érték)
23:                elágazás vége
24:            elágazás vége
25:        elágazás vége
26:    különben
27:        tovább ← p.gyerek[i]
28:        testvérB ← (i > 1 ? p.gyerek[i − 1] : ø)
29:        testvérJ ← (i <= p.n ? p.gyerek[i + 1] : ø)
30:        ha tovább ≠ ø ∧ tovább.n < t akkor
31:            ha testvérB ≠ ø ∧ testvérB.n ≥ t akkor
32:                BalTestvérbőlÁthoz(tovább, testvérB, p, i)
33:            különben
34:                ha testvérJ ≠ ø ∧ testvérJ.n ≥ t akkor
35:                    JobbTestvérbőlÁthoz(tovább, testvérJ, p, i)
36:                különben
37:                    ha testvérB ≠ ø akkor
38:                        ÖsszevonÉsSzülőbőlLehoz(testvérB, tovább, p, i − 1)
39:                        tovább ← testvérB
40:                    különben
41:                        ÖsszevonÉsSzülőbőlLehoz(tovább, testvérJ, p, i)
42:                    elágazás vége
43:                elágazás vége
44:            elágazás vége
45:        elágazás vége
46:        KeresÉsTöröl(tovább, érték)
47:    elágazás vége
48: eljárás vége
```

Felhasznált változók és függvények

- p: A vizsgálandó részfa gyökere.
- *érték*: A törlendő kulcs.
- RészfaMax(p : **M**) : **K**: Visszatérési értéke a részfa legnagyobb kulcsának értéke.
- RészfaMin(p : **M**) : **K**: Visszatérési értéke a részfa legkisebb kulcsának értéke.

6.4.4. Törlés segéd algoritmusok

Az áttekinthetőség kedvéért néhány művelet külső (esetenként csak egyszer hívott) függvényben lett megvalósítva.

Ilyenek például a megadott gyökerű részfa legkisebb, illetve legnagyobb elemének kiválasztása (6.9. és 6.8. algoritmusok). Ezek működése hasonló a bináris keresőfánál már tanulthoz, hiszen a fa legkisebb kulcsa itt is a legbaloldalibb elem (mindig a levélben), illetve a fa legnagyobb kulcsa itt is mindig a legjobboldalibb elem lesz (ami nyilvánvalóan szintén levélben fog elhelyezkedni). Ennek megfelelően ezek az eljárások csak egy egyszerű rekurziót tartalmaznak, ami folyamatosan halad a fában lefelé, amíg meg nem találja a megfelelő elemet, majd abban a szükséges kulcsot.

6.8. Algoritmus Adott gyökerű (rész)fa legkisebb kulcsa

Bemenet: p - **M**

```
1: eljárás RészfaMin(p : M)
2:     ha p.levél akkor
3:         vissza p.kulcs[1]
4:     különben
5:         vissza RészfaMin(p.gyerek[1])
6:     elágazás vége
7: eljárás vége
```

Felhasznált változók és függvények

- p: Az aktuálisan vizsgált B-fa gyökere.

6.9. Algoritmus Adott gyökerű (rész)fa legnagyobb kulcsa

Bemenet: p - **M**

```
1: eljárás RészfaMax(p : M)
2:     ha p.levél akkor
3:         vissza p.kulcs[p.n]
4:     különben
5:         vissza RészfaMax(p.gyerek[p.n + 1])
6:     elágazás vége
7: eljárás vége
```

Felhasznált változók és függvények

- p: Az aktuálisan vizsgált B-fa gyökere.

> **Megjegyzés**
>
> Az általunk használt algoritmusok nem optimálisak, például a minimum és maximum keresést követi egy törlés, ami tulajdonképpen ugyanazt az utat járja be. Ha tökéletesíteni szeretnénk a pszeudokódjainkat, akkor célszerű lenne ezeket az eljárásokat összevonni.

A legalább t darab kulcs biztosításához szükséges művelet a baloldali, illetve a jobboldali testvérből való kulcs áthozatala. Önmagukban ezek az eljárások nem bonyolultak, azonban a meglehetősen nagy mennyiségű adatmozgatás miatt (három csúcsban is változnak a kulcs, gyerek, és n értékek) meglehetősen hosszúak. Ezért ezek két eljárásban lettek megvalósítva: 6.10. és 6.11. algoritmusok.

Még szintén a t kulcs biztosításához szükséges lehet az elemek összevonását végző eljárás, ezt mutatja be a 6.12. algoritmus. Mint látható, ennek három paramétere van, a bal- illetve jobboldali összevonandó elem, illetve egy hivatkozás ezek közös szüleire és a kulcsra, amelynek ők a bal és jobb oldalán állnak.

Az összevonás tulajdonképpen a baloldali elembe való átmozgatását jelent, mind a szülőből lehozott új kulcs, mind pedig a jobboldali csúcsból áthozott kulcsok itt bővítik a már meglévő kulcsokat. Értelemszerűen ennek megfelelően növelni kell az n értékét, illetve át kell vinni a szükséges gyerek mutatókat is. A művelet során a szülőből törölni kell a lehozott kulcsot, továbbá a jobboldali elemet is töröljük, hiszen a későbbiekben nincs már rá szükség (a szülőből való kulcs törlést követően már hivatkozás sincs rá).

6.10. Algoritmus Baloldali testvérből elem áthozása

Bemenet: *tovább* - **M**, *testvérB* - **M**, *szülő* - **M**, i - **egész**

```
1:  eljárás BalTestvérbőlÁthoz(tovább : M, testvérB : M, szülő : M, i : egész)
2:      tovább.n ← tovább.n + 1
3:      tovább.gyerek[tovább.n + 1] ← tovább.gyerek[tovább.n]
4:      ciklus j ← tovább.n - 1-től 1-ig
5:          tovább.kulcs[j + 1] ← tovább.kulcs[j]
6:          tovább.gyerek[j + 1] ← tovább.gyerek[j]
7:      ciklus vége
8:      tovább.gyerek[1] ← testvérB.gyerek[testvérB.n + 1]
9:      tovább.kulcs[1] ← szülő.kulcs[i − 1]
10:     szülő.kulcs[i − 1] ← testvérB.kulcs[testvérB.n]
11:     testvérB.n ← testvérB.n − 1
12: eljárás vége
```

Felhasznált változók és függvények

- *tovább*: A B-fa egy csúcsa, amelyre tovább szeretnénk majd lépni (itt kell biztosítani az elegendő elemszámot). Pontosan t darab kulcsa van.
- *testvérB*: A *tovább* elem baloldali testvére. Legalább t darab kulcsa van.
- *szülő*: A *tovább* elem szülője.
- i: A szülőből a *tovább* csomópontra vonatkozó kulcs indexe.

6.11. Algoritmus Jobboldali testvérből elem áthozása

Bemenet: *tovább* - **M**, *testvérJ* - **M**, *szülő* - **M**, i - **egész**

```
1:  eljárás JobbTestvérbőlÁthoz(tovább : M, testvérJ : M, szülő : M, i : egész)
2:      tovább.n ← tovább.n + 1
3:      tovább.kulcs[tovább.n] ← szülő.kulcs[i]
4:      szülő.kulcs[i] ← testvérJ.kulcs[1]
5:      tovább.gyerek[tovább.n + 1] ← testvérJ.gyerek[1]
6:      ciklus j ← 1-től testvérJ.n - 1-ig
7:          testvérJ.kulcs[j] ← testvérJ.kulcs[j + 1]
8:          testvérJ.gyerek[j] ← testvérJ.gyerek[j + 1]
9:      ciklus vége
10:     testvérJ.gyerek[testvérJ.n] ← testvérJ.gyerek[testvérJ.n + 1]
11:     testvérJ.n ← testvérJ.n − 1
12: eljárás vége
```

Felhasznált változók és függvények

- *tovább*: A B-fa egy csúcsa, amelyre tovább szeretnénk majd lépni (itt kell biztosítani az elegendő elemszámot). Pontosan t darab kulcsa van.
- *testvérJ*: A *tovább* elem jobboldali testvére. Legalább t darab kulcsa van.
- *szülő*: A *tovább* elem szülője.
- i: A szülőből a *tovább* csomópontra vonatkozó kulcs indexe.

6.12. Algoritmus Két elem összevonása, és a szülőből egy kulccsal kiegészítése

Bemenet: *bal* - **M**, *jobb* - **M**, *szülő* - **M**, *i* - **egész**
Bemenet: *gyökér* - **M**, *érték* - **K**

```
1:  eljárás ÖsszevonÉsSzülőbőlLehoz(bal : M, jobb : M, szülő : M, i : egész)
2:      bal.n ← bal.n + 1
3:      bal.kulcs[bal.n] ← szülő.kulcs[i]
4:      bal.gyerek[bal.n + 1] ← jobb.gyerek[1]
5:      ciklus j ← 1-től jobb.n-ig
6:          bal.n ← bal.n + 1
7:          bal.kulcs[bal.n] ← jobb.kulcs[j]
8:          bal.gyerek[bal.n] ← jobb.gyerek[j + 1]
9:      ciklus vége
10:     ciklus j ← i-től szülő.n − 1-ig
11:         szülő.kulcs[j] ← szülő.kulcs[j + 1]
12:     ciklus vége
13:     ciklus j ← i + 1-től szülő.n-ig
14:         szülő.gyerek[j] ← szülő.gyerek[j + 1]
15:     ciklus vége
16:     szülő.n ← szülő.n − 1
17: eljárás vége
```

Felhasznált változók és függvények

- *gyökér*: A B-fa gyökere. A törlendő elem helyzetétől függően esetleg meg is változhat az eljárás végén.
- *érték*: A törlendő kulcs.

7. fejezet

Gráf

7.1. Gráfok felépítése

7.1.1. Alapvető jellemzők

Az eddig megismert dinamikus szerkezetek után (listák, fák), a következő lépést a gráfok jelentik. Egy *irányított gráf* nem más, mint a *csúcsok* halmaza, illetve egy ezen értelmezett bináris reláció (*élek* halmaza). Mint adatszerkezet, egyszerűen felfogható az eddig megismert egyéb dinamikus adatszerkezetek általánosításaként, hiszen azokhoz hasonlóan csúcsokból áll, amelyekből tetszőleges számú él mutat a többi csúcs felé. Ennek megfelelően mind a láncolt lista, mind pedig az előzőleg megismert fa adatszerkezetek is gráfok, a hamarosan bemutatásra kerülő műveletek (pl. bejárások) az előzőleg megismert adatszerkezeteken is jól használhatók (pl. a tanult fa bejárások alapvetően megfelelnek az ismertetésre kerülő mélységi bejárásnak).

Ezek előtt azonban vizsgáljuk meg, hogy milyen formában tudjuk eltárolni a gráfokat. A láncolt listák, illetve a fák tárolása erősen alapozott arra a tulajdonságra, hogy mindkét adatszerkezetben volt egy kitüntetett elem (*fej* vagy *gyökér* néven), és kívülről csak ezekhez rendelkeztünk hivatkozással, tehát ezeken keresztül értük el a többi elemet. A gráfok esetében azonban általában nem határozunk meg ilyen pontokat, sőt, irányított, illetve nem összefüggő gráfoknál még az sem biztos, hogy különböző csúcsokból kiindulva ugyanazokhoz a csúcsokhoz tudunk eljutni.

7.1.2. Alapműveletek

Itt is többféle implementációs mód létezik, és látni fogjuk, hogy az algoritmusaink működése szempontjából lényegtelen, hogy melyik tárolási módot választjuk (persze vannak teljesítménybeli különbségek), a lényeg, hogy az alábbi műveleteket meg tudjuk valósítani:

- Gráf szintjén szükséges műveletek (ahol G egy gráf, aminek típusa **GRÁF**):
  - *G.Csúcsok*: Visszaadja a G gráf csúcsainak a listáját.
  - *G.Élek*: Visszaadja a G gráf éleinek a listáját.
- Csúcs szintjén szükséges műveletek (ahol x és y csúcsok, amelyek típusa **CSP**):
  - *x.VezetÉl(y)*: Vezet-e él az x csúcsból az y csúcsba?
  - *x.Szomszédok*: Visszaadja az x csúcs szomszédait (azokat a csúcsokat, amelyekbe vezet él a megadott csúcsból).

A fentieket szükség szerint kiegészíthetjük további műveletekkel/tulajdonságokkal:

- A gráf csúcsainál bevezethetünk egy tartalom (*tart*) mezőt, ami a láncolt lista és a bináris keresőfa megvalósításhoz hasonlóan a gráf csúcsában eltárolt objektumot képviseli. Hasonló módon lehet *kulcs* mezőnk is ha szükséges.

- Súlyozott gráfok esetében a *VezetÉl*-hez hasonlóan bevezethetünk egy *Súly* függvényt is, ami megadja, hogy megadott csúcsba milyen súlyú él vezet (ha egyáltalán van ilyen).

Az alapműveletek implementációjával általánosan nem foglalkozunk, mivel azok erősen kötődnek ahhoz, hogy magát a gráfot milyen formában tároljuk. A következőkben, a tárolási módokkal együtt áttekintjük a fenti alapműveletek megvalósításának módjait is.

Ezeken túl szükség lehet további módosító műveletekre, pl. új csúcs felvétele, új él felvétele, meglévő csúcs törlése, meglévő él törlése, él súlyának megváltoztatása, stb. Részletesen ezekkel sem foglalkozunk, mivel a tárolástól függően ezek általában nagyon egyszerűen megvalósíthatók (vagy éppen nem megvalósíthatók, miként bizonyos tárolási módoknál nincs lehetőség a csúcsok számának növelésére).

7.2. Gráfok implementációja

7.2.1. Dinamikus tárolás

Ha nyomon követjük az eddig megismert adatszerkezeket, akkor jól látható a hasonlóság közöttük. A láncolt lista minden eleme eltárolt egy tartalmat, illetve egy darab referenciát a következő elemre. A ciklikus láncolt lista kivételével ez a hivatkozás nem mutathatott egy előző elemre. A fák esetében technikailag csak annyi különbséget láttunk (bár már ez is jelentősen megváltoztatta a műveleteinket), hogy minden elemnek lehet több „következő" eleme is, a bináris keresőfa esetében legfeljebb kettő, de pl. a B-fánál láttuk, hogy a gyerekek száma lehet ennél több is. A fánál azonban még mindig tudtuk, hogy nincsenek visszairányuló következő mutatók, tehát nem lehet egy elem bármelyik leszármazottja az ő őse is egyben, hiszen ezzel a legtöbb algoritmusunk végtelen ciklusba került volna.

Ha követjük az eddigi gondolatmenetet, akkor a gráfot elképzelhetjük egy olyan dinamikus adatszerkezetként, ahol minden egyes elemnek van egy tartalmi része, illetve tetszőleges számú mutatója, amelyek a szomszédaira mutatnak. Ezeket a hivatkozásokat eltárolhatjuk minden egyes csúcsban egy tömbben, ez azonban csak akkor valósítható meg, ha tudjuk az egy csúcsból kiinduló élek maximális számát. Amennyiben ez nincs így (mivel két csúcspont között több él is vezethet, így még a csúcsok száma se mindig kielégítő korlát), akkor esetleg egy láncolt listában is eltárolhatjuk ezeket a hivatkozásokat.

További esetleges módosítások:

- Irányított/irányítatlan gráfok: Mivel a programozási nyelvekben lévő mutató típusok mindig csak egyirányúak, így ezzel a módszerrel csak irányított gráfokat tudunk modellezni. Irányítatlan esetben minden él két mutatót jelent, amelyek a két irányt jelképezik.

- Súlyozott gráfok: Magában a gráf csúcsban is tárolhatjuk, hogy az egyes élek milyen súlyúak, vagy akár magát az élt is elképzelhetjük külön objektumként, aminek van súly tulajdonsága.

Az így elkészült adatszerkezet szépségét némileg beárnyékolja, hogy a gráf esetén nem elégedhetünk meg egy darab „fej-szerű" külső hivatkozással, hiszen a későbbi bejárásainkat bármelyik csúcsból el szeretnénk tudni indítani. Emiatt minden csúcsra célszerű felvenni egy-egy külső referenciát (amiket tárolhatunk akár egy hasító táblázatban is).

A módszer előnye, hogy dinamikus volta miatt optimális a helyfoglalása, illetve lehetőség van futás közben új csúcsok felvételére és meglévő csúcsok törlésére. Érdemes azonban megismerni két, a gyakorlatban elterjedt további megoldást.

7.2.2. Tárolás csúcsmátrixban

Amennyiben megengedjük azt a korlátot, hogy a gráf csúcsainak számát fixnek tekintjük, akkor jóval egyszerűbben kezelhető módszerekhez jutunk. A csúcsmátrixban való tárolás talán a legegyszerűbb tárolási módszer, még ha bizonyos esetekben nem is tekinthető hatékonynak. A gráf tárolásának alapelve, hogy létrehozunk egy kétdimenziós CS nevű mátrixot, amely sorainak és oszlopainak a száma megegyezik a gráf csúcspontjainak a számával (és minden gráf csúcsot hozzárendelünk egy-egy sorhoz, illetve oszlophoz).

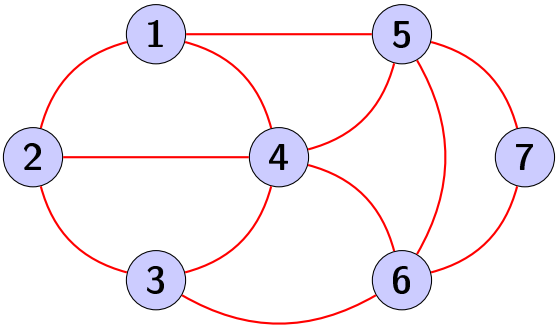
Ezt követően a $CS_{i,j}$ azt határozza meg, hogy az i. csúcsból vezet-e él a j. csúcsba. A mátrix tartalmazhat igaz/hamis értékeket (vezet-e él), vagy akár természetes számokat is (hány él vezet, 0 ha nincs, 1 vagy annál nagyobb, ha van).

A 7.1. ábra egy példát mutat erre a tárolási módra. Látható, hogy pl. a $CS_{2,3}$ helyen lévő 1-es azt mutatja, hogy van él, ami a 2-es csúcsból vezet a 3-as csúcsba.

További esetleges módosítások:

- Irányított/irányítatlan gráfok: Mindkét fajta gráfot egyszerűen ábrázolhatjuk. Irányítatlan gráfok esetében nyilvánvalóan minden $CS_{i,j}$ és $CS_{j,i}$ érték azonos lesz, így nem is szükséges eltárolni a teljes mátrixot, elég csak a felső háromszögmátrixot (ezzel a tárigény csaknem a felére csökken).

- Súlyozott gráfok: Egy súlyozott gráfot úgy is tárolhatunk, hogy $CS_{i,j}$ megadja, hogy az i. csúcsból a j. csúcsba vezető élnek mennyi a súlya. Ebben az esetben valamilyen speciális módon jelölnünk kell azt is, ha nem vezet él (ez alapos megfontolást igényel, hiszen pl. a 0 választása nem feltételnül szerencsés, ha szeretnénk 0 súlyú élekkel is dolgozni).



	1	2	3	4	5	6	7
1	0	1	0	1	1	0	0
2	1	0	1	1	0	0	0
3	0	1	0	1	0	1	0
4	1	1	1	0	1	1	0
5	1	0	0	1	0	1	1
6	0	0	1	1	1	0	1
7	0	0	0	0	1	1	0

7.1. ábra. Irányítatlan, súlyozatlan gráf és az azt reprezentáló csúcsmátrix.

Az alapműveletek megvalósításai:

- *x.VezetÉl(y)*: Egy lépéssel megoldható az ellenőrzés, mivel csak az x-hez tartozó sor, és az y-hoz tartozó oszlopban lévő értéket kell megvizsgálnunk. Ha itt *hamis* (vagy 0) szerepel, akkor nincs él, különben pedig van. Irányítatlan gráf esetén, ha csak a felső háromszöget tároljuk, akkor esetenként át kell alakítanunk a kérdést *y.VezetÉl(x)* formára.
- *x.Szomszédok*: Ez már egy jóval erőforrás-igényesebb művelet. Végig kell néznünk az X-hez tartozó sort a csúcsmátrixban, és ki kell válogatni azokat az oszlopokat (illetve a hozzájuk tartozó csúcsokat), amelyeknél a mátrix értéke nem *hamis* (vagy 0). Ez a lista lesz a tulajdonság visszatérési értéke.
- *G.Csúcsok*: Mindig feltételezzük, hogy valahol eltároljuk a csúcsok adatait. Itt ezt kell visszaadnunk.
- *G.Élek*: Ez szintén sok lépést igényel. Végig kell néznünk a mátrix minden elemét, és ez alapján tudjuk visszaadni az élek listáját.

Látható, hogy bizonyos műveletek nagyon hatékonyan megvalósíthatók ebben a tárolási formában, mások pedig kevésbé. Röviden összefoglalva a tapasztalatokat:

- Tárhely szempontjából: Mivel a létrehozott tömb mérete mindig a csúcsok számának a négyzete lesz, így ez mindig ugyanakkora. A tárolás hatékonysága tehát attól függ, hogy valójában hány élt tárolunk, amennyiben ez a szám kicsi, akkor a tömb sok felesleges helyet foglal.
- Sebesség szempontjából: Általában elmondható, hogy akkor célszerű ezt a tárolási módot választani, ha a gráfon végzendő műveletek főleg az élek meglétének vizsgálatát igénylik (mivel ez csak 1 lépést igényel), kevésbé a szomszédsági vizsgálatokat (ami viszont a csúcsok számának megfelelő lépést igényel).

7.2.3. Szomszédsági listában tárolás

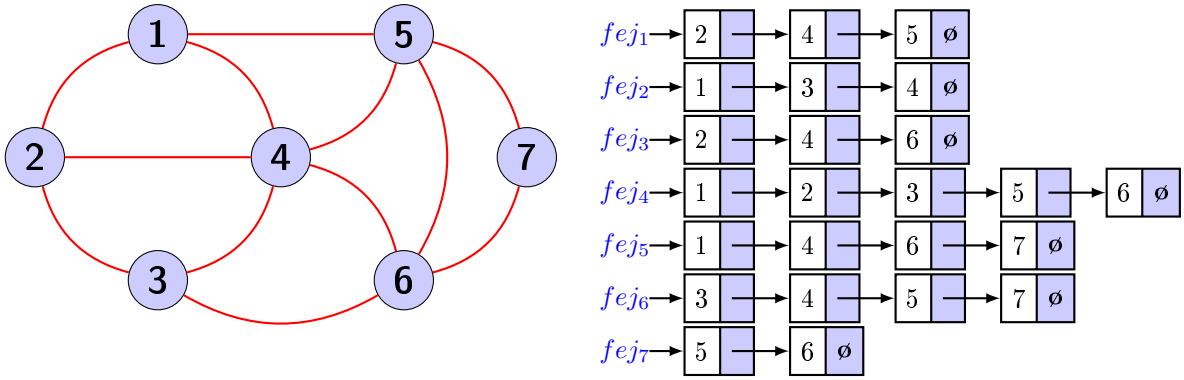
Az előzőleg felsorolt negatívumok miatt bizonyos esetekben célszerű lehet egy másik megvalósítást alkalmazni, ez pedig a szomszédsági listában való tárolás.

Az alapelvet jól szemlélteti a 7.2. ábra. Létre kell hoznunk egy, a csúcsok számával megegyező méretű L tömböt (esetleg hasító táblázatot), amely láncolt lista elem típusú hivatkozásokat tartalmaz (a gráf minden csúcsát hozzárendeljük a vektor egy eleméhez).

Az éleket pedig úgy tároljuk el, hogy az i. indexhez tartozó csúcsból kiinduló éleket (implementációtól függően magát az élt, vagy a cél csúcsot, vagy a cél csúcs indexét) az $L[i]$ láncolt lista tartalmazza.

Az alapműveletek megvalósításai:

- *x.VezetÉl(y)*: Az előző megoldással ellentétben ez egy meglehetősen erőforrás-igényes művelet, hiszen végig kell néznünk a láncolt listát. Rendezéssel persze valamennyire lehet ezen gyorsítani, illetve a listában csak annyi elem van, ahány él kiindul az adott csúcspontból, tehát valamivel jobb a helyzet, mint a csúcsmátrix szomszédkeresésénél.



7.2. ábra. Irányítatlan, súlyozatlan gráf szomszédsági listával ábrázolva.

- *x.Szomszédok*: Ez viszont itt könnyen megadható, csak vissza kell adni (esetlegesen kisebb átalakítások után) magát a láncolt listát, ami épp ezt az adatot tárolja.
- *G.Csúcsok*: Feltételezzük, hogy valahol eltároljuk a csúcsok adatait. Itt ezt kell visszaadnunk.
- *G.Élek*: Ez is hatékonyabb lesz ebben az esetben, csak össze kell fűznünk az összes láncolt listát az eredményhez.

Röviden itt is összefoglalhatjuk, hogy mikor érdemes használni ezt a módszert:

- Tárhely szempontjából: A láncolt listák *fej* változói és az elemek *következő* mutatói mind valamennyi tárhelyet igényelnek. Amennyiben kevés él indul ki az egyes csúcsokból, akkor általában még ezzel együtt is kevesebb helyet foglal a szomszédsági lista, mint a csúcsmátrix, az élek számának növelésével azonban ez a helyzet romlik, majd meg is fordul. Tehát ahhoz, hogy egyértelműen eldöntsük, hogy melyik foglal kevesebb helyet, tudnunk kell a csomópontok és élek számát, illetve a mutatók és a tartalom által foglalt helyet.
- Sebesség szempontjából: Általában elmondható, hogy akkor fogjuk ezt a tárolási módot választani, ha a gráfon végzendő műveletek főleg a szomszédsági kapcsolatok vizsgálatát igénylik majd (mivel ez csak 1 lépést igényel), kevésbé az élek meglétének lekérdezését (ez ugyanis 1..*csúcsokszáma* darab lépést is igényelhet).

7.3. Bejárások

Miként a többi adatszerkezetnél, úgy itt is a bejárások vizsgálatával kezdjük az adatszerkezet műveleteinek bemutatását. Tegyük fel, hogy már rendelkezünk egy, a fenti módszerek egyikével eltárolt gráffal, és ennek szeretnénk feldolgozni az elemeit. A bejárás célja itt is hasonló, mint az előző dinamikus adatszerkezeteknél, azonban attól némileg különbözik. Míg a láncolt lista és a fa esetében a bejárás kiindulópontja mindig a *fej* vagy *gyökér* volt, addig a gráfok esetében nem szoktunk kijelölni egy megkülönböztetett pontot, így a bejárás bármelyik csúcsból elindítható. Másik lényeges különbség, hogy míg a láncolt lista és a bináris keresőfa esetében a bejárás az adatszerkezet összes elemének a feldolgozását jelenti, addig a gráf esetében a bejárás csak azokat fogja elérni, amelyek az előbb említett kiindulópontból elérhetőek a gráf élei mentén (a feldolgozandó elemek alatt a gráf esetében általában a csúcsokat értjük, habár az élek ugyanúgy a gráf alkotóelemei, és elképzelhetőek olyan bejárások is, amelyek az élekre vonatkozóan is adnak valamiféle követelményt).

Ennek megfelelően tehát mindkét ismertetésre kerülő bejárás (szélességi és mélységi bejárás) esetében az algoritmus bemenete egy csúcs lesz, amelyből elindulunk. Az algoritmus pedig futása közben az összes, ebből a csúcsból elérhető csúcsot fel fogja dolgozni. A különbség pusztán az a két bejárás között, hogy különböző sorrendben érik el az egyes csúcsokat. A bejárás szempontjából nem kell azzal foglalkoznunk, hogy a feldolgozás maga milyen jellegű művelet, lehet hogy a képernyőre kiírás, lehet hogy módosítás, lehet hogy csak összegyűjti az elért csúcsok referenciáit egy tömbbe, stb. A többi adatszerkezethez hasonlóan az algoritmus ezektől függetlenül, általánosan értelmezhető lesz.

A bejárások széleskörűen alkalmazhatók egyéb gráfokkal kapcsolatos feladatok esetében is: komponensek keresésére, a gráf összefüggőségének vizsgálatára, stb. [1, 2, 11]

7.3.1. Szélességi bejárás

A *szélességi bejárás* alapelve, hogy elsőként mindig a kiindulóponthoz legközelebbi csúcsokat dolgozza fel. A 7.3. ábrán jól látható ez a sorrend. A kezdőpont után az ettől egy élnyi távolságra lévő csúcsokat dolgozza fel (ezek feldolgozási sorrendje már tetszőleges lehet), majd ezt követően a két élnyi távolságra lévőket, stb. A feldolgozás sorrendje egy hullám terjedéséhez hasonlítható, a közelebbi csúcsok korábban, a távolabbi csúcsok pedig később kerülnek sorra.

Működésének alapja egy sor adatszerkezet. Ez a sor tartalmazza azokat a csúcsokat, amelyeket már elért az algoritmus, azonban még nem lettek feldolgozva. Kezdetben a kiinduló csúcsot helyezzük el ebbe a sorba, majd egy ciklus segítségével addig fut az algoritmus, amíg ez nem lesz üres. Minden ciklus iterációnál kivesszük a soronkövetkező elemet, azt feldolgozzuk, majd az elem azon szomszédjait, amelyeket eddig még nem értünk el, elhelyezzük a sorba a későbbi feldolgozás érdekében.

Ezt a működést mutatja a 7.1. algoritmus. Az algoritmus első lépéseként az üres S sorba elhelyezi a *start* változó által tárolt csúcs, tehát a kiinduló csúcs referenciáját. Ugyanezt a csúcsot elhelyezi az F halmazba is (ez lesz a halmaz egyetlen eleme), amely a már feldolgozott, vagy legalábbis feldolgozásra váró elemeket tartalmazza (tehát azokat, amelyeket már legalább egyszer elértük az algoritmus működése során).

Ezt követően elindul a ciklus (4. sor), ami egészen addig fut, ameddig van elem a sorban. Ha elfogytak az elemek, az azt jelenti, hogy nincs több olyan csúcs, amelyik a kiinduló csúcsból elérhető, viszont még nem lett feldolgozva.

Az 5. sorban látható módon kivesszük a sorból az első elemet. Kezdetben csak a kiinduló elem van itt, tehát azt vesszük azonnal ki. A 6. sorban pedig már meg is történik ennek a feldolgozása.

Az aktuális elem feldolgozását követően az algoritmus egyesével megvizsgálja az elem minden szomszédját (7. sor), hogy melyik irányokba lehet folytatni a bejárást. Az egyes szomszédok közül bármelyik irányba továbbléphetünk majd, csak arra kell ügyelnünk, hogy ugyanazt az elemet ne dolgozzuk fel egymás után többször. Ez egyrészt hibás eredményt adna, másrészt ez az ellenőrzés azt is garantálja, hogy ne kerüljünk végtelen ciklusba. A feldolgozottságot egyszerűen tudjuk vizsgálni (8. sor), csak azt kell megnézni, hogy a vizsgált szomszéd benne van-e a már előzőleg elért (tehát feldolgozásra váró, vagy már feldolgozott) csúcsokat tartalmazó F halmazban.

Ha nincs, akkor a vizsgált szomszéd felé folytathatjuk majd a keresést. Nem lépünk rá azonnal, csak elhelyezzük a sorba a későbbi feldolgozás érdekében a 9. sorban látható művelettel.

7.1. Algoritmus Gráf szélességi bejárása

Bemenet: *start* - **CSP**,

```
1: eljárás SzélességiBejárás(start : CSP)
2:     S ⇐ start
3:     F ← {start}
4:     ciklus amíg (S ≠ ø)
5:         k ⇐ S
6:         Feldolgoz(k.tart)
7:         ciklus mind x ∈ k.Szomszédok
8:             ha x ∉ F akkor
9:                 S ⇐ x
10:                F ← F ∪ x
11:            elágazás vége
12:        ciklus vége
13:    ciklus vége
14: eljárás vége
```

Felhasznált változók és függvények

- S: Egy gráf csúcsokat tartalmazó sor, amely a feldolgozásra váró elemeket tartalmazza. Kezdetben üres.
- F: Egy gráf csúcsokat tartalmazó halmaz, amely a feldolgozott vagy még feldolgozásra várt elemeket tartalmazza. Kezdetben üres.
- Feldolgoz(*elem* : **T**): Tetszőleges feldolgozási művelet, amely minden csúcsban eltárolt tartalomra pontosan egyszer fut le.
- k, x: **CSP** típusú segédváltozók.

Az így elhelyezett elem feldolgozása majd a külső ciklus egy későbbi iterációjában fog megtörténni. A sor működéséből adódóan az 5. sor mindig az egyik legközelebbi csúcsot fogja visszaadni, ez pedig garantálja, hogy a szélességi bejárás valóban azt az eredményt adja, amit várunk.

Véges méretű gráfok esetében a még el nem ért szomszédok idővel elfogynak, ilyenkor már nem kerül a sorba új csúcs. A benne lévő csúcsok feldolgozását követően így a sor hamarosan üres lesz, ezzel pedig a külső ciklus feltétele nem teljesül többé, így véget ér a bejárás.

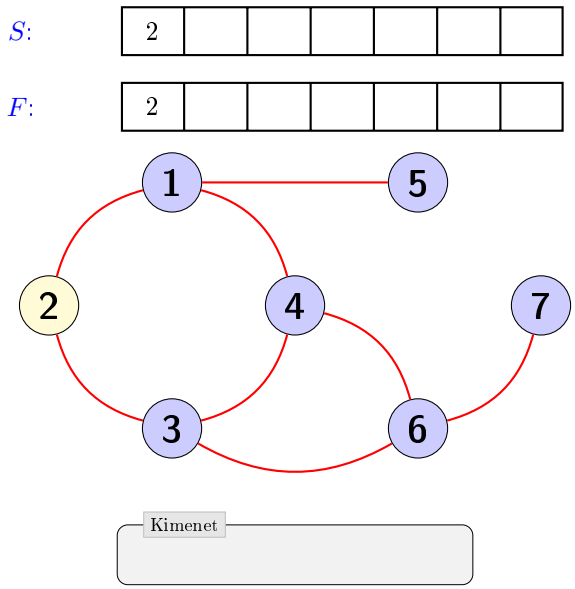
A szélességi bejárás működésére látható egy példa a 7.3. ábrán. Az ábra színezése minden egyes csúcs esetén az alábbi állapotokat különbözteti meg:

- Kék: A gráf csúcsai, amelyekhez még nem értünk el (kezdetben minden csúcsot ilyennek tekintünk). Tehét ezek még nincsenek benne az F halmazban.
- Sárga: Azok a csúcsok, amelyeket már elértünk egy másik csúcson keresztül, azonban még nem dolgoztuk fel őket. Tehát éppen benne vannak az S sorban, várakoznak a későbbi feldolgozásra. Emellett benne vannak az F halmazban is.
- Szürke: Azok a csúcsok, amelyek már feldolgozásra kerültek. Tehát benne vannak az F halmazban, de az S-ben már nincsenek benne.
- Piros: Az aktuálisan feldolgozás alatt álló csúcs (tehát valójában már szürke, csak a szemléletesség kedvéért kapott egy külön színt). Az algoritmusban a k változó értéke.

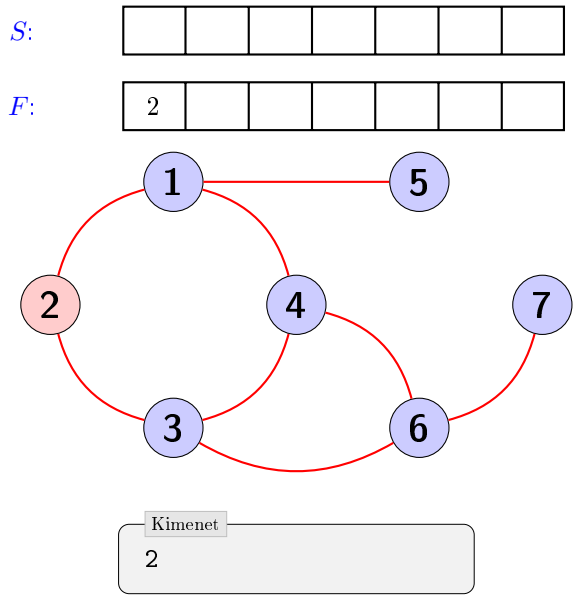
Érdemes tehát megjegyezni, hogy a szélességi bejárás csak egy komponens bejárására alkalmas. Amennyiben biztosítani szeretnénk, hogy a gráf minden csúcsát elérje, akkor egy újabb ciklussal a bejárást meg kell hívni minden csúcsra (illetve csak azokra, amelyeket nem dolgoztak fel az előzőleg futtatott bejárások).

Szintén gyakori módosítás, ha az algoritmust kiegészítjük néhány kiegészítő művelettel:

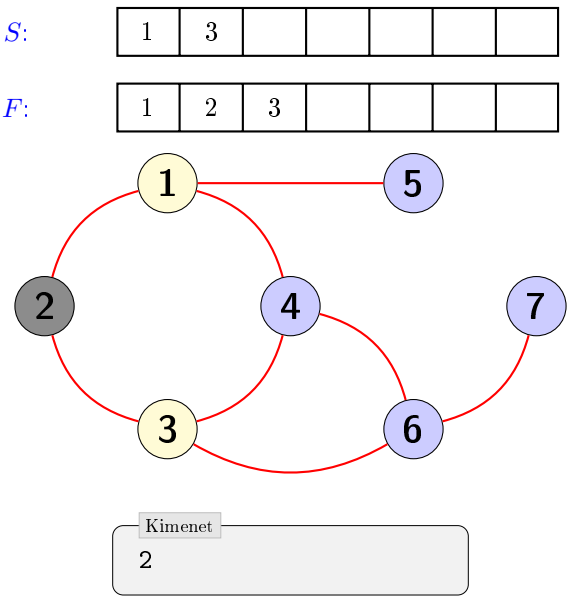
- Minden új csomópont felvétele esetén (tehát amikor találtunk egy új, még el nem ért szomszédot) eltárolhatjuk, hogy melyik csúcsból értük el ezt a pontot. Ilyenkor egy, a kezdőpontból kiinduló fát kapunk eredményül, amely mutatja az egyes csúcsok legrövidebb elérési útjait (ahol úthossz alatt a kiindulópontból ide vezető utakon található élek számát értjük).



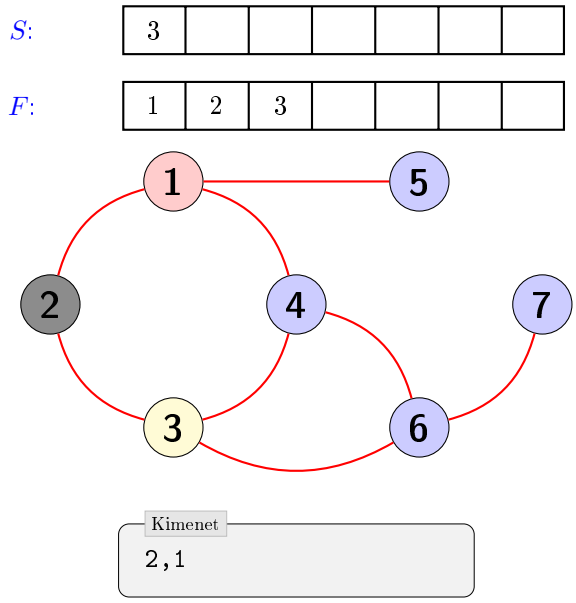
(a) Az algoritmus kezdeti lépéseként elhelyezi a kezdőcsúcsot (ami egy paraméter, jelen példában legyen 2) az S sorba és az F halmazba.



(b) A sorból kiveszi az első elemet, majd feldolgozza.

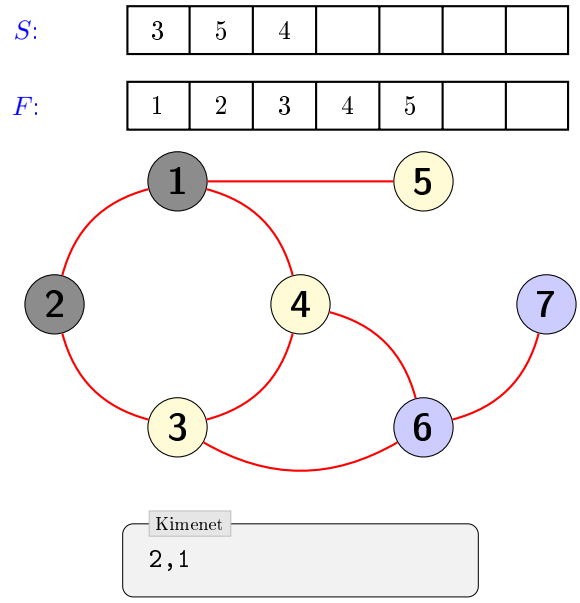


(c) A feldolgozott csúcs szomszédai bekerülnek a sorba és a halmazba (1,3).

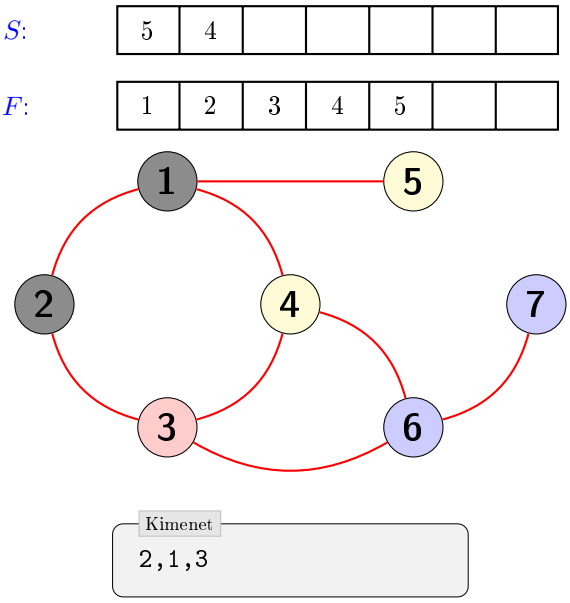


(d) A sorból kiveszi a következő feldolgozandó 1-es csúcsot, majd feldolgozza.

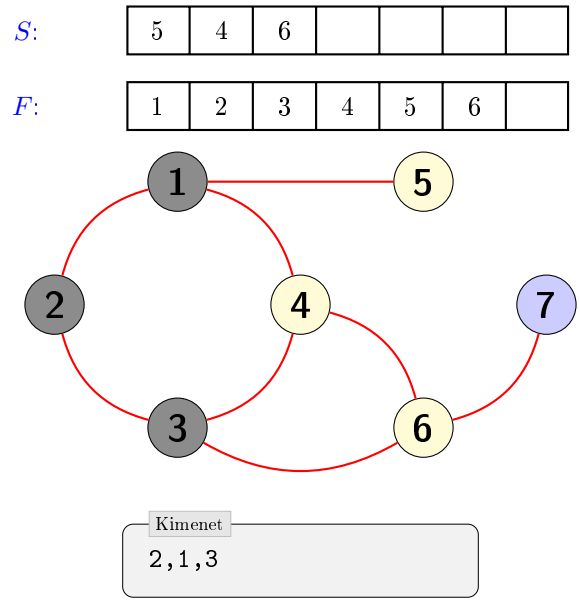
7.3. ábra. Gráf szélességi bejárása.



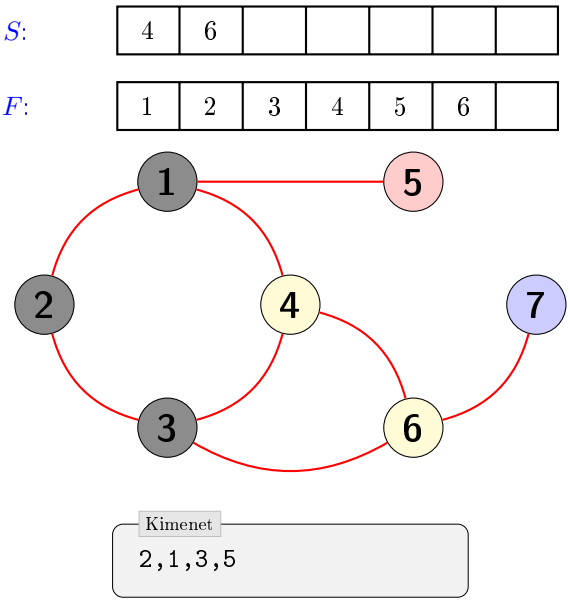
(e) Ennek a szomszédai (csak azok, amelyek még nincsenek benne) szintén bekerülnek a halmazba és a sorba (5,4).



(f) A sorból kiveszi a következő feldolgozandó 3-as csúcsot, majd feldolgozza.

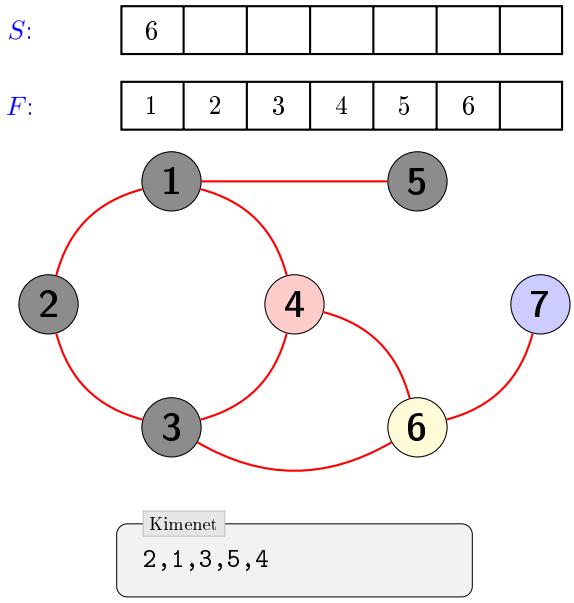


(g) Ennek a szomszédai (csak azok, amelyek még nincsenek benne) szintén bekerülnek a halmazba és a sorba (6).

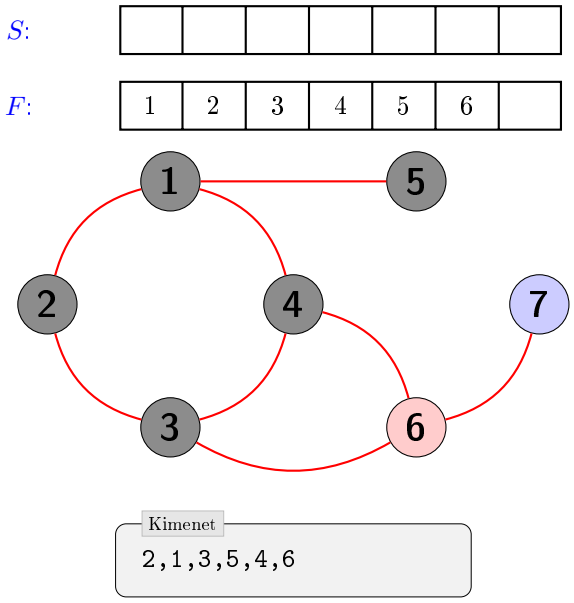


(h) A következő feldolgozandó elem az 5-ös. Ennek nincs új szomszédja, így nem bővül az S sor.

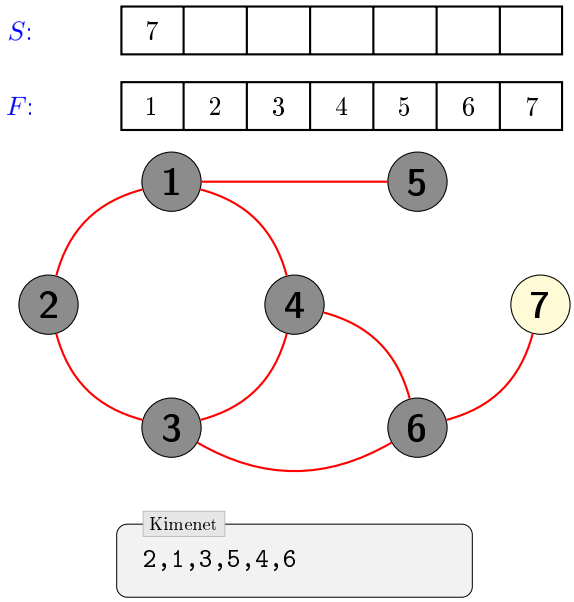
7.3. ábra. Gráf szélességi bejárása. (folytatás)



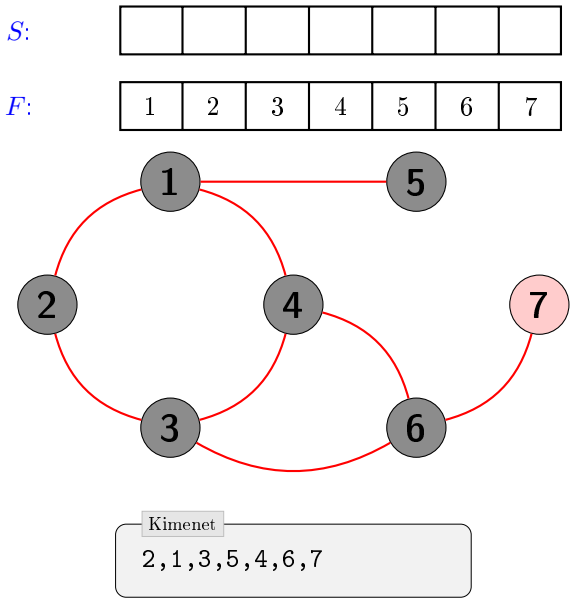
(i) A sorban a következő a 4-es elem. Ezt is feldolgozza, és mivel nincsenek még el nem ért szomszédai, itt sem kerül új elem a sorba.



(j) A sorban következő 6-os elem feldolgozása következik.

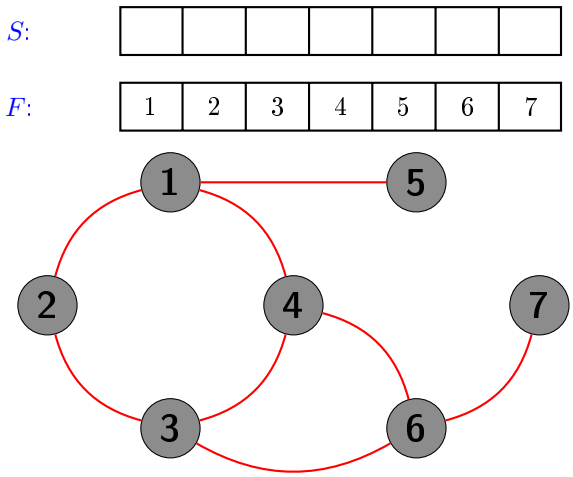


(k) Ennek a szomszédja még belekerül a sorba (7).



(l) Az egyetlen 7-es elem is kikerül a sorból és feldolgozásra kerül. Feldolgozatlan szomszédai nincsenek, így nem kerül a sorba új elem.

7.3. ábra. Gráf szélességi bejárása. (folytatás)



Kimenet

```
2,1,3,5,4,6,7
```

(m) Mivel üres a sor, nem teljesül tovább a külső ciklus feltétele, emiatt az algoritmus végetér. Láthatóan bejártuk az egész gráfot.

7.3. ábra. Gráf szélességi bejárása. (folytatás)

- Minden új csomópont felvétele esetén eltárolhatjuk azt is, hogy ő milyen messze van a kiinduló ponttól. Ez a távolság minden esetben egyszerűen számolható, hiszen mindig pont eggyel több, mint annak a csomópontnak a távolsága, ahonnan elértük az új elemet (kivéve persze a kiinduló csúcsot, ahol ez 0).

7.3.2. Mélységi bejárás

A *mélységi bejárás* ugyanazokat a csúcsokat fogja elérni, mint a szélességi, azonban más sorrendben. Az előzőleg megismert szélességi bejárás mindig a legközelebbi csúcsokat dolgozta fel először, és csak akkor látott neki a távolabbi csúcsoknak, ha már az összes közelebbivel végzett. A mélységi bejárás ezzel szemben elindul egy úton, és egészen addig halad előre a szomszédsági kapcsolatokon keresztül, amíg nem jut egy olyan csúcsra, ahonnan már nem tud továbblépni. Ilyenkor egy szinttel visszább lép, és az előző csúcsnál próbál keresni egy másik, még fel nem dolgozott szomszédot. Mindezt addig folytatja, amíg elfogynak a választható feldolgozatlan szomszédok.

Megjegyzés

Nem véletlen a hasonlóság a mélységi keresés alapelve, és a visszalépéses keresésnél tanult módszer között (folyamatosan halad előre, és zsákutcába kerülve visszalép egy szintet, és új irányt keres). A visszalépéses keresés tulajdonképpen a mélységi bejárás lépéseit végzi, csak a háttérben lévő gráfot nem a szokásos módon ábrázoltuk.

A szélességi bejárásnál egy sor biztosította, hogy a csomópontok feldolgozása abban a sorrendben történjen, ahogy először találkoztunk velük (ami így megfelel a kiindulóponttól való távolságnak), a mélységi bejárásnál verem segítségével tudjuk eltárolni a keresés során bejárt utat. Magát a vermet nem mi kezeljük közvetlenül, hanem az algoritmust rekurzívan adjuk meg, így a visszalépéses kereséshez hasonló megoldáshoz jutunk. A mélységi bejárás megvalósítását mutatja a 7.2. algoritmus.

Rekurzív algoritmus lévén találunk egy indító eljárást is, ez csak törli a már feldolgozott csúcsokat tartalmazó F halmaz tartalmát (11. sor), majd meghívja magát a rekurziót, első paraméterként átadva a kiinduló csúcsot. Második paraméterként mindig az F halmazt fogjuk átadni címszerint a következő szintekre, hogy ebbe tudjuk gyűjteni a már feldolgozott elemeket (objektumorientált megközelítés esetében ez persze szükségtelen lehet).

7.2. Algoritmus Gráf mélységi bejárása

Bemenet: *start* - **CSP**, F - **CSP halmaz**

```
1: eljárás MélységiBejárásRek(k: CSP, címszerint F: CSP halmaz)
2:     F ← F ∪ k
3:     Feldolgoz(k.tart)
4:     ciklus mind x ∈ k.Szomszédok
5:         ha x ∉ F akkor
6:             MélységiBejárásRek(x, F)
7:         elágazás vége
8:     ciklus vége
9: eljárás vége
```

Bemenet: *start* - **CSP**

```
10: eljárás MélységiBejárás(start: CSP)
11:     F ← ø
12:     MélységiBejárásRek(start, F)
13: eljárás vége
```

Felhasznált változók és függvények

- F: Egy gráf csúcsokat tartalmazó halmaz, amely a már feldolgozott elemeket tartalmazza.
- Feldolgoz(*elem* : **T**): Tetszőleges feldolgozási művelet, amely minden csúcsban eltárolt tartalomra pontosan egyszer fut le.
- k, x: **CSP** típusú segédváltozók.

Maga a rekurzió meglehetősen egyszerű, minden lefutásakor eltárolja az éppen feldolgozandó elemet az F halmazba (2. sor), majd elvégzi a paraméterként kapott csúcs feldolgozását a 3. sorban.

A továbblépéshez megvizsgálja az aktuális csúcs szomszédait egy ciklus segítségével (4. sor). A továbblépés egyetlen követelménye, hogy a kiszemelt csúcsot még nem dolgoztuk fel. Ezt egyszerűen tudjuk ellenőrizni az F halmaz alapján (5. sor), hiszen minden elért csúcs esetében a feldolgozás előtt közvetlenül elhelyeztük a csúcsot ide.

Amennyiben a szomszéd nincs benne az F-ben, akkor rekurzív módon újrahívja önmagát a függvény (6. sor). Látható, hogy a szomszédok ellenőrzése hasonló, mint a szélességi keresésnél, a lényeges különbség pusztán az, hogy a mélységi keresés nem tárolja az elérendő elemeket, hanem egy új csúcs felfedezése esetén azonnal átlép arra, és ott folytatja a feldolgozást.

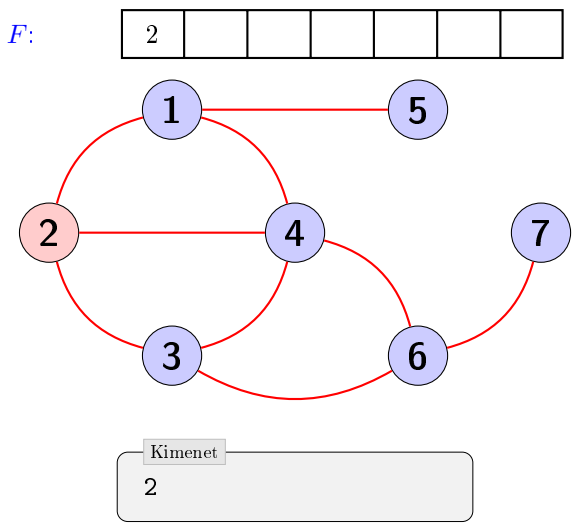
A mélységi bejárás működésére látható egy példa a 7.4. ábrán. Az ábra színezése minden egyes csúcs esetén az alábbi állapotokat különbözteti meg:

- Kék: A gráf csúcsai, amelyekhez még nem értünk el (kezdetben minden csúcsot ilyennek tekintünk). Tehét ezek még nincsenek benne az F halmazban.
- Szürke: Azok a csúcsok, amelyek már feldolgozásra kerültek. Tehát benne vannak az F halmazban.
- Piros: Az aktuálisan feldolgozás alatt álló csúcs (tehát valójában már szürke, csak a szemléletesség kedvéért kapott egy külön színt). Az algoritmusban a k változó értéke.

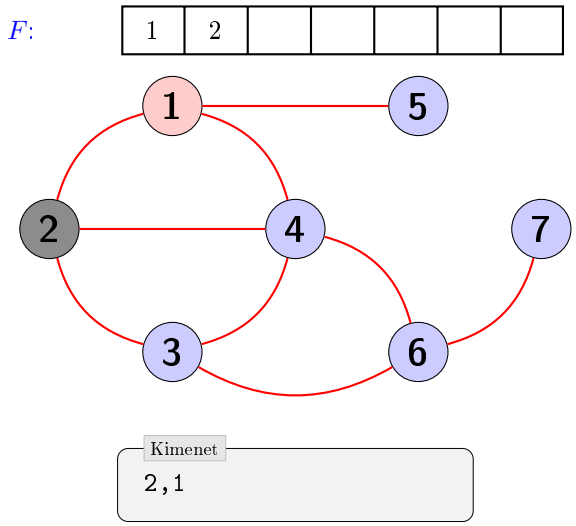
A mélységi bejárásra is igaz, hogy önmagában csak egy komponens bejárására alkalmas. Amennyiben biztosítani szeretnénk, hogy a gráf minden csúcsát elérje, akkor egy újabb ciklussal a bejárást meg kell hívni minden csúcsra (illetve csak azokra, amelyeket nem dolgoztak fel az előzőleg futtatott bejárások).

Az algoritmus kiegészíthetjük néhány kiegészítő művelettel:

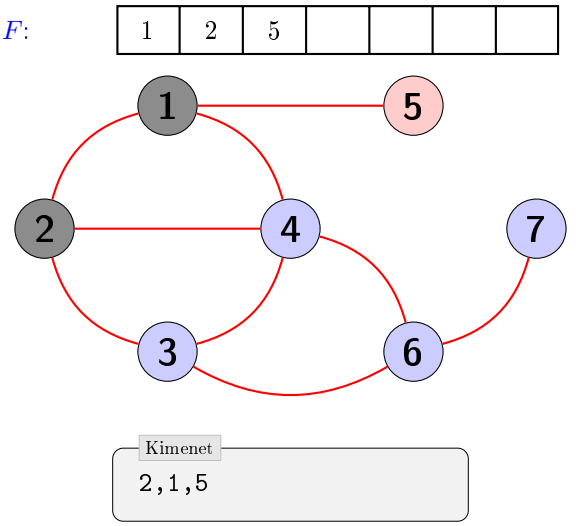
- Minden új csomópont felvétele esetén (tehát amikor találtunk egy új, még el nem ért szomszédot) eltárolhatjuk, hogy melyik csúcsból értük el ezt a pontot. Ilyenkor egy, a kezdőpontból kiinduló fát kapunk eredményül, amely mutatja az egyes csúcsok elérési útjait.



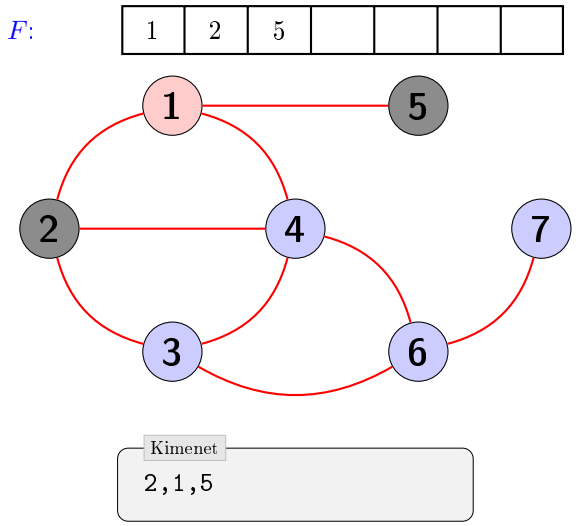
(a) Az algoritmus kezdeti lépéseként elhelyezi a kezdőcsúcsot (ami egy paraméter, jelen példában legyen 2) az F halmazba, és egyből fel is dolgozza azt.



(b) A 2-es elem első szomszédja az 1-es, ezért ide lép tovább.

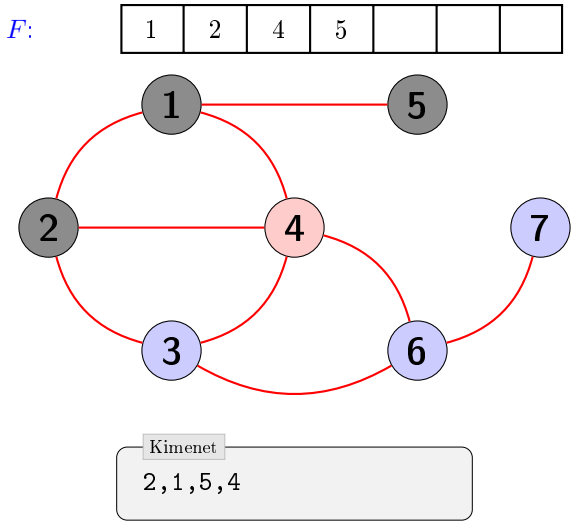


(c) Az 1-es első szomszédja az 5, ezért ide lép tovább.

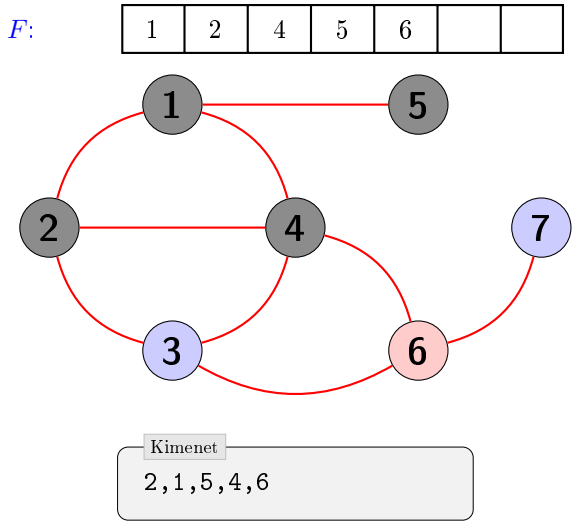


(d) Az 5-ösnek csak egy szomszédja van, az 1-es, de az már benne van az F-ben. Ezért visszalép a rekurzió.

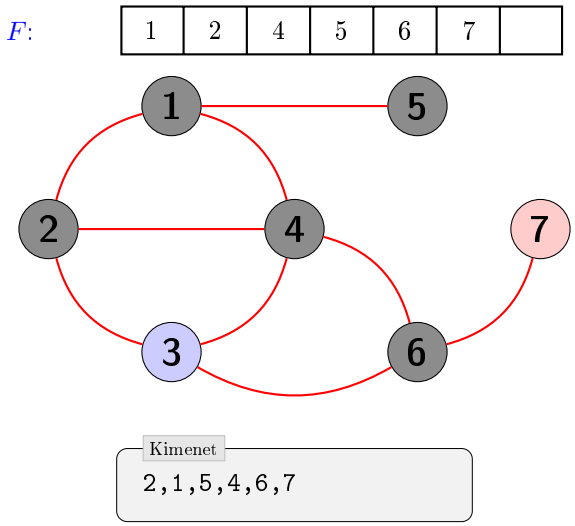
7.4. ábra. Gráf mélységi bejárása.



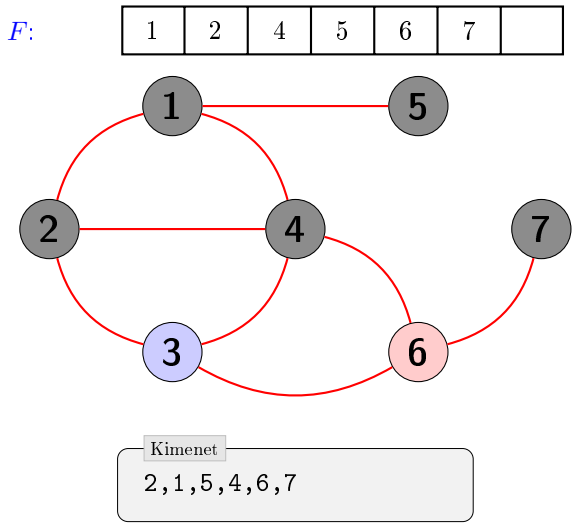
(e) Az 1-esről még tovább lehet lépni a 4-re, ezért újrahívja magát a függvény.



(f) A 4-es elem első szomszéja a 6, így továbblép ide.

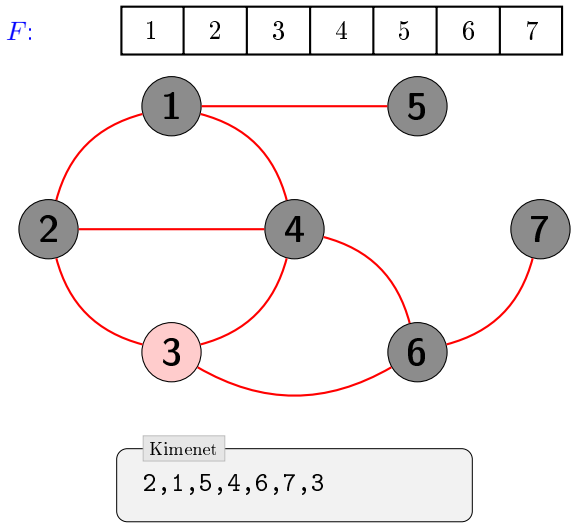


(g) A 6-ról pedig tovább lehet lépni a 7-re.

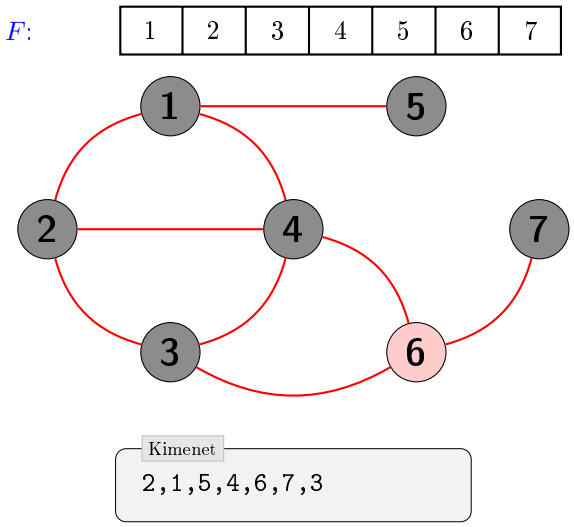


(h) A 7-nek már nincs olyan szomszéja, akit nem dolgoztunk fel, ezért visszalépés.

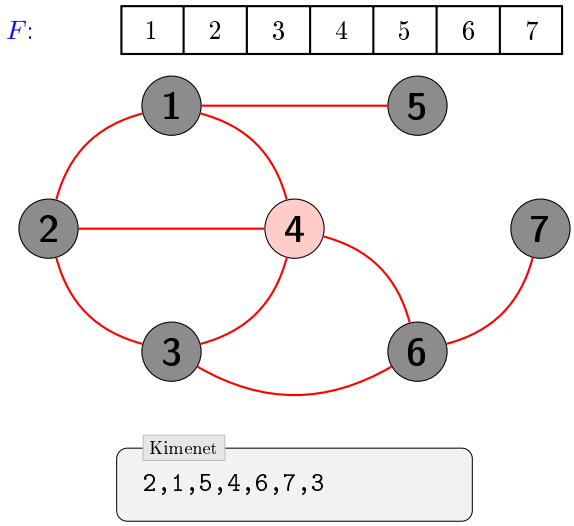
7.4. ábra. Gráf mélységi bejárása. (folytatás)



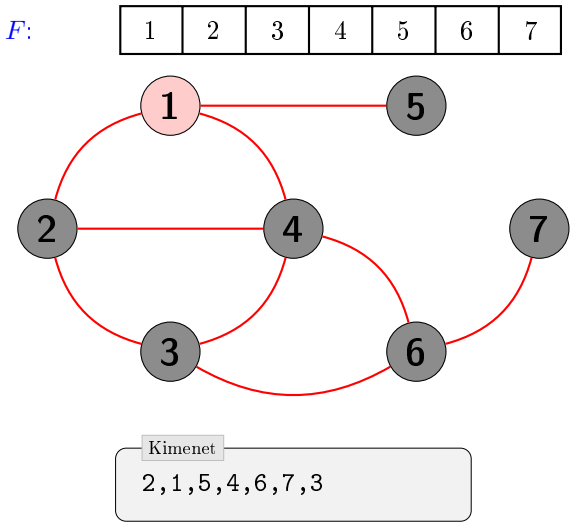
(i) A 6-osból el tudunk jutni a 3-ba is, mivel az még nincs feldolgozva.



(j) Mivel a 3-as minden szomszédja feldolgozott, ezért visszalépünk.

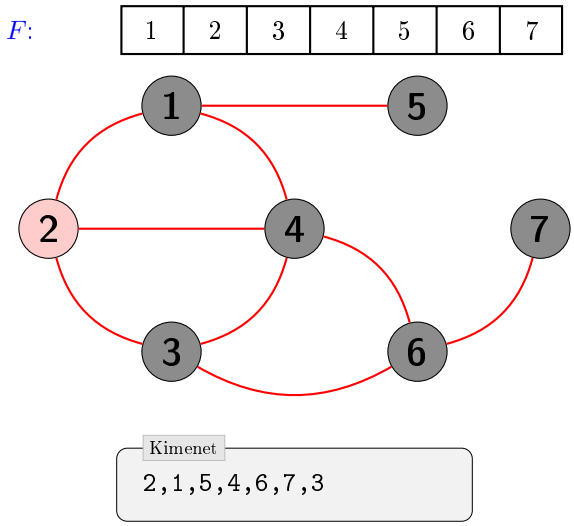


(k) A 6-osból is visszalép a rekurzió a 4-esre.

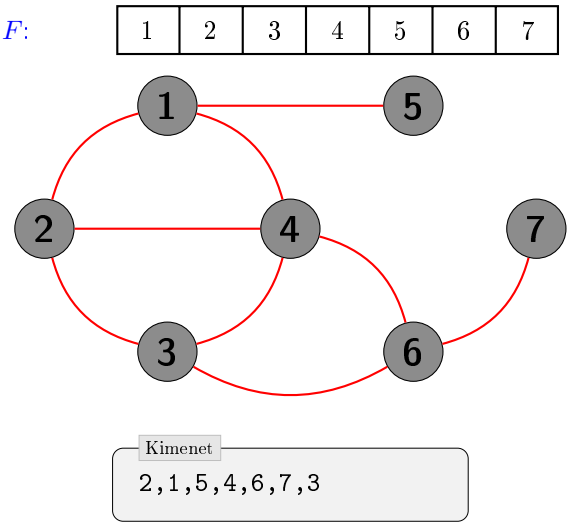


(l) A 4-esből is visszalép az 1-esre.

7.4. ábra. Gráf mélységi bejárása.(folytatás)



(m) Az 1-esből visszalépés a kiinduló elemre.



(n) A 2-esnek sincs már feldolgozatlan eleme, így az egész rekurzív hívás véget ér. Bejártunk minden csúcsot.

7.4. ábra. Gráf mélységi bejárása.(folytatás)

7.4. Legrövidebb út keresése (Dijkstra algoritmus)

7.4.1. Az alapelv

A szakirodalomban számos gráfokkal kapcsolatos problémára találhatunk algoritmusokat, az egyik ismert és a gyakorlatban jól használható példa a legrövidebb út keresése két csúcs között. Ennek egyik megvalósítása Edsger Wybe Dijsktra nevéhez kötődik, ezt a mohó algoritmust vizsgáljuk meg.

Feladatunk egy súlyozott gráfban az egyik csúcsból kiinduló legrövidebb utak megkeresése (legrövidebb alatt itt most azt az utat értjük, ahol az azt alkotó élek összsúlya minimális). Feltételezzük, hogy a gráf nem tartalmaz negatív súlyú éleket.

Az algoritmus alapelve, hogy egy szélességi bejárással végignézni a kiinduló pontból elérhető összes csúcsot, és minden csúcs esetében eltárolja az oda vezető legrövidebb út hosszát, illetve ennek az útnak az előző elemét (tehát, hogy honnan jutottunk el a kérdéses csúcshoz). A pszeudokódban ehhez két segédváltozót használunk:

- L: Ebben tároljuk el minden egyes csúcshoz a kiinduló ponttól való eddig talált legrövidebb távolságot. Ez lehet egy tömb, de akár egy hasító táblázat is, ahol a kulcs típusa gráfcsúcs, a tartalom pedig egy szám.
- E: Ebben tároljuk el azt az információt, hogy az egyes csúcsokat melyik csúcsból értük el a legrövidebb úttal. Ez szintén lehet egy hasító táblázat, ahol mind a kulcs, mind pedig a tartalom egy-egy gráf csúcs (a kulcs megadja, hogy melyik csúcsról van szó, a tartalom pedig azt, hogy honnan értük el).

Az eljárás működése az alábbi: a kiindulópontot tekintsük úgy, hogy elérhető, és 0 távolságra van (önmagától). Ezt követően elindítunk egy ciklust, amely az alábbi lépéseket hajtja végre:

1. Fogja a legkisebb távolsági értékkel bíró (L), még nem feldolgozott, de már elért elemet, legyen ez az u csúcs (az algoritmus indításakor ez nyilván csak a kiinduló pont lehet). Az u-t mostantól feldolgozottnak tekintjük.
2. Megvizsgálja u szomszédait. Amennyiben valamenyik x szomszédhoz talál rövidebb utat, mint amit eddig találtunk (tehát az u-hoz vezető út hossza + u és x közötti út hossza kevesebb, mint az eddig talált x-hez vezető út hossza), akkor módosítja x adatait úgy, hogy a L-be felveszi az új legrövidebb út hosszát, az E-be pedig rögzíti, hogy ezt az utat az u-n keresztül értük el.
3. Ezt követően addig ismételjük ezeket a lépéseket, amíg vannak fel nem dolgozott elemeink.

Miután lefutott ez a ciklus, belátható [2], hogy a L valóban azt fogja tartalmazni, hogy az egyes csúcsokhoz milyen hosszú volt a legrövidebb út. Ha pedig magára a csúcshoz vezető útra vagyunk kíváncsiak, akkor az E-n keresztül vissza tudjuk fejteni, hogy honnan jutottunk el az utolsó csúcsig, majd megnézhetjük, hogy honnan jutottunk ehhez az előző elemig, stb. Ezt addig folytatjuk, amíg el nem jutunk a kezdőcsúcshoz.

Gyakori módosítás, hogy csak két csúcs között keresünk utat. Ilyenkor ugyanezt az algoritmust elindítjuk a kezdőcsúcsból, azonban abban a pillanatban, amikor már a cél csúcsot kapjuk vissza a minimális távolság vizsgálatánál, be is fejezhetjük a keresést. Könnyen belátható, hogy a későbbiekben már biztosan nem fogunk jobb eredményt találni, hiszen a már feldolgozott csúcsokból rövidebb út nem létezik a cél csúcsba (ha lenne, akkor ez az érték szerepelne a L-ben), a nem feldolgozottak távolsága pedig már most is nagyobb (különben nem ezt a csúcsot kaptunk volna minimumként), negatív súlyú élek pedig nincsenek, tehát azokon keresztül sem számíthatunk rövidebb útra.

7.4.2. Dijkstra algoritmus pszeudokódja

A 7.3. algoritmus bemutatja az algoritmus részletes működését. Az első ciklus a felhasznált adatszerkezetek inicializációját végzi. A L hasító táblázat fogja tárolni az egyes csúcsokhoz eddig talált legrövidebb út hosszát, ennek minden elemét végtelenre állítjuk, mivel még nincsenek ilyen útjaink. Az E hasító táblázat fogja tárolni, hogy az előbbi minimális úton az egyes csúcsokat melyik szomszédjukból értük el, kezdetben itt is minden érték ø lesz, ezzel jelölve, hogy még nem értük el a csúcsokat. Az S prioritási sor kezdetben a gráf összes csúcsát kell, hogy tartalmazza, emiatt ezeket egyesével belehelyezzük. A konkrét

7.3. Algoritmus Dijkstra algoritmusa

Bemenet: G - **GRÁF**, *start* - **CSP**
Kimenet: E - **hasító táblázat**(**CSP**→**CSP**),T - **hasító táblázat**(**CSP**→**szám**)

```
1:  függvény DIJKSTRA(G : GRÁF, start : CSP)
2:      ciklus mind x ∈ G.Csúcsok
3:          L[x] ← ∞                          ▷ Távolság végtelen
4:          E[x] ← ø                          ▷ Nincs még előző elem
5:          S ⇐ x                             ▷ Csúcs a prioritási sorba
6:      ciklus vége
7:      L[start] ← 0
8:      ciklus amíg S ≠ ø
9:          u ← S.MinKivesz(L)                ▷ Legkisebb távolságú elem kivétele
10:         ciklus mind x ∈ u.Szomszédok
11:             ha L[u] + u.Súly(x) < L[x] akkor
12:                 L[x] ← L[u] + u.Súly(x)
13:                 E[x] ← u
14:             elágazás vége
15:         ciklus vége
16:     ciklus vége
17:     vissza (E, L)
18: függvény vége
```

Felhasznált változók és függvények

- L: Egy hasító táblázat, amelyik tárolja csúcsonként az addig talált legrövidebb út hosszát.
- E: Egy hasító táblázat, amelyik csúcsonként a legrövidebb talált úthoz tartozó előző csúcsot tárolja.
- S: Egy prioritási sor, amely a gráf csúcsait tárolja, és a legközelebbit vissza tudja adni.
- $S.MinKivesz(L)$: Az S prioritási sorból visszaadja azt a csúcsot, amelyhez tartozó L érték minimális. Egyben törli is ezt az elemet a sorból.
- u, x: **CSP** típusú segédváltozók.

prioritási értékeket itt még nem tudjuk rögzíteni, mivel azok a távolságokkal együtt változni fognak (a sor mindig a legkisebb távolsággal rendelkező csúcsot adja majd vissza).

Az inicializálást követően a 7. sorban beállítjuk a kezdőcsúcsot. Mivel az L az ettől való távolságot fogja mindig mutatni, így a kezdőcsúcshoz tartozó értéket kinullázzuk. A kezdőcsúcsnak nincs előző csúcsa, emiatt az E hasító táblázatot változatlanul hagyjuk (marad a $\emptyset$ érték).

A 8. sorban látható ciklus az előzőekben leírt módszerrel kezdi el az utak keresését. A ciklus megállási feltétele az, hogy elfogytak az S-ben lévő csúcsok. Egyelőre még a gráf összes csúcsa benne van a prioritási sorban, emiatt belépünk a ciklusba.

Minden ciklus iterációban ki kell vennünk a prioritási sorból a legkisebb távolsággal bíró elemet. Ezt a műveletet végzi el a 9. sor. Végignézi a sorban lévő csúcsokat, mindegyikhez megvizsgálja az L-ben hozzárendelt távolsági értéket, majd visszaadja az (egyik) minimális távolsággal bíró csúcsot (a konkrét programnyelvi megvalósításban persze elképzelhető, hogy ez egy kisebb szubrutin lesz, a pszeudokódban talán érhető az egysoros megoldás is).

Tudjuk tehát, hogy az u változóba betöltött csúcsnak már ismerjük a végleges távolságát. Vagy azért, mert a ciklus első iterációjakor az u tulajdonképpen nem más, mint a kezdőelem, így annak a távolsága 0, vagy pedig a fentiekben már ismertetett szabályok alapján. Ezt követően a belső ciklussal (10. sor) megvizsgáljuk ennek a csúcsnak a szomszédait (a ciklus futása közben ezekre mindig x-el hivatkozunk), és beállítjuk azoknak is a távolsági adatait.

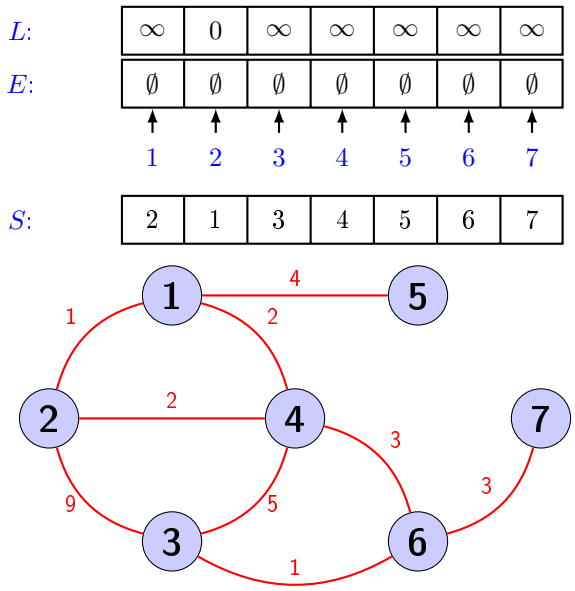
A 11. sorban lévő vizsgálat ennek a kulcsa, ugyanis itt ellenőrizzük, hogy az u csúcson keresztül elérve az x csúcsot rövidebb utat találunk-e, mint az eddig talált legrövidebb út. Ha még az eddigiek során nem értük el az x-et, az sem probléma, hiszen ilyenkor a $L[x]$ értéke az inicializáláskor megadott végtelen, tehát biztos, hogy ennél csak rövidebb utat találhatunk a későbbiekben.

Amennyiben azt találjuk, hogy az új út nem rövidebb, mint a régi, akkor nincs további teendőnk ezzel a szomszéddal. Azonban ha rövidebb utat találunk, akkor ezt a tény rögzítenünk kell. Egyrészt módosítjuk az L értékét (12. sor), hogy az valóban helyesen mutassa, hogy mi az eddig talált legrövidebb út hossza. Másrészt pedig módosítanunk kell az E értékét is (13. sor), hogy ezt a legrövidebb utat követve melyik csúcson keresztül értük el ezt a csúcsot.

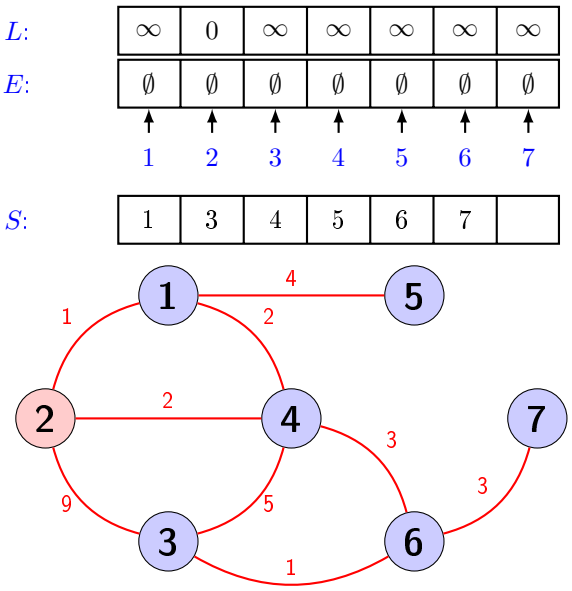
Miután átvizsgáltuk az összes szomszédot, akkor az u elem feldolgozása már véget is ért, a későbbiekben nem térünk vissza rá. A külső ciklus újabb iterációja betölti a következő minimális távolsággal rendelkező csúcsot, és folytatja annak a feldolgozását.

Amikor feldolgoztunk minden csúcsot, illetve azok szomszédait, az algoritmus futása véget ér. Az eredményeket a L és az E változók tartalmazzák. Ez a két táblázat a függvény visszatérési értéke, bár magához az út kereséshez elég lehet az E is. Hogy pontosan hogyan is tudjuk felhasználni az ebben lévő adatokat, arra hamarosan visszatérünk, előtte nézzünk meg az algoritmus működését a gyakorlatban.

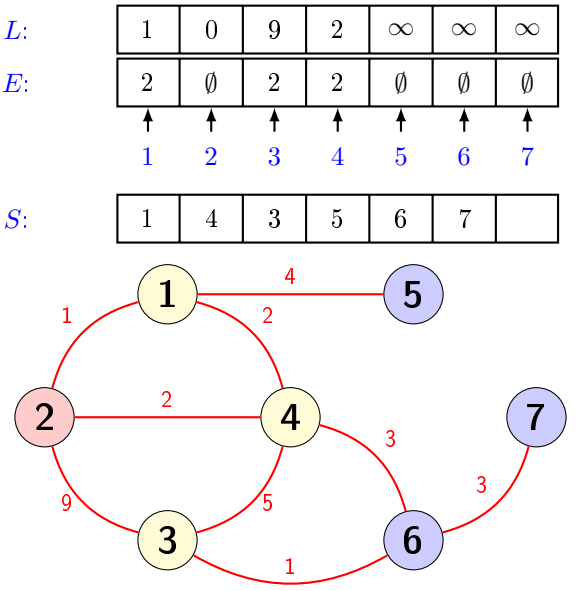
A 7.5. ábrán egy példát látunk, ahol egy irányítatlan súlyozott gráfban próbálunk legrövidebb utakat keresni. A kiindulópont a 2-es csúcs.



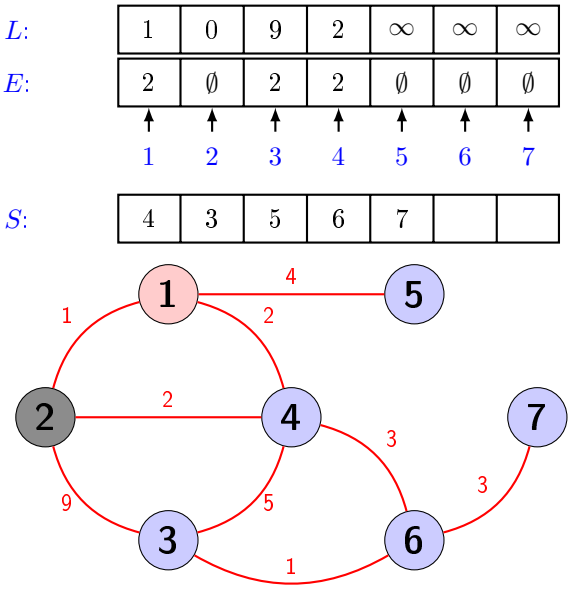
(a) Közvetlenül az inicializálást követő állapot. A kezdőcsúcs távolsága 0, megelőzője pedig $\emptyset$.



(b) A prioritási sorból kivesszük azt az elemet, amelyikhez a legkisebb L-beli érték tartozik. Ez a 2-es elem, ezzel kezdődik a feldolgozás.

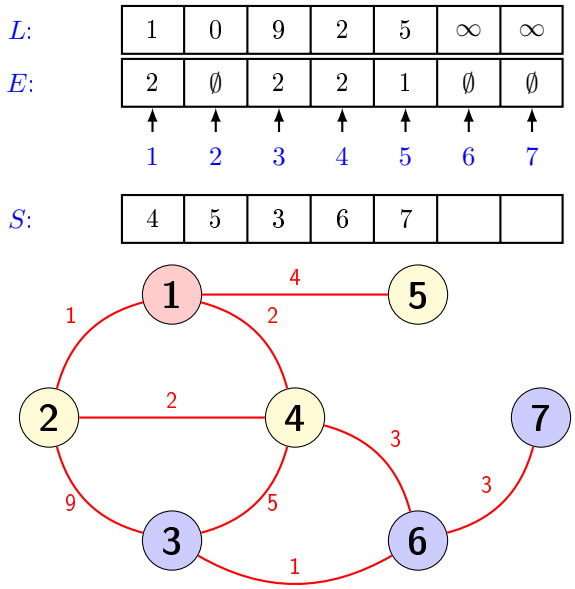


(c) Megvizsgáljuk a 2-es elem három szomszédját. Távolságunk nem más, mint a 2-estől indulú él súlya. Mivel ez mindig kisebb lesz, mint az inicializáláskor felvett végtelen érték, így ezzel felülírjuk az L-ben lévő értékeket, illetve beállítjuk az E megfelelő mezőit is, hogy honnan találtuk a legrövidebb utat.

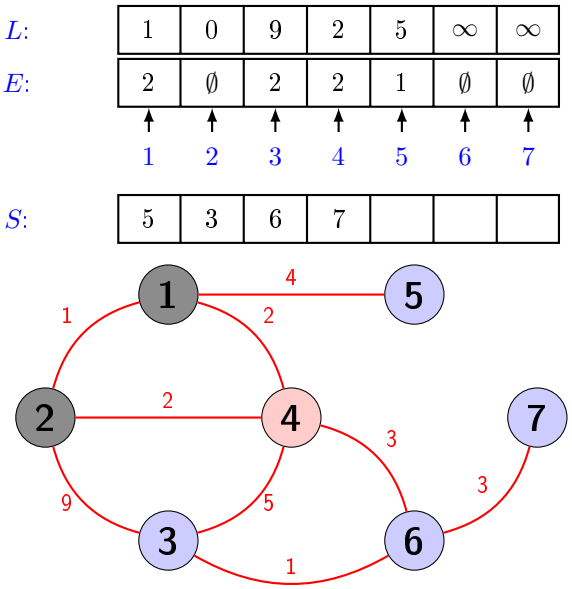


(d) A 2-es elemet ezzel feldolgozottnak tekintjük. A prioritási sorból ismét kivesszük azt, amelyik a legkisebb L-beli értékkel bír. Ez most az 1-es.

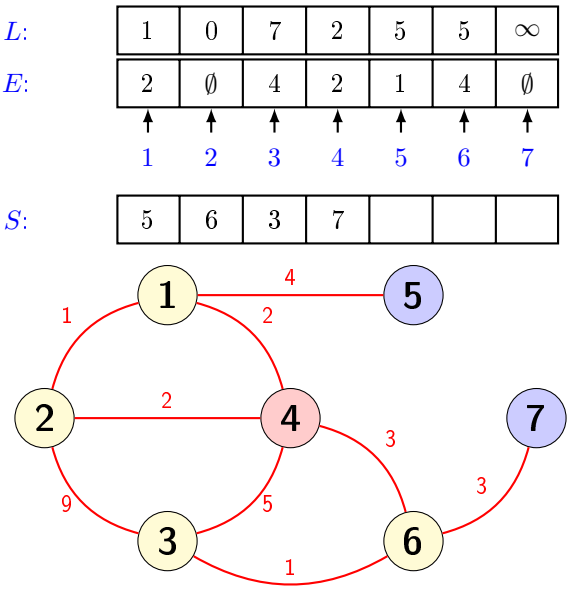
7.5. ábra. Dijkstra algoritmus működése.



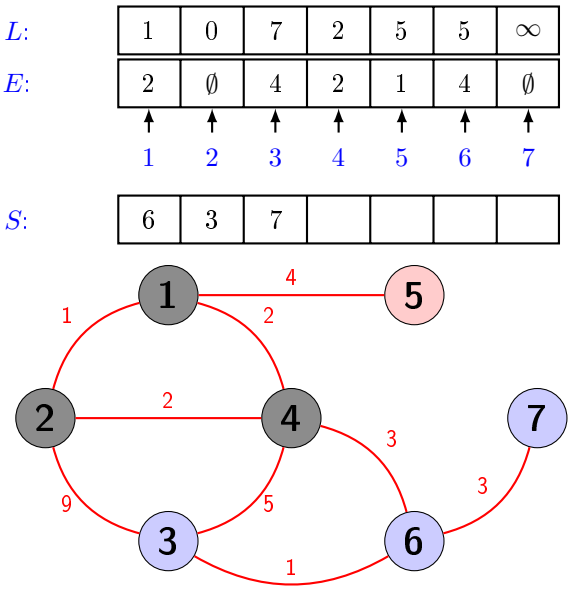
(e) Megvizsgáljuk ennek is a szomszédait, hátha az egyeshez vezető út hossza (1) + az egyesből vezető él súlya kisebb lesz, mint az eddig talált legrövidebb út. Az 5-ös elem esetében ez igaz, hiszen oda még nem találtunk utat. A 4-es esetében ez nem igaz, illetve a 2-es esetében értelemszerűen szintén nem lehet igaz.



(f) Az 1-est is feldolgozottnak tekinthetjük. Ismét ki kell venni a prioritási sorból a legkisebb L értékkel bíró értéket. Jelenleg ez a 4-es.

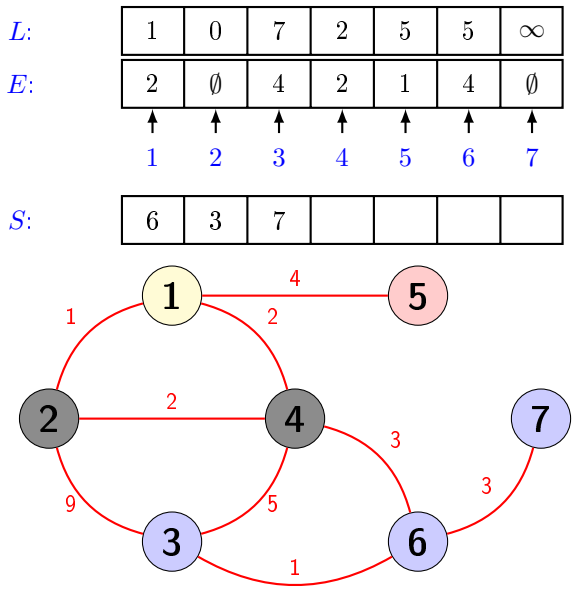


(g) A 4-es szomszédait is átvizsgáljuk, hogy találtunk-e a 4-esen keresztül haladó rövidebb utat, mint az eddigi legrövidebb. Az 1 és 2 esetében ez nincs így. A 3-as csúcsba így már 7 hosszú úton is eljuthatunk, tehát ez jobb mint az eddigi 9, emiatt cserélünk. A 6-osba pedig ez az első út amit találtunk.

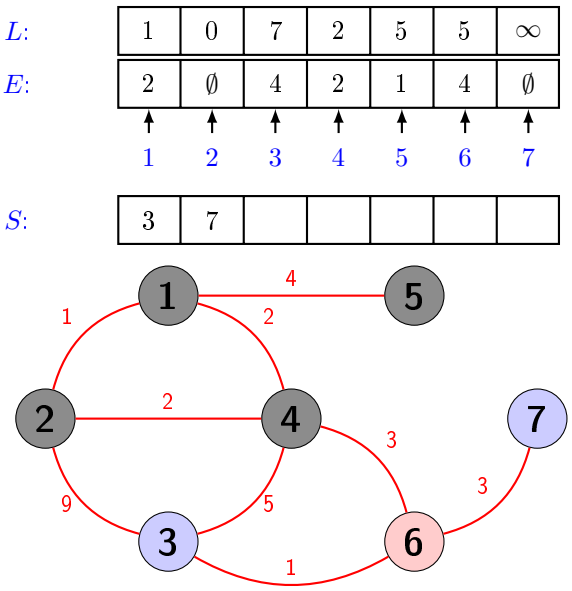


(h) A 4-es elemet ezzel feldolgozottnak tekintjük. A prioritási sorból ismét kivesszük azt, amelyik a legkisebb L-beli értékkel bír. Ez most az 5-ös vagy 6-os közül bármelyik lehet. Legyen az 5-ös.

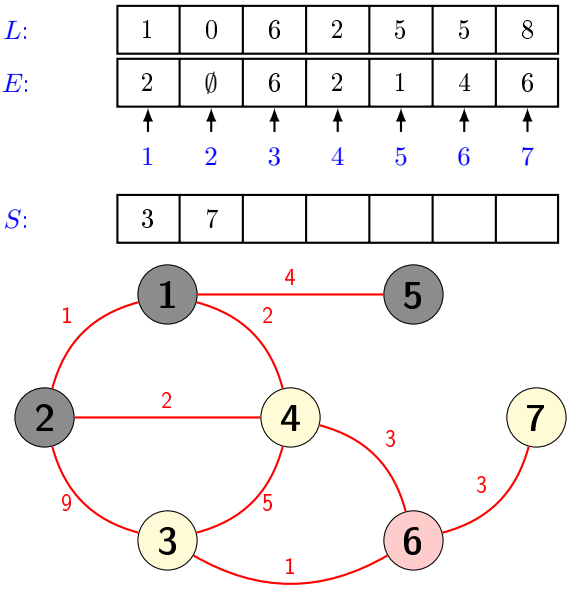
7.5. ábra. Dijkstra algoritmus működése.(folytatás)



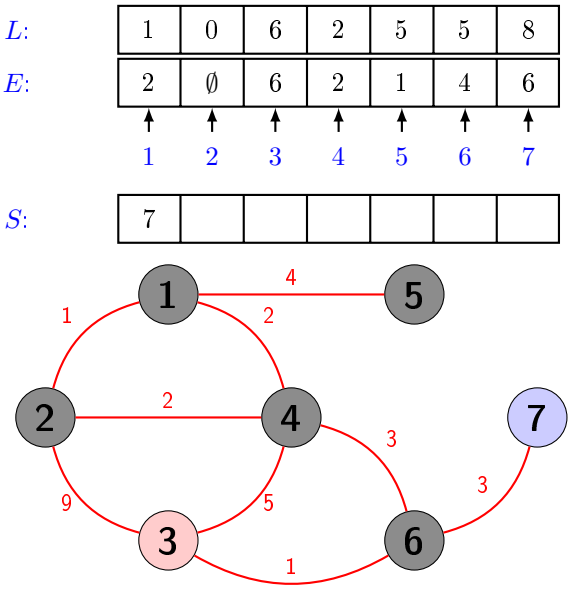
(i) Az 5-ösnek csak egy szomszédja van, és ide se találtunk jobb utat, így nem változik semmi.



(j) Az 5-öst is feldolgozottnak tekinthetjük. Ismét ki kell venni a prioritási sorból a legkisebb L értékkel bíró értéket. Jelenleg ez a 6-os.

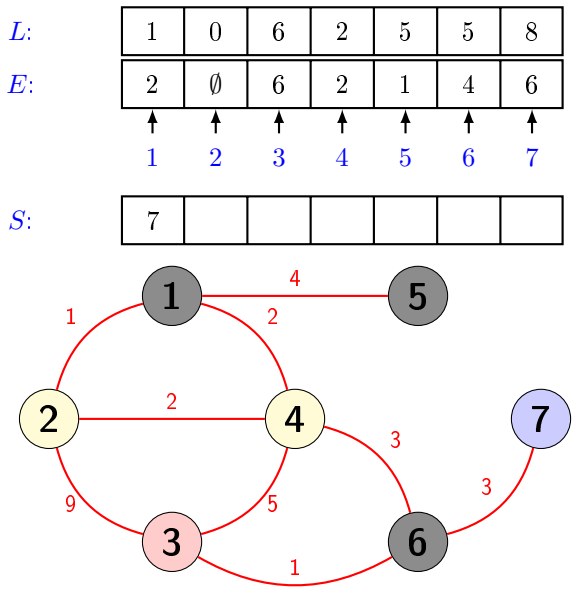


(k) Átvizsgáljuk a 6-os szomszédeit, hátha rövidebb a 6-osba eddig talált legrövidebb út (hossza 5) + a hatosból induló él hossza. A 3-as és a 7-es esetében ez igaz, ezért itt módosítjuk az L és az E értékeit.

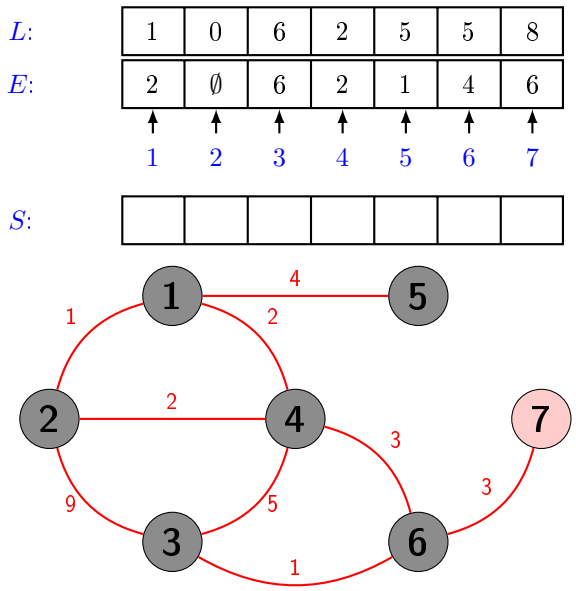


(l) A prioritási sorban már csak a 3-es és 7-es van. Ezek közül a 3-asnak kisebb az L-beli értéke, emiatt ez a következő.

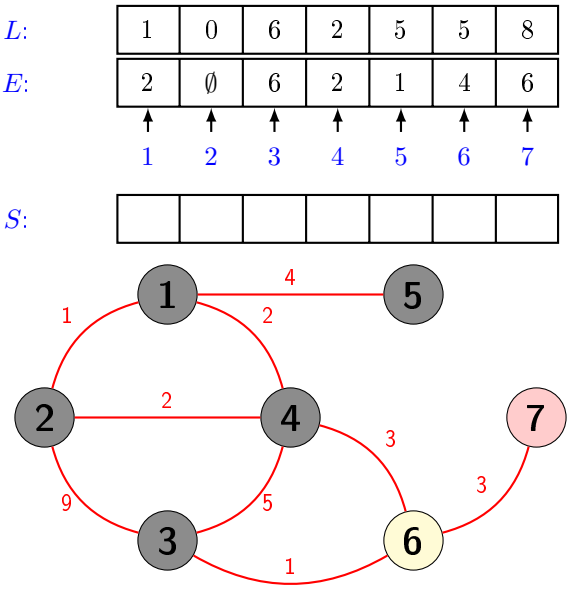
7.5. ábra. Dijkstra algoritmus működése.(folytatás)



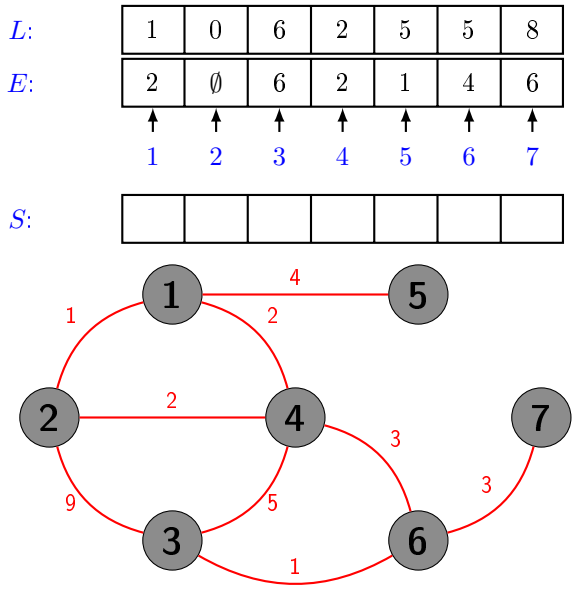
(m) Az 3-as szomszédai között nem találunk olyat, ahol javítani tudnánk a legrövidebb út hosszán.



(n) A prioritási sorban már csak a 7-es van, így ezt fogjuk feldolgozni.



(o) Mivel már mindenki mást feldolgoztunk, így értelemszerűen itt se találunk jobb értéket a szomszéd(ok) között.



(p) A sor üres, tehát az algoritmus futása végetért. A kimeneti adatok az E táblázatban vannak.

7.5. ábra. Dijkstra algoritmus működése.(folytatás)

7.4.3. Dijkstra algoritmus kimenetének feldolgozása

Habár a kiinduló feladatunk csúcsok közötti utak keresése volt, láthatjuk, hogy az előzőleg megismert Dijkstra algoritmus visszatérési értéke még nem pontosan ez. A visszatérési érték mindössze egy hasító táblázat, ahol azt tároljuk, hogy az egyes csúcsokba honnan érkeztünk a legrövidebb úton keresztül, illetve egy másik, ami a távolságokat tárolja. Ezek segítségével viszont már egyszerűen tudunk válaszolni az időközben felmerülő kérdésekre, pl.:

- Egy csúcsból melyik a legrövidebb út egy másik csúcsba?
- Egy csúcsból melyek a legrövidebb utak az összes elérhető csúcsba?
- Milyen messze van egy csúcs a kiinduló csúcstól?
- Milyen messze vannak az egyes csúcsok a kiinduló csúcstól?

A kiinduló pont tehát az lesz, amelyiket paraméterként átadtunk a Dijkstra algoritmusnak, a kérdés csak az, hogy melyik csúcs felé és melyik információra van szükségünk (maga az út, vagy pedig csak annak a hossza érdekes). A hossz ügyében nincs további tennivalónk, egyszerűen ki kell olvasnunk a szükséges értéket az L táblázatból. Ennek értéke az alábbi lehet:

- ∞: Azok a csúcsok tartalmazzák ezt az értéket, amelyeket nem sikerült elérni az algoritmus futása során, tehát ezekbe nem vezet út a kiinduló csúcsból.
- 0: Tipikusan a kezdőcsúcs értéke (persze más csúcsé is lehet, ha megengedjük a 0 súlyú éleket).
- Egyéb szám: A megadott csúcs távolsága a kiinduló csúcstól (tehát a hozzá vezető legrövidebb úton az élek súlyainak összege).

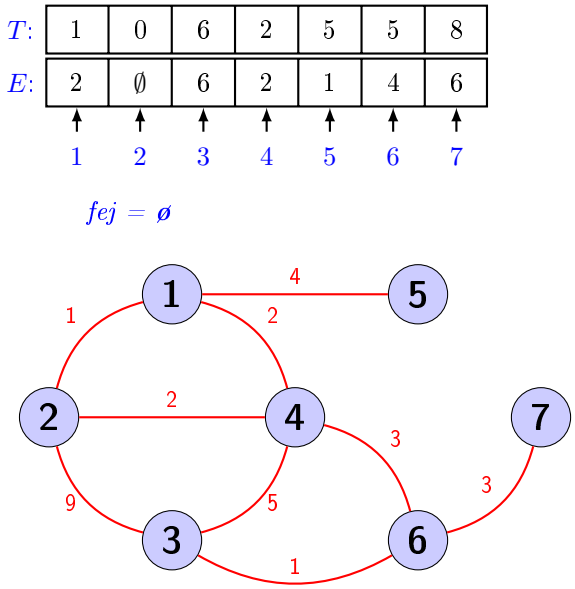
Érdekesebb kérdés az, ha magára a csúcshoz vezető útra van szükségünk. Ezt az utat az E táblázat alapján tudjuk előállítani. Ez minden csúcs esetében azt tárolja, hogy a hozzá vezető legrövidebb úton melyik volt az előző csúcs. Ennek megfelelően magát az utat is visszafelé tudjuk előállítani az alábbi lépéseket követve (az *ide* csúcshoz vezető utat keressük):

1. Töltsük be az x változóba az $E[ide]$ értékét. Ez visszaadja azt a csúcsot, amelyből az *ide* csúcsba tudunk majd jutni.
2. Mentsük el az x értékét, majd cseréljük ki az $E[x]$ értékével. Ezzel megkapjuk az aktuális x előtti csúcsot.
3. Addig ismételjük a fentieket, amíg az x értéke nem lesz **ø**, ez ugyanis csak a kezdőcsúcsra lehet igaz (az el nem ért csúcsoknál is ez az érték van az E-ben, de azokhoz nem juthatunk el ilyen módon).

A fenti módszerrel megkapjuk az utat alkotó csúcsokat visszafelé. Szükség esetén ezt a sorozatot meg kell fordítani, és ezzel megkapjuk a keresett utat. Ezt mutatja be a 7.4. algoritmus. Látható, hogy az utólagos megfordítás helyett egy láncolt listának az elejére beszúrását használjuk, ami ugyanazt eredményezi.

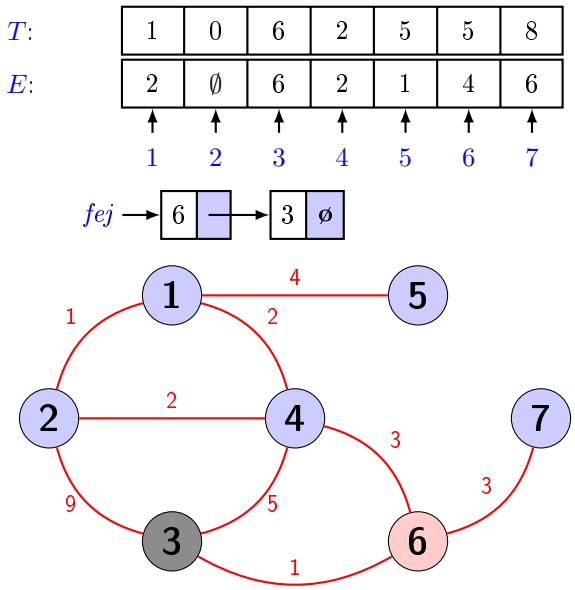
Amennyiben az összes útra van szükségünk, akkor ezt a műveletet kell megismételnünk az összes csúcsra.

Két csúcs közötti út keresésre (az előzőleg is feldolgozott gráfban a 2-es és a 3-as csúcs között keressük a legrövidebb utat) mutat egy példát a 7.6. ábra.

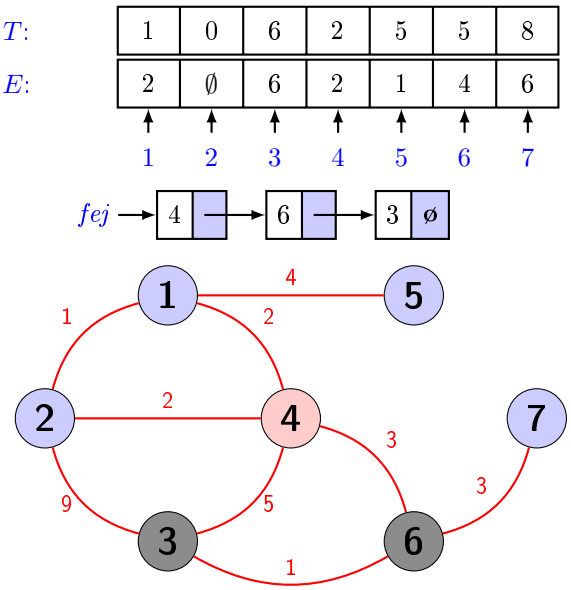


(a) Az algoritmus lefuttatja az előzőleg megismert Dijkstra függvényt. Mivel ugyanaz a gráf és a kiindulási pont, így a kimenet is ugyanaz marad. A fej mutató kezdetben egy üres láncolt listát jelez csak.

(b) Az x-be betöltjük a végcsúcsot, ami jelen esetben a 3-as. Az első ciklus iterációval ez azonnal bekerül a láncolt listába is.



(c) Az x-be betöltjük az E 3-hoz tartozó értékét, ami jelenleg 6. Ez egyben bekerül a láncolt listába is.



(d) Az x-be ismét bekerül az E x-hez tartozó értéke, ami jelenleg 4. Ez is bekerül a láncolt lista elejére.

7.6. ábra. Két csúcs közötti legrövidebb út keresése.

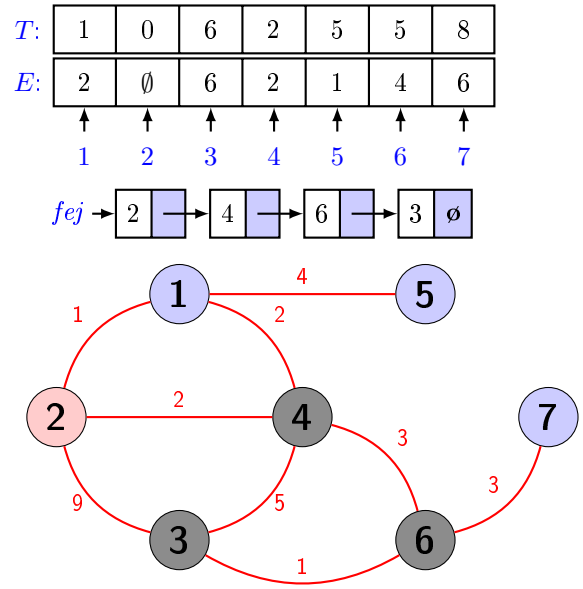
7.4. Algoritmus Legrövidebb út két csúcs között

Bemenet: G - **GRÁF**, *innen* - **CSP**, *ide* - **CSP**
Kimenet: *fej* - **M** (*az utat alkotó csúcsokat tartalmazó láncolt lista*)

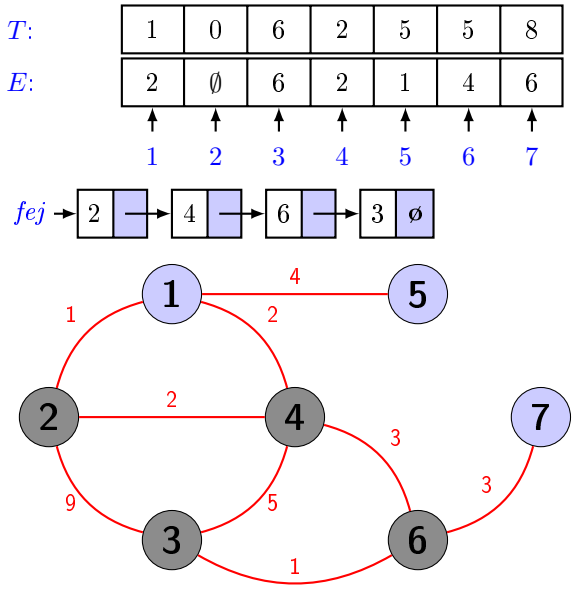
```
1:  függvény LegrövidebbÚt(G : GRÁF, innen : CSP, ide : CSP)
2:      (E, T) ← Dijkstra(G, innen)
3:      ListaInicializálás(fej)
4:      x ← ide
5:      ciklus amíg x ≠ ø
6:          ListaElejéreBeszúrás(fej, x)
7:          x ← E[x]
8:      ciklus vége
9:      vissza fej
10: függvény vége
```

Felhasznált változók és függvények

- Dijkstra(G : **GRÁF**, *start* : **CSP**):(**hasító táblázat**(**CSP**→**CSP**), **hasító táblázat**(**CSP**→**szám**)): Végrehajtja a Dijkstra algoritmust a paraméterként átadott gráfon és kezdőcsúcsból (7.3. algoritmus).
- x: **CSP** típusú segédváltozó.
- ListaInicializálás(*fej* : **M**): Inicializálja a láncolt listát (4.1. algoritmus).
- ListaElejéreBeszúrás(*fej* : **M**, *érték* : **T**): Felvesz a láncolt lista elejére egy új elemet (4.2. algoritmus).



(e) Az E segítségével ismét eggyel visszább lépünk, az x értéke most 2. Bekerül a láncolt listába.



(f) A ciklus következő iterációjánál az x értéke ø lesz, emiatt véget ér az eljárás. Visszatérési értéke a *fej* által mutatt láncolt lista, ami valóban a keresett legrövidebb utat tartalmazza.

7.6. ábra. Két csúcs közötti legrövidebb út keresése.(folytatás)

7.5. Minimális feszítőfa keresése

7.5.1. Prim algoritmusa

7.5.2. Kruskal algoritmusa

Irodalomjegyzék

[1] A. V. Aho – J. E. Hopcroft – J. D. Ulmann: *Számítógép-algoritmusok tervezése és analízise*. 1982, Műszaki Könyvkiadó. ISBN 963-10-4323-1.

[2] T. H. Cormen – C.E. Leiserson – R. L. Rivest: *Algoritmusok*. 2003, Műszaki Könyvkiadó. ISBN 963-16-3029-3.

[3] Gy. Y. Katona – A. Recski – Cs. Szabó: *A számítástudomány alapjai*. 2002, Typotex. ISBN 978-963-9664-19-7.

[4] D. E. Knuth: *A számítógép-programozás művészete - 1. kötet. Alapvető algoritmusok*. 1987, Műszaki Könyvkiadó. ISBN 963-10-7156-1.

[5] D. E. Knuth: *A számítógép-programozás művészete - 3. kötet. Keresés és rendezés*. 1988, Műszaki Könyvkiadó. ISBN 963-10-7206-1.

[6] D. Kotsis – G. Légrádi – G. Nagy – S. Szénási: *Többnyelvű programozástechnika*. 2007, Panem Könyvkiadó.

[7] G. Pap – P. Szlávi – L. Zsakó: *mikrológia 27 - Módszeres programozás: Rekurzív típusok*. 1998, ELTE TTK.

[8] G. Pap – P. Szlávi – L. Zsakó: *mikrológia 34 - Módszeres programozás: Adattípusok*. 1998, ELTE TTK.

[9] Sz. Sergyán: *Algoritmusok, adatszerkezetek I.* 2014, Óbudai Egyetem.

[10] S. S. Skiena: *The Algorithm Design Manual*. 2008, Springer. ISBN 978-1-84800-069-8.

[11] P. Szlávi – L. Zsakó: *mikrológia 38 - Módszeres programozás: Gráfok, gráfalgoritmusok*. 2001, ELTE TTK.

[12] P. Szlávi – L. Zsakó: *mikrológia 19 - Módszeres programozás: Programozási tételek*. 2004, ELTE TTK.

[13] N. Wirth: *Algorithms and Data Structures*. 1985, Prentice Hall. ISBN 0-13022-005-1.